

A++ 敏捷开发

Contents

前言 Prologue	19
I 案例分析	21
1 如何改善	23
敏捷开发过程改进案例	23
如何改善	28
第一堂课	28
丰田故事	30
水面下的管理思路	31
改善质量	38
结束语	41
附件	43
现代汽车生产	43
“ 5 Why ” 实例	45
5S 法	46
II 团队与自我改善基本功	47
2 改进从团队开始	51
美国费城 Weisbord 故事	51
处在 X 思维的主管	52
启发	54
芬兰计划	55
参考 References	58
3 克服拖延症	61
附件	66
锻炼之道 The Truth about EXERCISE	66
参考 References	67

III 自我管理开发过程	69
4 三点估算	73
估算是一个范围，不是一个数	73
从单点到三点估算	73
总结 + 解读分析结果	75
利用蒙特卡洛模拟 10 个步骤（三角形分布）的总分布	76
分析 10 个步骤模拟结果	77
问答 Q&A	78
附件	79
蒙特卡洛 (Monte Carlo) 模拟	79
5 量化管理从个人开始	83
如何降低项目进度偏差	83
从量化培训开始	84
从估算、策划到监控	84
如何改善	86
附件	88
挣值分析法 (Earned Value)	88
如何简单记录工时	89
个人软件过程 Personal Software Process (PSP) 简介	89
使用挣值分析监控案例	91
参考 References	94
6 估算软件规模	95
为什么要估算规模	95
为什么不应用代码行数 (LOC)	95
怎样估算简化功能点 (SiFP)	97
从故事点转简化功能点	98
附件	101
简化功能点 (SiFP) 简介	101
逻辑文件 (Logical File)	106
基本过程 (Elementary Process EP)	107
实例：识别基本过程 (EP)	108
案例 A: 开发 (Development) 项目	109
答案与解读	110
案例 B: 优化维护 (FEM) 项目	110
答案与解读	111
与国际功能点 (IFPUG) 的偏差	112
参考 References	113
7 策划与监控	115
如何管理每轮的活动依赖和人手分配	120
总结	123
附件	123
项目概览 POS 模板	123

参考 References	124
-------------------------	-----

IV 开始过程改进之旅 125

8 获取高层支持 129	129
案例	129
第一版方案书	130
第二版与中层确认，然后向高层汇报	131
9 寻找改进机会 133	133
破冰 - 变革 - 稳定	133
成功要素	135
A 公司过程改进之旅	136
回顾进展缓慢的原因	138
如何能加快破冰	139
后记	142
附件	143
睡衣工厂实验 Harwood Manufacturing	143
粮食分配实验	143
互动工作坊 (workshop)	144
参考 References	145

V 如何做好迭代回顾 147

10 二八原则 151	151
怎样开始	152
附件	154
二八原则 (80/20)	154
开发项目工作量（成本）分布	156
公司高层不一定关注质量	157
11 分析根因 159	159
根因分析的误解	159
按团队水平因材施教	161
黄带水平	162
绿带水平	163
如何分析与对应各类由于人员引起的缺陷	165
总结	167
附件	168
机场延误数据分析	168
FMEA 实例	169
12 软硬兼施 173	173
回顾流程	175
内部教练如何做好现场辅导	176

团队迭代回顾根因分析常见问题	177
附件	178
Asch 1951 群众压力实验	178
Brehm 1956 决策影响实验	180
KJ 分析方法步骤	181
游戏: 应与否	183
功能性分组帮助团队成长实例	184
敏捷回顾互动练习 1: 时间表 (Timeline)	185
敏捷回顾互动练习 2: 颜色点 (Color Code Dots)	186
13 从团队实验到持续改善	189
收集哪些数据	189
缺陷排除率	191
缺陷排除预测模型实例	191
怎样开始	197
如何能简单有效地收集编码缺陷与返工工作量数据	198
从迭代复盘到持续改进	198
总结	199
参考 Reference	199
VI 软件工程质量	201
14 精益与敏捷	205
2017 年	205
每小步验证	206
精益? 是敏捷的概念吗? 为什么 2001 年提出的敏捷宣言和 12 条 原则都没有提精益?	207
浪费 (Waste)	208
非开发人员如何应用精益	209
系统工程 V 模型	210
参考 References	210
15 TDD 与重构	213
测试驱动开发 (Test Driven Delivery TDD)	213
如何以数据说话, 持续改善	214
管理软件开发团队的困难	214
设计	215
参考 References	218
16 评审与结对编程	219
背景	219
2 天培训课	220
代码评审	221
培训设计	221
培训重点	221

经验分享	222
总结	222
附件	223
Ex 5	223
17 持续集成	225
验收测试	225
测试自动化	226
持续集成	227
团队协作、不断替换	228
持续交付	229
附件	230
持续集成	230
技术债务例子	231
参考 References	231
18 软件需求	233
需求并不是关于需求 (Requirements are not really about requirements)	233
需求分析	233
识别利益相关者	237
非功能性需求	238
其他最佳实践	238
用户故事卡	238
做好需求评审	239
总结	239
附件	240
利益相关者	240
客户声音: Kano Diagram	242
线上续护照的异常场景	242
参考 Reference	243
19 协作改进整个系统	245
需求与研发	245
编码以外	246
小结	247
改进整个系统 (System Thinking)	247
西南航空 (SW Airline)	247
IBM 信贷 (IBM Credit)	248
Value Stream Map 价值流图	249
总结	249
附件	250
练习答案	250
参考 References	250
20 从 AI 到最佳设计	251

两个故事	251
向煤矿工人学习	252
1951 年, 贝尔实验室的会议室	253
开发团队如何应对 AI 大趋势	255
总结	259
附件	259
英国煤矿生产问题的研究	259
ChatGPT 自动生成代码质量研究结果重点	263
参考 References	265
VII 度量与分析	267
21 取消量化目标	271
建立基线	272
控制图的应用例子	274
软件开发应用实例	276
总结	279
附件	279
中心极限定理 (Central Limit Theorem CLT)	279
如何画 ImR 控制图	279
各种控制图	280
过程能力 (Process Capability)	281
参考 References	282
22 教小孩统计分析	283
从考试成绩开始	283
分析例子: 连续与分组数据	289
总结	291
附件	292
如何画茎叶图 (Stem and Leaf Plot)	292
箱线图 (Box and Whisker Plot)	292
卡方检验 (Chi Square test)	293
参考 References	293
23 做问卷调查	295
案例: 为某全国快餐连锁做问卷调查	295
度量分析计划	299
总结	300
参考 References	301
24 预测模型	303
从数据开始	303
挑选最佳	304
自动化	306
总结	307

参考 References	307
-------------------------	-----

VIII 总结 309

25 创新：从个人到公司 313

400 年前的重大数学发明	313
原创精神 Spirit of Creating	315
从个人到公司	317
音乐播放器	317
iTunes 网上商店	318
公司如何创新	319
万事起头难	320
从濒临破产变为回报最好的投资	320
成功要素	321
总结	323
附件	324
Pixar 制作第一部电脑动漫：玩具总动员 (Toy Story)	324
参考 References	325

26 北京手记 327

创作泉源	329
参考 Reference	331

27 根与翼 333

根	333
翼	334
共通点	337
TED 演讲	338
东北虎	340
总结	340
参考 References	341

IX 附录 343

28 附录 A：绅士俱乐部过程改进 345

原因分析与解决	345
参考 References	348

29 附录 B：过程改进工作坊 349

背景	349
案例	350
总结	353
附件	353
互动工作坊 (workshop)	353

Dedication

To Maria: Thank you for your support and love. Otherwise, I could not complete this project.

----- Edmond

序 Preface

这是一本从事敏捷开发和过程改进的人员的必修之书，作为本书的早期读者，有两个创新让我印象深刻。其一是本书的写法，类似《金刚经》体，全文多是以对话的方式，让读者读起来轻松流畅，少了长篇大论的说教，而是加入了真实的、与不同 IT 企业高管、中层及基层开发和技术人员的对话，在问答中将普遍问题进行归纳总结，并提出解决方法。其二是案例和数据的引用，虽然很多是引用书籍中的案例和数据，宋老师能结合自己多年经验，对案例和数据的引用力求精益求精，每个案例都颇为经典。阅读和理解这些案例能加深对会话中问题的领悟，也加深了对相关建议和解决方案的理解。

读过很多管理类的书籍，有些书籍将管理数据自始至终贯穿如一，例如德鲁克的《卓有成效的管理者》；有的则以故事的形式给企业管理者以启示，例如《目标》。而本书的内容始终如一地贯穿了过程改进的主题，通过引用一些小故事，宋老师将多年来在该领域的培训和咨询的经验跃然纸上，让书籍活跃起来，使读者读起来不至于太累。诚然，一千个人有一千个哈姆雷特，具体有什么收获，就请大家在本书的阅读中去体会吧。

除此以外，给我感触更深的是宋老师严谨的写作态度，每次他写完一章，都会发给不同的管理者阅读，邀请他们提供改进的意见和建议，每一个建议都会得到反馈并在相关章节中加以完善。这种态度是非常值得尊敬的，毕竟在他才是该领域货真价实的专业人士，而我们这些先期读者只是普通的从业者。最后，预祝本书能帮助到那些有志于在管理上提升的从业人员。

杨立东《微管理》作者，四维世景科技（北京）有限公司总经理
2023 年 3 月于北京

第二版序 Preface to the Second Edition

从 2023 年底第一版定稿后，在使用《A++ 敏捷开发》的材料与团队交流时，发现有些内容容易引起误解，第二版针对这些部分做了修正和完善，但总体架构和章节基本没有变动。改动较大的部分包括：

第五部分《如何做好迭代回顾》：

里面的 4 个章节都做了大幅修改，按企业领导解读的视角，先用《二八原则》选择针对软件开发的方面去分析，然后介绍了不同的软件根因分析的技巧后，第三章针对人性部分，如何确保团队成员的投入和积极性，最后第四章里加入了实际的团队常见的问题，对应解决方法和成功案例等。

最后第八部分：

在公司创新的具体实例后加入了 26 章《北京手记》，利用 3 场表演说明个人每天听、看、读、写对改善、创新的重要性。最后 27 章《根与翼》也加了实例说明根与翼，最后总结这部分的概念如何帮助敏捷团队和公司持续改善、保持竞争力。

感谢 Acknowledgments

首先感谢我的“写作伙伴”杭州的徐洪洋，北京的洪一海、王国军、王绍军、赵丽在百忙之中也抽空看我的初稿，指出错误，提出建议。因自己是难以看到自己的不足，如果没有他们，难以确保作品的质量。

也感谢朋友圈里各位老师、行业精英们分享经验和意见，包括北京的杨立东、武宏伟、纪钟涛，天津的韩淑军，上海的周青龙，杭州的程文进、姚钧、胡蕊莉，成都的杨杰等。

如果没有与这些老师、客户、学生和朋友的面对面详细讨论，并在项目过程中反复试验与反馈，不可能写出这本书。这些宝贵经验帮我验证了各种敏捷思路与量化管理的可用性，也让我更加清晰地理解了质量改进的重点。

最后感谢无锡的陈镜庆、秦瑜不断提出修改意见，并帮助我最后完成本书的编辑；

也感谢我香港大学老同学李启良先生对 Mediawiki 服务器、Linux、LATEX 等平台的技术支持。

前言 Prologue

前言 Prologue

We are uncovering better ways of developing software by doing it and helping others do it.

–Agile Software Development Manifesto

敏捷宣言已经有超过 20 年的历史，国内越来越多软件开发团队开始采用敏捷和迭代，但总体效果参差不齐。有些年轻的团队成员听到敏捷不需要文档，以为也不需要注重代码质量，包括代码可读性，导致后面发布的软件产品问题多多，难以维护。要编写出高质量的代码，人本身能力非常关键，但软件工程快速发展，导致程序员人数快速增长，其中很多缺乏专业工程师素养，所有若要敏捷开发真正起作用必须先提升程序员能力。

没有数据就无法管理，但很多敏捷团队只是走流程（每天站立会议、看板并非敏捷的重点），缺乏度量，所以管理者一听到敏捷就觉得不靠谱，要求团队用回传统的瀑布开发方式。

这 20 年来，中外都出版了很多关于敏捷的书，但绝大部分都没有深入去探索以上的问题。这本书就是希望通过解读各种敏捷最佳实践，加上敏捷以外的其他知识，帮助大家理解并更好使用敏捷，提升软件的质量和总生产率，让团队成员与公司管理层获得双赢。同时也希望管理层通过这本书能了解敏捷开发的要素，并能使用敏捷开发模式，帮助公司提升软件产品质量，同时降低成本，来提高公司的竞争力。

某技术总监陈总在 5 天差距分析的最终总结会里问开发团队：

1. 你们知道自己是每天产出多少代码行吗？（生产率）
2. 你们知道平均修复一个系统测试的缺陷花费多少工作量（人时）吗？
3. 有没有发生过：代码开发完成，系统测试人员尝试运行，但跑不动的情况？（开发人员不能借口说不懂业务，或需求不清，因为你写完代码后，自己连最基本确保代码能运行都没有去做。我还未问你们做开发的有没有做好单元测试/集成测试。）
4. 你们做测试的知道测试覆盖率是多少？

陈总有丰富开发经验，他最后总结说：“我以前自己做开发时也是被问过这些问题，我当时也不懂，但我能知耻而后勇，XXXX。先知道现在的不足，然后按评估老师的最佳实践，持续改进。”

- 如果你像陈总下面团队一样，不懂以上问题的答案，这手册可以帮你自己找答案。
- 如果你（是管理者或客户/用户）不满意团队的软件开发成果，这手册可以帮你找出根因，启发团队改善。
- 如果你已经很了解，恭喜你！但应还可能从这手册里找到一些你未知但有用的方法。

1980 年，STROUSTRUP 先生不满意当时很流行的 C 语言，因为在 70 年代，已经有如 SmallTalk 之类的面向对象语言，C 是基于传统步骤式的语言，没有面向对象的功能，所以 STROUSTRUP 先生就在 C 的基础上，加上有类功能的 C (C with Classes)。过了 4 年，他同事觉得这个名字不好，就改成 C++，也出了一本书叫《C++》，与本来经典的 C 很相似。后面 C++ 就成为了面向对象的常用语言。

A++ 继承 40 年前 C++ 的思路，希望在敏捷 (Agile) 开发的基础上，加上增值的元素，帮助团队与管理者做好软件开发，让客户满意。

Part I

案例分析

Chapter 1

如何改善

敏捷开发过程改进案例

5 月

A 公司一直为某电信公司专门提供客服、线上播放等 IT 业务。张工是公司的中层管理者，管理好几个开发团队，有 5 位项目经理向他汇报。他听说老同学的团队都开始用敏捷开发，很感兴趣，便参加了几次敏捷交流会，觉得可以解决很多开发团队的问题，尤其是可以快速交付给客户。他便提建议给部门经理推动敏捷开发，找咨询公司做相关培训，例如 SCRUM Master 内部培训，然后全面开展实施。开始时，部门经理有些怀疑，问：“听起来很吸引，但后面那些工程文档都不做，会不会影响质量和交付？客户都是专业做电信的，不缺钱，但是对质量要求很高。”张工解释说：“只要利用敏捷把过程变成迭代，快速交付、改善工程的问题不难，主要是人的问题。”部门经理听到敏捷可以比以前更快速交付，因之前客户经常抱怨项目延误，他也希望可以改变，就答应了。

8 月

开发组长王工，三十出头，毕业后一直做开发，一年前晋升为组长。周五下班后与朋友喝酒，很开心地说：“太兴奋了。我们团队刚参加了两天 SCRUM 培训，并指出我们以前按传统瀑布式开发的种种问题。我们会两周一次迭代并割接上线，还会与用户代表定期交流，不再会像以前几个月交付后才发现开发出来的功能不合需求。现在团队合作不像以前只按工种工作，也会跟产品经理、业务方面更充分合作，给客户带来更高价值。工作方式也会改变，以前要写需求规格说明书，现在简单化成产品需求卡片，以前我们要做详细项目计划和甘特图。现在会改成用燃烧图和“KANBAN”。每天用便利贴去写要开发什么内容贴在白板上，我们会改名叫 SCRUM Team。

工作区周围都有 SCRUM 的海报，提醒我们 SCRUM 的重点。团队也没项目经理了，自己管理自己。部门经理会变成产品负责人，敏捷开发方式让我们团队自己做决策，不仅仅是技术方面，项目相关的也由我们项目组一起讨论决定。”

9 月

王工的朋友老杨周五晚喝酒时问他：“你们团队使用敏捷 SCRUM 后，效果如何？”

王工立马面露笑容，充满自信地跟大家说：“我们培训后就使用 SCRUM 的方法，每 2 周一个冲刺，每次冲刺前都会用故事点来估算每个功能多大，然后按本次冲刺的资源，估计可以完成多少功能。然后用看板来监控模块完成的情况，哪些正在开发中？哪些已经完成？团队和管理者都可以一目了然，不用像以前……。”

“听你这样说，我也要尽快提议领导引入敏捷开发。”

“我还没说完，我们每天早上也按照 SCRUM 的规定站立会议，每人说自己完成了哪些任务，今天做什么……。”

10 月

喝酒时老杨问王工：“你们项目如何？”

王工听完，没有立马回应，把注意力放在窗外的大街上，想了一下，然后说：“我们本应上周要完成一次冲刺后的割接上线，但被推到下次了。”

“为什么？”

王工说：“我们按培训学到的做冲刺计划会议，按照产品的待办事项列表，团队利用扑克牌一起估算每一事项所需要的时间。我们总共 8 位开发人员，其中有一半是刚毕业不久，但大家刚上完培训，很有信心，虽然技术主管张工对我们的估算有些顾虑，觉得我们太理想，但大家刚培训完敏捷，张工也希望让部门经理尽快看到敏捷开发可以加快速度，我们就按这‘进取式’估算开展两周冲刺。但因新人多，编码水平有限，虽然大家已经尽快把开发出来的代码交给系统测试人员手工测试，依据测试发现的缺陷修正再测试，但割接上线期限前 4 天还有很多 Bug 没改好，最后 3 天，基本是天天加班，最终到上线期限仍然有不少问题，最终割接前测试，还是不能达到客户要求的水平，没办法上线。大家确实都尽力冲刺了，但未能达到我们本来希望的结果。”

“你们开发人员在提交代码到系统测试之前，有没有自己先自测？”

王工说：“我们敏捷开发每天早上站立会议上都用看板，记录每个模块的进度完成情况，开发编码人员都说自测过。”

“如果开发人员能自己把握好交付代码的质量。不应该等到系统测试才发现这么多问题要解决。你们团队是怎么做自测？有记录吗？”

王工说：“这我就没有详细问了，因为我们学完 SCRUM 后，大家更关注开发速度，用看板保证按计划进度，不延误。但没想到后面缺陷会这么多。而且有些问题改完以后还会导致另外一些 Bug 发生。到了最后，因为赶时间，大家对修改 Bug 已经麻木了。”

“你们之前不是都有代码评审和扫描的习惯吗？”

王工说：“之前我们确实比较多评审，除了评审核心设计和代码，我们还要求组长抽查新人开发的代码，并且要求提交代码前必须通过扫描。但我们为加快速度，没有硬性要求必须先通过扫描。经你一问，我想这也可能是出现很多 Bug

的原因之一。”

“你们有没有事后迭代复盘？复盘可以帮团队分析之前有哪些不足，后面改进。”

王工说：“我们都交付不了，哪里有心情来做回顾、复盘。最后几天冲刺，大家每天都没睡好，大家都只想回去好好睡一觉。”

11 月

部门经理之前收到客户总监电话，投诉一些技术缺陷，导致好几次不能按计划上线，问为什么正在交付的软件质量变差了？

张工被问到是什么原因时无法回答，只能说立马回去探索原因，尽快汇报。

张工从部门经理办公室出来后，找其中一位项目经理李工喝茶，问他对今次敏捷改革的意见。

两人都赞同敏捷开发应能帮助团队提升，当张工问李工有什么可以改善的地方，李工说：“让团队自主是件好事，但因为我们专注做这块业务已经很多年了，本来业务的变化不多，只是一些功能微调，所以开发人员尽量不去修改核心代码，怕改动了反而会影响投产，无法切割。为了不触及原始代码，新代码都只能在核心的外围去写，但这种做法效率很低，无法有效长久维持，到时会被迫重写整个产品。而且懂老代码的开发人员大部分都离开了，代码维护越来越困难。

敏捷教练强调团队自主管理，给团队空间发挥。但 SCRUM 的教练缺乏软件工程的基础，只懂项目管理过程。所以他们也解决不了软件相关的问题。只是把精益管理怎么做迭代、怎么做回顾这些基本过程再解读一下，解决不了实际问题。”

张工说：“很赞同，这是必须要解决的问题。但现在燃眉之急是要解决主管提出的客户投诉不能按时交付这紧急问题。不然我们以后也不敢再提敏捷开发了。”无论张工或李工也没有能去总结出什么好的解决方案。现在推行敏捷才刚刚 3 个月，绝不能打退堂鼓，回到本来的状态。但应怎么解决敏捷带来的问题，挽回部门经理与客户的信心呢？

:= = = 从上面案例看到，本来管理层希望利用敏捷开发，加快软件开发的交付、减少延误、令客户更满意。但因为只注重项目进度是否延误，没注意如何改善软件开发本身的质量，也因为团队成员能力不足，开发出来的软件缺陷比以往还多，导致后面大量返工，恶性循环，后面更导致延误和客户投诉。

因为以往没有关注软件的设计与维护，导致软件难以修改，开发人员都不敢改动任何代码，怕可能会引起系统崩溃。

问题出在哪里？是否不应采用 SCRUM 这种敏捷开发方法？

敏捷开发有多种方法 (SCRUM 只是其一)。

因为张工的团队不仅仅要管好项目进度，也要确保软件产品的质量。SCRUM 只包括项目管理部分，不全面。

是否加上其他敏捷开发方法，如极限编程 (XP eXtreme Programming) 便可？(因它的发明者 Kent Beck 本身是一位精通面向对象的程序员，所以 XP 不仅仅关注项目管理，也包含编程的最佳实践。)

想避免张工这些问题再发生，就要系统地解答以下问题：问题出在哪里了？那些敏捷开发的实践没有做好？

首先要清楚有哪些敏捷开发的实践。虽然敏捷宣言已经有超过 20 年历史，也越来越多开发团队使用敏捷，但敏捷这个词却越来越被泛滥使用。如果你细问某使用敏捷开发的团队，可能发现他们只是每天站立会议和规定每四周发布。就

算一些使用 SCRUM 的团队，也像张工情况一样做过培训，团队也不一定全部按照 SCRUM 的实践执行。Ken Schwaber，《Scrum Guide》的作者之一，说大概七成企业不是完全依据 Scrum Guide，都只是挑选某部分执行。

针对缺乏完整的敏捷实践，幸好 Michael Neumann 在 2022 年先搜索 2010 年到 2021 年期间关于软件开发实践的资料，再从搜索得出的 867 篇筛选，按是否与软件开发和敏捷实践相关、有没有经过同行评审、是否是英语、是否少于 3 页等条件，最后选出 37 篇文章和书籍（例如包括《Scrum Guide》，《XP Explained》），最后从里面的关键字得出综合敏捷实践列表，我按出现频率最高（85%）做出表 1-1:

敏捷开发实践汇总列表

	频率	实践
118	93	测试 Agile Testing (含 TDD)
	25	评审 Review meeting
174	59	进度监控 Track Progress
	46	待办列表 Backlog
	37	每天站会 Daily Standup Meeting
	32	策划发布 Release planning
	53	持续集成 Continuous Integration
105	35	频繁迭代 Small / Frequent releases
	35	迭代 Iteration process
	35	回顾 Retrospective / learning loop
	32	文档记录 Documentation
	31	客户协作 Customer Collaboration
	28	重构 Refactoring
	28	结对编程 Pair Programming
48	24	估算 Agile Estimate
	24	用户故事 User Story
	24	Collective Code Ownership
	23	编码规范 Code Standard
55	22	自主团队 Empowered Team
	17	沟通 Communication
	16	激励 Energized work

表 1-1

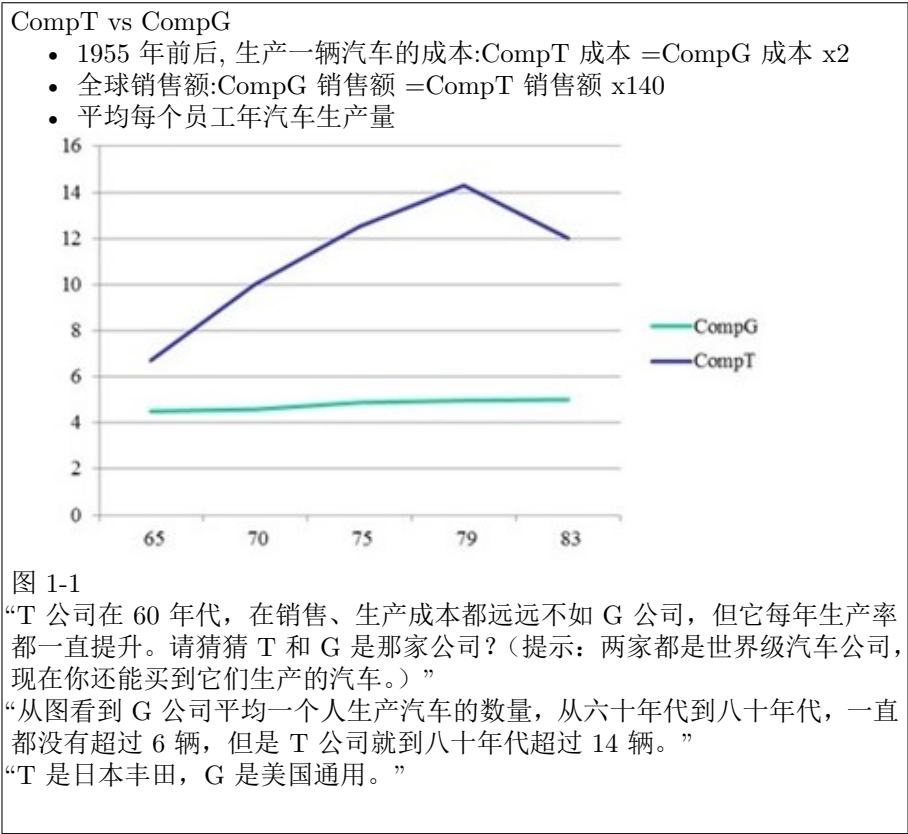
注释：频率是在 37 文件里总共出现次数，这列表的总频率是 719，原本完整列表的总频率是 840，占 85.6%。

如何改善

谈如何依据敏捷实践列表来解决张工的问题前，必须先抓到重点。这么多敏捷开发的方法、框架，最关键和重要是哪些实践呢？先邀请进入培训室，听我给学生讲的第一堂课。

第一堂课

“先问你以下关于汽车公司的问题： ”



为什么丰田汽车能从一家在战后本来快破产的小公司，可以超越那些美国的巨人公司，如通用、福特等，成为世界上最大的汽车公司呢？我们在说这个之前，首先要了解汽车生产的历史，汽车发明之初属于小众的玩意，需要定制生产，所以非常昂贵。之后福特先生使用了工业生产的生产线概

念，大规模大量生产汽车，把汽车的平均价位降到中等家庭可以负担的水平。

福特 Model-T

它是第一辆一般人买得起的汽车。首辆现代汽车是由德国奔驰 (Benz) 于 1885 制作，但是这些第一代汽车都非常昂贵。

老福特是一名工程师，对机械特别感兴趣，白天是爱迪生公司的总工程师，到了下班后就研究汽车发动机。经过 10 年的不断优化，最终开发出 4 冲程自动汽车发动机，他的发明大大降低了汽车组件的成本。1908 年 Model-T 首次推出市场，售价是 850 美金，虽然已经比同期的其他厂家汽车便宜，但还是超出一般工人家庭可以负担的水平。

为了进一步降低成本，老福特觉得必须想其他办法。汽车组装一直都是作坊式运作，所以组装汽车需要超过 12 个工时。为了降低组装时间，必须要学其他工厂，如啤酒厂、面粉厂，它们都已经是流水式生产线生产 (Assembly Line)。福特把整个 Model-T 组装拆分成 84 步骤，并培训工人只做一项工作，还聘用了工程顾问 Taylor 先生设计整个流程，提高效率，最终可以一个半小时完成组装一辆汽车。

生产线生产帮助福特公司把成本和售价降下来，在 15 年期间 (1913 ~ 1927)，福特公司总共生产了 15,000,000 辆 Model-T。虽然大量生产能直接降低生产成本，但这种做法也要付出代价：

- 为了生产所有都是同一个款式同一个颜色，黑色
- 为了提高生产的效率，也增加了组件的库存量（为了提高效率、降低成本，大部分组件都要批量生产 (Batch production)，例如轮胎。从右下图看到工厂里存在大量轮胎。）

福特 Ford Model T 生产 (~1910 – 1920)



↑
**Ford
Model-T** →



图 1-2

Federick Taylor 与科学管理 (Scientific Management)

十九世纪末美国很多工业快速扩张，Taylor 先生发现当时很多工厂生产力很低，例如钢铁厂。他是很落地、实干的人，就想有什么好方法可以提高生产力，工人也多赚些钱。他发现很多工作都未设计好，未能好好利用工人，他针对工作本身做很多科学研究，测量每个工作应该多长时间，怎么做最好，然后按他的最佳设计把工作细分，也同时要求公司提高员工的奖励，希望工人看到因科学管理，提高生产，个人也受益。1890 年，他加入了 Bethlehem Steel——钢铁业的大公司，帮它设计了很多奖励制度，也设定了一些叫工业工程师 (Industrial engineers) 岗，发现生产力可以提升 30-40%。后来公司被收购，他也被辞退，便去做顾问。当时都没有顾问这个职业，他可算是全球首位工程顾问。

丰田故事

二战后 50 年代，丰田汽车规模很小，经营很困难，在破产边缘。但丰田创始人喜一郎先生明白美国生产线的弊端，意识到未来的汽车生产必须是 Just-In-Time(JIT):

JIT = 每个工作步骤所需配件按生产需要到达（不晚到也不早到），把生产过程中的配件降到零

- 每一辆都是按客人订单订制：例如颜色、配置、左右钛等
- 从钢材原料开始，整个生产线零等待、零浪费

远景 Vision

- 0 defect 零缺陷
- 100%value added 百分百的增值
- One-piece flow, in sequence, on demand 按订单的流水线生产，过程中零浪费

喜一郎先生安排总工大野耐一先生去美国汽车公司考察。

总工大野耐一先生从美国的超市（非汽车公司）得到如何做 Just-In-Time 的启发，回国后就开始在丰田全力推动。开始时，很多人都觉得 Just-In-Time 这个愿景好像是远不可及的梦想。而到了今天，现代汽车生产都基本做到了当年喜一郎先生和大野耐一先生的 JIT 梦想。（详见附件“现代汽车生产”）

为什么丰田可以超越西方的巨头汽车公司？不会是日本人比欧美人更聪明吧？会不会是有很多新的技巧？比如看板那些方式。但欧美的汽车公司也不是傻瓜，他们会模仿。所以要了解为什么日本丰田公司能超越，并行成了如今的一些全球生产线标准，就得了解丰田背后的系统。

水面下的管理思路



图 1-3

为什么丰田能成功地把汽车生产做到 Just-In-Time，超越西方的巨头，使日本汽车制造过程成为世界“标准”？

但我们不能单从表面看这“系统”的方法和技巧，例如大家都熟悉的看板管理（他的竞争对手通用、福特肯定都学过），更重要是了解背后的管理思路。我们看看下面丰田七个习惯，开始了解水面下的“系统”：

容易实现的目标不是好目标

1. 不以“我们公司”作主语
2. 重复五遍“为什么”
3. 持续改善没有停止
4. 成长比成功更重要
5. 以忙碌为耻
6. 从心里相信“大家的力量”

容易实现的目标不是好目标

不是“削减一成”，而是通过“取消一个零”来发现浪费

“如何把原先需要 3 小时的工作，改用 3 分钟完成”

如果听到上司以上要求，你该怎么回答呢？如你回答：“这太强人所难了”、“绝对做不到”，请读以下丰田故事。

汽车生产线不可能只生产一个型号的汽车，但要生产线上面的机器改变配置，从生产某个型号到另一款，通常需要更换的时间长达 2-4 个小时，必须要尽量压缩更换时间，才能提高生产效率。所以丰田工程师在大野先生指导下，开始研究如何可以压缩更换时间。他们发现装置更换包括 2 类：

内部装置更换，必须要在机器停下时更换

1. 外部装置更换，可以在机器运转过程中进行更换

因此，要缩短时间，就要把 2 种更换分开，尽量用外部替换的方式。同时对内部和外部的装置替换进行改善，最终可以把更换的时间压缩到一个半小时。工程师给大野先生汇报改的是经过 4 个月不断摸索和试验的结果，原本以为大野先生会很满意，但大野先生说：“既然能缩短到现在的水平，如果你们继续改善，时间应该能压缩到 3 分钟。”改进组大部分人都觉得做不到，但其中一位资深工程师说：“如果我们能想办法把所有内部装置改为外部装置，3 分钟也不是不可能。”改进团队就按这个思路开始做改革，同时研究各种切割工具、模具，并且减少多余操作，尽量用一个动作就能完成所有更换。通过 5 个月时间，做了超过 100 轮改善，最终达到了大野先生定下的目标。

容易实现的目标不是好目标，丰田这个原则能否也用于我们软件开发的过程改进呢？

学员：应该也可以吧。

我：是的。比如希望减少后面在验收或者最后系统测试发现的缺陷数，我们的目标就不应该仅仅是降低 5%、10% 这种没意义的小变化。你估计可以定多高合理呢？

学员：最多降个 25% 到 30%。

我：不。从我们实际操作例子来看，如果他之前没做过过程改进的话，在做过几轮迭代后，缺陷数能达到减半是很正常的。例如本来完全没有做正态代码扫描，后面增加了这个检测，或者本来没有注重代码人员自己做单元测试，后面做了，通常都能把后期的缺陷减半。如果不能减半的话，管理层是没有兴趣继续支持这种过程改进的。做培训、监控等着投入成本也不小，如果没有可观的回报，肯定高层会叫停，持续不下去。大部分软件开发公司，因为缺乏数据、度量，很少有标杆的概念，但也不是全部。尤其是客户对质量有要求的公司，比如证券、银行、保险这些软件公司，开发经理会问标杆这个问题。从他们开始关注标杆，就会了解到管理层对持续改进、度量或者提升等，都有一定要求。大家对质量或者生产率、工作量等，都会有具体的提升要求，并定期监控。公司才会提高本身的竞争力，不会很容易被淘汰。

如果大家针对每次迭代的系统测试缺陷密度都进行统计，就可以开始建立团队的内部标杆或者基线，作为后面的参考。日后在做项目估算的时候，便可以依据以往的历史数据，更准确去做好新一轮迭代的策划和估算。

这个思路也不仅是用在缺陷，也可以用于工作量、工期等的估算。

刚刚说的只是其中一种标杆管理，除了参考团队自己的以往数据和趋势，也应该可以在公司内部与其他团队比较，或者和公司的主要竞争对手比较，例如找行业中最优秀的公司去对比，就会知道自己的不足。

丰田一般会选行业里最强的对手作为基准（标杆）。例如，1965 年，美国通用汽车公司是世界顶尖。例如销售额的规模，丰田与通用是 1:60，完全不在同一个层次上。成本上丰田对通用是 1:0.5，成本是通用的两倍。

丰田把当时顶尖的通用公司当成了自己进行标杆管理的对象。丰田就思考怎么激发内部去改善。例如，同一个零件的成本在丰田是一千日元，而通用的成本是四百日元，丰田就会用四百日元作为成本基准，把差额六百作为不必要花费，要求团队立马改善，尽量想办法达到标杆。

标杆管理 (Benchmarking)

很多公司都会用百分比来设定改善目标，但改善了百分比，不一定代表质量有改善，例如某快递公司去年送包裹未按时送达占 8%，今年是 6%，好像已经改善了 2%。但去年托运的数量是 500 万件，所以 8% 是 40000 件。今年的数量是 750 万件，所以今年的 6% 就是 45000 千件。所以今年包裹延误增加了 5000，这样非但没有任何改善，反而服务变得更差了。所以更重要是看数字本身。丰田（如下图）很注重各种标杆，例如内部标杆、竞争性标杆等。软件开发也应该同样利用数据来制订量化目标。

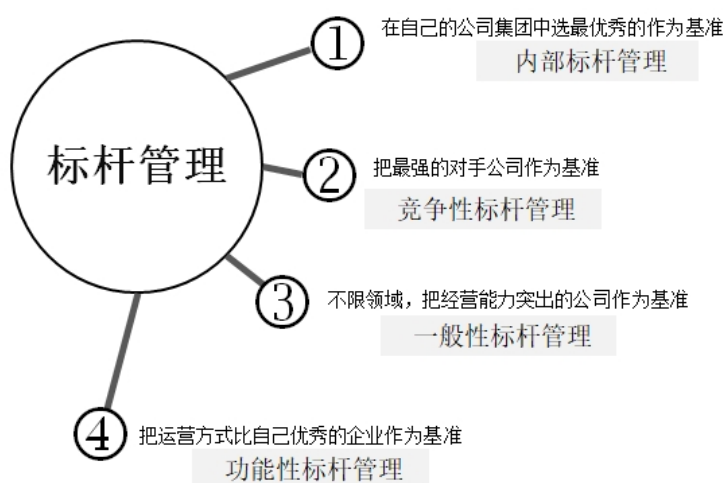


图 1-4

(图 1-4 参考岩松义人的《为什么是丰田》)

某家专门开发金融软件产品的公司：
 研发总监问我“你接触这么多国内的企业，觉得有哪家优秀的，可以作为我们的标杆？我们尝试寻找，但还未找到。有些优秀的公司，但它们业务跟我们不同，我们的产品是面对企业，也非嵌入式软件（我们不生产硬件）。”
 “确实难找，我真没遇到过，但为什么你只看国内公司？”

从以上对话看到越来越多高水平开始用标杆管理，但标杆必须是比自己高很多才有作用。

不以“我们公司”作主语

- 不是从“专业”的角度，而是从“顾客”的角度生产产品

创业大忌——闭门造车，所以丰田的原则，对客户有用就一定做出来，但对客户无用或者不想要，就绝不生产

有些汽车公司技术很厉害，主要精力用于做到技术领先。但丰田的原则是，如果对客户有用就一定要做出来。因为如果不是以客户的需求为主导，就很容易导致生产出来的产品，并非客户所需要的。

例如宅急便是一家日本专门送货搬家的公司，他们发现市场上的送货车不方便驾驶员使用，便提出要求配送车应该便于装卸货物，且方便驾驶员上下车，也易于在车装卸区上下工作。例如，让驾驶员在车里可以直接从座位走到后面的货物区进行操作。一般汽车公司听完这种特殊要求，就会反驳说你们不熟悉汽车设计，这种奇怪的设计违背了很多汽车设计的原则，整个车的架构不稳定，且会更耗油，建议你还是用我们这种标准设计好的送货车，而且价位也合理。而当丰田听完这个需求，却说如果你们觉得有这个需要，我们的工程师就跟你一块设计能满足你们需求的车。后面确实为宅急便公司生产出了合适的送货车。丰田汽车确实在汽车行业比竞争对手更灵活，更尽量满足客户的个性化要求。其他汽车公司虽然嘴上说“一切为了客户”，但总是想生产能让公司标榜最新技术的车型，最终还是主要考虑企业的利益。

必须从“顾客”的角度看，不单从工程师的视角看——这不仅适用于汽车制造，在软件开发行业也适用，但要平衡，有时候不要误认为客户很清楚自己的要求，因为他们也没有软件开发的经验。例如客户频繁提出的需求变更，因为他们不知道这种频繁的变更，反而会导致软件公司大量返工，对最后能否按时交付有很大的影响。所以软件开发在需求方面永远是优化，尽量花精力去把它做好，不然会影响后面工作。但如何做好，不仅是听客户的需求，并需要以顾问的角度指出客户的误区。

重复五遍“为什么”(5 Why)

最重要的不是追究责任而是找出根因。

五个“为什么”是一种找根因的方法，实例可详见附件“5 Why”：

- 不停留在“原因”上，而要找出“真因”，彻底改善
- 不追究责任，而是追究原因

丰田要求检查人员不仅仅检查过程与结果、做统计报告、交给负责的工程师，也必须分析产生不合格的原因，使同类的问题不再发生。检查人员不仅仅是“考官”，也作为“内部教练”，指导生产的持续改善。反过来，如果下属因出错导致失败，立马被上司骂，觉得很难受，就会从反省变为排斥。如果公司有这种“责难，追究责任”的风气，坏消息就会被隐瞒。所以丰田有一条原则是“Hard news first（先说坏消息）”。当工程师找出了问题就必须要求他查找原因、提供数据，但很多时候这个工作并不简单，但大野耐一先生决不放弃，必须找到根因。他说：“做到一半是不行的，只有一个期限——“到完成为止。”

帮一家香港公司做软件开发质量改进，公司历史悠久，但公司主业不是做软件开发，而是承担办公楼的机电工程项目，例如电梯、空调、机房或数据中心的建设。公司管理层大部分就是工程师出身，比较严谨。推动这改进项目的公司高管刘先生说，“我们公司这么多年有一个弊端——员工都是报喜不报忧。如果有什么成功案例是都会尽力在公司内宣扬，但是有什么错误问题出现就绝口不谈，导致很多宝贵的经验教训没有保留下来，供日后参考。”

持续改善没有停止

“请猜猜丰田汽车在 90 年代初，平均每员工一年提多少条改进建议?”

“大概每月一条，12 - 15 条。”

“老师这样问应不止，多一倍，25 条左右。”

丰田当时大概 8 万员工，一年里共有 4 百万条改进建议，平均每员工每年提 50 条。

丰田某部门在一年半时间，成功地把平均零件替换时间从 90 分钟缩短到 30 分钟，但再过了半年会问这几个月只缩短了几分钟，丰田管理者便会问为什么。

人有的时候很想“往前跑”，但有的时候也想“停下来休息”。对丰田而言，比起以为“已经胜利了”而在睡午觉的兔子而言，他们更推崇不倦不怠一直向前进的乌龟。

某家成都的软件开放公司总经理是从日本回国的，对质量很有要求，开始时，推进了很多质量改进的方式，确实把大量缺陷减少到原来的一半。过了不到一年时间后就没有再持续改进了。当总经理问我是什么原因时，我就会用丰田的故事说明持续改进必须从高层开始，且一直坚持下去。

所以改善能否成功，大家能否把改进养成习惯，取决于管理者能否坚持，不单是依赖学到什么技巧。

成长比成功更重要

- 要培养人才，“改变体制”比“改变人”更有效。

有些工厂只依赖张贴标语、海报，希望可以减少工地的事故发生率，但丰田不注重喊口号，而是动手干实事，包括机械保安、设备保安等。

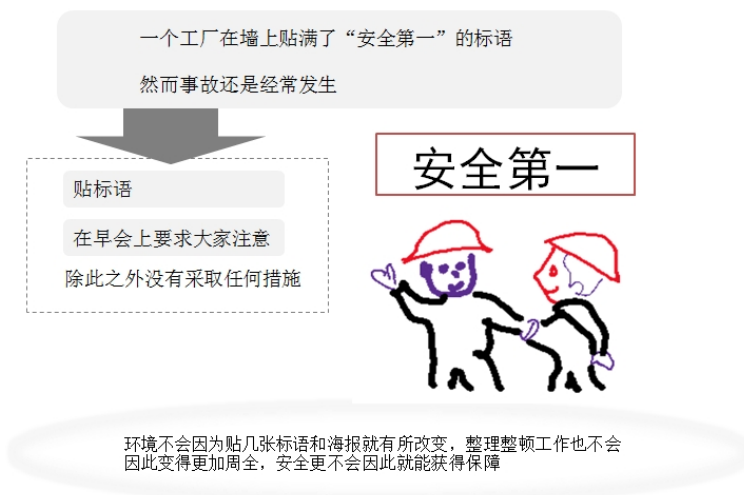


图 1-5

所谓 5S 指的是：整理、整顿、清扫、清洁及修养。（详见附件“5S 法”）无法做到这些基本原则的企业及个人是无法生产出优质产品。

在这些基本原则中，丰田尤其注重“整理和整顿”。

“处理掉不需要的东西是整理。使得任何时候都能把想要的东西拿出来叫整顿。只是把东西放整齐的则叫做排列。我们要求在生产现场必须要做好整理和整顿。”

所以如果有机会进丰田的生产场所，你会感觉到很清洁很干净，没有大堆废物或者零件堆在里面，这也直接影响到工作的效率。

这种 5s 的好习惯，其实不仅仅适用于工厂生产，对个人也适用。

因为我经常要去客户现场做培训，每次离开的时候都会很仓促，常常遗漏一些小物件，比如充电器等。我就按 5s 的要求，给每件东西都安排一个背包或者手提袋固定的地方。

以前坐飞机时也常常会把一些电子产品 (Kindle 等) 或者耳机遗忘在座位前的口袋。学习了 5S 后，我都会在下机之前检查一下，确保东西把所有东西放回它的固定位置，后面就没有再遗漏东西了。

所以当我在酒店大厅听见前台接到客人电话，说在房间遗漏了 iPhone，请前台快递给他，心想 5s 习惯应该能帮到他。

成长不是仅仅按既定方向去做，必须思考动脑筋，并了解背后的目的才能真正取得效果。

在七八十年代，汽车公司开始引进自动化机器人提高生产效率。

但大野先生对引进机器人是很慎重的，他觉得如果只是觉得机器人用起来很方便，可以替代工人去完成工作，就表示那家公司没有有效的使用机器人，要真正用好机器人，必须让它与工人的工作合拍，辅助工人才有意义。

所以丰田不但要求人的工作和机器人的工作要明确分工，并且要二者能和谐的存在。例如丰田当要决定，最后在组装工序中引进机器人，他们工程师都会慎重考虑，要经过很长的一段时间做准备，里面分析如果人会采用什么方式去做？什么工作是机器人做得了，什么是机器人做不了。例如，哪些零件更适合用机器人装配。

另一方面，回顾七八十年代的美国通用公司，它就花了很多钱用 IT 把生产自动化，但最后没有产生什么效果。估计就是因为没有想清楚，为什么要自动化，自动化怎么帮到人的工作并产生效果，只是相信用技术、高科技就能提供生产率。二战后，日本经济萎缩，丰田辞退了不少员工。1950 年，朝鲜战争爆发，需要大量增产，但丰田选择了只增加设备，而不增加人手，大野先生也借这机会，完善丰田生产方式，成功找到了以不增加人手为前提的增产办法。

这些丰田的成长故事也适用于软件开发团队，例如，有些软件开发团队盲目地想推自动化测试，却没有想好哪一类测试应自动化，哪一类更合适手工测试，最后因效果与投入不匹配，以失败告终。

以忙碌为耻

- 不吝惜智慧，但要吝惜汗水

这一条是最常受到开发人员或者开发主管认同的一条。

学员：你说的太对了。

我：其实长期每天超过 12 个小时的工作，实在是受不了。对身体、对产出物的

质量都有不良影响。因为每个人的精力有限，长期加班很不健康，反而会导致失去冲劲，只希望在最快的时间完成任务而不注重质量。反思一下，其实有这种长期加班的话，可能表示本来策划没做好，人手没分配好，才会出现。加班变成常态，大家反而会觉得反正尽力最快完成任务，还需要加班，没有什么好处，不如拉长一点来做。大家没有动力去提升生产力，反而会导致躺平使生产力下降。很多人混淆了“有动作”和“在工作”，说自己“一直在努力工作”，不管上班时间的有多长，如果没能够创造出利润，那么就不能称之为工作，也不能称之为一直在努力。

丰田是把“动作”和“工作”分开考虑的。在丰田看来，“就算一直在动也不代表那个人在工作”。省掉徒劳动作，把“在动着”转化为“在工作”。

如果问一些年轻员工：“每天工作一小时左右，你们能做到吗？”

听了这句话后，他们会抱怨：“算上加班时间，我们一天工作 9 个小时，他那句话是什么意思？”

确实，他们在公司待了很长时间，但如果把“有动作”和“在工作”分开考虑，9 个小时中，真正在工作的时间可能只有 1 小时。

关键的不在于流着汗在公司转了多长时间，而要把工作区分成“徒劳”和“有价值”。

从心里相信“大家的力量”

- 不是靠“一个不平凡的人”，而是依靠“一百个平凡人”来创造亮眼的成绩。
- 生产产品就是培养人才。

有一位丰田员工被问到“丰田生产方式到底是什么”的时候，这样回答：“就是在人的智慧建起的基础上，立起了自动化和及时生产这两根支柱。”

他所说的“人的智慧”是指“在一线工作人员的智慧”。

人了不起的智慧所谓丰田生产方式其实就是要建立起一种体制，把“人了不起的智慧”引导出来，使得这些智慧能在生产一线得到充分发挥。所以说“丰田生产方式源自人的智慧”。

企业相信员工的智慧提升以后，员工们的干劲、使命感、责任感都会随之而生，最重要的是，员工们会因此逐渐对工作抱有自豪感。一旦员工们的意识改变了，理所当然地，企业的竞争力也将获得很大的提高。

大野耐一先生说：“改良是指通过投入资金使情况变好，改善是指通过动脑筋使情况变好。”

A 先生因为自己在推行生产改革过程中没有得到技术部门的配合，所以就做不下去，他很苦恼，就去找大野先生。

大野先生就请他去参观丰田生产线的工作情况和附近一个协作企业的工厂，之后大野先生问他觉得如何。

A 先生说：“我觉得今天听到的和看到的，跟丰田方式所强调的重点好像不太一致。”

大野先生：“就连我，也是一直都在忍耐。不是单靠权力就能解决。要想得到对方的理解和信任，必须先拿出诚意去找人家。”之后，A 先生使用诚意去找对方协商，不久以后，他成功地实施了生产改革。

从以上故事可以看到，千万不要以为有权利就可以使生产改革一帆风顺。因为改变人的习惯是很困难的。我在大陆和香港所有的质量改进的项目，期间走过很多弯路，最后依赖高层一直坚持，才取得一些效果。

例如大陆很多企业都挑选本月之星，觉得这种对员工是激励。其实很多时候适得其反，我常常跟学员说这种做法反而会影响了其他没有被选上本月之星员工的士气。

所以日本公司有个原则：不会奖励个人，但可以奖励整个团队。

= = = =

“老师，不好意思，真是有点迷茫，本来我们不是讨论敏捷开发吗？为什么跟我们讲丰田汽车？我们又不是开发汽车相关的软件？”

我们不是应先找出针对那些重点入手吗？从丰田故事能否得到一些启发？

“用 5W 找出根本原因。”

“应该以忙碌为耻，拒绝加班。”

“应该向龟兔赛跑中乌龟学习。”

这同学说完后忍不住不停地笑，其他人觉得他太过分了。

你猜得比较接近了，但还不是不对。我们不是应该找出水面下的管理思路，而不是水面上的技巧吗？列表中除了最下面的框，其他上面的都是技巧，虽然有作用，但不是核心。如果员工没有积极性，团队缺乏自主能力，缺乏沟通，只是学好如何写好代码，定期结对编程，能帮我们解决软件开发的问题吗？

“了解了，如果管理者不支持和关注团队，员工个人没有动力，学好这些技巧可能只是短期有作用，不长久。”

如果没有持续改善（KAIZEN）为核心支持，大家不觉得需要不断回顾，如何能在迭代有提升，学好技巧起不了作用。

依据实践列表，除了上面的技巧外，更要学好持续改善，包括个人和团队，团队如何能做好迭代回顾，找出根本原因和对应措施，在下个迭代尝试提升。

其实持续改善历史悠久，质量大师如 Dr Deming, Dr Juran 都已经一直使用，辅导过很多不同行业的公司，也可以用于软件开发。

改善质量

质量大师 Dr Juran 用财务管理比喻质量管理，同样也有策划、控制和改进，然后基于这大框架再细分：如何制订质量目标、度量等。

质量策划包括：

1. 设定目标，包括外部和内部目标。
2. 识别内部需求。
3. 依据客户需求制订产品的功能特征。
4. 制订产品和过程的目标。
5. 设计过程来达到这些目标，并验证过程能力。

质量控制和质量改进

按质量大师 Dr Juran 定义，质量包括两部分：

- 满足客户需要（包括外部客户和内部客户）

- 没有缺陷

这定义不仅仅适合于制造业，也适用于软件开发、IT 业。后面我们会针对如何降低缺陷返工或技术负债去做改进。大量缺陷都在后面系统测试或验收时才发现是很多开发团队的通病，所以建议先从减少长期浪费入手（缺陷导致返工，是一种浪费）。

例如下图，发现缺陷比率为 20% 左右，这就是过程的能力，这是策划的时候已经定好的，过程控制没有什么可以做，只是当缺陷有变化，比如特别高的时候，需要做一些纠正措施。例如希望把缺陷从 20% 降到 3%，这就必须驱动一系列的改进计划。改进计划也必须按项目管理方法推行与监控，没有其他办法。

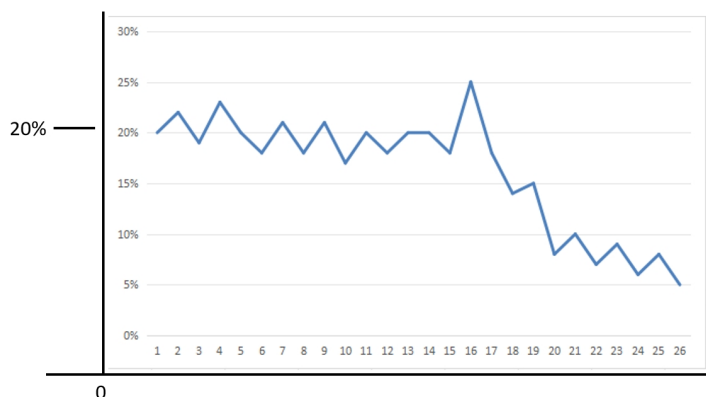


图 1-6

所以如果希望敏捷开发有效，不仅仅是走迭代过程，确保进度没偏差，还要确保软件产品质量，也应该用裘兰博士的质量管理思路去看敏捷过程，才能更全面了解如何确保软件开发的质量，控制交付工期。

最佳实践，如果没有质量目标、度量衡量和策划，都只是空想，对长远提升企业文化、员工素质没有任何帮助。

例如我们想改善团队的策划和估算。首先要识别客户，哪些是主要干系人——甲方有什么要求、内部部门经理有什么要求。然后从他们的诉求变成过程的功能和特征，但如果特征只是描述，没有数字也没有意义，所以要配合可衡量的度量单位，和用什么方式去收集那些数字，然后依据目标定过程要怎么去做，怎样改。例如：

- 是否应该从本来的瀑布策划方式制定大的总计划，改用精益的思路，分成迭代
- 每次估算应该怎么做、如何监控等
- 然后改进过程要不断从实验中优化
- 最后还要用数字证明有显著改善，然后在其他团队推广

不是单纯“空降”某敏捷流程（例如 SCRUM），便能加快团队的发布速度。所以虽然极限编程里每一条实践都是最佳实践，也必须配合质量策划和监控改

进才会有效果。

乔布斯 85 年离开苹果，自己开创 NEXT 公司，他很注重质量，便邀请了裘兰博士 (Dr Juran) 从美国东岸飞到西岸，帮他们公司做辅导、改进过程。以下是节录他接受访谈时，对美国企业、质量和裘兰博士的观点：

美国已经富裕了很多年。很多企业都忘记要获得成功，还是要关注基本功，包括教育。现在我们美国很多企业面临困难，处处感觉被日本领先了。其实不是日本针对我们，而是我们作为美国企业家应反思一下，为什么我们的战略比日本差，为什么我们的策划不如日本？我们知道裘兰博士多次去日本，帮助日本企业提升质量。现在他回到美国，希望把他的经验带到美国企业，提升竞争力，可以再一次成为世界第一。

我觉得裘兰博士很实在，不是泛泛而谈。我们的工程师也深受他这种风格影响。无论我们之中谁问他问题，总裁还是工程师，他都会全心全意用自己的知识解答。所以工程师和我都很希望用他那套方法来做提升。质量提升的道理其实很简单，是一个重复的过程，然后我们需要不断去看，有哪些无效的环节要省略，哪部分要重新设计，不断试验、提升，就这么简单。重点是所有的提升都应该是科学化的，有数据而不是泛泛而谈。

一般管理层的思路是：我是领袖，你们应该听我命令。但应该是反过来，让应如何做好的决定权利放在团队手上，做改进不需要请求管理层的批准。改进是工作的一部分，整个架构扁平化，自己管控日常过程，每位工程师应像以前的工匠，愿意花精力不断做好。然后能以自己最后做出的优质工艺、产物自豪。

从以上丰田故事看到，要有改进必须从高层确定目标出发，再配合一些管理原则和管理者（比如大野耐一总工）的驱动，才可以有效果。员工本身有没有动力也要考虑，但仅仅靠这些还帮不了企业实现真正改进，还需要一些具体的方法和实践，因为所有改进都是由试点项目开始公司的第一小步。

对应综合敏捷实践列表，A++ 除了使用持续改善最底下框里 3 项和迭代回顾外，也会针对其他 3 个框讨论：

为什么等到客户使用时，才暴露一些很棘手的 BUG，但没能预先在内部测试或评审发现？

为什么应先写测试用例才开始写代码？

为什么不同人估计的故事点差异这么大（超过 30%）？

为什么每轮迭代都是最后 2-3 晚疯狂加班？但还会经常不能按期交付（切割），要延到下一轮？

因不少实践已经有很多很好的书籍举例说明，其他如持续集成，结对编程，重构等也会提及，但不会详细讨论。详见表 1-2。

敏捷开发实践汇总列表

实践	对应章节	
	A++重点专注	有覆盖
测试 Agile Testing 评审 Review meeting	18	
进度监控 Track Progress 待办列表 Backlog 每天站会 Daily Standup Meeting 策划发布 Release planning	7	
持续集成 Continuous Integration		16
频繁迭代 Small / Frequent releases 迭代 Iteration process 回顾 Retrospective / learning loop	10, 11, 12, 13	
文档记录 Documentation		
客户协作 Customer Collaboration		19
重构 Refactoring		15
结对编程 Pair Programming		17
估算 Agile Estimate 用户故事 User Story	6	
Collective Code Ownership		
编码规范 Code Standard		
自主团队 Empowered Team 沟通 Communication 激励 Energized work	2, 3, 5, 8, 9, 20	

表 1-2

结束语

若要团队利用敏捷开发帮公司提升没有捷径，只能像乔布斯说的，要达到不断改进的目的就要考虑有没有可靠的数据。因为没数据就没有管理，也不知道现在什么水平，没有标杆，更无法知道几个月后确实有没有改善。所以度量与分析是基础之一。不是所有公司或者团队都适合做敏捷开发，所以也要考虑公司的文化、管理和团队本身的能力。（2 章）

很多软件开发团队都缺乏数据，但软件开发与工业生产不一样，很多数据还是要靠个人收集的，例如返工的工作量，所以也要看如何可以把可以让个人开始有收集数据的习惯。(3 章、5 章)

如果个人没有动力，没有好习惯，公司也无法改善。有了这些基础，就可以看团队可以怎么做好估算和计划。(4 章、6 章、7 章)

虽然敏捷宣言强调淡化策划，因为它是针对传统的瀑布式开发，一开头做 3-6 个月的项目计划。但是如果我们把项目拆分成 2-4 周的迭代，每次迭代还是要做估算和策划。首先要如何说服高层支持，因为没有高层支持，将过程改进立项，后面肯定不会产生任何效果。(8 章)

有了高层支持和立项后，改进的过程肯定也不会顺利，因为要改变员工原有的习惯是很难的一件事，而且不会像童话故事一样，挥动一下魔术棒，下个月就立马有效果。(9 章)

要做好团队的迭代回顾，其实是分析数据，找出根本原因，制定改进方向的最佳时机。(10 - 13 章)

和前面说的一样，我很同意要做好敏捷开发，不能用 SCRUM 那种方式，只是针对项目管理做好迭代，还必须要软件工程师的基础。后面我们会从需求一直到产品集成、持续交付等，如何把测试驱动开发这个精益的原则用上。(14 - 20 章)

市场竞争激烈，单是靠持续改善完善现有的过程，减少缺陷不一定能够支撑公司的长远发展，还是必须要有创新，才有出路。在最后部分会探索个人和公司如何创新。(25 - 27 章)

在软件开发里，尤其是持续交付、持续集成方面，配置管理起很重要的作用，但因为篇幅有限，我们在本书中暂时不涉及配置管理、版本管理相关的内容。

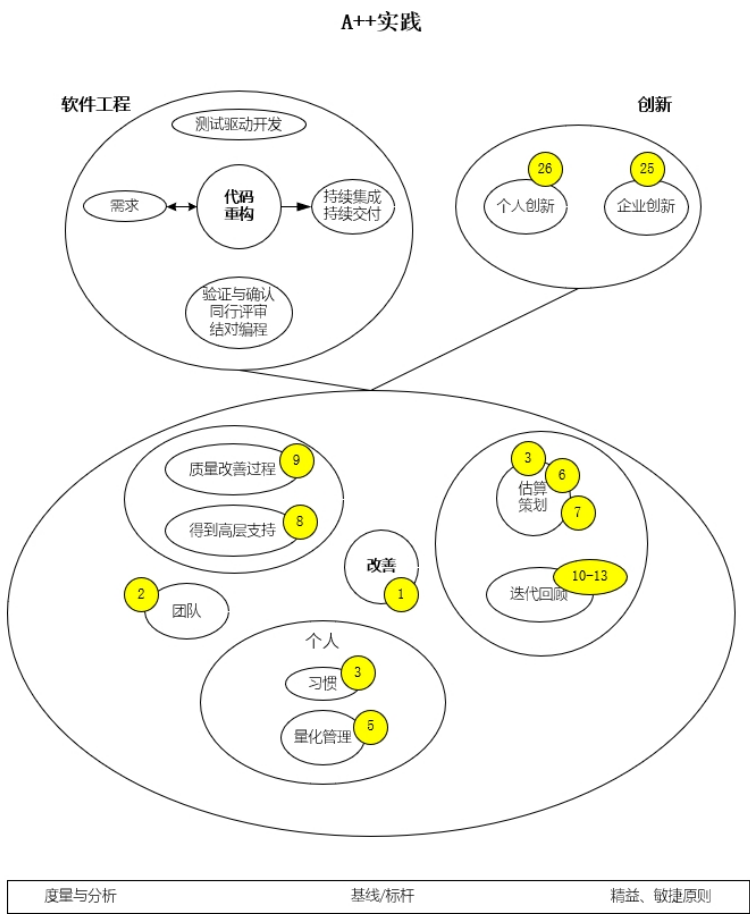


图 1-7

附件

现代汽车生产

今天 Just-In-Time 已成为汽车制造的主流，例如：日产在英国牛津郡 (Oxfordshire) 专门生产 mini 车的工厂便能做到：

- 从钢材原料到生产出汽车只需要 24 小时
- 整个生产线没有任何中间等候，每 68 秒出一辆
- 每天生产 1000 辆车



图 1-8

挑战：每一辆汽车都不同——颜色、设备、左右驾驶座等

- 生产线上每一辆汽车都按照客户需求订制
- 组件不早不晚按需求准时到达生产线
- 这便需要信息化系统把客户订单转换成生产信息

客户订单与生产交付同步

<生产线的步骤固定，每一步的时长也固定，支持从客户下单到交付的生产线流程。>

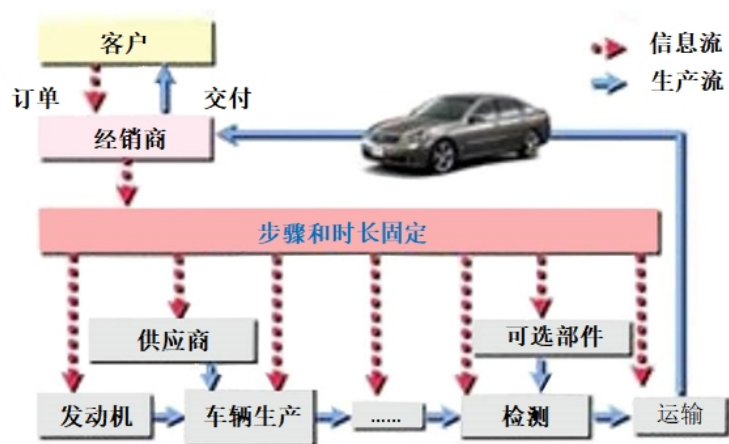


图 1-9

例如生产线上每一辆的颜色都可以不同：

400px



图 1-10

“ 5 Why ” 实例

大野耐一先生见到生产线上的机器总是停转，虽然修过多次但仍不见好转，便上前询问现场的工作人员。

(1W) 问：“为什么机器停了？” 答：“因为超过了负荷，保险丝就断了。”

(2W) 问：“为什么超负荷呢？” 答：“因为轴承的润滑不够。”

(3W) 问：“为什么润滑不够？” 答：“因为润滑泵吸不上油来。”

(4W) 问：“为什么吸不上油来？” 答：“因为油泵轴磨损、松动了。”

(5W) 问：“为什么磨损了呢？” 答：“因为机器打磨金属零件，空气混进了铁屑等杂质，并掉进机器油缸里。”

经过连续 5 次不停地问“为什么”，找到问题的真正原因（润滑油里面混进了杂质）和真正的解决方案（安装过滤器）。由现象推其本质，因此找到永久性解决问题的方案，这就是 5W。

针对软件开发的 5W 例子：

1W：问：为什么显性 Bug 多？

答：开发自测不够。

2W：问：为什么开发自测不够？

答：1) 任务太多，赶着完成；2) 只测自己改的 Bug，没想到覆盖相关方面。

3W：问：为什么要赶着完成任务？

答：预留时间本身不足，还有临时任务的加入。

4W：问：为什么不能给任务做好估算并预留临时任务时间？

答：光做计划中的任务，都完不成，临时任务太多，要加班，要不简化流程。

5W：问：为什么不尽早提出进度滞后的风险，计划没提前同步及随时反馈问题？

答：有些计划内容没同步及达成共识，有些任务类型，如，开发自测，没拆解出来或者没明确标准，也没有预留临时任务时间。

措施：计划中关键活动要罗列并明确完成标准；计划不能排太满，预留一部分

临时任务的时间；计划要组内同步达成共识。

5S 法

本来 5S 是用于工业生产，例如日本的生产工厂很注重洁净，东西要放在容易找到的固定位置。其实这对生产线员工有很重要的心理作用。如果整个环境都很脏，必然会影响人工做好的动力，如果东西乱放，也容易找不到。5S 不仅仅适用于制造业，也适用于服务业，每件东西必须放在固定位置；这也可以用于个人管理，我以前常常在出差去客户现场时，经常忘记把一些东西，如鼠标或插头。但后面我固定了每件东西都应放哪里，临走收拾时就确保那些东西不会遗漏掉。平常工作有一个洁净的环境，也能降低工作压力，提高工作效率。

400px

5S		实例 Example
Organization	整理 Seiri	倒掉垃圾、长期不用的东西放仓库
Neatness	整顿 Seiton	30秒内就可找到要找的东西
Cleaning	清扫 Seiso	谁使用谁清洁、管理
Standardize	清洁 Seiketsu	维持无污染状态
Discipline & training	修养 Shitsuke	严守标准、遵守已决定工作习惯

表 1-3

Part II

团队与自我改善基本功

改善都应从团队开始，并要改善本来的工作习惯的不足，但改变工作习惯并不容易。首章用实例介绍公司团队如何从“破冰”到变革，到稳定，以及成功要素。

团队的提升依赖于个人的成长与提升。敏捷开发假定每位成员都具备极高的经验与能力，如果团队成员都是刚毕业的新手，不建议使用敏捷开发。后面第 3 章介绍提升个人效率的方法。

Chapter 2

改进从团队开始

上一章介绍了丰田方式中七个水面下的习惯，那么公司如何有效地开展和推广这些理念呢？有哪些需要注意的误区？让我们从美国东海岸一家小型印刷公司的故事说起。

美国费城 Weisbord 故事

在上世纪 60 年代，复印机尚未普及且价格昂贵，因此有不少公司专门为各类客户提供印刷服务。故事的主角 Weisbord 是美国东岸一家印刷公司老板的儿子，他没有接受过正式的管理培训。他的好友 Don 已经在一家国际大公司工作了 20 年，向 Weisbord 推荐说：“很多大公司已经开始推行团队自主管理，以提高生产率。你可以先读读《X-Y 理论》（McGregor 著，Human Side of Enterprise）这本书。”

X 理论：

对 (员工) 人性的假定

- 懒散 / 不想工作，尽可能避开责任
- 依赖 / 被动希望被指挥，工作安稳最重要
- 因此主管必须要把工作细分，并要紧密管控才会达到结果

Y 理论：

假定

- 关注工作，希望负责任 (take responsibility)
- 希望成长，有成就
- 如给予机会，会尽力把事情做好

处在 X 思维的主管

Weisbord 原本只是好奇，想看看这本书有多厉害，结果一个周末便一口气读完了。他意识到自己的公司到处都是 X 理论式的管理：

- 工作分得很细，每个员工不清楚全局
- 任务都是依赖主管分配
- 员工遇到问题、困难，交给主管处理
- 员工上下班打卡

他认为新方式可以提升公司的管理，因担心自己难以推动这项改革，便正式邀请好友 Don 加入公司。

通过分析，他们发现公司业务的最大问题出在订单处理部门（Order Processing），每个订单被拆分成五到六个独立的任务，员工仅负责被分配到的任务，如：

- 筛选客户邮寄地址，邮寄印刷样本
- 输入订单。
- 检查客户的信用
- 发起内部生产订单
- 用打字机开发票邮寄出去
- 收到的支票，对上那些应付账

公司平均每天要处理 200 到 300 个订单，但由于任务划分过细，一旦某个环节（如订单录入）有一两人缺席，便严重拖累整个流程，效率可能因此减半。为解决这一问题，Weisbord 计划对部门进行改革，将订单处理部门分成多个 4 至 5 人的团队，再将公司的 2 万个客户按地区分配给各团队。每个团队负责自己区域内的客户，并配备打字机等设备，希望以此提升部门的灵活性，提高生产效率。

他与五位小组主管讨论改革方案，其中两位表示支持，两位持中立态度，剩下的一位认为方案不会成功。

Weisbord 最终决定启动这一改革，与 Don 一同开展部门重组的工作。

开始团队制

开始时问题非常多：

- B 物流公司运送出错，送到另一个城市，怎么办
- D 团队误解了生产的步骤，怎么办
- C 团队的样板工人，刚入职 3 周，对我们公司产品线不熟识，怎么办

原因很简单：原来每人只熟悉自己负责的一小块工作，遇到问题都由小主管处理。而在自主团队中，没有主管的支持，每个团队都需自行解决问题，感觉就像瞎子摸象。

但问题确实太多了，无奈之下，Don 建议每周开会讨论，但团队从来没有定期会议的习惯。起初，Weisbord 认为开会浪费时间，不如把时间用于生产更实际些。然而，问题依然层出不穷，这种情况延续了一个月。

Weisbord 开始对 X-Y 理论产生怀疑，觉得这些理论可能只适合在象牙塔，而难以在实际公司运作中落地。Don 也无更好的应对策略，Weisbord 甚至萌生了放弃改革、恢复原有架构的念头。尽管 Don 建议再耐心等待一两周，Weisbord 因团队持续的效率低下、公司运作受到影响，打算在下次会议上宣布撤销改革。

到了第五周，与以前开会一样，Weisbord 先问：大家有什么问题？

会议室一片沉默。几分钟后，一位组长说道：“没有问题。”

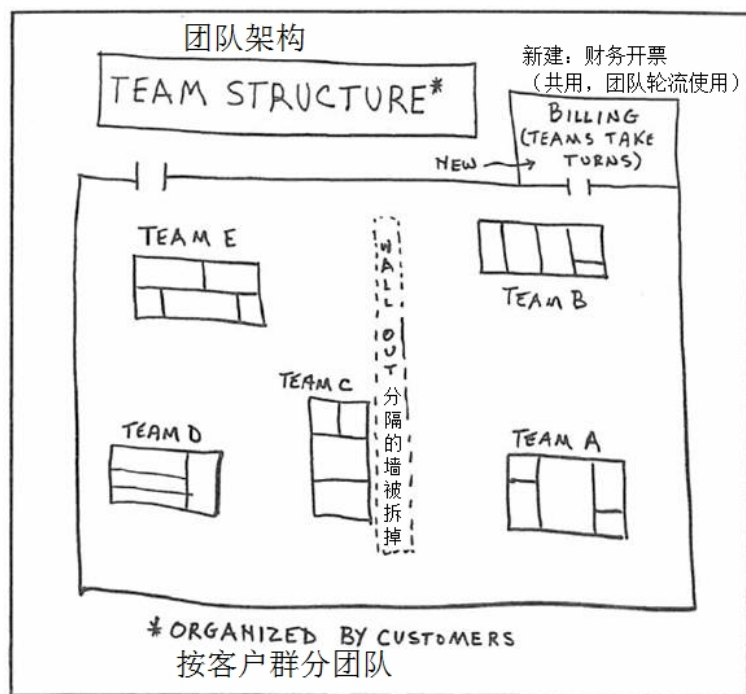
Weisbord 感到惊讶：“怎么会没有？之前一直问题不断。”（心里想，一定是你们也放弃了，连问题都不愿提了。）

另一位组长回答：“本周我们确实没有什么问题，因为遇到的问题在之前会议里解决了。”

效果

改革最终带来了显著成果：每天的订单处理量从不足 300 单提升至 400 单，员工的出勤率也显著改善，缺席率几乎为零。

改革后的订单处理部平面图：



（在团队改革前，小主管们为减少在处理订单时的摩擦，曾在部门内加了一道隔墙，到后来团队一致认为拆除隔墙更有助于沟通协作）

Weisbord 事后总结道：任何变革和重组都需要时间来磨合，期间管理者的持续支持至关重要。如果管理者不能坚持到底，直面并解决遇到的困难，改革很容

易半途而废。

启发

如果员工具备知识技能，管理者便应尽力给员工平台，让团队自主发挥，才能不断提升竞争力，应对变化万千的市场需要，让员工与公司都受益。管理者不仅仅是制订目标、计划，发号施令的角色，而更应该是一位导师，辅助团队成长。这故事也引出推行丰田式改善的 4 个常见误区：

- 不要指望一蹴而就
- 只能坚持到有结果，不能半途而废

本来 Weisbord 和 Don 误以为会很快就有效果，预估不了会出现这么多问题，如果他们没有坚持到底，就会以失败告终。

- 最重要是培养人才，但训练比教育更重要

大野先生认为：教育式教对方他们本来不知道的东西。训练是让对方重复练习已经知道的东西，让他们牢牢记住该怎样做。要做好训练，领导或内部教练必须能够身先士卒带头动手做。不然会容易被批“光会嘴巴说，但什么都不干”。

- 让员工自己找答案

过程改进需要每一个部门都参与，要由团队自己去找答案，而不是直接告诉他们怎么做，才可以提升公司的最终目标，

大部分已经是九零后了，很多员工都有自己的想法，很难要求他们像以前只听从主管指挥。所以也不能再依赖专家（外部或内部）和顾问解决问题，必须所有员工参与改进整个系统。上面费城故事就是领导让团队记住自主和学习，完成订单处理部门从传统主管式管理改组成团队自主管理的例子。当时，改成团队制后开始每周开会，不仅仅解决团队的疑问，其实也促进团队之间的交流，了解其他团队的问题。如果管理层认同这种新的管理模式，组织架构从本来的功能划分变为小组独立自主管理外，还有什么具体方法促进公司走向？所有员工参与改进整个系统？

回顾过去，放眼未来

最后依据培训的主题，让大家讨论并制定具体的改进目标和行动计划。

但若要有效果，管理者必须定期监控：

改进效果如何

遇到什么困难计划

计划要不要调整

若不及时监控，团队会觉得只是一般的培训，学完便完事了。

工作坊可以促进团队间的了解，例如：

- 产品开发部原来不知道现场实施工程师对新产品的安装和使用还是有很多误解。

- 后面开发部除了做安装手册也录视频，减少现场工程师误解。
- 开发工程师从现场实施工程师了解到有些因为现场测试不通过，导致不能按时发新版本的主因。
 - 原来客户验收有一套标准的流程，包括他们的测试数据和步骤，但测试人员交付前的检测都只是按自己理解测试，与客户验收流程有出入。

芬兰计划

要具备什么条件才能让敏捷开发团队有好成绩？

再邀请你来听我讲 ACP 的第一堂课。

(ACP = Agile Certified Professional 是美国 PMI 推的敏捷管理个人资质。)

???	???
94 年 由两位史丹福研究生创立	96-98 年 由两位史丹福研究生创立
1996年上市	2004年 上市
员工数 超过 8千 （2016）	员工数 超过 7万
产品领域:	一些产品包括:
Mail, Search, Web portal, Directory,	Android
News, Messenger.....	Chrome

表 2-1

“请问大家左右两家美国西岸 IT 公司是什么公司？”

估计你们都能猜到右面的是谷歌 Google，但不一定能认出左面的是 Yahoo！因为后者在 2016 年已经被收购。在 2001 年时，谷歌是刚成立不久的小公司，雅虎是首批互联网公司，当时已经是有名的上市大公司，市值比福特、通用汽车公司还高，并不断收购公司扩大业务。但也养成了很多“大公司”病——官僚化管理。例如很多决策或改进都需要管理层多层审批，有很多 X 管理的味道。

谷歌 (Google) 公司是由两位史丹福大学研究生 Larry Page， Sergey Brin 创办。在 2000 年,Eric Schmidt 加入并担任 CEO（他之前是 Novell 的 CEO，以前在 SUN Microsystems 工作）,Jonathan Rosenberg 也加入并出任产品经理（他之前在 Apple 当产品经理）。

谷歌的管理很扁平，当 Eric Schmidt 第一天进公司当 CEO，他本来是 Novell 的 CEO。因为谷歌公司快速发展员工很多，办公空位不够，人事部经理好不容易才为他找到一个办公室，但过了几天有一位工程师来问：“我那边的办公地点是在太剂了，我看你的办公室应该还可以容立多一个人。我可否搬过来过来？”虽然 Eric Schmidt 觉得很奇怪，但因为刚进公司，就接受了。

当时谷歌 (Google) 还是很小的公司，但因在搜索引擎上有所突破，预计会被微软攻击和收购。董事会主席 Mike Moritz 便请刚进公司的 CEO Eric 和产品经理 Jonathan 准备了一份详细的商业计划书来应对，代号“芬兰计划”。俩人经过两个多月细心准备，约创始人 Larry 开会解释计划详细内容，Larry 拿到计划

书快速看了一下，便立马问他们俩：“你们以往有试过超出你们的计划吗？如果没有，建议还是先跟我们那些工程师聊聊吧。”

后面他们俩发现 Google 的工程师都是精英，无论技术或业务方面的能力都很强，都有很多创新思路。了解到如果还是按传统的计划驱动反而会限制团队创新能力。

Eric 与 Jonathan 后面把谷歌的独特文化写成书 “How Google Works”，他们把公司里这些优秀人才统称为聪明创意人 (Smart Creatives)。

当我讲到这里时，便把书的目录派给学员，然后讲解书里相关章节，介绍谷歌公司文化：

七的规则 (The Rule of Seven)

一般公司组织架构都会限制主管直属下属人数，谷歌当时也规定人数为 7，但规定直接下属不能少于 7，可以看到他们组织架构多扁平。

河马 HiPPO

河马 (Hippopotamuses) 是攻击力很强，非常危险的动物。

谷歌公司用河马的英文简称 HiPPO (Highest Paid Person's Opinion)，比喻企业类内的“大王”按自己的主观判断，雷厉风行也是非常危险。

例如在谷歌的内部讨论会，做决定不看提出意见者在公司的地位、权威。高管提出的建议，如果缺乏充足的数据支持，也可以被工程师推翻，例如创始人之一 Sergey 想某工程部协助开发某新产品，但他的方案缺乏有力数据支撑，工程主管不同意。Sergey 没有用他的权威强制要求（他绝对有这权力），他让步妥协，只需要一半人手参与，但工程主管还是不同意，最终大家客观比较各种方案的优劣，Sergey 的方案未被采纳。

谷歌有个特点，每次开会都会用 2 个投影仪，其中一个专门投放数据的。很多公司管理层都会嘴巴说用数据说话。

两个披萨的规则 (Bezo's two pizza rule)

两个披萨应该让团队够吃，团队人数不能太多，妨碍有效沟通。

员工工作饮食生活在一起 (Work, eat, live together.),(Keep them crowded)

这跟敏捷开发要求团队必须面对面沟通，一起工作，效率才会高如出一辙。

所有重组在一天内完成 (Do all reorgs in a day)

因为市场变化快，组织必须频繁重组才能跟上。为了减少重组对工作的影响，规定每次重组必须快速完成。

针对客户 (**Focus on the user**)

制定几乎达不到的目标 (**Set {almost} unattainable goals**)

这 2 条与前面丰田的文化类似。也验证了持续改善的原则不仅适用于制造业。

70/20/10 规则

- 70% 的时间用于和核心业务
- 20% 用于具备潜力的新产品
- 10% 是用于全新的产品（例如在初步构思阶段）

20% 规则

员工都可以用 20% 的时间用于自己有兴趣的项目。

以前惠普公司（创始人 Hewlett 和 Packard 本身都是工程师）也规定公司实验室七天 24 小时开放。欢迎工程师任何时间进实验室做自己感兴趣的项目，不一定局限于公司安排的项目。很多创新的产品不是高层可以通过想象得出来的。例如谷歌的 Android，就是其中一个员工特别喜欢玩手机操作系统，开始的时候，他是用自己时间创建出来，后面成为谷歌产品的其中一条主力。

如项目不能达到预期效果，便立马叫停 (**Fail well**)

例如谷歌 Sydney 分公司推出新一代电子邮箱服务，虽然里面有很多创新，但一直没有达到预期的用户数量。便被叫停。

频繁交付和迭代 (**Ship and iterate**)

因为市场和需求快速变化，企业必须跟上节奏，尽量减少浪费。

这条与上一条类似，都是精益和敏捷开发的原则。

====

2000 年时，谷歌在快速成长期。我通常会在敏捷培训课的第一天借用谷歌的故事说明团队敏捷开发的成败依赖于公司文化。

讲敏捷课时，某学员听完敏捷精益的概念后，说她的经理要求她输出一份 3 个月的详细工作任务分解，且要求活动的颗粒度细到 5 天之内，因为她的经理按计划管理的思路很重。她很困惑，软件开发变化这么多，怎么可以做出这种详细计划呢？

这其实有点像我们要在一个足球赛还没开始之前，要预先写赛后报告一样没意义，都是猜。

我说：如果老板有这种思路的话是无法推动敏捷开发，其实敏捷开发有一个很重要的概念，因为需求变化这么大，我们的范围是不固定的，所以基于精益的概念，在有限的时间里，我们要制订这个冲刺在 2 周里面可以完成哪些故事。做完以后再依据过去的的数据进一步策划下一个阶段就更实际了，不是全都靠猜测。这种思路更能让客户尽早看到实际的项目进展，所以敏捷特别重视可以运行测试好的故事，而不是一堆需求文档或者没测试过的代码，因为需求没有给客户看过，还不算是完成，只是一个中间的状态。所以如果经理知道软件开发的这种特性后，就会在那个策划的三角形——时间、成本和规模范围之间适当取舍。如果我们规定时间为 2 周，应该可以在那些故事里面挑选最有价值，可以在 2 周完成的故事，完成它并展示给客户。这就有了 MVP（最小有价值产物）的概念，就会避免团队为了只追求进度，而导致最后做了一大堆没价值的东西。

早上讲完谷歌故事，学员下午做互动练习时便说自己公司里确实有不少“河马”横行。

从以上两故事看到，优秀的公司高层，无论东西方，都理解公司的发展依赖“大家的力量”和各人的智慧，同心协力，“大王”文化会妨碍公司持续改进，难以改善。从以上故事看到，如果公司缺乏敏捷精益的文化，管理层也不理解，单靠团队参加培训，不会有改进效果。这也是开头案例里张工推行敏捷开发失败的原因之一。

从 Weisbord 先生的故事，了解到 XY 理论，管理层要赋能与团队，公司才有机会提升，但这只是第一步。

从谷歌例子看到，公司文化、员工的能力都是关键因素。当时谷歌每位员工骨子里都已经包含了敏捷开发和精益，不仅仅是公司材料的条文，而是实际落到员工的行为上。当你踏进谷歌公司范围，跟员工接触，立刻能感受到。但有多少会像谷歌这样，用行动表现。所以我在培训 3 天 ACP 敏捷开发管理时，首先就以谷歌公司为例，说明如果公司没有类似的敏捷文化来支持团队，学了各种精益和敏捷的原则和方法，也会不会起什么效果。

团队的成长与能力取决于成员的能力，下章先探索一些能帮助个人提升的方法。

参考 References

1. McGregor, D. *The Human Side of Enterprise*.(1960)
2. Weisbord, M.R. *Productive workplaces revisited : dignity, meaning, and community in the 21st century*. (2004)

3. Schmidt, Eric *How Google Works*.(2001)

Chapter 3

克服拖延症

技术总监问：现在我遇到最大的难题就是如何提升下面技术人员的能力，如果他们全都是高手，我就很轻松了，但实际上高手最多只有 1/3，其他都是中低水平。你接触过这么多软件开发团队，有什么好方案？

我：你可以先听听以下故事。

=====

小李：你平常办公时间一直都很忙，还可以腾出晚上和周末时间，把客户遇到的问题，如何解决等，汇总成分享文章，每两周公众号发布，很厉害呀。

我：其实你也可以做到。要成为专业软件工程师，除了要学习软件工程相关的知识与技能外，个人有没有高效率的习惯其实更重要。

我在 5 年前教项目管理时参考过一本效率小册（详见参考 Reference），这本小册罗列了 99 个小技巧，每个技巧都不超过一页纸，我自己也一直用这小手册提醒自己。

目录

Part 1	Part 6
Beating Procrastination	Productivity Hacks
克服拖延症	产生效率黑洞
Part 2	Part 7
Becoming Organized	Doing the Right Work
做事更有条理	做对的工作
Part 3	
Staying Energized	
保持活力	
Part 4	
Getting Things Finished	
把事情完成	
Part 5	
Automate Your Routine	
让你的日常工作自动进行	

例如，第一章克服拖延症，这里的内容几乎全部都有帮助。

周/日目标 (Weekly/ Daily Goals)

我每天都会订计划，早上希望完成哪些功能，下午完成哪些。当然这个计划也会按实际的进展调整。

周 / 日目标是个人时间管理的基本功。每一天第一件事不是回邮件，而是仔细想想今天要完成什么任务，每一周的开始，也应该想我本周希望完成什么任务。不然的话，每天的时间就很容易被琐碎的小事吃掉，一事无成。

背后体现的道理很简单，要把时间花在重要、但非紧急的活动上，效率才会体现出来。

	紧急	非紧急
重 要	危机 紧迫的问题 受截止时间驱使的项目	针对风险未雨绸缪 建立关系 挖掘新机会 规划、娱乐
非 重 要	打扰，电话 邮件，报告 会议 紧迫的事情 受欢迎的活动	费时的琐事 邮件 电话 轻松快慰的活动

限定时间 (Timeboxing)

把每天的任务安排成时间段，每一段不应超过 1.5 小时。
一般人可以专心集中的时间段都不会超过 60 分钟，小孩可能更短。如果老师叫你星期五 5 点钟交卷，你不会提前交，都会等到最后 10 分钟，甚至最后 5 分钟。所以如果我们把一天的时间切开，分成 1~1.5 小时时间段，自然有动力，希望在时间之内完成任务。
我们写代码的时候应该也是用同样的原理。例如某些编程活动尝试了多次，但没有进展，有时总共会花超过 10 小时。所以每次当我发现某编程工作超过了 2 小时，我就会先做其他事情。

分解任务 (Dissolving tasks)

个人学习编程经验：因为都是练习题，所以每一个功能都比较细，不会超过 20 行。如果我们平常做开发时，也必须要把一些大、复杂的功能预先细分才有效率。

加强自律 (Building Self-Discipline Muscles)

晨礼 (Morning Rituals) 日常运动 (Make an Exercise Routine)

不要以为编码是一个单纯的脑力活，整天坐在屏幕前面敲代码就可以。如果人的体力、精力没有配合上也会出问题，好在我每天早上一直坚持 30 ~ 40 分钟的轻量运动，然后晚饭前 0.5 ~ 1 小时的骑单车或者慢跑的习惯。中间也不是整天坐着，一段时间会走一走、喝点儿橙汁等，以确保身体不断在动，这样才不会困，保持动力。
贝多芬每天都会通过去外面散步来获得一些创作的灵感，然后他会立马把这些写在本子上，用于后面的音乐创作。

我：身体健康，精神状态也同样重要，你每周有锻炼的习惯吗？

小李：没有，每天都太忙了，虽然一直觉得身体不如几年前了，也知道锻炼好，但无法抽出时间。

我：我也很忙，但深知定期运动对身体非常重要，我一直按以下 2 种方法保持个人身体状态—

- 工作时尽量避免长期坐下来，因我主要做培训、咨询、评估，所以可以大部分时间站着或在走动。(NEAT#)。
- 尽量每天 7 点吃早餐前跑圈，疾跑 1.5 ~ 2 分钟，休息半分钟，重复这循环 4 ~ 6 轮。(HIIT#)。

不会分心的工作场所 (**Create a Distraction-Free workplace**)

轻策划、迭代、再策划 (**Ready , Fire, Aim!**)

30 年前，软件开发都是一些大型的项目，整个架构要设计好才动手去写代码。现在反过来，需求变化极大，开发都需要敏捷，轻文档、轻计划，尽可能写好代码，做一些功能给客户，从反馈优化下一轮。我这次的几天开发也是用同样原则，没有花时间在有些设计或者文档。想直接把代码写出来，并通过单元测试，节省了很多耗时间的工作。把有限的时间都放在写好代码上。

不断清洗 (**Churning**)

万事起头难。我假期重学编码时也是遇到同样问题，不知如何入手，太久没看写代码的书了，很多基本的都不知如何入手。所以我开始的时候不会直接尝试写题目里面的功能，而是重写一些书本的代码，看看结果怎么样，然后逐步提升。写一些基本功能，慢慢有了习惯，调整过来了，后面就越来越顺。好比一台旧的水泵，刚开始抽上来的水总是有难喝的铁锈，只要不停止抽水，当污水最终都从系统中抽出后，就能发现底下的净水。

要有好的土壤 (**Remove your Hidden Roadblocks**)

在含盐量高的土壤里种植物是结不出果实的。浇水、平衡在阴凉处和阳光下的时间都抵不过根部吸入的毒素。如果我们没有积极性，就可能是土壤的问题。如果没有足够的积极动力，就不会在长假专注写程序，也不会定期要求自己写分享文章。所以要有明确、很想达到的目标驱动。像作曲家希望写出很多经典的优秀作品一样，会不满足于现在的状态。觉得自己的灵感或者创造力没有发挥出来，成为可以保留下来的东西。也是这种驱动力让我可以一直努力做这件事。

摒弃拖延恶习 (**Quit your Procrastination Vices**)

长假里，大部分人都会把时间用于看视频或电视剧，而我正好没有这个习惯，也一直没有玩网络游戏的习惯，否则肯定完成不了。

最终我用日程记录 (Timelogging)，把整件事和什么活动、时间花在什么地方都记录下来。

小李：我看你上面列出的技巧，我大部分都还没做到。

我：不要紧，我 6 年前刚开始定期写文章时跟你一样，但只要不放弃，一直往既定目标努力，不良习惯最终都会改正过来。我常常说人的潜力是极大的。舒伯特你听过吗？

小李：好像是很有名的作曲家。

我：是的，但他 31 岁就去世了，你猜他一生一共写了多少首歌和音乐作品。

小李：我记得中学时，老师介绍过他的艺术作品，如《鳟鱼》，但他 31 岁就死了，我猜 100 ~ 200 首歌？

我：他一生写了超过 460 首歌曲（时长 >24 小时）。除了歌曲，他还写了其他作品，如 9 首交响曲（1 首未完成，1 首只有草稿）、20 室内乐、120 钢琴曲等，每一类都包括大量经典作品，对后世影响深远。

小李：如果粗算一下，他一生约有 600 个作品，算他有 16 年时间作曲，平均每月要完成 3 个作品，真是不得了。

我：虽然他的作品有大有小（从一首歌到 45 分钟的交响曲），他确实生产率极高，而且他最后的 7 年身体一直都不好，所以他那个时候肯定不会像我们现代“996”方式工作。他每天主要是早上用来写作，傍晚便去休息散步。但他会同时做多个创作项目。如果项目没有灵感，就暂时放下来，创作其他作品。他著名的未完成交响曲就是个好例子，只有两个乐章（一般交响曲都是 4 个乐章）。所以他是使用高效技巧的一个成功例子。每个人都有自己的理想，但如果没有高效率来执行，理想只是天马行空、天方夜谭，不会有任何成就。除了以上这些技巧外，保持整洁也重要。你有没有试过想找某东西，找半天都找不着？

小李：确实经常发生，而且还会遗漏东西。我上次出差便忘记了 iPhone，后面回北京后电话联系当地酒店前台后，我找当地同事去酒店取，然后快递给我，烦死了。

我：有听过 5S（5S 法 #）吗？例如，如果你把东西都放固定地方，就可以避免同类问题再发生。如果你一直在一个很乱的环境工作，会导致心情烦躁，对工作、身体都不好。

（# 详见附件”锻炼之道”的多了解 NEAT、HIIT；5S 法详见第 1 章附件。）

总监：我大概懂你的意思了，要提升技术人员的能力应先改变他们的习惯——如时间管理，才有机会提升。

=====

总监：我大概懂你的意思了，要提升技术人员的能力先要改变他们的习惯，有良好的习惯—如时间管理，才有机会提升。

即时笔记 (The Capture Device)

总监边听边在本子上记下那些重点。高效的人都会有工具帮他记录想到的灵感、想法、项目、待做事项等，不会仅仅靠大脑记忆。你提出一个要求，他会立马写在小本子上，你会觉得他应该会按你要求去处理，但反过来如他只是口头说会处理，你会担心很可能没有下文。但我看有些领导，身边只拿个手机，除非他们的记忆力超人，否则我估计他每天都会忘记不少重要事项。

很赞同杭州某高级经理的总结：

人一定要自律！你说的小技巧确实能起到很大帮助，而且我基本都会使用，但如果不养成习惯，想起来使用下，最终还是改不了拖延症。所以要解决拖延症，一定从根源做起，还是得靠自己，需要培养自己意志力、专注力，坚持好习惯，改掉坏毛病。

人分高低，但并非取决于基因、种族，主要取决于她后天的习惯、自律与努力。要养成良好习惯要从小开始，深受家庭和教育的影响，所以百年树人。

与公司改进一样，改变个人习惯很难，这些技巧可以帮助个人改善。

附件

锻炼之道 The Truth about EXERCISE

想大家都同意和相信：“多运动，便多烧耗卡路里，便能帮助减肥，降低体重。”

某国家给市民的健康指南：每周起码做 150 分钟中强度锻炼，或 75 分钟高强度锻炼。

但不是每个人都能每天抽时间做锻炼，有什么更好方法？

不一定只依赖去健身房锻炼，平常工作生活少坐多走、站着工作、开会，甚至小动作等都有帮助。

N.E.A.T. (Non-exercise activity thermogenesis) 小实验：教授使用有电子传感器的底裤，记录记者、咖啡厅女服务员、商务人员 3 人一周每天正常工作中消耗多少卡路里。

发现：

- 女服务员最好。因每天都非常忙碌（尤其是早餐时段），送餐、接单、做咖啡等等。
- 商务人员第二。虽然有很多时间坐下来，但每天都会走一公里路见客户，而且每周二、五下班后会去健身锻炼。
- 记者最差。每天无论工作或家里，大部分时间都是坐下不动，所以他看起来不胖，但其实体内存有大量脂肪，集中在肝、肾等内脏。

这实验告诉我们：如果每天一直坐下不动很不好，就算每天下班后晚上都去健身锻炼也帮不了。

后面记者听教授建议，改变习惯，定期站起来走动。如与同事交流与尽量边走路边交流，减少长期坐下来；多爬楼梯，少用电梯；骑单车，不开车等方式。后面，从实验数据分析，发现这些改变帮他增加每天卡路里消耗接近一倍，到 500 水平。

研究发现不断大量健身锻炼不一定对每个人都有效。有 20% 会没有效果，另一端对 15% 的人会非常有效。这跟人的基因密切相关，所以多锻炼不一定都有效。

=====

实验发现，如锻炼能快速提升心跳率到最高，然后休息，反复做 4 6 轮，效果不会比大量健身锻炼差。例如每天做几轮 40 秒的冲刺，把心跳速度快速提升到极限，效果可以比长时间的缓步长跑更好。

HIIT (High Intensity Interval Training) 小实验：教授与助手教记者使用运动单车做 HIIT，用尽全力练 20 秒，休息，再同样做两轮；每周 3 次。记者按教授要求完成了 4 周 HIIT 锻炼，虽然帮他提高了血液分解糖份的能力，减少糖尿病风险；但提升不了他的最高带氧运动量。教授解释这是因记者的遗传基因是属于没有效果的 20%。

参考 References

1. Young, Scott. "The Little book of Productivity." 《超效率手册》

Part III

自我管理开发过程

如何估算工作量一直都是软件开发的挑战。大家都知道应分析历史项目数据，作为后面项目估算的参考。但首先要不同项目的数据可以相互比较。如能客观衡量项目规模大小，便可用规模作为分母把不同大小的项目归一，让不同大小的项目可比。本部分会讨论应使用什么来估算项目规模。

软件开发工程师自己收集数据并分析，是团队量化管理的基础。敏捷开发团队可以从个体软件过程 (PSP) 开始。本部分也会简单介绍估算工作量、工程师如何收集数据并分析等的挑战与最佳实践。

估算本应是一个范围，不应单是一个数，会先用一个例子介绍三点估算法，并与模拟结果比较，衡量三点估算法的正确性，是否可用。

Chapter 4

三点估算

估算是一个范围，不是一个数

唐工：你估计完成开发用户登录模块要多少天？

小李：3 天。

唐工：能在 3 天完成的可能性有多高？

小李：可能性很高。

唐工：可否量化一点？

小李：可能性为 50% 60%。

唐工：所以很有可能不止 3 天，要 4 天了。

小李：对的，其实也有可能要 5、6 天，但我估计机会不大。

唐工：你信心有多少？

小李：难说，有 95% 的信心可以在 6 天之内完成。

唐工：所以有可能要用上 7 天了？

小李：这样说吧，如果所有可能出问题的都出了问题，甚至会 10 天或 11 天，但这种概率很低。

唐工再问小李：是能否给我一个确实能完成这个模块的日期？

小李：正如我前面说，很可能 3 天可以完成，但有可能 4 天。

唐工追问：你可以说 4 天吗？

小李：也有可能 5、6 天。

唐工结束对话：OK，请你尽力 6 天之内完成这个模块。

唐工貌似请求，但实际是要求小李承诺这个模块要在六天之内开发完。假如这个模块的开发时间超过 6 天，唐工就有依据说小李没有尽力导致延误了。

所以从以上对话，可以看到作为开发专业人员，必须分清估算和承诺。作为专业人士，我们不应该给一些没有把握的承诺，误导对方。中国老话说“一诺千金”就是这个道理。

从单点到三点估算

从上面的例子可以看到，一般的单点估算是很容易被误导，以为那个天数是有把握达成的，所以我们最好从单点估算变成三点估算，除了估算最可能的天数，

还有最佳和最差共三点。但项目是由一系列的任务组成（如第二任务依赖于第一个任务的完成），如何计算所有任务的总天数？下面用例子说明如何用 3 种使用三点估算估计的方法（A、B、C）估算总天数：

A) 假定都是正态分布，用模型估计：

先用 PERT 方程式计算每一步的预计值与标准差：

预计值 (Expected Value EV) = (Best + 4xMost Likely + Worst) / 6

标准差 (Sigma) = (Worst - Best) / 6

Step	Best 最佳	Most Likely 最可能	Worst 最差	EV	Sigma
1	1	3	12	4.167	1.833
2	1	1.5	14	3.5	2.167
3	3	6.25	11	6.5	1.333
		10.75		14.168	

表 4-1

如果假定是正态分布，按以上预计值和标准差，使用蒙特卡洛模拟，从下图可看到，95% 置信区间是 8.02 ~ 20.37

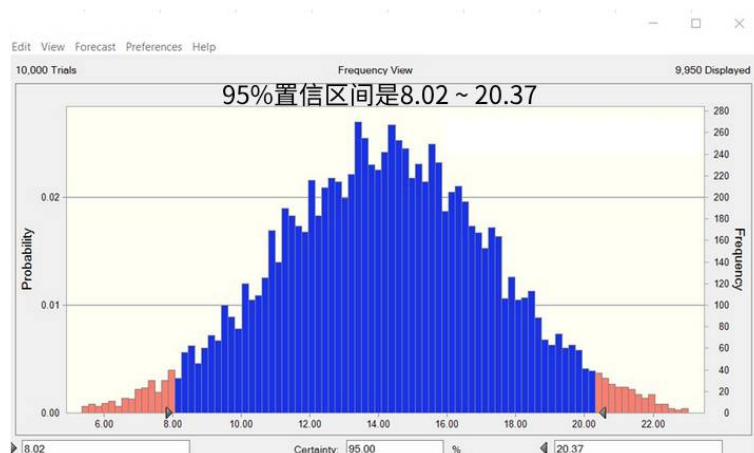


图 4-1

B) 直接用 PERT 方程式计算总天数的均值与标准差：

如不用模拟，直接把 3 步的均值加起来：

$$4.2 + 3.5 + 3.6 = 14$$

计算 3 步总方差：(方差 = Sigma^2)

假定：总方差 = 每步方差的总和

$$\text{总方差} = 9.77$$

$$\text{Sigma } \sigma = 3.13$$

95% 置信区间计算公式为：均值的总和 $\pm 2\sigma = (4.2 + 3.5 + 3.6) \pm 2 \times 3.13 = 14 \pm 6.26 = 7.74 \sim 20.26$

结果与蒙特卡洛模拟预测类似。

C) 假定都是三角形分布，用模型估计：

如果用三角形分布，95% 置信区间是 10.38 ~ 26.45

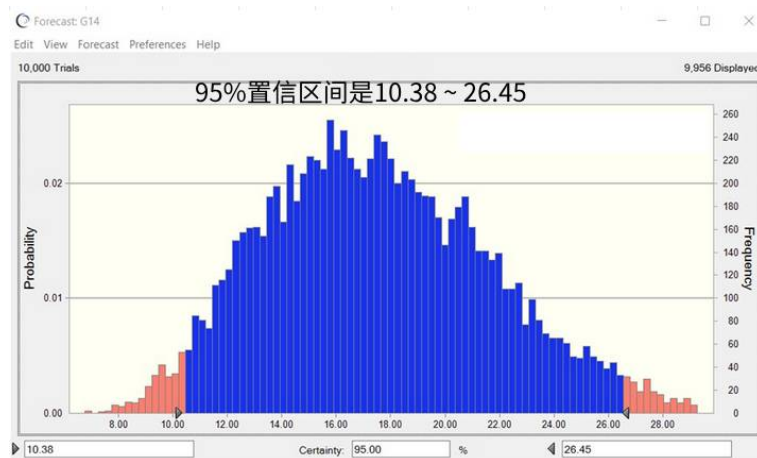


图 4-2

总结 + 解读分析结果

- 如果假定每一步的分布都是一个正态分布，就可以用头两个方程式计算每一步的平均值跟标准差和方差，用方程式可计算 3 步的总均值大概是 14。也可以用方程式计算标准差，总的标准差 (sigma) 是 3.13 左右。
- 也可用蒙特卡洛模型（假定步骤都是正态分布），得出很类似的正态分布，总的平均也接近 14，95% 的置信区间是从 8.02 ~ 20.37，接近上面算出的均值 $\pm$ 两个标准差数值。
- 但因 3 个步骤都是明显往右偏，所以不能假设它们是正态分布，更合适的是使用三角形分布，然后用蒙特卡洛估算“加”起来的分布，看见最后的图明显是类似往右有个尾巴，能更正确反应 3 个步骤加起来的天数的估计分布。
- 跟假定正态分布的结果比较，很明显看到用三角形分布结果往右偏，上限是 26.45（比正态分布的 20.37 高）。不是正态分布的话，左面就没有长尾巴，所以就会比本来正态分布的下限高，下限是 10.38（比正态分布的 8 高）。
- 从这简单例子看到，如果我们要把三点估算加起来，尤其是非正态分布的话，就不能用简单的方程式，或者假定它是正态分布来计算，需要用蒙特卡洛模型假设三角形分布才能真正反应总体的分布。

从这 3 个偏左分布步骤例子看起来好像有些偏差，但不是很严重。如果我们看见用十个步骤都是偏一边分布，总分布会如何？是否相差会更远？

利用蒙特卡洛模拟 10 个步骤（三角形分布）的总分布

如果每步都估算天数，10 个步骤的总天数就是 500 天（把 10 个估算值加起来）。

但如果每个步骤都是三点估算：

Process 过程	天数 Durations		
1	27	30	75
2	45	50	125
3	72	80	200
4	45	50	125
5	81	90	225

表 4-2

很明显看到每一步都是偏左的分布，所以可预计总天数应不止 500 天，但估多少才合适？

假定每步骤是三角形分布，用模型估计重复 5000 次，得出下面分布：



图 4-3

Process 过程	天数 Durations				
1	27	30	75	37	80
2	45	50	125	61.66	13.33
3	72	80	200	98.66	21.33
4	45	50	125	61.66	13.33
5	81	90	225	111	24
6	23	25	63	31	6.66
7	32	35	88	43.33	9.33
8	41	45	113	55.66	12
9	63	70	175	86.33	18.66
10	23	25	63	31	6.66
		500		617.33	

表 4-3

A) 用 PERT 方程式计算每一步的预计值与标准差：

Step	Best 最佳	Most Likely 最可能	Worst 最差	Expected	Sigma
1	27	30	75	37	8
2	45	50	125	61.6666667	13.333333
3	72	80	200	98.6666667	21.333333
4	45	50	125	61.6666667	13.333333
5	81	90	225	111	24
6	23	25	63	31	6.666667
7	32	35	88	43.3333333	9.333333
8	41	45	113	55.6666667	12
9	63	70	175	86.3333333	18.66667
10	23	25	63	31	6.666667
		500		617.333333	

表 4-4

得出 95

B) 假定是正态分布，按以上预计值和标准差，使用蒙特卡洛模拟，得出的总分布的结果几乎一致，都是左右平均分布的正态形。

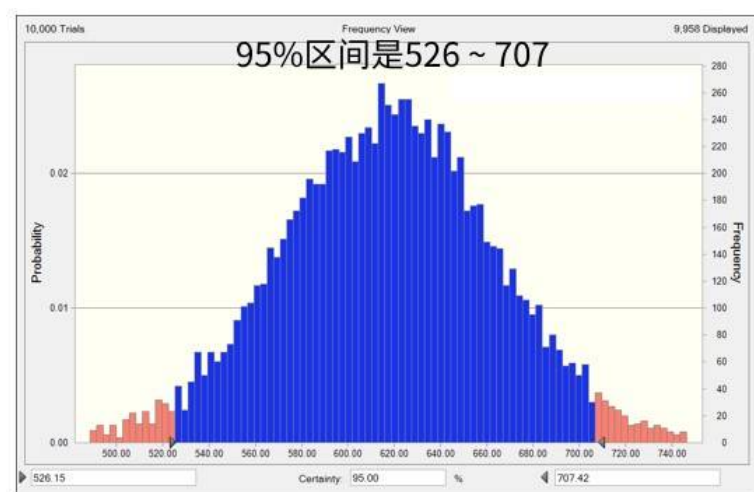


图 4-4

分析 10 个步骤模拟结果

- 为什么用三角形分布模拟出来不是偏左的分布（类似前面 3 步结果），而是一个正态分布。

以上实验验证了“中心极限定理”，无论本来是什么形状分布，如果随机抽样够多，样本的平均值分布接近正态分布。所以如果本来只是 3 个步骤的时候还是可以看出是三角形偏左，但到了用 10 个步骤相加时，得出的分布便非常接近正态分布。

(中心极限定理会在后面数据分析里用上, 例如通过画控制图判断过程是否稳定)。

- 实验结果也验证了当每一步都类似正态分布可以用 PERT 公式计算每一步的预计值和标准差, 然后计算总结果的分布 (不需要蒙特卡洛模拟), 但如果非正态分布 (如偏左的三角形分布) 便需要使用蒙特卡洛模拟, 不然预估会有偏差 (类似上面 3 步模拟的结果)。

问答 Q&A

问: 为什么要花这么多精力去研究分布, 我们日常不都是单点估算吗?

答: 例如你觉得把整个公司的人均生产率从 1.14 一年后提升到 1.21 算不错吗?

(注) 问: 不是非常好, 还算可以。

答: 如果本来的分布和提升后的目标是如下图, 你觉得怎么样?

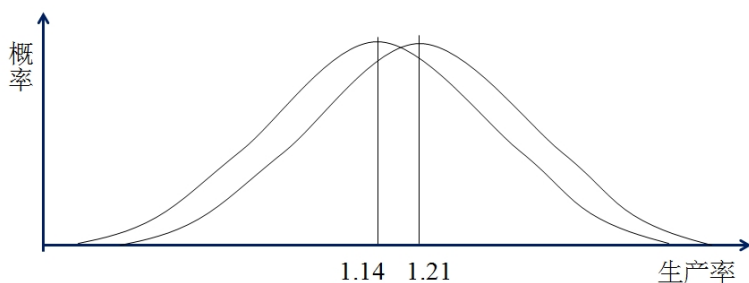


图 4-5

问: 提升就太微小了。

答: 从这简单例子看到所有估算都应包含两部分: 分布和中间趋势 (例如平均值)。

另一例子: 假如我们预估生产率的分布是如下图, 达到或超越 1.14 这目标的概率是 65

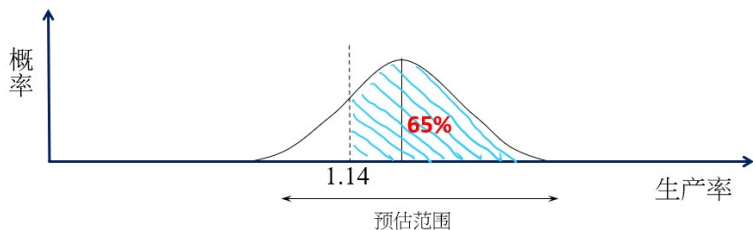


图 4-6

但如果告诉你目标是 1.14 只是目标平均值, 分布是如下图:

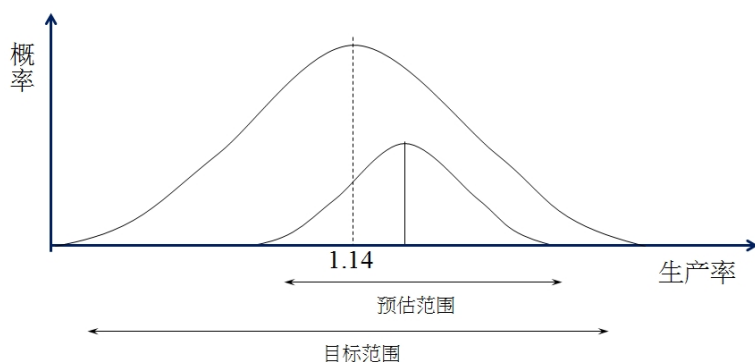


图 4-7

预测的生产率分布完全在目标范围之内。

(注：生产率单位每人天产出代码的功能点数，类似有效代码行数都是衡量软件规模的单位。)

在下一部分，我们会看到如何使用 PERT 三点估算的实例。

附件

蒙特卡洛 (Monte Carlo) 模拟

当结果不能用数学公式计算的时候（例如是三角形分布），可以用电脑随机模拟结果。例如：

- 计算 3 个步骤的总共人天，每个步骤都是三角形分布，我们就用电脑的随机功能模拟，让随机功能的结果按三角形分布。
- 第一次模拟：步骤 1 得出 1.3，步骤 2 得出 1.2，步骤 3 得出 2.0，得出 3 个步骤的总工期是 4.5 人天。
- 第二次模拟：步骤 1 得出 1.5，步骤 2 得出 1.15
- 如果我们模拟 1000 次、10000 次，便能模拟出总分布。
- 因为是电脑随机模拟，出来的结果会有些偏差，但差异不会太大。（例如上面 10 步三角形分布的模拟结果偏差都没有低于 0.3

5000 次；

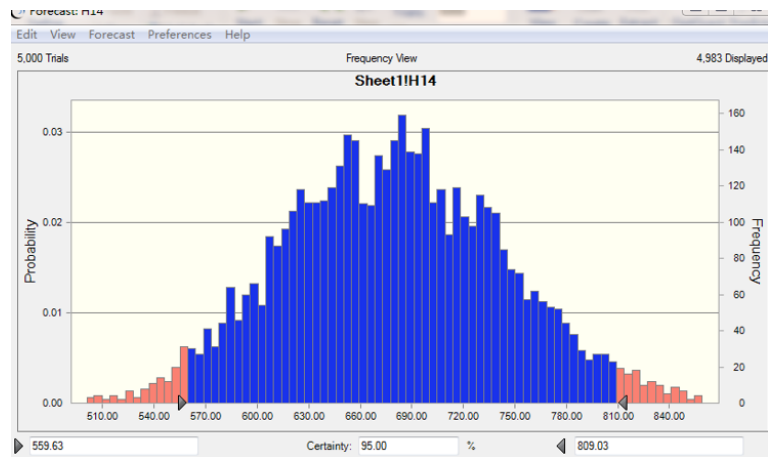


图 4-8

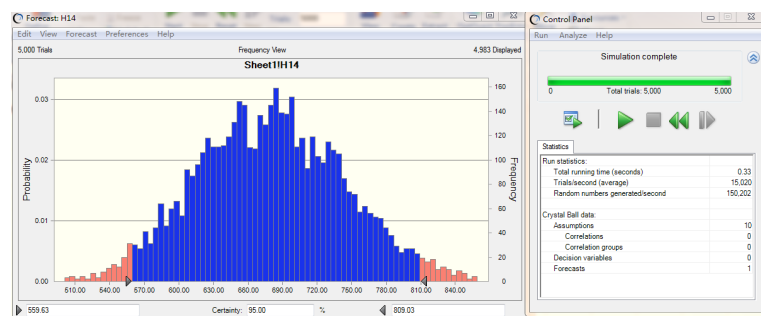


图 4-9

10000 次;

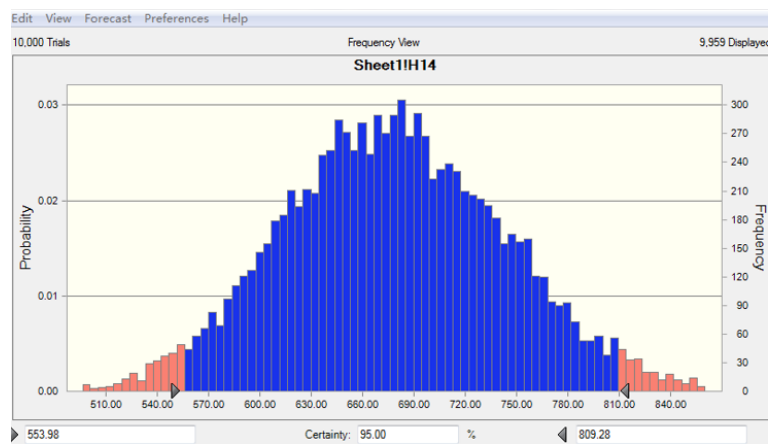


图 4-10

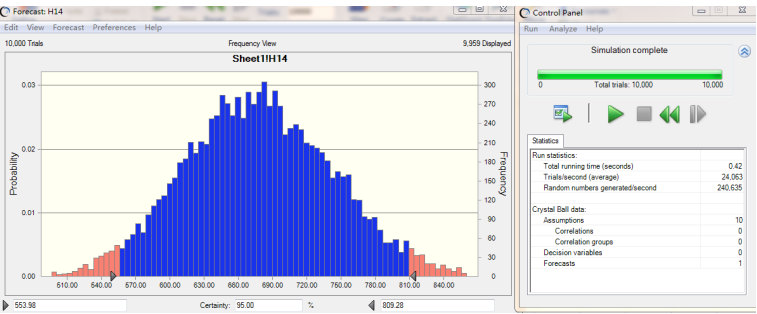


图 4-11

Chapter 5

量化管理从个人开始

我：你们管理层和客户都比较关心项目的进度，项目是否能按时完成？请问你们过去的项目如何？

开发：我们现在就是走敏捷开发，两周一个迭代。每次迭代前，我们聚一起开会，把所有用户故事按优先级排序，估计这个迭代可以完成哪些任务？然后放在看板的待办事项里。每天我们会做站立会议，监控实际的进展。

我：你们都能按计划在冲刺迭代里把所有任务开发好吗？

开发想了一下，说：我们也有延误出现，有些模块不能在计划的冲刺里完成。

我：为什么？

开发：我们花了很多时间修正系统测试暴露的问题，有一些是因为前面没有把需求分析透导致的返工。

我：请问你们是怎么估计每个任务应该花多少时间？

开发：我们会一起讨论，然后用敏捷的扑克牌方式，集思广益来估计。

我：除了你们每天站立会议监控项目的进展，你们要不要统计一下项目延期的水平是多少？

开发：其实我们也不算是延误，有些模块不能在本迭代完成，就放在下一个迭代去做。

我：软件开发有一个系数叫每人的生产率，就是开发人员一天能产出多少有效代码？你们有统计吗？

开发：你说的是否是敏捷开发燃烧图里的速度？我们有画，但发现变化波动很大，所以后面我们也没有再用。

如何降低项目进度偏差

敏捷开发不一定能帮助团队按期交付，开发人员也不清楚自己的生产率是多少。要减少项目的延误，首先取决于估算是否准确。但如果只是依赖个人经验、头脑风暴去估计，很可能低估，因为可能没有考虑开发以外的因素，比如缺陷、需求等问题。在冲刺里，不能交付所有计划的模块，等同于延误。要做好估算，就需要有数据。数据需要从个人收集的历史数据作为参考。IT 公司是否有注意这方面？我们可以先了解他们的新员工入职培训。

我：请问你们如何培训新入职的开发人员？

培训师：先上基础理论课，培训公司的代码规范、框架和复用的模块。然后进行个人或小组练习，我们会简化一些简单的开发项目里面的某个模块，让他们照着做个小项目，然后我们就会对他们的成果进行反馈，让他们知道自己的开发水平如何，是否掌握了我们所培训的内容。

我：他们会收到什么反馈？

培训师（想了一下）：依据他们是否掌握了培训内容的重点来打分。

我：怎样打分？

培训师：我们依据评判标准打分，分数从 1 到 5，1 最低，5 最高。

我：评判的标准可以说说吗？

培训师：（想了一下）依据他们是否掌握了培训内容的重点打分。

我：你们公司不是已经开始推行量化项目管理，是否应该也配合量化管理，不仅仅靠老师主观判断来打分？

培训师：可否举些例子？

我：例如他们质量方面的缺陷数，如果有做单元测试或者评审相关阶段的缺陷排除，项目管理相关所花费的工时，从而可以反算出编码的效率等客观的量化指标。

从量化培训开始

很多公司都没有与量化软件开发项目管理相关的培训，这样如何能希望他们在项目中做好量化管理？入职不仅需要培训开发的技巧、怎么写好程序、做好面向对象设计，也应该学到要一直度量自己的开发过程，包括所花的工时、过程中发现的缺陷数等。

在新员工培训时，如何可以加入这些量化的元素呢？

在先培训质量相关的技术指标，比如什么叫质量成本、什么叫排除率等，然后做实战练习时，要求他们除了写程序，还需要记录所花的工时，自己项目检查时发现的缺陷数、测试的缺陷数、评审所花的时间等。也要求开发人员除了提交程序以外，提交开发计划报告，内容包括实际所花工时、本来预估的工时、过程里发现的缺陷数、在哪一类过程等。帮助他们培养量化度量的习惯，以避免过了试用期，开发的习惯已经养成，后面难以改变。

从估算、策划到监控

挣值分析 (Earned Value, EV) 是常用的项目监控技巧，如果简单用百分比监控项目的进度或工作量偏差，因缺乏明确定义，难以比较，挣值分析把计划、实际等都统一用成本来计算，没有项目管理工具也可以简单手工计算（EV 公式与示例，详见附件）。

Humphrey 先生出版了不少软件工程 PSP（注）的书，他利用自己如何用 EV 方法监控自己写书的进度偏差，来说明开发人员也可以用 PSP 里的 EV 方法监控自己的开发进度。

注：个人软件过程 Personal Software Process(PSP) 与团队软件过程 Team Software Process(TSP) 都是 Humphrey 先生当卡内基梅隆大学软件工程研究院 (CMU SEI) 时提出的方法，帮助提供软件开发的质量和效率。）

我也试用挣值法来监控写这本书的进度，确实能帮我预估能否按期限完成交付和监控进展（详见附件案例）。下图是 PSP 估算与监控的流程：

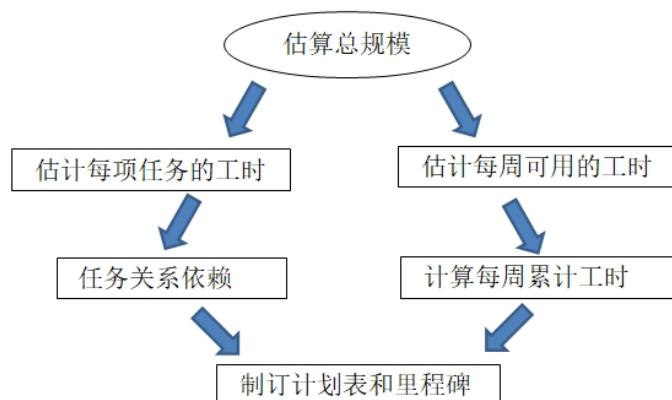


图 5-1

可以把敏捷的燃烧图方法看成挣值分析法的简化版，挣值分析也可以按现在的速度来预计完成的时间（燃烧图假定每一轮迭代的速度不变）。

传统大型项目每个时间段投入的工作量会有变化，挣值分析法可以按当前的进展和投入预计完成时的总成本。

想了解如何使用高中低三点估算和 PERT 方程式估算总工作量与范围，详见附件里的案例。

很多软件开发项目延误的原因是以前没有收集历史数据，导致估计所需人时都很理想。我也有同样问题：从我的挣值分析实例看到，因我是第一次写书，之前也没有养成统计章节所花的实际工时的习惯，所以在 11 月初做估算时，理想地认为最多花一个月完成，但过了 3 周就发现，实际比本来计划延误很多，原因包括：

- 实际可用工时没有想象这么理想，有很多意外事情都要处理。
- 低估了完成一章节所需时间：
 - 例如第一个章节 TDD 实际所花的工时与估计差不多，原因是这个章节是现有的，只是做了一些调整，所花时间不多。
 - 但到第二个章节 MGR 就不一样了，全部内容都重新写；如何从实际客户现场的场景总结，与理论结合也很花心思。

所以下次当你遇到团队项目延误问题，可以问项目经理以下问题：

1. 如何估算项目工期
2. 如何估算每一项任务所需的工时，依据是什么？
3. 如何收集实际工作量？

4. 如何监控累计到现在的完成情况？
5. 有没有统计以往的历史数据？如何用来帮助做好估算？

你可能会发现类似我用挣值分析估算和监控暴露出的问题。

如何改善

软件开发和工业生产不同，工作量数据必须靠个人自己收集，量化管理依赖个人习惯，所以 Humphrey 先生在 90 年代推出了 PSP，教导开发人员如何一步步自己收集数据、自己估算、自己监控。例如记录花了多少工时、发现多少缺陷、完成多少代码（从最基础开始，逐步改善），尽量减少缺陷返工，提升开发效率。（详见附件 PSP 简介。）

也正因为大部分的团队都没有收集数据的良好习惯，没有 PSP 的基础，所以就很难要求他们在冲刺回顾时拿出数据，分析根因、在下一轮迭代改善，从定性提升到量化管理。当开发人员养成好习惯，就会更清楚知道自己的速度和质量水平，不会出现本章开头的场景，开发人员对自己的质量水平或生产率一无所知。

有些企业深知量化管理对企业发展很重要，所以在新员工培训时，不仅仅教他们技术技能，也教他开始自己每天记录工时、缺陷数等习惯。因为工作习惯养成后，便难以改变。反之，如果周边的人都有记录数据，自己也觉得应做记录，逐渐形成公司文化。

问答 Q&A

资深敏捷顾问：我大约 20 年前对这个东西比较熟，PSP 包含 2 个重点：

1. 关于个体的估算内容，这个我看您的文章里边表达得比较充分。
2. 关于缺陷记录分类统计和根因分析的内容，这一块实际上相当于是 CMMI 里需要个体配合的地方。我看您几乎没有提到。

我：非常赞同，若要做到本手册后面提到基于缺陷数据的量化根因分析并改进，就要依赖个人记录缺陷与返工工作量，碍于篇幅有限，这里先用人时的估算与监控带出 PSP 的思路。

顾问：我自己也是在某军工航空项目里边用过一次。我们当年计划少、跟踪多，大家本来就缺少生产率的概念，所以其实也不知道每天能干什么事，但只要记下每天干了什么事，然后比对这个基数做就可以了，无论快或慢，都需要看看到底是什么原因。

我：您觉得这种方法有用吗？

顾问：因为开发依赖人的创造力，写新程序其实很难准确估计所需时间，但我后来发现如果估算是由组长负责去做，就能起很大的作用。例如有一次，我看有一个程序员花了一个月时间用 Java 写了某模块，接近 1 万行代码。虽然我不熟悉 Java 语言，但觉得代码有点问题。后面我就找另外一个比较资深的组长，我们 3 个人一起看代码。评审并删除无用的代码，最后剩下不到 100 行代码就可以实现了。如果当初我们先做好这模块的估计，就可以避免一个月人时的浪费。所以我建议还是需要估算，但是应该由有经验的技术组长负责。

我：很好，你很熟悉敏捷，例如 SCRUM 里用扑克牌方法估算，你觉得怎么样？

顾问：我先跟你讲个故事。我曾参加某冲刺估算会议，他们用扑克牌方法估算“数据显示”故事点，其中一位估计是 8，另外一位估计是 40。为什么会差这么多呢？第一位解释只需要展示数据，确实该工作量不大。但第二位理解就不一样了，包括整个数据的分析、输入和展示等，整个过程工作量确实很大。从这里可以看出，如果没有统一理解需求，就难以估算。所以那次以后，需求分析应分成“实体”和“行为”，清晰地写出功能需求，减少误会。（下章功能点估算规模里会说明“实体”和“行为”是什么。）

我：赞同。Humphrey 先生在 PSP 书里强调，要参考历史数据来做好估算，其实也可适用于敏捷估算。不应靠人的主观判断。

顾问：是的，所以我前面强调必须要由经验的主管去负责估算。估算很重要，但是要程序员自己去估很难。所以 PSP20 年前的思路，还能适用于现在的敏捷开发团队，帮助他们做好估算。

PSP 里强调要评审设计、代码并分析缺陷，当时因为只靠人手，所以会很耗时。现在，我们有类似 SONAR 的静态扫描工具，可以像机器人一样，做本来耗时的代码评审工作。但 SONAR 只能针对一些基本语句问题，针对整个 OO 设计，我们会建议用我们的工具去扫描，补充 SONAR 扫描的不足。跟 PSP 代码评审的思路一样，所有扫描出的问题都必须修正。有些团队觉得问题太多了，只处理掉那些重大的问题，剩下一些可能不会直接影响到功能的问题暂时不处理。我不赞同这思路，这么多的问题其实都是累积出来的，如果从一开始都一直有定期处理清空，就不应该累积大量问题，而且这些问题遗漏下来还是对程序有隐患。

我：很赞同，归根到底还是开发人员缺乏保证质量的意识。最近有个团队也是用工具扫描代码，然后我问他为什么找出的部分问题不处理。他说例如定义某个变量，但是没有用，虽然被扫出是问题，觉得不影响程序的运行就不处理。我解释说，这样就好像放了一个地雷，后面还是会爆炸的，所以还是应该要处理。

PSP 要求评审代码，分析每个发现的缺陷，讨论原因，然后改进。有些管理者听过 PSP，但认为 PSP 成本太高，只适用于高价值、高质量要求的项目。如果他们理解 PSP 能最终帮团队把缺陷返工减半，能降低开发成本 20%，便不会再觉得 PSP 不适用于公司。

估算很重要，要做好，不仅仅要参考历史数据，也需要对需求有共同的理解。功能点方法是估算软件规模的一种标准，下一章会用实例介绍。

附件

挣值分析法 (Earned Value)

用最简单的例子，来说明挣值分析中 PV、EV、与 AC 的意义。

PV：计划完成多少？

AC：完成工作的实际成本是多少？

EV：实际完成了多少工作？

公式：

进度偏差 $SV = EV - PV$, $SPI = EV / PV$ 完成占计划完成的百分比

成本偏差 $CV = EV - AC$, $CPI = EV / AC$ 完成的价值占实际已花成本的百分比

本文实例主要关心进度偏差，所以没有计算 AC

以下面“使用挣值分析监控案例”为例：

截止到 22.11.20，PV 累加本应是 27.5，EV 只有 3.1

所以进度偏差 $SV = EV - PV = 3.1 - 27.5 = -24.4$ 工时

$SPI = 3.1 / 27.5 = 11.27\%$

用 PERT 公式估算头三周任务累积工时的 95% 区间（工时）：

不用 MonteCarlo 模拟，直接把 12(=1+4+7) 章节任务的 ExpectedValue 加起来

计算 12 步总方差：(方差 = σ^2)

总方差 = 每步方差的总和

总方差 = 1.69 ($\sigma =$ 总方差的平方 (Sq.Root))

EXCEL 公式为：SQRT(SUM(所有方差范围)) = SQRT(SUM(H3:H15))

95% 范围下限 (=55.8)

EXCEL 公式为：SUM(所有 Expected 范围)-2× 总方差 = SUM(F3:F15)-2×I15

95% 范围上限 (=62.5)

EXCEL 公式为：SUM(所有 Expected 范围)+2× 总方差 = SUM(F3:F15)+2×I15

	A	B	C	D	E	F	G	H	I	J	K
1										95%区间(工时)	
2		章节	最佳	最可能	最差	Expected	Sigma	Sigma^2		下限	上限
3	22.11.7	TDD	2	2.5	3	2.50	0.17	0.027778	0		
4	22.11.14	MGR领导的任务	4	5	8	5.33	0.67	0.444444	0		
5		PSP	3	4	5	4.00	0.33	0.111111			
6		M6	4	5	6	5.00	0.33	0.111111	0		
7		M5	4	5	8	5.33	0.67	0.444444	1.07	20.0	24.3
8	22.11.21										
9		IC	4	5	6	5.00	0.33	0.111111			
10		CM	8	9	12	9.33	0.67	0.444444			
11		SiFP	3	4	5	4.00	0.33	0.111111			
12		INNOVATE创新	4	5	8	5.33	0.67				
13		MitOCW与Python	2	2.5	3	2.50	0.17	0.027778			
14		克服拖延症	2	2.5	3	2.50	0.17	0.027778			
15		CI	6	8	12	8.33	1.00	1	1.69	55.8	62.5

表 5-1

如何简单记录工时

如前面“克服拖延症”里提到，每人根据自己的习惯，采用每日 ToDoList 的方式，然后在开始前，简单记一下时间，结束时也记下时间。然后每日/每周统计每任务总工时：

日期	章节	时间	产出物	工时
11.25	PSP	8:15 ~ 10:10	capScreen	2hr
	SiFP	14:00 ~ 17:00	Q.A draft	3hr
	INNOV	20:15 ~ 21:00		1hr
11.26	PSP	9:00 ~ 11:00	PV ~ 95% range(xls) Screen.Caps	2hr
	SiFP	12:30 ~ 17:30	INNOV ... (1126)	5hr
	INNOV		PROD ... (1126)	
	PROD		Wbs	
		18:00 ~ 19:00		

表 5-2

如只是个人，不一定需要项目管理工具；如果是团队，可把自己手工记录上传到项目管理工具，方便团队记录与监控。

个人软件过程 Personal Software Process（PSP）简介

PSP 的简单介绍

- **PSP0** 基础 - 工时：计划与实际对比；每阶段引入多少缺陷；排除了多少缺陷
- **PSP0.1** 加入代码行统计——计划与实际对比；代码规范
- **PSP1** 加入使用 PROBE 方法做规模估算
- **PSP2** 加入设计与代码评审的计划与统计
- **PSP3** Cyclic process 先做策划与总体设计，然后多轮开发，有点类似迭代开发。

PSP 跟 CMMI 成熟度模型类似，也是按部就班一步步，帮助软件工程师利用度量改进自我的开发过程。

PSP 0.1

第一步 PSP 先估计写某个模块所需要的时间和实际花的时间，记录所有的缺陷，包括因为需求、设计或者实现引起的问题，记录修复时间，从开始发现问题直到缺陷被解决，提升到 PSP 0.1 策划不仅仅是估计所需时间，加入了规模的概念，本来 PSP 是用代码行数来估计规模，也可以使用简化功能点方式估计规模大小。

在 PSP0.1 的阶段还没有用规模做策划、估算，只是记录实际的规模，里面包括基本规模，就是在开发之前软件系统本来有多大。也记录删除、改变、增加、复用的规模数等。最后总的规模数应该是等于基本、新增、删除、复用相加。

PSP 1

下一步，PSP 1 就有规模计划的概念，跟依据 0.1 的规模定义一样，我们在做开发之前，先估计刚才的所有规模数，并用回归方程估算对应的工作量或者进度，包括偏差的范围。

在 PSP 0.1 增加了一个过程改进建议的环节，依据上一次迭代的数据，发掘下一轮可以完善的改进过程。也加入了编码标准，这个跟我们前面章节提到代码规范的概念相同。所以在 PSP0.1，除了计划，也需要有一个叫过程改进经验教训的部分，每一轮都应该有复盘回顾的概念，来提升个人的开发过程。

PSP1.1

PSP1.1 基于的 PSP1 的基础，加入挣值分析法去监控实际的项目进展，加入了 PV 和累加的 PV，计划多少和 EV 挣值和累计的挣值，反应实际完成多少。从那些系数也可以算出一个叫 CPI 成本性能指数，来反应完成与计划怎么比较，是否有延误，预计什么时候可以完成。详细可以用挣值分析法估计写书完成时间的实例。

PSP2

到 PSP2 就开始增加评审的概念，包括设计评审、代码评审。这些同行评审是很有效的方式，避免缺陷等到出事才暴露、出现问题，因为如果只依赖测试暴露缺陷的话，只是告诉你这个程序跑不通，你还是要看代码才知道问题在哪里，怎么修正？但评审就直接看代码，首先它可以让很多缺陷在评审就发现，不用等到测试才暴露问题。评审也可以找出一些不一定测得出来的问题，例如有些银行对系统质量要求很高，核心的算法不能有任何错误，他们都会要求核心模块必须走专家评审，因为他们也知道很多核心算法的问题不能单靠测试来发现和处理。也是这个原因，很多公司也会要求代码必须走静态扫描，利用扫描机器人评审代码，预先发现一些明显代码违规的语句问题，弥补单靠人手去评审的不足。

有了基础，再加入缺陷密度的概念，即缺陷数除以模块的规模大小，本来 PSP 是使用代码行的，我们也可以用功能点取代代码行，因为单看缺陷数无法比较，无论个人或者整个团队或者团队之间，所以必须除以规模大小，才可以把系数

归一，变成一个项目组之间，人之间可以比较的系数。也引入了排除率的概念，如果我们只是做了评审，但是都是 0 发现，你的缺陷排除率就是零了，一点效果都没有，所以排除率可以看成是当前评审发现的缺陷数除以当前系统内总共引入的缺陷数，比率越高越好。也有阶段或者过程的排除率，就是这个阶段开始时总共有多少缺陷，然后在这个阶段里我们找出多少，作为分子的一个比例。还有一个就是缺陷排除的速度，按每小时评审的时间找出多少缺陷来判断评审中，可以找出缺陷的速度有多快，评审的效率有多高。

评审质量是 PSP2.1 的概念，2.1 也增加质量成本（COQ）概念，与前面敏捷回顾篇里强调软件开发缺陷发现越晚，返工量越会以几何式上升的思路一致。

PSP3

接近敏捷迭代的思路，把程序分成不超过几百行的模块，然后把模块按优先级分到不同的迭代，但还需要有需求与设计。敏捷增加了精益的思路，因为需求可能会有变，所以可以暂时不做后面迭代的估算，只先针对当前的迭代去做策划、估算。敏捷里面的迭代回顾跟 PSP 的回顾如出一辙。

大家可以参考 PSP 书里面的教材和练习，比如在书中要求学员自己编写程序，自己记录时间。虽然现在开发语言统计方法跟 90 年代有差异，但原理还是共通的。如果对新员工难以安排学校式的一周课程，也可以考虑用自学的形式，按要求去记录学习、写报告。

学 PSP，培养好习惯所以编程人员可以用 PSP 的原则，要他们记录。因为现在是写程序，不是玩游戏，所以更贴近实际。也要求后面的程序，让他分析自己的返工成本、质量成本、缺陷排除等，让后面项目团队回顾时有真实数据做分析。

例如在新员工培训时，练习中要求他们有编码规范的概念，按照公司的规范参考来写程序，在每一次做开发之前，都先简单计划一下时间和预估缺陷数。开发完后，记录实际的时间与缺陷数，因为虽然不仅仅是写一个程序，下一次开发时，可以参考以往练习的数据来更好估计，会看到自己的估计越来越准。

使用挣值分析监控案例

今天 11 月 8 日，离年底要交付全部初稿的时间不到 2 个月，书的内容也完成了超过一半，但剩下的时间因为年底密密麻麻很多评估，而且评估就是早上 9 点开始到下午 5、6 点结束，只有中间一些空档时间和周末可以自由安排，所以我就用同样的思路来策划估算是否能在年底完成要求。第一步算出现在到年底可以用在写书的时间，按每周估计：

	计划工时	累积工时
	Plan Hrs	Cum. Hrs
22.11.7	4	4
22.11.14	24	28
22.11.21	38	66
22.11.28	16	82
22.12.5	21	103
22.12.12	38	141
22.12.19	26	167
22.12.26	21	188
23.1.2	38	226
23.1.9	14	240

表 5-3

第二步从历史的数据估算剩下的章节，每个章节需要多少小时，写出计划的完成顺序，然后依据原计划可以使用的时间，按最佳使用算出每个章节可以在哪周完成，写在表格的右面空白处，从而估算何时可以完成所有章节，如果确实不能在 12 月底完成，就可能要重新调整、策划，是否有一些章节优先级比较低，暂时不放在第一版。

因觉得单点估算很难定一个点数，便用三点估算法。每一行估计都有三点：最差、最佳、最可能，用 PERT 方程式估计预计 (Expected) 和标准差 (Sigma)。挣值分析法所有东西 - EV、PV、AC 都是用钱来结算（这个例子我们就是用所花工时），例如我们估计了每个章节所需要的工时，利用三点估算计算出每个章节预计的工时，加起来得出预计总工时是 80.33 工时（用 PERT 公式计算）。例如第一个章节 TDD，预期工时是 2.5，除以 80.33 就得出 0.031，我们就用 3.1

作为 TDD 的 PV（所有任务完成的总 PV 大概是 100）。同样方式算出 MGR 章节的 PV 是 6.6。得出每一个章节对应的 PV 后，我们就可以计算累计的 PV 数：

Task 章节	最佳	最可能	最差	Expected	Sigma		PV	Cum. PV
TDD	2	2.5	3	2.50	0.17	0.03	3.1	3.1
MGR领导的任务	4	5	8	5.33	0.67	0.07	6.6	9.7
PSP	3	4	5	4.00	0.33	0.05	5	14.7
M6	4	5	6	5.00	0.33	0.06	6.2	20.9
M5	4	5	8	5.33	0.67	0.07	6.6	27.5
IC	4	5	6	5.00	0.33	0.06	6.2	33.7
CM	8	9	12	9.33	0.67	0.12	11.6	45.3
SiFP upd	3	4	5	4.00	0.33	0.05	5	50.3
INNOVATE创新	4	5	8	5.33	0.67	0.07	6.6	56.9
MitOCW写Python	2	2.5	3	2.50	0.17	0.03	3.1	60
克服拖延症	2	2.5	3	2.50	0.17	0.03	3.1	63.1
CI	6	8	12	8.33	1.00	0.10	10.4	73.5
RDM	6	8	12	8.33	1.00	0.10	10.4	83.9
Opening开章	3	3.5	4	3.50	0.17	0.04	4.4	88.3
最终检查	8	9	12	9.33	0.67	0.12	11.6	99.9
								99.9
		70.5	Sum_Exp	80.33		1.00		

表 5-4

估计哪一周可以完成哪些章节，得出总体的进度计划。例如第三周累积有 66 工时，应可完成总共 12(=1+4+7) 个章节，因用 PERT 三点估算，累积工时 95% 范围是（55.8 ~ 62.5）。

	章节	95%区间（工时）		计划工时 Plan Hrs	累计工时 Cum. Hrs	策划价值 PV	累计策划价值 Cum. PV	计划哪一周 Plan Wk
		下限	上限					
22.11.7	TDD			4	4	3.1	3.1	1
22.11.14	MGR领导的任务			24	28	6.6	9.7	2
	PSP					5	14.7	2
	M6					6.2	20.9	2
	M5	20.0	24.3			6.6	27.5	2
22.11.21				38	66			
	IC					6.2	33.7	3
	CM					11.6	45.3	3
	SiFP					5	50.3	3
	INNOVATE创新					6.6	56.9	3
	MitOCW写Python					3.1	60	3
	克服拖延症	48.1	53.6			3.1	63.1	3
	CI	55.8	62.5			10.4	73.5	3

表 5-5

监控

按实际完成的情况就可以填 EV 挣值，挣值的算法很简单，必须整个章节完成才算有挣值，不接受部分完成。比如第一周虽然 TDD 章节已完成九成，但实际是到第二周开始才完成，所以第一周的挣值为 0。到了第二周才把本来的第一章完成，所以里面才放上它的 PV(=3.1)。监控的好处，好比要准备半年后的马拉松，开始长距离慢跑（LSD）锻炼，再进一步做连续马拉松配速训练，每周 3 天，每次 5-15 公里，速度是多少？比如平均每公里花费 6.25 分钟。有了数字才知道现状与标准的差距，才有动力对自己说现在离比赛不到 12 周，必须加强锻炼才能赶上。

要让项目有更大的机会按期完成，便需要监控，这样可以预计自己离最后交付差距多少，如果按现在的进展要到哪一周才可以完成，都可以从数字预计出来。但是我们写书也靠灵感，跟写程序类似，可能顺序不一定能按本来的执行，怎么办？

其实道理也一样，你可以按新的顺序更新一下本来的计划，把实际的填上。如果你是用简单的 Excel 表，这个改变可能只花几分钟，很快就能调整好顺序。例如现在看到的第二章节，本来计划放在后面才写，但因刚好与客户接触后产生了灵感，便放在前面写了。

	章节	计划工时 Plan Hrs	累计工时 Cum. Hrs	策划价值 PV	累计策划价值 Cum. PV	计划哪一周 Plan Wk	实际工时 Actual Hr.	实际哪一周 Actual Wk.	挣值 EV	累计挣值 Cum. EV
22.11.7	TDD	4	4	3.1	3.1	1	2	2		
22.11.14	MGR领导的任务	24	28	6.6	9.7	2	7		3.1	3.1
	PSP			5	14.7	2	5.5			
	M6			6.2	20.9	2				
	M5			6.6	27.5	2				
22.11.21		38	66							
	IC			6.2	33.7	3				
	CM			11.6	45.3	3				
	SiFP			5	50.3	3				
	INNOVATE创新			6.6	56.9	3				
	MitOCW写Python			3.1	60	3				
	克服拖延症			3.1	63.1	3				
	CI			10.4	73.5	3				
22.11.28		16	82							
22.12.5		21	103							

表 5-6

要估算任务工时便要依赖以往类似活动的历史记录，如果不是每天记录，过后无法记住。每周依据个人的习惯，每天记录实际所花小时数，然后统计每周累计数。

参考 References

- 1.Humphrey, Watts S. "A Discipline for Software Engineering."
- 2.Humphrey, Watts S. "PSP: A Self-Improvement Process for Software Engineers."

Chapter 6

估算软件规模

为什么要估算规模

规模可以帮我们：

1. 依据历史数据策划，例如估算工作量、工期
2. 归一 (Normalize) 不同项目作比较
3. 知道现在水平

依据历史数据策划先把项目分成组件，参考以往类似的组件所花工作量，估算整个项目的总工作量。规模大小可简单看成是组件的数量。如果是新开发，以前从未做过同类开发，就只能靠个人经验直接估算工作量或工期，但是如果以前做过类似的工作，就可以参考以前的历史数据估算。

规模可以帮我们把不同项目归一。例如验收测试缺陷数无法比较，但缺陷率（缺陷数/规模）便可以比较；生产率（规模/所花总工时）便可比。

有了归一后可比较的系数，个人/团队便更清楚当前的水平（质量或生产率）是否在上升或下降。

为什么不应用代码行数 (LOC)

在 1996 年前，IBM 一直使用代码行数估算规模。之前一直都使用近似机器语言的 Assembly Lang，但为了提升效率，引入了高层语言 PL/S，发现不能再代码行数来估算规模，因 PL/S 只需要更少代码行数也能完成同样功能：

	Assembly	PL/S
代码行数	17,500	5,000
工作量（月）	30	12.5
工作量（工时）	3,960	1,650
每月代码行数	583.33	400.00
等同的 Assembly 代码行数	17,500	17,500
等同的 Assembly 人月	583.33	1,400.00
功能点	100.00	100.00
功能点/人月	3.33	8.00

表 6-1

上表是 IBM（1968 ~ 1975）对两种编译器的统计：

每个月 PL/S 产出代码行数 (400) 反比 Assembly(583) 少，但是如果用功能点数便能真正反映生产率的提升：

PL/S:8.00 对比 Assembly:3.33

只适用于编码

代码行数只能反应编程的工作量，但编码仅占项目总工作量的一部分，如果把项目按工作量分成以下 5 个部分，代码行数只能用于第二部分编码 (25%)，其他 4 部分 (30% + 20% + 15% + 10%) 都不合适。

	活动	成本 (%)
1	发现与修正缺陷	30
2	编码	25
3	支持类文档	20
4	会议沟通	15
5	项目管理	10
	总计	100

表 6-2

与功能点比较

能估算好项目规模可帮助我们更好估算工作量，规模应具备以下条件：

1. 要和工作量密切相关
2. 要容易数得出来
3. 容易在项目早期可以估算到

功能点比较符合以上条件，只要功能需求明确，就可以估算出对应的功能点数。因功能点估算已经是国际标准，基于功能点的度量数据可以与其他国家的标杆对比。

怎样估算简化功能点 (SiFP)

有了功能性需求，并识别出系统范围，就可以开始估算功能点。

1. 数据功能（简称实体）的数量 $\times 7$ （实体是系统要管理的数据）
2. 业务功能（简称行为）的数量 $\times 4.6$ （行为可以简单看成是增删改查等功能）

简化功能点数 = 上面 1 和 2 的总数

例如，附件例子案例 A 新开发项目：

- 识别出 2 个实体和 6 个行为，得出简化功能点数：

$$2 \times 7 + 6 \times 4.6$$

$$= 41.6$$

- 案例 B 功能优化维护项目：增加 1 个实体和 3 个行为，使用功能优化与维护公式：

$$1 \times 7 + 3 \times 4.6 \quad (\text{增加 1 个实体和 3 个行为}) \quad \text{因可能有些功能删除，有些功能变更，先} \\ = 20.8$$

- 以上优化维护功能点数主要是用于估算本期开发项目的工作量，因为无论变更功能或者删除功能，都会导致有开发工作量，所以不能是零或负数。国际功能点的规定很简单，都加起来。

- 然后再用应用 (Application) 公式计算优化维护后系统的功能点数：

$$41.6 + 20.8 \\ = 62.4$$

- 因案例 B 没有功能删除，也没有功能变更，优化后的功能点数是功能点数只需要首次开放功能点加上优化维护的功能点数 (41.6 + 20.8)，但如果案例 B 有功能变更，那些功能点便不能加到最后的应用功能点数。(如果有功能点删除，便需要减。)

从故事点转简化功能点

因故事点只是每个团队自己定义，无法用来作为组织级标准衡量规模，所以很多公司（尤其是银行）转用功能点来衡量软件开发规模。它不仅可用于公司内，也适用于行业标杆，功能点也适用于公司内或公司间结算，例如软件维护期开发工作量变化很大，也难以事前预估，所以有些银行会按最终开发出来的功能点数结算使用部门付开发部的费用，减少争议。

虽然功能点分析源自 70 年代，但由于计算较复杂，一直未普及。有些人觉得国际功能点协会 (IFPUG) 的功能点算法 (FPA) 太复杂。针对这问题，国际功能点协会简化本来的功能点算法，推出简化功能点 (SiFP)，减少了估算的工作量与学习难度 (例如培训可以从以往的 2 天半降到半天)。因简化功能点不考虑实体/行为的复杂度，它与本来功能点的估算有 $\pm 15\% \sim 20\%$ 偏差 (详见附件)。因敏捷团队是按每轮迭代估算 (而不是一次性估整个项目)，偏差就可以接受 (例如 2 周一个迭代，偏差大概 1.5 天)，所以 SiFP 能适用于敏捷多次迭代估算。所以越来越多团队开始从故事点转简化功能点，但由于不熟悉功能点估算是从用户角度估算，而非从开发工程师的视角，团队初次估算 SiFP 通常有误。下面是某团队从故事点转简化功能点的案例：

客户：我们以前一直使用故事点，为了更好做量化管理，我们新的项目开始使用简化功能点 SiFP。

我：好的，看一下你的估算表，为什么这两个功能要分成两个行为？

客户：这两个功能都挺复杂，估计需要很多开发工作量。我们以前用故事点估算时，也会分成两个故事点。

我：请注意，在功能点估算是否是一个行为，取决于它算不算是基本过程。如果俩行为互相依赖，不能单独作为基本过程，就不应该分开为 2 个功能，不然便可能不断细化，无法得出公认的功能点数。很多团队刚开始用功能点，与你们一样，没有弄清楚基本过程的概念，还是从工程师的角度，估计开发时间的工作量来判断是一个功能还是两个功能？在故事点用这种方式估算习惯，但因为功能点是要同一套功能需求，不应有功能点数估算差异，所以不能用估计开发难易程度来判断。

客户：可否举个实例？

我：以我们日常的网约车为例，是否可以分成以下 8 个“行为”？

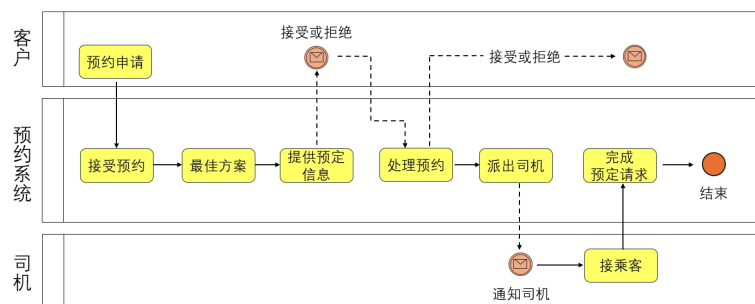


图 6-1

1. 预约申请
2. 接收预约
3. 选择最佳方案
4. 提供预约信息
5. 处理预约
6. 分派司机
7. 接乘客
8. 完成预约请求

客户：是的，每个行为都要花一定开发工作量。

我：其实按 SiFP 只有 4 个行为，因开头 4 个和最后 2 个都要合并才正确（详见附件），因如果要能独立成行为，必须符合基本过程条件。

基本过程不是随便定的，它有规则，比如从网约车案例我们可以看到，“预约申请”本身不能算一个基本过程，虽然预约申请可能要很复杂的开发工作，但是因为它不能独立存在，必须依赖其他的行为才算完整。

例如某公司财务系统有打印支票功能（用来付供应商），你觉得打印支票本身是否算基本过程？

客户：听完你网约车例子，应该不算。因打印支票只是付款流程的一部分。

我：正确，好比银行账户转账，更新账户余额和展示账户余额这 3 个用例不能单独，必须合起来才算完整。算实体也应使用同样概念。例如某人事管理系统，除了管理职工信息外，还有员工家属信息，因家属信息必须关连到员工信息，不能独立存在，所以家属信息本身不算是一个实体。你是否觉得你这表里的某些

实体应合并？

客户：听完你的解释，我们这次计算里确实有些实体应合并。

但如果客户需要管理员工家属信息，我们在建数据库时确实要增加额外表格管理相关的字段，会增加不小工作量，所以如果和只需要管理员工信息，同样是算一个实体，同样的功能点数，是否不合理？

我：很理解你的疑问，也正是很多工程师功能点的困惑。你在之前不是说过在估算开发工作量时常常与客户谈不拢，客户常常觉得你们估算的工作量过大，应不需要这么多。所以功能点估算的目的是希望从客户的角度，依据功能需求，按已定的规则，让任何人都能算出同样的功能点数（不仅依赖工程师估算）。所以有些工程师学功能点时，像你一样未能抛开软件开发工作量的想法，不觉得应只按功能点的规则来计算。

客户：理解你的意思，但你刚才这个例子确实有额外的工作量，是否会导致会低估了项目的工作量？

我：预算工作量除了考虑功能点也要看团队的历史生产率数据。如果以往项目里，很多实体都比较复杂，都包含几个相关的数据表，反映在项目的生产率历史数据里，作为新项目工作量估算的参考。

我：你们的功能点估算表没有明确区分新的迭代怎么处理变更与删除。

客户：我们表中有计算每次迭代的总功能点数。

我：你们可能有计算每迭代的功能点，但难以看清后面迭代对应之前的变化。优化与维护 (Enhancement) 功能点是用来估算本迭代的工作量，而应用 (Application) 功能点就是迭代后产品的功能点数。而且应该用表格形式把变更和删除累加在原本上一轮的功能上面，才可以更好看到迭代与迭代之间功能上的变化。

所以不要误以为可以像之前故事点估算，按个人理解写上各种不同复杂程度的故事点例子便可。国际功能点手册里包含很多计算实例，来解释功能点计算，才能确保对同一套功能需求，每人都能估算出同样的功能点数。我刚刚的例子全都可以从国际功能点手册里找到，你们不需要自己研究再‘发明’公式。学功能点估算与写程序类似，必须多动手试，例如按我发你们的 6 个案例，按案例的功能需求，计算 SiFP 功能点数。

（大家也可以看附件里有 2 个简单案例：

1. 首次开发：管理潜在客户（公司和联系人）信息
2. 优化功能与维护：首次开发后，添加帮助 (Help) 页功能，和添加报表

)

总结

虽然简化功能点 (SiFP) 比传统国际功能点 (IFPUG) 简单易学，但开发人员还是容易用工程师的视角来估算（本应用用户的视角），导致计算错误。所以要多看案例并做练习，才能把握（能参加培训会更好）。

附件

简化功能点 (SiFP) 简介

它是做什么的？

应用软件开发的需求可分成 3 类：

- 1. 功能性需求
- 2. 技术需求
- 3. 质量需求

第二类和第三类归为非功能性需求。功能点主要是针对功能性需求，目的是提供对客户有意义的功能点数，来客观地衡量软件规模。

该如何去做？

简化功能点 (SiFP) 主要分为两类度量：

- 1. 数据功能——实体 (逻辑文件 Logical File)
- 2. 交易功能——行为 (基本过程 Elementary Process)

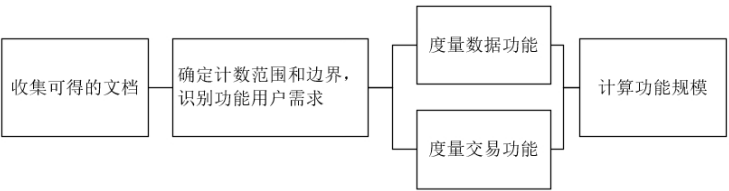


图 6-2

简化功能点估算步骤：

- 1. 确定功能点分析类型
- 2. 识别分析范围和应用边界
- 3. 计算数据类型功能点
- 4. 计算交易类型功能点
- 5. 计算功能点

1 确定功能点分析类型

3 种计算类型：

- 1. 开发 (Development)
- 2. 优化功能与维护 (Functional Enhancement Maintenance FEM)
- 3. 应用 (Application)

开发 (Development) 功能点数包括首次应用开发的功能点，加上数据迁移或转换的功能点。

优化功能与维护 (FEM) 计算对现有应用程序的优化与维护，包括增加、删除和变更功能，也加上数据迁移或转换的功能点。

应用 (Application) 计算已经安装应用软件，例如优化与维护后的软件，提供给用户的功能点数，

2 识别分析范围和应用边界

IFPUG 包括两类数据功能点（因 SiFP 是基于 IFPUG 功能点）：

- 内部逻辑文件 (ILF)
- 外部接口文件 (EIF)

和 3 类交易类功能点：

- 外部输入 (EI)
- 外部输出 (EO)
- 外部查询 (EQ)

以下例子表示数据类和交易类功能点与应用软件系统（参考案例 A）的关系（虚线标示系统边界）：

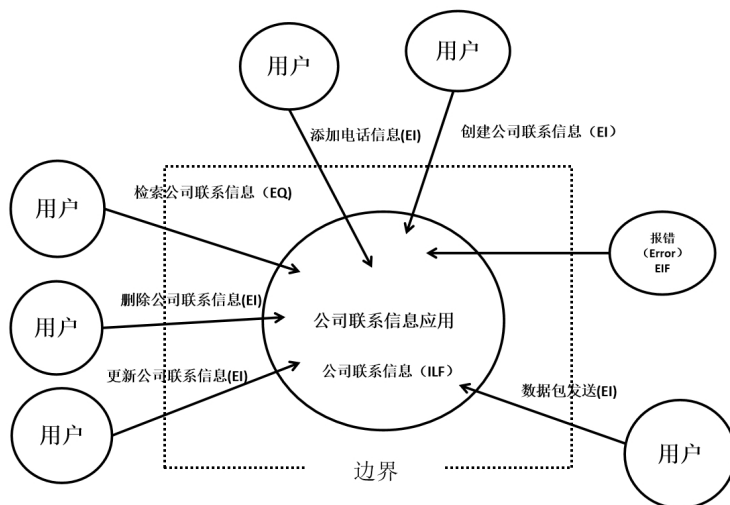


图 6-3

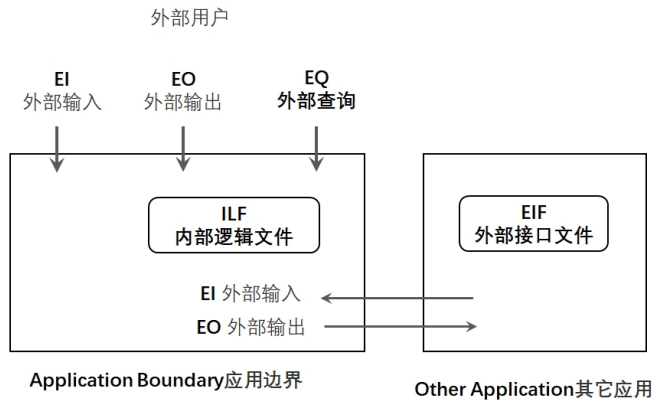


图 6-4

计数范围由计数目的决定。它标识将被划分大小的系统、应用程序或应用程序的子集。它可以包括通过购买软件包来满足的功能；它可以包括所有将被外包的应用程序；它可以被限制为应用程序中执行特定目的的函数，例如报表。应用程序边界是被测量的应用程序与外部应用程序或用户域之间的边界。

IFPUG 制定了确定边界的具体规则：

- 边界是基于用户的观点用户应该能够用用户的语言定义应用程序的范围和业务功能
- 相关应用程序之间的界限是基于单独的业务功能，而不是基于技术考虑举例，微软办公软件目前包括 Word、Excel、PowerPoint 和 Access；每个都是 Microsoft office 套件中的独立应用程序
- 被增强的应用程序的应用程序边界保持原样当然，除了添加的功能可能会扩展边界，删除的功能可能会缩小边界。功能优化不会成为它自己的应用边界，而是按计数应用程序的领域定义（如微服务）。如开发项目和优化项目可能包括不止一个应用程序，功能点计数时应参考上图例子先画好边界，然后独立计算每个应用程序的功能点数

有关应用程序中制定 3 边界定义的混淆。

虽然功能点计算的一致性非常重要，但如果计算一致但不正确，功能点计算几乎没有什么好处。例如，在确定应用程序边界时。边界不应该在软件程序或软件模块级别建立。功能点似乎只在更高的应用程序级别上才相关。

- 但也可能在过高的层次上进行计数，例如，某会计系统可能包含几个独特的和独立的应用程序（例如，应收帐款、总账和应付帐款），它们可能具有内部关系。这些应用程序应该分别计算
- 反过来，某工厂生产系统可以由许多从属但独立的应用程序（例如，车间计划，物料库存，工作计划）组成，这些应用程序分别开发，维护和使用，并且可能具有内部关系

3 计算数据类型功能点

- 先计算逻辑文件数
- 关于计算规则，详见“逻辑文件”
- 按对应公式计算功能点数:(每个逻辑文件 = 7.0 简化功能点)

4 计算交易类型功能点

- 计算基本过程数
- 关于计算规则，详见“基本过程”
- 按对应公式计算功能点数:(每个逻辑文件 = 7.0 简化功能点; 每个基本过程 = 4.6 简化功能点)

5 计算功能点

- 功能点数 = 数据类型功能点数 + 交易类型功能点数

Development Project Function Point Count 开发项目功能点计数

开发项目功能点计数由两个功能点数组成:

1. 应用程序的功能点数，由 EI、EO、EQ、ILF 和 EIF 组成
2. 转换功能，通过软件将以前的数据转移到新的 ILF 中（该组件通常包括旧数据文件的输入作为 EI 计数或将数据输入到已计数的新 ILF 中，可能还包括转换报告的 EO）

$$DFP = UFP + CFP$$

- DFP 是开发项目功能点计数。
- UFP 为功能点计数。
- CFP 是数据转换所包含的功能点的数目。（注 1）

Enhancement Project Function Point Count 优化维护项目功能点计数

功能点计数也由两个功能数组成，但它们有些不同:

1. 应用程序未调整的功能点计数，由 EL、EO、EQ、ILF 和 ELF 组成，它们是:
* 由增强项目增加 (以前不存在的功能，例如：一个新的 EQ，三个新的 EL，一个新的 ILE，一个新的 EQ 和一个新的 EO)

- 由增强项目更改（以前存在的功能，但现在有不同的字段，FTR 或需要不同的处理）
- 由增强项目删除 (已从应用程序中删除的功能，例如：已删除的报告)

2. 转换功能，通过软件将以前的数据转换为新的 ILF（该组件通常包括旧数据文件的输入 [作为 EL 计数或将数据输入到已计数的 ILF 中]，可能还包括转换报告的 EO）

增强项目功能点计算公式如下:

$$EFP = (ADD + CHGA + CFP) + DEL$$

- EFP 是增强项目功能点计数
- ADD 是增强项目增加的功能的未调整功能点数
- CHGA 是经增强项目修改的功能的未调整的功能点计数
- CFP 是数据转换所包含的功能点的数目（注 1）
- DEL 是那些在增强项目中被删除的功能的未调整功能点计数

（注 1：这简介不包含数据转换的计算与实例。）

Application Function Point Count 应用功能点计数

应用程序功能点计数由一个功能组件组成（转换工作功能不包括在应用程序计数中，因为它是开发工作的一部分，而不是应用程序一旦建立）：

1. 应用程序的功能点计数，包括 Els, EOs EQs, ILFs, EIFs

有两种不同的时间段估算应用程序的功能点数：

1. 当应用程序最初交付
2. 当增强项目改变了应用程序的功能值时（当安装增强项目时，必须更新以前的应用程序功能点计数，以识别对应用程序的修改）。增强项目可以修改应用程序

下面是两种不同的计算公式：

Initial Application Function Point Calculation 初始应用功能点计算

$AFP = ADD$

- AFP 是初始应用程序功能点计数。
- ADD 是开发项目所安装功能的未调整功能点数。

Application Function Point Calculation after an Enhancement 优化与维护后的应用功能点计算

- 增加（新）功能，增加应用程序的功能点大小
- 变更，通过增加或减少本来开发功能点，但不影响应用程序的功能点数，只影响了优化维护功能点数
- 删除功能，会减少应用程序的功能点，但反而会增加优化维护 (Enhancement) 功能点。

$AFP = (UFPB + ADD + CHGA) - (CHGB + DEL)$

- AFP 是应用程序调整后的功能点计数
- UFPB 是增强项目前应用程序的功能点数
- ADD 是增强项目增加的功能的未调整功能点数
- CHGA 是经增强项目更改的功能的未调整功能点数（此部分反映更改后的功能点值）
- CHGB 是改进前未调整的功能点数（此分量反映改进前的功能点值）
- DEL 是那些被增强项目删除的功能的未调整功能点计数

逻辑文件 (Logical File)

下面在计算实例里简称“实体”以方便理解

- 逻辑文件满足内部和外部数据存储要求的功能。它是用户可识别的逻辑相关数据组或控制信息组，
- 2 类逻辑文件：
 - 内部逻辑文件 (ILF)：在度量应用程序的边界内维护和/或引用。是在被度量应用边界内部维护的
 - 外部接口文件 (EIF)：被度量应用程序所引用，但在另一应用边界内维护

术语定义：

- 用户可识别 (**User Identifiable**)：对过程和/或数据组的需求，得到用户和软件开发人员双方的认同和理解，并达成一致。例如，财务应用程序可能需要一个支票账户记录
- 逻辑相关 (**Logically related**)：要求每一组与相关描述逻辑上一致。逻辑文件不应为了维持其存在而依赖或归因于另一个逻辑文件，这些相关逻辑文件应合并（特别是因业绩或执行原因而建立的组）。例如，地址表可能属于更高级别，如客户文件、账单文件、库存位置文件或员工文件
- 数据 (**Data**)：应用程序中维护的描述和/或数据组。例如，每张支票的支票号码、金额、日期、收款人、备忘录记录和账户号码都可能在支票账户记录中维护
- 控制信息 (**Control information**)：应用程序里影响基本过程的数据。它定义处理那些数据 (what)、何时处理数据 (when) 或如何处理数据 (how)。对于 ILF，这些数据、规则或参数在应用程序中存储和维护。举例，在恒温器上设置温 I 度以控制加热或空调的时序
- 维护 (**Maintained**)：通过应用程序的基本过程修改数据。数据或控制数据可以通过诸如添加、更改、删除等。逻辑文件可以被多个应用程序维护，但在每个应用程序只能计算为一个 ILF
- 基本过程 (**Elementary process**)：具有意义的最小活动单位。它可以更新多个逻辑文件

注意:

逻辑文件包括两类不同的用户需求数据:

1. 功能性数据
2. 非功能性数据

功能性数据是用来满足用户功能需求的数据。例如销售、银行账号、供应商、人员等信息。

非功能性数据主要是为了满足易用性（如支撑下拉菜单所需的数据，可输入数据的上下范围等），或性能（如用于查询数据的索引），或可维护性（配置参数）。

只有第一类功能性数据才算是逻辑文件。

功能点估算只包括功能性需求，国际功能点 (IFPUG) 有 SNAP 计数手册，专门针对非功能需求。

基本过程 (Elementary Process EP)

基本过程是对用户有意义的最小活动单元。例如: 添加员工的用户需求包括建立工资和家属信息。只有添加所有员工信息, 才能创建员工信息记录。单独添加一些信息使添加员工业务处于不持续状态, 只有员工工资和家属信息都添加后, 这个活动单元才能完成且业务处于稳定状态。

识别基本过程

为了识别基本过程, 需要执行以下活动, 把功能用户需求分解为最小活动单元, 使其满足下面条件:

- 对用户有意义
例如功能用户需求要求在教育中添加新员工的能力
- 构成一个完整的事务
例如: 用户定义的员工信息包括工资和家属信息。如果家属人数大于零, 添加员工信息时必须包括家属信息。本例中, 添加员工信息 (不包括添加地址、工资和家属信息) 不满足本规则
- 自包含
例如: 除非输入所有的必需信息并且完成所有处理步骤, 如验证、计算、更新 ILFs, 添加过程才是自包含的
- 让应用程序的业务保持持续状态
例如添加员工的用户需求包括建立工资和家属信息。只有添加所有员工信息, 才能创建员工信息记录。单独添加一些信息使添加员工业务处于不持续状态, 只有员工工资和家属信息都添加后, 这个活动单元才能完成且业务处于持续状态

识别活动单元为基本过程需要满足以上所有规则。

识别基本过程主要目的

基本过程的主要目的可识别为下列情形的一种:

- 改变应用行为
- 维护一个或多个 ILFs
- 呈现信息给用户

实例：识别基本过程 (EP)

下面是预约网约车过程：

活动名称	描述
预约申请	客户发出预约请求到预约系统。预约信息包括：日期、时间、上车的位置和目的地位置。
接收预约	预约系统收集客户预约请求，并把预约数据记录在数据库。
检查是否有可用的车/司机	预约系统从预约数据库中查看是否有合适的车/司机，如果能找到合适的时间、日期，能配合预约请求的话，看看司机是否有空档。如能找到，在系统更新内部状态为是，否则为否。
寻找其他可选	如果状态是设有，预约系统继续在数据库搜索有没有接近的预期时间，否则写没有合适请求的出发地点和目的地。
提供预约信息	预约系统自动发出通知到客户，是否有合适空档或者提供接近的日期时间。
处理预约	客户回复预约系统接受或拒绝，预约系统把反馈记录在预约数据库。如果反馈是接受，预约系统会继续统计客户详细预约信息。如果客户拒绝，预约系统就会回复收到，并终止过程。
分派司机	如果客户接受，预约系统会指派司机到已确认的日期、时间、上车地点和目的地。预约系统把这个记录在数据库中，并发送信息通知相关司机。
接乘客	司机按约好的日期时间，上车的地点接乘客，然后发信息到预约系统，通知乘客已经上车。
完成预约请求	预约系统在数据库中记录已经完成整个过程。

表 6-3

分析能否满足所有 **EP** 识别规则，判断能否独立成为基本过程 (**EP**)，部分例子

预约申请

1. 是否对用户有意义，是客户功能需求的一部分。是
2. 是否构成一个完整的事务？否：预约申请本身不是一个完整的交易，因为过程必须也包括预约请求信息，收到其他可选的档期这些步骤，都不可以分离
3. 是否自包含，可以独立存在？否：例如接受预约申请，查看是否有档期，查看有没有其他接近的档期等，都是一些必需的相关步骤去完成这个基本过程
4. 是否让应用程序达到稳定状态？否：整个业务需求只能在收到预约信息，发送、接受、处理、反馈给客户才算是完成稳定状态

分派司机

1. 是否对用户有意义，是客户功能需求的一部分。是
2. 是否构成一个完整的事务？是：分配到司机是一个完整的交易，包括收到司机的确认，把信息记录在系统中并通知司机
3. 是否自包含，可以独立存在？是：分配到司机本身可以独立存在
4. 是否让应用程序达到稳定状态？是：因为当司机被分配后，是完全满足业务的需要

接乘客

1. 是否是客户功能需求？是
2. 交易是否完整？否：接乘客本身不算一个完整的交易，因为预约系统必须也记录这个信息
3. 是否自包含，可以独立存在？否：确认预约申请是下面一个必须执行的过程，来完成这个基本过程
4. 是否让应用程序达到稳定状态？否：整个业务需求只能在预约系统确认沟通，已经接到乘客，然后系统也把记录更新到预约系统才算完成

业务过程/活动	基本过程 (EP) 必须满足所有EP识别规则
• 预约申请 • 接收预约 • 检查是否有可用的车/司机 • 寻找其他可选 • 提供预约信息	如果单独来看每个活动，不能满足基本过程的条件。它们必须要结合在一起，才能满足所有基本过程的条件。基本过程包括左面所有过程/活动。
• 处理预约	这业务流程、活动满足所有基本过程的要求，可以当成一个基本过程。
• 分派司机	这业务流程、活动满足所有基本过程的要求，可以当成一个基本过程。
• 接乘客 • 完成预约请求	如果单独来看每个活动，不能满足基本过程的条件。它们必须要结合在一起，才能满足所有基本过程的条件。基本过程包括左面所有过程/活动。

表 6-4

案例 A: 开发 (Development) 项目

公司计划构建应用程序来维护对其功能点课程感兴趣的公司信息。

要维护的公司联系信息，按逻辑分组，包括以下数据：

- 公司
- 联系人姓名
- 职位
- 初次联系日期
- 街道地址
- 城市
- 状态
- 邮政编码
- 电话号码
- 传真号码

该数据最初将在收到查询表示对课程感兴趣时创建。员工将能够通过屏幕，使用以下命令创建、更新和删除信息。创建和更新将维护以上所有的 10 个字段。删除只需要输入公司和联系人的姓名。

下面是公司联系数据中包含的其他字段，但要用另外单独更新：

- 发送材料日期
- 电话联系日期
- 笔记、备注

这些字段将由两个独立的基本流程维护：

- (1) 发送信息包时，发送信息包的个人将使用一个单独的屏幕，使用功能键输入公司、联系人姓名和发送数据包的日期。
- (2) 邮件寄出后两周内，回电确认收到并答复问题。合同完成后，来电者将使用屏幕，使用功能键输入联系人的公司名称、电话联系日期和备注。电话联系人的日期将用作辅助键（第二种记录类型），以保存多次出现的信息并更新公司联系人数据。

由菜单驱动的用户界面导航，提供以下六个功能：

- 创建公司联系人
- 检索公司联系
- 更新公司联系

- 删除公司联系
- 已发送的材料包
- 已完成的电话联络

除了检索公司联系之外，其他所有功能都已经描述。输入公司、联系人姓名和功能键后，检索功能便显示公司联系数据中维护的所有字段。

任何功能都可以从外部维护的错误（Error）文件返回错误信息，该文件有四个字段，其中一个字段包含具体错误信息。

答案与解读

共个实体：

1. 公司联系信息
2. 报错文件

你可能会问：那些电话信息是否也应该是一个实体？（因需要花工夫开发）这不应该是一个实体。因为电话信息必须依赖公司联系信息挂在一起，不可以独立存在，就好比我们要电话信息，假如也要维公司联系信息，这个电话信息就不能算另外一个实体，因为没有公司联系信息，电话信息是不能单独存在的。区分原则不是根据是否要产生开发的工作量，而是从用户角度看，这个实体能否独立存在和维护。否则功能点的估算就只是根据个人对开发工作量的估计，而不是从用户角度看功能的客观判断。

创建、检索、更新、删除等 4 个功能和管理发送资料包和电话联系 2 功能，共 6 行为，加 2 实体。为方便理解，建议先画下图，把行为与实体关联：

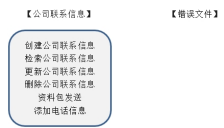


图 6-5

所以按简化功能点每个实体 ×7，每行为 ×4.6 得出 41.6(=2×7+6×4.6) 简化功能点，详见下面列表：

功能点名称	功能点类型	功能点	功能点值	功能点权重	功能点值	功能点值
创建公司联系信息	数据	1	1.000	1	1.000	1.000
检索公司联系信息	数据	1	1.000	1	1.000	1.000
更新公司联系信息	数据	1	1.000	1	1.000	1.000
删除公司联系信息	数据	1	1.000	1	1.000	1.000
资料包发送	数据	1	1.000	1	1.000	1.000
添加电话信息	数据	1	1.000	1	1.000	1.000
总计		6				6.000

表 6-5

案例 B: 优化维护（FEM）项目

应用程序交付后，Caren Garmus 希望利用已经由外部应用程序维护的帮助子系统来提供两类帮助页面：

1. 字段级

2. 屏幕级

每个屏幕上都可用这两个级别的帮助功能，并且将从单独维护的外部帮助文件访问帮助文本。

在所有情况下，Help 都少于 5 个字段。

Mary Herron 说她想要一份每周的逾期联系报告（详见下表），识别出：

- 那些联系没有收到资料包
- 那些在邮寄后两周内没有接到电话联系

该报告将每周从公司联系数据文件中检索数据，不需要更改公司联系数据文件。

日期： xxxx.xx.xx 页码： xx
从 xxxx.xx.xx：
公司
联系人
首次联系日期
数据包发送 xxxx.xx.xx
计划电话联系日期 xxxx.xx.xx
过期总数

表 6-6

答案与解读

另外加了一个要管理的实体：

1. 帮助文件

也可参考 IFPUG 关于 ILF/EIF (实体) 的识别要求, 必须符合以下条件:

- 1. 数据的集合必须是逻辑相关的并且是用户可以识别
- 2. 这些数据或者控制信息必须是在本应用的边界内被维护

总结:

- 实体方面增加了 1 个实体,
- 行为增加 4 个行为

所以动态功能点是增加的功能点 $25.4 (=1 \times 7 + 4 \times 4.6)$

数据类功能	Data Type	类型	TYPE	SiFP	MULT	操作类型	SiFP 合计
公司联系信息		实体	ILF	7	1		
错误文件		实体	EIF	7	1		
帮助文件		实体	EIF	7	1	ADD	7
交易类功能 Transactional functions							
Company contact data							
查公司联系信息		行为	EI	4.6	1		
查公司联系信息		行为	EQ	4.6	1		
新公司联系信息		行为	EI	4.6	1		
查公司联系信息		行为	EI	4.6	1		
数据包发送		行为	EI	4.6	1		
添加电话信息		行为	EI	4.6	1		
Error file							
Help file							
现场帮助		行为	EQ	4.6	1	ADD	4.6
屏幕帮助		行为	EQ	4.6	1	ADD	4.6
逾期		行为	EQ	4.6	1	ADD	4.6
联系报告		行为	EO	4.6	1	ADD	4.6
总计							
					1		25.4

表 6-7

与国际功能点 (IFPUG) 的偏差

以案例 A 为例子:

- 使用 IFPUG 数 EI、EO、EQ、ILF、EIF 每类都有规则判断复杂度, 高中低, 对应得出功能点数。例如 ILF 公司联系信息可分成 2 组, 公司信息和联系信息如打电话、资料包等; 总共包括 13 类字段, 数据组 (Record Elements Type RET) =2, 字段 (Data Element Types DET) =13, ILF 复杂度为低。同样思路, 创建、查询、更新也都是 2 组共不超过 15 DET 字段, 按定义都是属于中等, 但删除只需要共 4 字段, 所以复杂度低, 加起来得出调整前功能点数 FP=35 (详见下表)
- 使用 SiFP 估算实体和行为数, 计算得出 FP=41.6

Complexity 复杂度	Function type 类型				
	ILF	EIF	EI	EO	EQ
Low 低	7	5	3	4	3
Average 平均	10	7	4	5	4
High 高	15	10	6	7	6

表 6-8

IFPUG	低 L	中 M	高 H	UFP	SiFP	数量 Count	FP
外部输入 (EI)		1	4	19	交易类功能 (行为)		
外部输出 (EO)							
外部查询 (EQ)			1	4		6*4.6=27.6	
内部逻辑文件 (ILF)		1		7	数据类功能 (实体)		
外部接口文件 (EIF)		1		5		2*7=14	
IFPUG 功能点数				35		简化功能点数	41.6

表 6-9

- SiFP 得出的实体数量，与行为数量都一样（实体 (=ILF+EIF)=2；行为 (=EI+EO+EQ)=6）
- 因为简化功能点只是不区分实体与行为的复杂度 (高中低)，取平均值。因这案例 A 复杂度偏低，所以用 IFPUG 考虑复杂度时功能点数比 SiFP 低，但如果综合公司多个项目，复杂度有高有低，平均的差异便会降低

参考 References

1. Brigido, Sergio. "FPA Rules interpretations for Elementary Processes ", IFPUG Whit paper. (2022)

2. SiFPA Assoc. "Simple Function Point Functional Size Measurement Method, Measurement Examples ", SiFP-0.1.00-EX-EN-01.01 (2014)

3. Garmus, David. 'Function Point Analysis', Addison-Wesley (2001)

Chapter 7

策划与监控

首次拜访 H 公司负责 IT 服务的总经理，胡总。

公司包括好几个部门和子公司，主要做电力相关的软件项目，也服务集团内信息化需求。胡总说希望用 12 个月时间改善开发项目的质量。

我简单介绍了很多开发团队都因为前面工作没有关注质量，导致大量缺陷在后期才暴露，除了影响质量，也导致大量返工。他们赞同这总体改进方向后，我继续说若要有效果，必须跟做项目一样要策划与监控。但很多团队因没有做好 WBS（工作任务分解），无法做好计划和监控开发过程中的偏差（进度、工作量）。

胡总说：“上周刚与甲方开会，问我们项目的进展，偏差多少，因缺乏相关项目信息，说不出来、所以项目组长须要了解项目管理的步骤，每步做什么？有什么产出？”

我：“赞同。项目经理不仅要制定项目计划和进度表，也要依据实际任务的完成情况监控，汇报项目的偏差，例如延误了 15%”

总监说：“还要明确 15% 是什么意思。”

我：“所以建议项目管理都使用挣值分析计算偏差百分比，避免误解。必须有机制知道任务实际是否完成，才能监控项目实际的进展情况。例如：任务的产出物必须通过同行评审才算是 100% 完成。

“过程改进的项目计划应该如何制定？有什么步骤？”技术研究中心主任问。

（中心负责指导和辅助各开发部门成立改进小组推进过程改进）

“步骤跟软件开发项目类似，也是包括：

- # 先了解需求、项目的目标、成功要素，立项

- # 按项目和团队特性选择项目生命周期

- # 工作任务分解和规模估算

- # 成本预算、时间安排、分配资源

- # 按里程碑监控

但因为需求比较明确，规模固定，有些地方可以简化。”

总监说：“你就跟老师学习一下，学好了可以后面辅导项目团队，如何做好估算、策划和监控，解决当前无法提到项目偏差这难题。”

会议结束后，我便与主任继续讲解如何开展，制定计划。

=====

与软件开发项目不同，今次过程改进项目的需求和目标很明确。

但还需要跟所有干系人开会确认，并写出带出项目概述（一页纸内），里面包括：里面包括：

* 目标、预计结果、度量、里程碑、主要活动

* 人员：组长和组员

因为你们第一次做，不熟识，这模板已经把已经知道的内容填上，你填空便可，但有些地方需要你按实际项目情况写。（详见附件）

成功准则非常重要，因最终必须能为公司省钱，所以概览里明确会预计 3 个月剩余多少钱（或返工工作量），后期暴露的缺陷数会减多少。

可以按以下考虑因素为过程改进选择项目生命周期： * 需求是否明确？

* 过程中会不会有很多需求变更？

* 是否会探索很多新的技术？或者已经很成熟，做过同类项目很多次？

* 团队成员的经验和能力

“虽然需求很明确，出现需求变更的可能性低。但我们没有做过类似的项目，团队成员也没有相关经验。”主任说。

所以就不合适选择传统的瀑布式生命周期——前期花很多精力做好需求，写需求/设计文档。因为大家都没做过，也无法写出合适的过程文档和指南，应该用有限的人力资源投放在试点——学习如何收集数据；分析根因；提出对应纠正措施；并马上在下一代尝试。所以应选择迭代生命周期，如果对应的软件开发项目也是按迭代生命周期，每 2-4 周一个迭代，两者就可以同步了。

虽然有很多不同类型迭代生命周期，我们可以简单按原型法迭代的方式。可以把每一轮试点改进当成做一轮原型，方便理解。

如果是全新的研发项目：

需求和技术方面都存在很多不确定的变数，而且团队成员能力强，便应相信团队成员有能力在每轮冲刺，开发出优秀产品为公司增值。采用敏捷项目生命周期，减少使用传统的策划指标在里程碑监控。（和前面谷歌创始人 Larry 先生对芬兰计划的评价一样，传统计划方式反而会妨碍了员工的创新。）

迭代项目管理

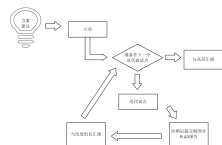


图 7-1

因为需求较明确：针对如何能让缺陷预先发现解决，减少大量后期返工建议类似原型法，使用迭代项目生命周期：

* 立项 (如收集需求)

- * 准备在下一个迭代做试点 (如策划下一个原型)
- * 迭代试点 (如开发原型)
- * 回顾后提交根因分析 A3 报告 (如交付原型)
- * 与改进组长汇报 (除了 A3 报告, 也包括项目监控) (如收集客户反馈)
- * 里程碑与高层汇报 (与计划对比) (如最终交付)

工作任务分解 WBS

可以按这种迭代方式做工作任务分解。

例如:

以上试点的迭代活动, 再加上培训、汇报、开会、写经验教训 wiki

WBS 的第一层应包括:

- * 迭代准备与策划
- * 迭代里的准备 (包括收集数据)
- * 一起做迭代回顾, 分析根因
- * 写每轮迭代 A3 报告, 更新缺陷和返工趋势图
- * 参加会议, 与高层、EPG 汇报
- * 培训
- * 在 Wiki 写经验教训、过程、指南等

因为项目不大, 活动应该都不会超过五人天, 不用再细分

定了 WBS 以后, 需要记录在项目管理工具里,

每个活动除了开始结束日期外, 都应该有明确的人员和产出物。

每次迭代汇报就是一个里程碑。

假如你们四个分公司的改进目标需求都是一样, 你们的项目规模应该是一样, 所以就不需要像软件开发项目按输出输入多少估算功能点数。

但每个小组还是可以依据他们以前的同类项目数据 (历史数据), 更准确估算下轮迭代某活动, 例如在 wiki 写迭代 A3 报告, 的工作量。

如果除了返工工作量, 也想估算返工成本, 可以按员工人均均为工时的成本计算, 得出项目的返工成本。

== == == ==

如何监控项目偏差?

很多敏捷教材包括 SCRUM, 使用:

- * 燃尽图: 按过去的故事点完成趋势预估项目的完成日期, 与本来计划完成日期差多少。但因项目规模大小不同, 不能用偏差的天数, 例如会晚 15 天完成, 代表项目的偏差率。
- * 使用看板把项目的进度可视化: 干系人可从看板可以看到哪些开发活动到了什么阶段, 哪些已经完成, 但还是给不了延误百分多少。

有些团队 2-4 周一轮迭代，策划时只是利用故事点估计每一轮迭代能完成多少模块的开发，没有再细分到开发活动。

因为没有细分到任务 (task) 并估计每条任务需要的人天，很容易出现帕金森现象，大家都只会看到最终迭代结束日期的节点，导致延误。

“今天才周二，离本冲刺结束日还有 3 周多，今周不如先研究我负责这模块所需的技术，下周 3 左右开始写也来得及。”开发人员会这样想。

没有考虑任务间的依赖关系，例如：测试用例的执行必须要对应的模块开发完成后才可以开始；但测试用例须要尽早写好，给开发人员参考。所以要满足管理层关于项目偏差监控的要求，和减少任务的延迟，需要有 WBS 管理功能的项目管理工具，配合挣值分析才能有效策划与监控项目。

监控可视化

在传统项目管理除了燃尽图，还有以下方法：

- 1. 甘特图
- 2. 按里程碑的挣值系数趋势图
- 3. 红黄绿灯

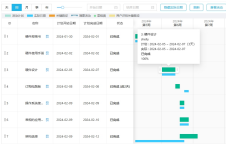
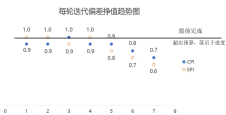


图 7-2

从甘特图看到每个活动的实际开始与结束日期，与计划的偏差。

2. 按每次迭代的挣值系数趋势图（如果每轮迭代都按计划完成，所有报告也完成，进度偏差应是零。）



从上图看到 C（CPI），和 S（SPI）都是每个迭代越来越差（往下），预算超出，进度落后。

3. 红黄绿灯



图 7-3

第一个灯代表表示计划进度，绿色为进度计划正常，黄色是需要关注，红色是进度落后，需要改善。

第二个灯代表费用，绿色为进度费用正常，黄色是需要关注，红色是费用超支，需要改善。

第三个灯代表质量控制，绿色为进度质量正常，黄色是需要关注，红色是质量有严重问题，需要改善。

以上只是监控项目活动执行情况，有没有偏差。更关键应看每组返工工作量和缺陷数量的趋势，是否有改善。如发现与本来的目标差异很大，便应与高层开会讨论，是否应调整目标与计划。

如果 WBS 项目管理工具能支持以上可视化监控功能，如大家都每周五按 WBS 填写工时表，组长不须要画额外工作量，所有人，包括高层等，都可以即时在系统看到以下图表和系数监控项目进展

::= == ==

我只是用某个小组的计划来解释，比较容易理解，但其实你们中心也要制定你们的计划，因为你们有很多前期准备工作，也需要用计划来监控。

如何用于软件开发

总体思路不变，例如：开头还是要与干系人确认出一页纸的项目预览 (POS)。然后按项目需求选择合适的项目生命周期。过程改进项目，因为需求明确，选用迭代原型法便可以。从范围（需求），进度、成本、质量和人力资源等要求，判断这软件开发项目应使用那类项目生命周期。如果范围（需求）明确，不允许修改，进度成本和人力资源便应依据项目需求制定时间和资源安排（瀑布式）。敏捷开发是假定时间与人手固定 (Time Boxing)，但范围可变，团队便应请简策划，在每轮迭代里先开发最有价值的功能，给客户的价值最大化。如下图：

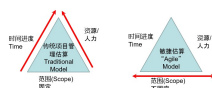


图 7-4

如质量（减少遗留到客户的缺陷）重要，便应制定项目质量计划，为各评审与测试活动定目标缺陷范围和方法选择。例如，需求评审要所有团队成员参加外，也邀请业务专家参加；评审前要所有参与者使用评审检查单预先找出问题。

如果团队因需求不确定，选择敏捷开发方式，虽然项目启动后还是要准备 WBS 工作任务分解，但不需要像传统瀑布式把几个月里面的所有活动都细分，减少浪费，只需要先制定第一级分解便可以。功能需求也要须要分优先级，例如：可以用 MoSCoW 分 4 类。也需要定每一轮的时长，例如 2 周，和预计总共要多少轮才能完成。经过与干系人确认审批，便可以开始每一轮的策划。如下图的上半部分：敏捷项目框架 (Agile Project Framework APF) 流程：

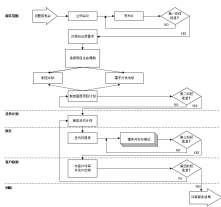


图 7-5

Fig 11-9

每个项目由多轮迭代组成，开头的发版范围除了 POS 外，也应包含：

- 需求分分解 (Requirements Breakdown Structure)，并分优先级
- 总体 WBS 和依赖关系

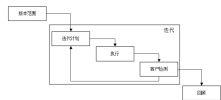


图 7-6

Fig 11-10

与传统瀑布式项目管理不同，每轮迭代只需要做本迭代的 WBS，不会花精力在未来未知的迭代上，满足精益思路，也可以有 WBS 管理的好处，两者取得平衡。

如何管理每轮的活动依赖和人手分配

某敏捷团队希望开发一款新的游戏，每两周一轮迭代，他们按项目所需要的需求（故事点），制定了项目的总体发布计划，第一周，团队成员包括开发与测试人员，针对某一功能一起头脑风暴，按照第一轮迭代的故事估算工作量，如下图：

	活动	人时
A	状态管理类编程	16
B	为 A 做自动化测试	10
C	让机器人追踪没有阻挡的环	12
D	为 C 做自动化测试	12
E	虽然玩家试图阻挡，但机器人还是继续建环	8
F	为 E 写测试用例	4
G	为 E 做自动化测试	4
H	让机器人阻挡玩家建环	12
J	为 H 写测试用例	4
K	为 H 做自动化测试	2

表 7-1

因为估计总共需要 84 人时，按每工作日每人有 6 小时工作，（减出沟通开会等时间），也考虑团队成员有哪天会请假或者被其他项目占用时间，然后直接问这三位成员是否可以。

你觉得以上的敏捷项目管理方法（参考 Cohn 先生在他书中的案例）有什么不足？

“好像没有考虑活动间的依赖关系，也不知道自己负责的活动从哪一天开始，哪一天结束？”

如何用 **APF** 方法完善

先把活动工作量的颗粒度从小时改成每半天：

	活动	人时	
A	状态管理类编程	16	2
B	为 A 做自动化测试	10	1.5
C	让机器人追踪没有阻挡的环	12	1.5
D	为 C 做自动化测试	12	1.5
E	虽然玩家试图阻挡，但机器人还是继续建环	8	1
F	为 E 写测试用例	4	0.5
G	为 E 做自动化测试	4	0.5
H	让机器人阻挡玩家建环	12	1.5
J	为 H 写测试用例	4	0.5
K	为 H 做自动化测试	2	0.5

表 7-2

使用 WBS 项目管理把依赖关系，每个活动的开始日期，结束日期再加上人员安排，例如：可以在墙上的白板或大白纸，用便利贴、水笔画上本迭代里活动间的依赖关系，在下面按每轮 2 周，填活动安排，包括每活动上三名开发或测试人员资源的，如下图：

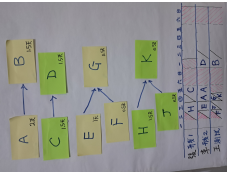


图 7-7

不需要用软件项目管理工具，便可以做到两周迭代的详细计划。在两周的人员安排表只需要分到半天不需要分到小时，每天早上定期十五分钟站立会议，每人汇报实际的进展情况，并更新白板上的计划。可以最简单低科技做到每轮策划与监控的作用，大家一看白板上的图就知道现在项目今次迭代的当前状态。能否在两周内完成承诺的组件。

因为每轮迭代只有两到四周，用大白纸和便利贴，这些低科技方法便可以有效策划和监控，改善之前敏捷策划方法的不足，包括：

- 每活动有开始和结束日期
- 活动间的依赖关系
- 人手分配

也尽量减轻管理工作的负担。

迭代结束并与客户展示确认后，可以按迭代完成情况算出当前的 EV 挣值 AC 实际成本 PV 计划，例如：如果计划里的所有模块都已经完成，客户也接受，也没有增加其他的模块开发，EV 便等于 PV，进度没偏差。

如果公司改进组和开发团队用以上方法策划与监控项目，也同时满足 CMMI 里不放级实践要求。（详见附件）

若要全面了解各类项目管理方法与技巧，从需求到结项，便要参考有全面覆盖敏捷和传统项目管理的书籍，例如 Wysocki 的 Effective Project Management。

你们项目组也可以用这次机会，开始养成定计划，然后按里程碑监控的好习惯。后面可以使用同样步骤，加上功能点规模估算，制定开发项目的 WBS 计划。

总结

你自己可以首先使用刚才的原型法制定公司总体的过程改进计划（要注意刚才我举的例子只是改进小组的计划，并不是公司总体计划），并要求项目组针对改进目标下制定自己的改进项目计划，他们后面可以再使用 APF 的方法，针对软件开发项目制定计划和监控，后面便应可以回应开会时胡总要实时监控项目偏差的要求。

附件

项目概览 POS 模板

机会或问题描述

- 由于缺陷没有得到适当的审查或测试而造成的长期浪费
- 质量问题：顾客拒收，顾客不满，影响 H 品牌形象
- 由于资源大部分时间用于“技术债务”（返工以纠正缺陷）而导致的延迟

目标

- 通过在早期阶段提高缺陷去除效率，例如评审和测试
- 结果和时间框架：
 - * 将迭代中的系统测试缺陷密度从当前（2024 年 10 月）的 xx/FP 提高到 2025 年 1 月的 yy/FP
 - * 将每次迭代的总返工工时从当前（2024 年 10 月）的 xx 人-hr/FP 减少

到 2025 年 1 月的 yy 人-hr/FP

- 示例：
* 经过问题诊断和补救措施，将导致 ST 缺陷在实施后 3 个月内减少 50%。初步计算估计节省的努力（成本）潜力约为。总项目工作日的 30%（由于在需求和开发阶段通过早期移除缺陷减少了返工工作量）

预计结果/度量/时段/行动

- 针对试点项目，培训团队如何做好迭代回顾分析缺陷（Eva 11 月 15 日）
- 从 JIRA 收集迭代缺陷数
- 返工工作将在 sprint 回顾中手动收集（在 Nov 培训里跟团队说好大家应如何收集返工工作量）
- 目标：回顾时一起分析根因（有 ATM 现场辅导），在下一轮迭代使用纠正措施，希望能把迭代系统测试缺陷数减半

关键里程碑

1. 将测试重点放在容易出错的模块上（10 月 10 日/ Peter）
 2. 设计测试用例培训（11 月 10 日/ John）
- 注意：详细的时间表请参考 Jira
- === 成功标准 === “我们为什么要做这个项目？”，衡量这个项目的业务结果“我们（和我们的客户）必须做些什么才能说项目是成功的？”
- * 由于“技术债务”导致的返工工作显著减少
- * 在 UAT 和用户开始使用产品后的前两个月发现的缺陷显著减少（注）
- 注：参考过程改进项目建议可行性研究中的缺陷返工和 ROI 估计中的数字

项目经理与组员

- 组长：陈乔
- 成员：李明明、华志、胡天

参考 References

1. Wysocki, R. *Effective Project Management* 6/e (2012)
2. Cohn, Mike, *Agile Estimating and Planning* (2006)

Part IV

开始过程改进之旅

所有有效的改进，团队必须先获得管理层的关注与支持。

第 8 章先介绍如何以高层的“语言”分析改进对公司的价值，获取高层支持，投入资源，成立改进组，跟其他项目一样立项管理。

第 9 章继续用实例说明过程改进项目的主要成功要素，包括要做好根因分析。

Chapter 8

获取高层支持

我：对过程改进来说，最重要的成功要素是什么？

客户：最难的是如何得到高层的支持，这不仅仅是嘴巴说说而已，而是要切实地给人、给时间。高层往往不清楚什么是质量改进的重点，但他们对员工的人均收入、利润（比如员工可为公司盈利的时间占比。如果少，就表示这个员工对公司的盈利贡献不够。）等这些财务指标都非常清楚。

我：非常赞同。我们可以利用评估机会来引起高层对质量改进的重视，但往往评估组只说软件开发的各种问题，难以引起他们注意。一般人，尤其是高层，一听到问题都会觉得比较烦，没有动力听下去，更不要说有对应的改进行动了。

客户：你说的挺有道理的。确实我们每年都会有一些评估，发现了不少问题。高层也都会参加评估会，并同意这些问题需要改进。但很多时候过后就没下文了。

我：质量大师 Dr Juran 有丰富的过程改进经验，他深知要说服高层真心投入、支持改进，就必须用高层的语言打动他们。如果与高层交流，不能引起高层关注，就难以获得立项，做改进。

这原理大家都懂，但有些质量经理觉得要收集足够数据才提建议给高层。但其实不用，只需要有足够信息让高层相信这改进回报高，值得去做便可。

案例

我问公司内部评估组，高层有哪些关注点？他们就列出来，如人均收入、人均利润等商务指标。

影响这些指标的因素很多，有些是软件开发团队无法控制的，例如销售人员与客户的关系。

影响这些指标的因素很多，有些是软件开发团队无法控制的，例如销售人员与客户的关系。

质量经理：项目进度的偏差，遗漏给客户的缺陷数。

我：这些我们都叫性能目标。这些指标是如何影响高层目标的？质量经理：比如项目延误，成本就会超支。

我：是的。如果我们过程改进希望改善质量、减少缺陷，你们觉得会对成本有什么影响？

质量经理：改进要投入工作量，肯定会增加成本。但长远应可降低成本。

我：很多高层都不熟悉质量管理，所以很少关注和监控团队的缺陷。他们通常还是会觉得质量改进是好事，但要花成本。我做到客户满意的水平就可以了，不要追求十全十美，也正是这个误解，才导致他们认为大部分缺陷在系统测试或验收阶段才发现是正常、是常态。如果是开发软件产品的公司会好一些。在软件工程领域，因为缺陷发现越靠后，返工工作量是前期发现同样缺陷的 15-40 倍。所以如果我们能把返工缺陷预先发现并处理，必然会大量降低返工工作量，与相关成本。

理论上也可以用缺陷排除率来估算改进后的缺陷分布（会在后面迭代回顾里细讲），但高层一般不会注意这些细节，沟通越直接越简单越好，所以只需要简单估计改进后的数量，大家觉得合理便可。

第一版方案书

质量经理做了第一版的方案书。



表 8-1

我：挺好的。你在投资方面是保守型吗？

质量经理：不是啊，我还会定期买股票，因为单是靠银行定期，利息太低了。

我：但是看你在质量改进方面的目标很保守。比如你看，说如果做了改进以后，后面的缺陷可以降低 10% 到 20%，然后最后算出来节省不到 5 个点。你估计高层会买单，投资你这个项目吗？

质量经理：确实有道理。

我：你这里有几个地方没做对，例如从我们过去的经验，因为很多团队在前面几乎没做好单元测试或代码扫描，反之，系统测试或者验收测试的缺陷几乎可以减少一半。低的目标不是好目标。还有，你评估在后面阶段返工的工作量也太低了，只是比前面发现的高一点点。你尝试依据这 2 个思路再调一下吧。

质量经理：你有所不知，我们虽然有些行业参考数据，但没有实际项目数据支撑，是否需要先找些实际数据才可以做方案，不然好像说不过去。

我：在初步方案阶段，其实不需要实际数据的支撑。你可以想象，目的是要让管理者有依据做决定，你可以利用实际数据把节省算得更准确，但对高层决策没有太大帮助。所以在初始阶段，合理的估算就足够了，不需要花精力在没有价值的事情上。

第二版与中层确认，然后向高层汇报

做了第二版后，大家觉得这确实比之前更有说服力了，质量经理再跟中层确认。

项目A：规划120人月，总成本300w					
里程碑	单个缺陷返工工时/h	原缺陷数	原返工工时/h	改善后缺陷数	改善后返工工时/h
需求评审	1	10	10	15	15
设计评审	1	5	5	8	8
UT阶段	1	45	45	60	60
SIT阶段	20	240	4800	120	2400
UAT阶段	30	100	3000	50	1500
总计			7860		3983
成本占比			39.30%		19.92%

改善效果：成本下降19.39%，约58w

问题	需求变更 评审不充分（例如：设计评审） 单元测试（自动化脚本） 开发人员技能不足 版本管理问题
影响	项目延期、成本增加
总人月	原计划120人月 影响后130人月
总成本	原计划300w 影响后325w
造成损失：	成本增加8.3%，约25w
	改善后的返工工时是原来的 51%

表 8-2

质量经理跟高层汇报，并得到高层的认可后，高兴地说：以前都以为过程改进应先自己默默耕耘，先在试点取得效果，再跟高层要资源后再推广。现在看，还是应该一开始便提具体改进方案并立项，才有机会成功。

我：管理层是过程改进最重要的干系人，必须一开始便得到他们支持。你后面有什么计划？

质量经理：既然他们都同意，我就找项目试点，对吗？

我：虽然试点很重要，但不是第一步。你们一直有过程改进小组吗？

质量经理：懂你的意思，但实际没有。团队有关质量问题便会直接找我。我：应让团队自己依据迭代数据分析根因，下一轮做改进。但有些需要跨功能或组织级改进，如完善复用框架，或完善规范/指南与相关检查单等，便需要依靠过程改进小组来推动。如果你们没有，便应尽快举办工作坊互动培训，邀请所有过程的利益相关者参加，一起讨论未来改进重点并制订具体短期长期目标与计划。

质量经理：你说的有道理，请发我相关案例，打铁趁热，我尽快跟高层说。

结论

从以上简单案例，得到的经验教训：

- 1. 有效的质量改进必须从获取高层支持开始。
- 2. 中层要从高层关注点（¥）提改进方案。
- 3. 在做改进方案时应根据现状估算，暂时不需要实际项目统计数据，以节省资源和时间。
- 4. 如果确实有高层真正支持的话，就必须正式立项，再配上具体的团队、分工和详细计划，否则整个改进计划还是一纸空文。
- 5. 立项后，可以用 2 天工作坊培训启动，大家一起制定目标和行动计划。

反馈

某 CTO：方向很好，但高层不仅关注如销售额这些商业指标，也关心交付效率和质量相关指标。

公司老板针对于研发除了关注成本外，也关注以下内容：

- 人员：人均薪资、人均产能、人员参项率（有收入的项目）、人员闲置率
- 进度：是否能正常交付
- 质量：缺陷率、严重缺陷数

例如，公司都会用人均产值，人均交付金额等指标。但有些公司业务很稳定，公司不一定关注创新：例如，针对重复性工作，是否能基于以往项目形成公共组件或低代码平台，来大幅度提升生产率。

质量方面，如果客户能参与使用，例如产品定义，需求的质量会大幅度提升（不仅依赖评审、测试）。为什么客户不愿意深度参与？是否因为没有让客户可以高效参与的工具。

例如，需求调研过程，90

所以不应只关注如何尽量提前发现缺陷

我：是的。想要过程改进成功，首先必须得到高层的支持，很多高层不一定关注质量，只关注项目是否按时交付和财务指标。

所以质量改进方案必须强调能节省多少人力成本，不仅仅是质量提升，才有机会得到高层支持。

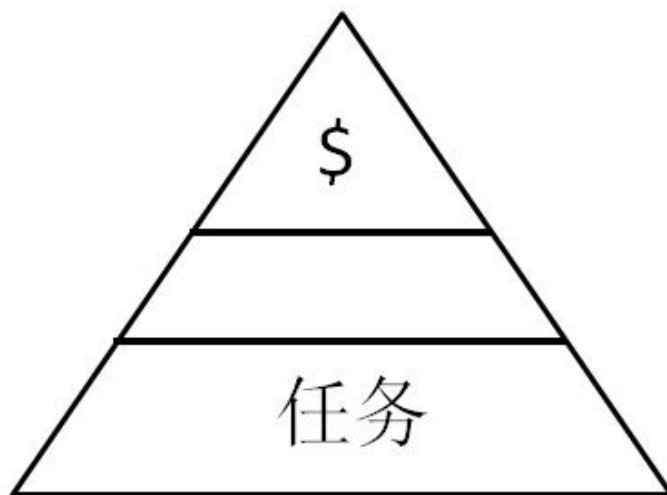


图 8-1

大量缺陷在后期才暴露是最常见的质量问题。提前发现并消除这些缺陷最容易能提升项目质量，并降低成本（因大幅减少返工）。所以上面案例使用这质量改进作为例子，但如果面对像你这种通情达理的高层，就可以更全面，从质量、创新等多维度估计改进能如何为公司带来价值。

Chapter 9

寻找改进机会

如果贵公司计划在 8 个月后进行质量改进（如 ISO 或 CMMI），但当前缺少必要的过程文件和指导方针，该如何应对？

三年前，我们确实遇到了类似的情况。幸好咨询公司告诉我们不必担心，他们根据其他已经通过评估的公司整理了一整套完整的流程、模板和指南，确保能够满足评估要求。评估前，我们安排项目组根据过程和模板准备了相关的项目文档，一个月后顺利通过评估。”

请问你们现在还继续使用吗？

评估通过后，团队便恢复了原有的工作方式，不再使用咨询公司提供的那套文档了。但我在电脑上还是保留了备份。”

从以上对话中，我们看仅仅要求团队使用第三方的最佳实践，常常难以落地，对流程和质量的提升也没有实际帮助。那么，为什么团队会抗拒这些原本非常有价值的模板和指南呢？背后的原因是什么？

破冰 - 变革 - 稳定

一些美国企业，如通用和福特，相信科学管理。例如，老福特曾邀请 Taylor 先生优化汽车生产线，但他的优化主要集中在工程层面，忽视了一线员工的参与，这导致许多一线工人抵触由“工程师”用科学方法设计的最佳方案。

为什么过程改进的落地如此困难重重？又为何遭到工人的反对？要解答这些问题，需要先了解人类的心理。

心理学教授 Lewin 发现，任何过程改进，例如费城 Weisbord 先生的自主团队改革，都需经过以下三个阶段：

第一、破冰（Unfreezing）阶段，通过提供新的信息和数据，减少团队的反对声音，动摇原有习惯，促进“破冰”。

第二、变革（Moving）阶段，促使团队改变态度、价值观、架构和行为等。

第三、固化（Refreezing）阶段，在取得成效后，持续维护改进成果，避免回归旧习。

增量过程改进如下图所示，显示每轮都有提升。

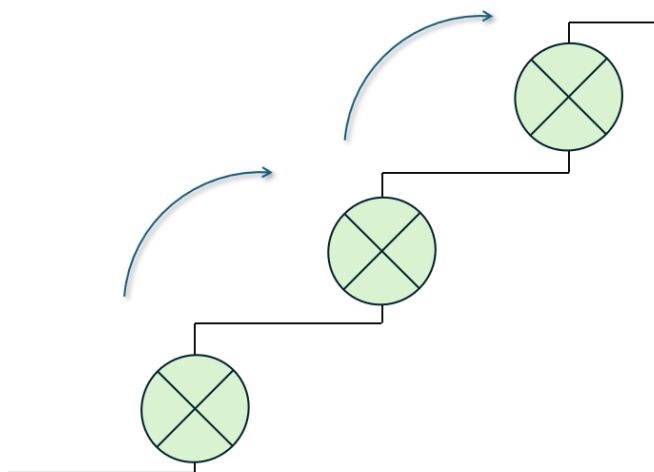


图 9-1

Dr. Lewin 在分析二战粮食分配实验（见附件）时发现：“必须让人们参与分析和讨论，才能激发他们采取行动的动机。We are likely to modify our own behavior when we participate in problem analysis and solution, and likely to carry out decisions we have helped make.”

美国弗吉尼亚州的 Harwood 工厂也做了一个实验，允许团队自主安排工作以应对季节性市场需求的变化，结果表明让团队自主策划效果最佳（详见本文附件“睡衣工厂试验”）。

这两个实验都验证了集体讨论与自主管理确实能够提升团队的生产率，同时也印证了实验前的观察：不同工人有各自不同的工作方式。

自己找答案

丰田强调“自己找答案”和“生产产品就是培养人”。因此，即便过程改进小组知道如何有效解决一线员工问题的方法，他们也不会直接提供答案，而是鼓励员工自主思考寻找解决方案。

为什么呢？

因为丰田管理层的信念就是：人的智慧没有尽头。过多干预不仅会阻碍员工发挥无限潜能为公司增值，还容易传递“员工必须服从指挥”的信息，助长企业内的“大王”文化，类似于前述谷歌的“河马”文化。

鼓励员工通过动脑思考和总结经验教训，是培养人才的必经之路。

从丰田的案例和 Dr. Lewin 的实验可以看出，群体参与的重要性。让员工有机会参与或影响决策，他们就有动力推动改进。反之，如果只是要求员工遵循专家的“最佳方案”，则难以激发他们主动进行实验和分析数据。下面另一个实验可以让我们更理解：

自己做实验，分析数据有家工厂，很多主管不愿意雇佣 30 岁以上的女性操作工，认为她们的生产效率不如年轻女工。若我们作为顾问，直接提供数据证据，能改变这种看法吗？不太可能。所以顾问只能鼓励他们自己进行实验，证明那些 30 岁以上女工的生产率确实比年轻女工的要低。所以顾问只能鼓励他们自己进行实验，证明那些 30 岁以上女工的生产率确实比年轻女工的要低。从此，这些主管不再拒绝聘用年长的女工了。但那些没有参与实验的团队依旧不愿意改变他们的观念。

成功要素

从此，这些主管不再拒绝聘用年长的女工了。但那些没有参与实验的团队依旧不愿意改变他们的观念。以下是他的经验分享：

在为企业做访谈和诊断时，一般的顾问仅关注企业遇到了什么问题、有什么困难及其解决方法？而更为有效的是跳出这个圈子，从更高的视角去思考这家公司有哪些改进的潜力？公司内部是否有能够持续推进这种改革的人？是否存在一些可以吸引全员参与的改革方向？随着环境快速变化，顾问的角色不应再局限于帮助客户解决技术问题或提供诊断方案，而是转向挖掘员工的潜力，让他们主动思考并解决自身遇到的问题。只有这样，才真正对企业有价值。”

他提出了三个成功的关键因素：

Leadership commitment 领导支持

有没有领导的支持？

我接触过很多公司，如果高层难以联系或并不关心质量问题，甚至认为可以委托他人处理，推进就会变得困难。常见的情况是领导本身业务出身，也不关注软件工程，认为软件开发并不复杂，市场和销售才是重点。或者，领导只是口头上支持，实际上没有定期监控改进的进展，对培训也不太关注，最终很难取得实质性效果。

商机 Good Business opportunity

公司有没有一些持久的商机？

当前软件工程的竞争激烈。如果公司没有持久的方案、产品或专注的业务方向，而是靠客户关系做些定制开发的项目，就很难投入资源进行提升。

员工动力 Energized people

公司里面有没有中层愿意投入时间，真心推动改革？

如果谷歌没有 Smart Creatives 这样的员工，就不会有如此多的谷歌创新产品。如果员工都是负面的心态，推动改进将困难重重。

关于第一点领导支持，前面已经分享了如何用高层的语言（金钱、成本）吸引他对改进感兴趣，然后估计能节省多少，并正式立项。

有了这些基础，便可以开始质量改进之旅。

A 公司过程改进之旅

今年 4 月初，我首次拜访 A 公司高层，听他们过程改进需求。

COO 万先生最后一位到，看来 50 多岁，中等身材，身穿有公司标志的长袖外套，工程师打扮，留了一点胡子，精神饱满。他非常忙，只能空出半小时。

我们甲乙双方各 2 位，开始围着小圆桌交流。

COO 谦虚地说：“我没有软件开发项目经验，一直都是管理基础建设项目。我之前见过任正非，当时公司帮华为在香港建设数据中心。”

万先生是高层团队成员之一，与集团主席、副主席、总经理、财务总监等共 7 人组成高层团队。

我简单介绍了 CMMI 是用于衡量公司的过程成熟度模型，软件开发供应商一直质量水平差异很大。接着说：“软件项目不同于硬件项目，交付后会根据需求不断变动。如果开发期间不注重质量，又缺乏评审和测试，产品会有许多缺陷。由于软件本身设计不佳，最终引发大量返工工作。就像埋下了大量地雷，后面需要还债，这叫‘技术债务’（Technical Debt）。编码人员为了省时间，只顾写代码，缺乏文档，其他人难以看懂，几周后甚至自己也看不懂，导致后续还债更加困难。所以虽然小学生都可以写程序，但不一定能写出灵活、高质量的程序。”看到 COO 深有感触，他望着旁边的老王（部门经理）说：“你说得对，我们现在也正在还债。”

接着，他介绍了集团的组织架构：主要有 4 个业务部，最大的是工程部，承接大厦的电机工程，例如电梯、空调、自动化等；专门做政府软件项目的部门叫科技部（约 80 名员工），虽然业务部门，如工程部也逐步拓展了软件服务部分，但占总体业务收入不到 5%。

COO 预计咨询顾问可能怀疑他们做过程改进的决心，便解释说之前已经跟其他董事会成员说过：“虽然现在软件开发占公司业务量不多，但包括软件部分的项目（如办公楼的自动化）越来越多，并有不断上升趋势，这将是未来核心竞争力的重要部分。所以必须预先练好内功，才有能力在未来承接更多的软件相关项目。”

他再强调公司不仅仅是为了拿到 CMMI 证书，更重要是希望改进我们软件开发过程。

项目启动后，从 4 月底到 5 月初为他们的 6 位评估组成员，做了 8 天培训。

很佩服万总的魄力和支持，在第一天培训开始前，他特意找了公司总经理来培训现场给各学员说话，打气。虽然万总每天都非常忙，但他还是尽量抽一些时间来听课。

但因为 COO 万总确实太忙了。我们解释了如果没有一位有经验的人领导改进组，后面难以推进，他便联系老刘，请他担任组长。

老刘进 A 公司已经 16 年了，年龄与他接近，是整个集团培训中心的负责人。

6 月底，公司安排了午饭期间，我与 COO 安排的过程改进组长刘先生在培训中心旁边的 Ibis 酒店首次见面。刘先生问：“请问过程改进小组是做什么的？虽然我一直负责工程项目和质量，但对软件开发并不熟悉，这样能胜任吗？”

“非常欢迎你来当组长，我们非常需要像你这样有经验的管理层来推动过程改进。你不用担心不熟悉软件开发，我已经和相关的项目经理、评估组成员、QA 等进行了几天的深入培训，他们很清楚需要改进和欠缺的地方。组长的主要职责是协助这些改进组，一方面为他们争取所需要的资源，另一方面向高层汇报，让他们了解改进的价值并获得支持。”

老刘问：“软件工程的质量问题与一般工程项目有什么不一样吗？”

我解释：“现在我们周围的日常产品大多都包含软件，因为软件比硬件灵活，功能更新也容易。以前在硬件上做变动很困难，而软件则不同。正因为软件容易更新，我们在写程序时必须确保代码易于增加或删除功能。如果程序写得不好，任何改变都可能导致整个程序的大幅改动或重写，这就是所谓的‘技术债务’。”老刘说：“这跟我们做工程项目很不一样，很多项目在交付前，业主还是会提很多修改。比如某些管道怎么走，因为这些修改不影响建筑结构，所以我们还是会按要求修改。”

旁边的小李（A 公司项目经理）补充道：“我听说去年某项目团队，因为前期过程，如单元测试、集成测试等都没做好，也缺少报告与文档，最终导致客户在验收时拒收。团队尽力补救，但那些阶段已经过去，无法弥补。管理层为了避免项目烂尾，影响声誉，承诺把整个项目从头再做。客户因为有了经验教训，特意找了第三方公司监督项目的过程。所以我们非常同意前期如果不注意质量，后期就要还债。”

后面边吃边聊中，了解到老刘一直保持运动。年轻时他喜欢打壁球，但因为壁球需要前后快速移动，容易伤到膝盖，所以年纪大了改为跑步。他每天 6 点起床，7 点前跑步。如果周末不用上班，他早上会跑步大约一小时。

过程改进小组正式成立，并由老刘担任组长，负责监督每月的小组会议，尽管会议有纪要和待办事项，但 3 个月过去了，进展并不明显。

== ==

9 月底，过程改进小组开第二次会议，参与的人数比上次多，有四、五位组长参加，大家积极参与讨论，组长熟悉了整个过程改进的监控。例如用了 2 小时讨论配置管理的问题和改进后，组长与大家确认是否所有要改进的点都讨论完毕，并把具体的改进工作分配到对应的人，大家决议用 2 周一轮冲刺的方式来评估进展。

参与的人也主动提出要做什么改进的工作，整个气氛比上次好多了，但还是没有形成一个具体任务工作分解（WBS）的计划。

因为过程改进如果没有分到任务的计划，很容易会延误，后面就没有冲劲。但外部顾问暂时难以直接把问题说出来，唯有静观其变，原先的总体计划是 4 月份启动，10 月份做预评估，但到了 9 月份大家发现进展缓慢，所以调整计划把预评估延到 2 月份，也计划在 11 月初在香港，安排了我到现场做一天补充培训。

11 月份一天培训

早上 5 个部门的领导，加上万总、老刘都参加，除了 2 位，其他 5 位都是首次见面。我先跟他们说因前面评审和测试不足，大部分缺陷都在后面测试时才发现处理，导致大量可以避免的返工。也介绍了“技术负债”。我继续解释该如何选择——第一个改进项目的方向很重要。因为最终必须为公司省钱，管理层才会继续支持，所以必须能节省成本或工作量。也必须可度量。我接着解释为什么建议应针对如何能尽早发现并解决缺陷，避免后面大量返工。这样便能不仅仅是质量最优（留给客户的缺陷数最少），也能降低总工作量。

各领导都纷纷表示赞同。

午饭后，大家开始做互动练习，但大部分领导没有参加，主要是项目经理和评估组成员。在培训结束前最后一小时，邀请万总和老刘来现场，跟 5 个项目组长说好要星期一提交改进计划，但过了期限没有任何回应，过了 3 天万总亲自发邮件催促，再过 2 天才有一组回邮件发计划，但已经晚了一周，再过几

天另一组也发了计划，其他 3 组一直没有反应。

这两组计划。其中一组只提交了进度表；另一组误以为只需要写一下过程文件，满足两个月后的评估需求。所以没有一个合格的计划。我立马反馈给高层管理说明改进计划应包括什么内容，例如必须要有量化目标，明确团队成员和改进的里程碑和相关的培训和辅导等。

回顾进展缓慢的原因

除了高层支持和人员参与外，也需要有工具配合。例如：项目管理工具。如果已经有明确量化改进目标，也有所需要的资源，包括人员和工具，我们还需要有一张地图，在敏捷开发过程改进中，可以参考 Scrum、XP 极限编程、CMMI 等。

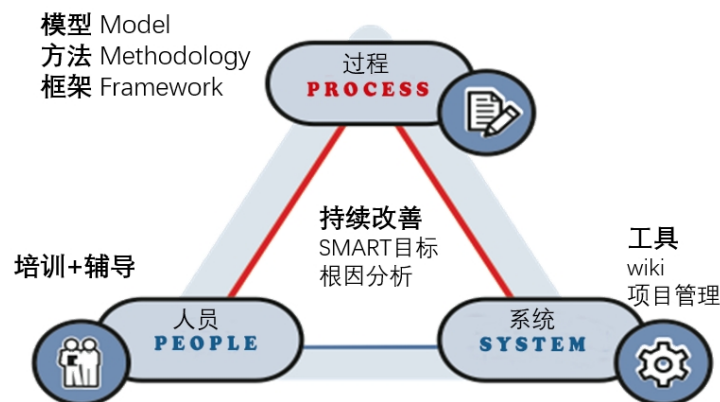


图 9-2

例如如果依据 CMMI 里针对估算、策划和监控的实践要求，便能列出项目管理工具需要以下项目管理工具应具备的基本功能：

- * 有日期和可交付成果的 WBS 管理
- * 个人更新工时表，更新任务实际开始/完成日期
- * 只有经过同行评审，任务才能 100% 完成
- * 实时项目状态：例如红黄绿、挣值分析（进度变量%，成本变量%）

A 公司的过程改进在哪些地方出了问题？

一直没有把过程改进当成一个项目来管理，进展缓慢，但因没有项目实际与计划对比的监控，大家都不知道确实延误了多少。（只知道现在是有延误）

* 虽然 COO 从头一直尽力推动，但部门级领导参与不多。因为软件开发人员都忙于做项目，从 5 月份开始一直都是由共 6 位从部门安排的代表参加。例如很多部门经理只是 11 月初来参加了上午半天培训。下午到做互动练习是他们的手下做项目的工程师来参加，到了最后要制定行动计划和目标时，部门经理都不在

* 虽然在 5 月份做了共 8 天培训，但一直没有改进计划，导致培训内容与实际改进行动难以对应

* 虽然做了五天专题培训，包括测试、功能点估算、需求、项目策划与监控、配置管理等，但因为没有立马在项目使用，没有效果

综上所述，目前的状况不是特别意外，虽然大部分参加培训的成员开始时都相当积极，但碍于他们的领导缺乏投入，导致没有进展。因为大部分团队，不论组长或者组员都没有经历过过程改进，所以开始时走很多弯路。

虽然培训很重要，但最困难是改变人的习惯，培训本身不能改变团队的思路和做事习惯，如果公司不把过程改进作为项目来管理，投入资源并定期监控，培训不会为公司带来任何价值，员工也没有动力参加。

如何能加快破冰

传统咨询辅导的方式：先诊断发现差距，针对不足做培训，在某些项目试点，然后推广，再评审。前后会跨越好几个月。以 A 公司为例，除了要解决管理层参与、人员有充分时间投入与工具等基础问题外，如想改进能尽早有效果，考虑所有干系人聚在一起，做 2 天工作坊，大家针对改进主题讨论，出目标与计划。针对以上传统做法的不足，如果把所有干系人聚在一起，做 2 天工作坊，一起回顾过去，放眼未来，制定具体量化目标和行动计划，便能让“所有员工参与改进整个系统”便能改善各种沟通不畅的问题。把本来需要几周时间沟通，压缩到 2 天。

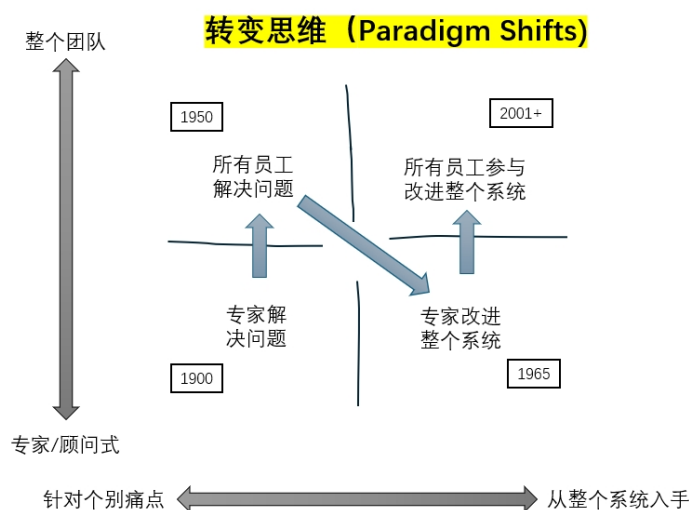


图 9-3

(参考 M. Weisbord, Productive Workplaces Revisited)

过程重组

因为 IBM 的机器很昂贵，大部分客户需要安排分期付款。针对这客户需求，IBM 公司成立子公司专门提供贷款服务。整个过程共 5 个步骤：初步审核、查看公司的财政背景、专人制定的贷款的利率、法务看合同条款、文员把合同打印好交给销售人员，平均需要六个工作日，但有时候可以长达两周。在这段时间，客户可能会找到更好的贷款渠道，或者竞争对手导致客户改变主意，所以一线销售人员一直觉得时间太长，并会不停催促。

为了解决这个问题，2 位 IBM 高级经理专门头脑风暴研究是否可以缩短整个过程。他们先实验，把这 5 个部门的工作集中一起做，发现总共只需要 90 分钟（一个半小时）。中间大部分时间浪费在部门之间信息交流、等待。他们发现开始提供贷款服务时，认为每一个步骤都需要专业人事处理。但他们发现除了少数特殊情况，大部分交易都可以标准化处理。例如把各种情况列出来，并预先先定好对应的价位和利率，和标准合同模板等等，都预先记录在电脑中。有了这些信息化支持，这种改善做完以后，这些本来由专家负责的工作便可合成由一个人负责。

结果把本来六七天的流程减到 4 小时。

这类改善并非渐进式，而是改革式，从基础总体看问题，因为不能仅仅是按现在的流程做优化，必须从整个流程从头到尾看才可以取得效果。

“我们开发人员天天加班，尽力满足需求，但产品经理提的需求都写的很模糊，又经常改变，后面系统测试发现很多缺陷，不能但怪罪我们开发人员。”

“我们测试人员要做好测试真是很困难，前面开发人员没有做好自测，写完代码就扔给我们测试，肯定会发现很多缺陷，但项目经理又要求我们赶上在发版日前完成，真不晓得应怎样做。”

大家都可能听过以上的声音。跟上面 IBM 的案例一样，不能单靠某部门去改善总体效果，需要各部门都参与，才可以提升，最终达到公司的目标。

若要所有员工参与，有效改进整个系统，要注意以下 4 条原则：

让团队自己寻找改进方案

上面粮食分配实验也验证了团队只有兴趣执行自己集体讨论的分析结果，而非听从“专家”意见。所以必须让大家参与、动脑筋，才有动力采取行动。

让每位团队成员都有充分机会表达意见，集思广益，得出更好的纠正措施。

整个团队（所有相关干系人）参与

整个团队分析全局问题。例如如果只是从测试人员的角度，他可能以为缺陷都是来自开发；需求分析人员（或产品经理）会觉得自己做得很好，因需求评审里没有什么问题被发现。但如果团队所有成员聚在一起（包括项目经理、需求、设计、编码、测试），让团队所有岗位一起分析，才可以全局看问题，才可能发现不少问题是因为需求没有明确，导致后面做出来的功能并不是客户要的。后面可扩大参与回顾的干系人，例如客户代表，可以更全面分析。

看全局，寻找大家的共同点

避免局部最优 (Sub-optimization)，避免盲人摸象，最终希望可以全面从每个角色的视角全面分析。（这点可看成是上点整个团队参与的延续）

某公司管理层一直非常关注度量，对各过程，包括需求、开发、测试等，每个过程都订了七八个 KPI 指标，并每月监控。但各个过程按指标做好不一定代表总体效果会好，必须所有相关岗位和部门一起讨论，大家才能从全局看问题。要达到像丰田减一个零的目标必须研究总体过程，并需要过程里的所有岗位人员都参与。

回顾让数据提供者能即时收到反馈

有数据才具体，谷歌不是开会都用两个投影仪吗？所以之前要尽力收集数据，软件开发与工业生产不一样，大部分的数据必须开发人员自己收集，不能单靠机器/系统（例如：返工工作量）。除了硬数据（如缺陷率），也要看软数据（如组员心情、感受等）。因为不单是培训，要大家讨论出行动计划，定目标，所以大家立马看到是否合理，立马给反馈，不是靠邮件传达，前后来回要好几周。以上 4 点就是 2 天一起互动工作坊的重点，但不会有天上掉下烧饼，要有效必须做好准备和满足一些必须进入条件，详见附件。

== ==

过程改进不是两、三个月就出成绩，开始时，员工可以关于改进有想法，写文章分享，但要这个作为习惯继续下去，是需要有些新的想法和思路，这可以借助公司内部培训，比如有没有过程域相关的参考资料、内部分享会、学习小组，定期有内部教练的辅助指导，定期体检、和诊断。让员工容易共享资料、心得、经验教训的平台，如 Wiki，也可以促进过程改进。

定期与大家分享/汇报

客户完成了评估后，举办项目团队评估后总结会，安排了其中的 3 个项目团队汇报他们在过去半年，如何利用所学到的方法达到 CMMI5 级评估要求。非常高兴地看到他们已经掌握了我们老师所培训的技能——利用模板从需求计算功能点，然后从功能点估算出项目工作量、测试用例数，再利用小工具测量代码质量，并且预估最终的缺陷密度等。

3 位项目经理每人都针对以上内容发表了 1 小时的演讲，分享实战心得。

项目团队把学到的东西用于项目中，配合度量数据，取得具体的质量提升，且达到了相应的评估要求。



图 9-4

后记

12 月与现场咨询顾问对话

咨询顾问：“客户（A 公司）现在人手紧张，都忙于交付项目，可能希望先简单快速完成级别评估，后面才慢慢实行真正的改善。”

评估师：“建议继续坚持做好改善才可以。因为有评估才能有效驱动团队抽出时间来做改善，很多领导都会说请放心，我们先快速拿证，后面才真正做改善，但我从未见过企业做到。你觉得我发你的过程改进资料有实用价值吗？”

咨询顾问：“你建议的各种方法论，如果给一些较成熟的软件开发公司是合适的。只是现在各团队，实在太忙，有心无力。”

评估师：“所以问题核心是要腾出时间，这也是要有计划的目的。我们自己要相信他们员工和管理层是具备所需的能力。从首次见面看得出高层万总很有魄力，做事有决心，相信他有能力解决中层、部门经理不投入、不配合这个关键问题。其他需要解决的问题都会水到渠成。”

== == ==

咨询顾问与 COO 万总开会汇报，并了解到为什么项目经理王工第一位提交改进计划，原来他的领导一直关注 CMMI 的进展。万总决定立马内部开紧急会议并敦促其他 3 位领导，要求每个小组必须按照要求针对缺陷与返工每迭代做质量改进。

回顾过去 6 个多月，虽然启动了，也做了培训、定期开会，但一直没有任何实际的改进。所以必须具备关键成功因素，如：领导的关注和支持、团队有时间和精力推进、制定计划、安排人手、按计划执行、一轮一轮逐步改善，就是这么简单。但确实要做到并不容易。

附件

睡衣工厂实验 Harwood Manufacturing

二战后，在维京尼亚州有一家专门做睡衣的工厂 Harwood。

难题：

要按不同的季节需求，做不同款式的睡衣，但往往有些女工习惯了某种工作方式，效率很高，但是要她改变工作方式，就接受不了，很难转换。

实验——比较 3 种变更方式：

按传统方式，依赖工程师去设计每一轮的变更。团队只需要按设计去做

1. 每个团队有代表与管理层去讨论如何变更，然后执行
2. 自己去策划自己的变更，也提建议。也安排那些女工每周跟那些生产率高和低的女工一起讨论不同方式的优劣，自己讨论怎么改变

实验结果

* 第一组按传统强制要求她们改变，花了 3 个月的时间，并引起很多不满，效果也不理想：生产率降低了 20%，也发现 10 名女工里面有一位后面就辞职不做了

- 第二组效果中等，需要大概 2 周才恢复到本来的水平
- 第三组发现 2 天后就已经返回本来变更前生产力水平，后面慢慢的上升提高到 40%。例如某实验组就把本来一天 45 件，5 天后提升到 87 件——一个本来无法想象的数字，后面甚至提到 90 件，超越那些工业工程师本来的目标

粮食分配实验

二战时，美国虽然不是战区，也需要控制粮食供应。政府面对的难题：

- 怎么让那些家庭的消费习惯，满足战时的食物供应

比如当时有些食物是要分配的，如何让家庭多吃一些供应充足的食物，避开紧缺的食物。他们实验发现，首先要知道整个过程是家庭里哪个人做这一块的决定。

研究分析发现，绝大部分的家庭都是由家庭主妇决定购买什么、储存什么、怎么做菜，丈夫没有太多的意见，通常是由妻子决定。

实验设计：

- 把家庭主妇分成 2 组：
 1. 专家跟一组家庭主妇宣扬某种食物营养丰富，对家人的身体都会有好处
 2. 另一组家庭主妇没有专家指导，只是给她一些营养的数据，邀请主妇分成小组自己讨论决定怎么去做

结果：实验发现第一种方法没有效果，第二种效果却很好。

互动工作坊 (workshop)

原则

- 高层的支持 (Top Level Buy-in)：高层参与整个过程，也表示高层的重视。
- 从整系统入手 (Improve Whole Systems)：要企业改变成功，仅仅抓一个小模块是没用的，要求岗位相关人员都参与，让所有人都看到整个相关系统。
- 破冰-让人了解需要改变 (Unfreeze – need for change)：因为是工作坊形式的互动讨论，经过 2-3 天可以让所有相关人都了解改变的前因后果，和参与决策
- 整体参与 (Everybody solves problems)：整个团队都参与到讨论中，包括高层。而不是传统的高层策划再向下推广，不然没有参与，就没有承担的心，提高学员对改进的承担 (commitment)。
- 自己动脑筋：主持人/领导不会直接告诉你答案，而是通过一些讨论和分享的方式，引导学员自己找出痛点，分析并找出解决方案；每组都可以天马行空地出主意，但也会面对来自不同等级人员和评委的残酷批评
- 做实验/试点/游戏 (Experiment/ pilot)：把平常要半年到一年的改进项目压缩到 3 天，但关键的重点都一起讨论过。但也好像一次有奖游戏，激励团队的参与。

特点

- “把所有的相关者凑在一起 ‘Whole System’in the room’——所有相关人员都参与。例如：项目管理过程改进参与者不仅仅是项目经理，还有利益相关者，包括团队成员、供货商、客户代表、上下游、其他支持类部门等，可能都要参与

(目的：所有与系统相关的人都能对全局有概念，而不仅仅看到自己的部分)

- 顾问/老师不直接给答案/方案，只是引导、帮助学员找出方向。
- 用分组竞赛形式提高成员参与的积极性
- 所有人都可以提出批评和意见，不仅仅是从上到下的决策/教育，接纳所有的思路和意见（像一个镜子）
- 分 4-8 组，8-10 人一组，需要有一个大而光亮的会议室，方便小组互动，大白板和架子，各种颜色的水笔，墙壁可以贴上小组的白纸等
- 培训期间里都最好不受外界打搅，例如每人都上交手机



图 9-5：每组有自己的“地盘”大家并行讨论



图 9-6: 时间控制很重要, 老师在屏幕投出余下分钟控制不超时

成功关键要素

1) 预先要有充分准备:

- 建立行动委员会负责准备这次活动, 活动的具体设计, 在公司推进整个准备活动
- 主持人/协调人-对整个策划/准备/设计做辅导, 现场活动包括 (~ 6 - 7 天):
 - 2 天的策划会议
 - 1 天详细设计
 - 2-3 天工作坊现场辅导 (包括事前的场地准备)
 - 1 天的事后回顾/跟进

2) 参会人员是否能够代表各相关角色? (能否包括整个系统, 整体参与?)

3) 高层是否全力支持和参与?

参考 References

1. Weisbord, M. , *Future Search*
2. Hammer, M., Champy, J. *Reengineering the Corporation*

Part V

如何做好迭代回顾

怎样避免同类问题的重复发生？因为有很多问题背后的原因是整个系统的不足，而不仅仅是个人不关注、不努力，做好根因分析可以帮助我们找出背后的根因。

怎样做好根因分析？二八原则、五个“为什么”（5Why）等，大家可能都听过。在网上也可能找到相关的理论知识。但很多人还是有误解，没有弄清楚应该如何利用这些技巧来找出根因。

针对软件开发的过程改进，也可以使用根因分析，帮助软件开发团队避免问题的重复发生，以提升团队的质量和效率。

迭代回顾 (Retrospective) 可以让团队每走一小步（一个冲刺），便立马回顾过程中有哪些不足、分析根因，以便在下一小步中得到改善。

要做好迭代回顾，除了要了解根因分析的技巧，还要了解回顾背后的原则，配合团队互动练习，再配合工具做好数据分析，才能使团队有实际的提升，让老板满意。

这部分会讨论：

- 根因分析的主要元素。
- 软件开发团队应针对哪方面入手，才能最容易取得效果。
- 迭代回顾前应如何做好准备。
- 内部教练引导团队做回顾时应注意什么。
- 如何可延续，让团队（甚至组织级）开始建立标杆。

Chapter 10

二八原则

团队成员协作，利用项目数据，分析根本原因，制定纠正措施，并立马尝试，判断是否有效，是改善的“基本功”。10-12 章会探索里面的注意事项，13 章会看两家公司的实施情况和常见问题。

如果已经获得高层支持，高层赞同目前返工工作量太多并愿意投入资源进行过程改进，我们首先应该针对哪些方面进行改进呢？

软件开发的特点是，超过 95% 的成本都是人力成本。因此，我一般会从二八 (20/80) 原则开始（附件里有二八原则简介），识别应针对什么做改进，才能最容易取得效果。软件开发里什么工作的占比最高？

请把过去一年的软件开发项目，按照不同工种占项目总工作量的比例，从多到少排个序。

1. 编码与代码设计
2. 交付后的所有工作，包括维护、更新与缺陷修正
3. 交付前的评审，静态扫描，测试与缺陷修正
4. 项目管理与监控

可以参考软件开发度量专家卡珀斯 • 琼斯 (Capers Jones) 先生在 2012 年的研究（详见附件中的“开发项目工作量 (成本) 分布”），看看你的选择与典型分布有多大差距。

软件开发项目最大的工作量通常是花在找出与修正缺陷上（开发里的“测试与评审”），Capers Jones 发现，对于一个超过 10,000 个功能点、计划使用 25 年的大型系统来说，有接近一半的工作量是与找出并修正缺陷相关。

很多项目中的缺陷大部分还是到后期才发现，这导致了大量返工。如果能提前发现并解决这些缺陷，就可以大大降低研发成本（不仅仅是提升产品质量）。

对于一个软件项目来说，发现缺陷最多的是在系统测试阶段，其次是在验收测试阶段（很多公司都是如此）：

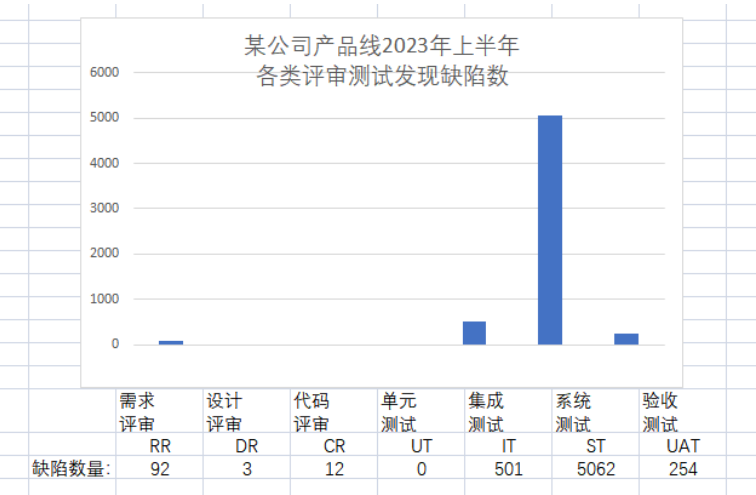


图 10-1

但很多开发人员误以为编码是项目的主要工作，却忽视了大量因质量问题引起的返工。

前面在“获取高层支持”那章里缺陷越后面发现，返工量越高，如果是在前面评审或单元测试发现的缺陷返工量不会超过一小时，但到了系统测试或验收，返工量平均不会低于 20 人时。所以在软件开发，大量缺陷在后面测试或客户使用时才发现不单影响产品质量，也会导致大量返工。

只有当管理层意识到尽早发现并排除缺陷的重要性并引起重视，公司才有机会改进。

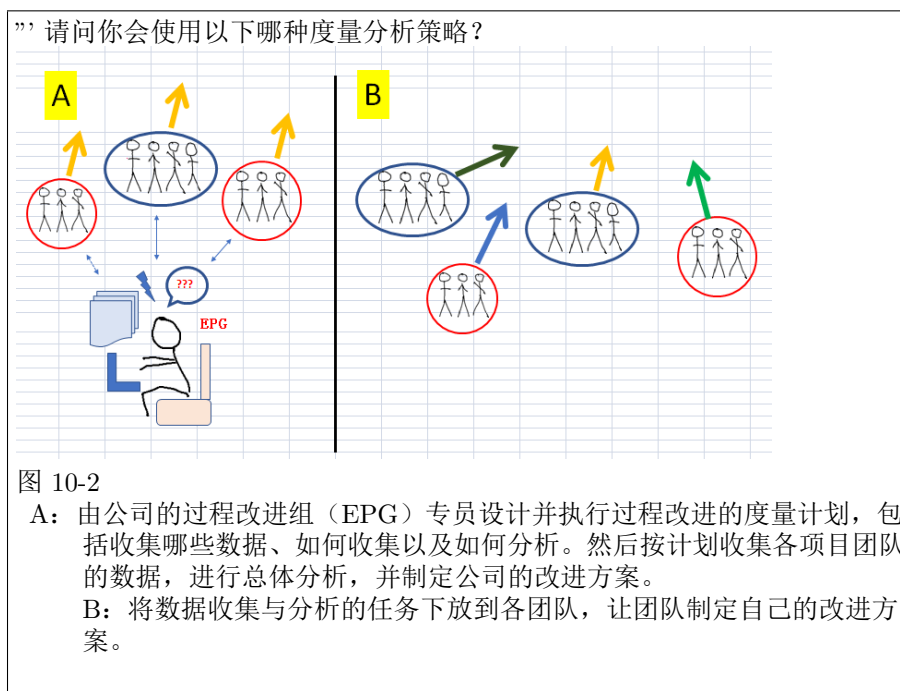
有些偏业务的领导可能会质疑，客户不一定关注缺陷密度，只要达到可接受的水平就可以了。

我解释：“在软件开发中，减少后期测试才发现的缺陷，不仅仅能提升产品质量，更重要的是能降低研发的总工作量，所以最终能帮助公司省钱，减少开发人员加班。”（详见附件“公司高层不一定关注质量”）

怎样开始

没有度量便无法谈改进，所以应先考虑如何采集修复缺陷相关的数据。然而，在探索数据收集与分析之前，我们更需要先确定度量分析的策略。

但在探索数据收集与分析之前，我们更需要先想好度量分析的策略。应怎样做？我与质量部经理张工和项目经理讨论了上面工作量排序和那阶段发现缺陷数最多后，现场问他们以下问题：



项目经理：我认为度量与分析需要专业知识，要由专人来做，我选 A。

我问：但提供数据的人（图中 EPG）需要很长时间才能收到反馈。例如，从团队成员提交里程碑数据，到项目经理填报，再由度量分析人员收集、整理多个项目的数据，可能需要一到两个月时间。再经过数据分析，管理层评审，最终到团队成员收到反馈可能共两三个月。由于每位提供数据者都希望尽快得到反馈，如果反馈的时间很长，那么团队成员还会有动力继续收集数据吗？

项目经理：估计确实可能会导致缺乏动力。

我：无法确保数据的准确性，会导致分析没有意义。A 场景很难确保数据的正确性。但 B，因团队自己分析数据，会好些。

质量经理：从去年开始我们进行度量分析以来，就发现员工会对数据造假，导致结果失真。度量什么，他们就造假什么，所以我们想通过工具代替人工，不仅能提升效率，还能帮助判断数据是否合理。现在我们的主管很反感度量，因为每次度量就会有人造假——大家了解算法原理后，就开始造假。比如我们搞了红黑榜，但很多人聪明得很，就会想办法作弊。我们有很多数据统计分析，比如看测试用例与需求的比例。尽管客户发现的缺陷比例降低了，但由于我们对质量特别注重，产品经过多年的逐步演化，过程很复杂，导致软件缺陷的修复很耗时，客户不太满意。而且公司要求交付的频率要比以前高了很多，我们团队做这些分析都忙不过来，所以希望利用自动化工具加快速度。

我：确实有不少管理层误解了独立的用途，当度量数据被用于公司考核指标，便里面出现你说的情况。度量分析人员需要什么数据，提供数据的人都会按理想的指标填写，反正公司级的总体分析人员也无法验证数据的准确性。

除了数据收集问题，你们这种总体分析策略可能还有另一个问题：没有考虑各个项目的特性并不一样。现在你们对几十个项目的总体趋势进行分析，但每个

项目的差异可能很大，根本原因很可能不同。例如，你上次展示过一些测试数据，同样是测试用例比例数，你们采集的数据范围从最低的 0.3 到最高的超过 200，但你们取平均值 5.1 来做分析。所以自动化工具不一定能帮到你。

但如果把数据分析下放到团队自己做，便灵活多了；也正因为他们有参与收集和分析讨论，过程改进组便可以节省很多沟通（数据收集）成本与失真。反而应该重新定位自己是内部老师，辅导团队怎么做好数据分析，效果会更好。

项目经理问：团队自己讨论便可以得到改进吗？不需要领导监督？

我：如果团队有能力，就应该放手让他们自己收集数据、利用数据进行分析并做出改进。但大部分团队还是没有做过度量分析，没有概念。所以质量经理应把自己重新定位为团队的教练，去辅导他们如何收集与分析数据，如下图的分工：右面的内部教练熟识各种度量分析技巧，辅导左面团队做改进。千万不要以为这项工作会比以前自己做分析更轻松，其实需要更熟悉整个过程，才能真正辅导好团队。要公司从定性管理提升到定量管理，最大的挑战是如何让项目团队动起来，所以应要让管理层了解并赞同需要让团队自主分析数据，自己寻找纠正措施。最终也要团队自己定目标，定计划，定期与上层汇报改进效果。例如，当你要健身锻炼减肥，也请了教练，最终因自己没有恒心，最终没有效果不应该把责任推卸到教练一样道理。

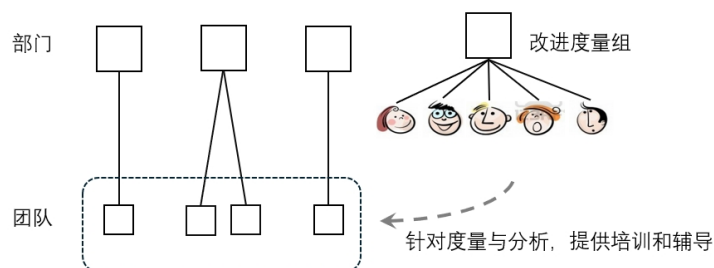


图 10-3

下一章会继续探索团队协作与分析根因；团队如何利用迭代冲刺数据，在敏捷冲刺回顾，做好根因分析，得到提升。

附件

二八原则 (80/20)

帕里托 (Pareto)，16 世纪意大利威尼斯人，他发现虽然威尼斯很富有，但财富并非平均分布，80% 财富在 20% 手里，他也发现很多其他分布都非平均。

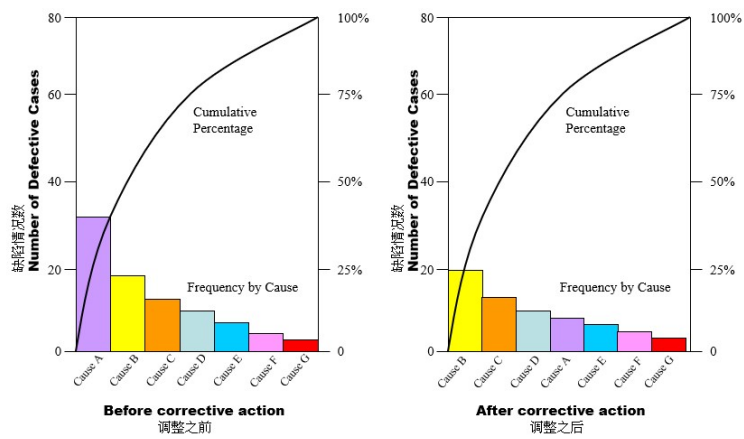


图 10-4

因为过程改进需要公司额外的资源投入，必须有针对性地找出最容易取得改进效果的原因，才可能拥有最大的成功机会。所以，应该利用二八原则去识别影响最大的几个因素。以上图为例，原本 A 类问题出现最多，针对这类问题做过程改进之后，A 类问题就少了很多，下一轮便应针对最多的 B 类问题做改进。有些人以为，只需要对某个维度做分析即可，并没有从多个维度进行分析。例如，某纸制产品工厂会计数据显示，8 成的产品问题相关成本都归属于 5 类。例如，某纸制产品工厂会计数据显示，因纸质量问题 (Broke) 被客户退货类的损失最大，共 5560 千美元，占总损失的 61%。（其他损失类包括材料成本高、生产线停顿等。）针对这类损失，发现里面八成是由 6 个产品引起（共有 53 个产品），共 4480 千美元，占 5560 的 80%。针对这 6 个产品把损失按缺陷种类细分，发现 B 产品容易断裂缺陷 (Tear) 的占比最高，612 千美元，我们就应该针对这一问题研究如何改善。

质量成本(千美元)							
产品 类型	Trim 裁切	Visual defects 视觉缺陷	Caliper 尺寸	Tear 撕碎	Porosity 透气度	Other 其他	Total 总计
A	270	94	-	162	430	364	1320
B	120	33	-	612	58	137	960
C	95	78	380	31	74	62	720
D	82	103	-	90	297	108	680
E	54	108	-	246	-	62	470
F	51	49	39	16	33	142	330
						总计:	4480

表 10-1

依据以上数据，可以按产品画帕累托图：

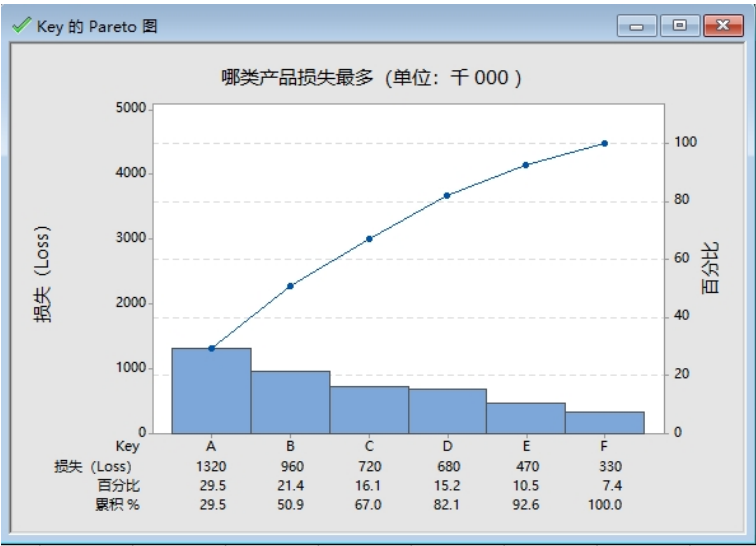


图 10-5
也可以按产品和缺陷类画帕累托图：

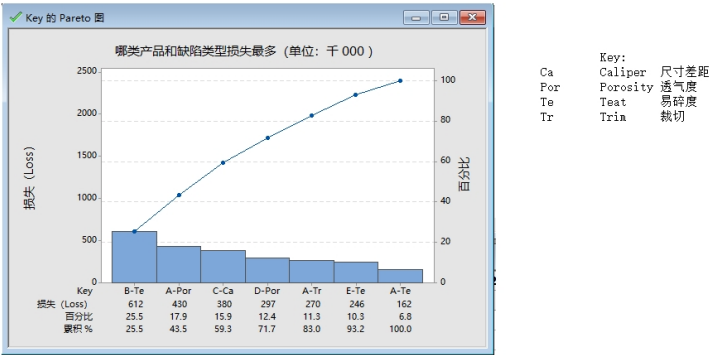


图 10-6
帕累托图除了能帮我们分析数据外，也能用于与高层沟通，让他们一目了然更好理解问题的主要源头，希望得到高层的支持。

开发项目工作量（成本）分布

参考 Capers Jones 先生 2012 的例子，汇总成以下比例：

	30%	25%	20%	15%	10%
66%	测试与评审 100 (26%)	编码 83 (22%)	文档 66 (17%)	开会沟通 50 (13%)	项目管理 33 (8%)
13%	发布后修改缺陷 20 (5%)				
21%	维护 32 (8%)				

表 10-2

测试与评审一般占开发工作量的 30%，测试与评审一般占总质量成本（包括发布后维护与改缺陷工作）的 66%，把所有工作量都加起来，测试与评审还是占最大 (26%)，编码第二 (22%)。

注意：测试与评审包括所有与缺陷相关的成本，包括单元测试、静态扫描、评审与相关的缺陷修正，而编码只包括设计与编写代码部分。例如有些人会觉得比例应该是开发 30%，测试和 Bug 修改 25%，需求和设计 20%，项目管理和沟通 20%，文档 5%。但如果按上面的定义，开发部分很可能已包括单元测试、静态扫描与改正缺陷工作，如把这些都归到测试评审里会变回类似上图的比例。

公司高层不一定关注质量

问：在传统 IT 的公司，无论是缺陷 (Bug) 率还是其他质量指标对业务的影响的相关性都不容易度量，只有重大事件才会让公司在客户面前失去信任，所以高层不会真正的重视质量。

答：理解，但软件 BUG 的暴露越往后，返工的工作量就越高，而且是几何级数的增加（例如在单元测试或评审发现都可以 1 人时内解决，在系统测试时发现通常要花 20 人时，在客户使用后才发现便更高）。但在很多 IT 公司，大部分缺陷都是在系统测试、甚至到验收测试才发现。如果能把软件缺陷的发现前移，把通过系统测试、验收测试发现的缺陷减半，便可以大量降低质量成本，提升开发生产率。所以我建议利用缺陷数估算返工的工作量来引起老板重视。（而不是仅仅说降低缺陷率）

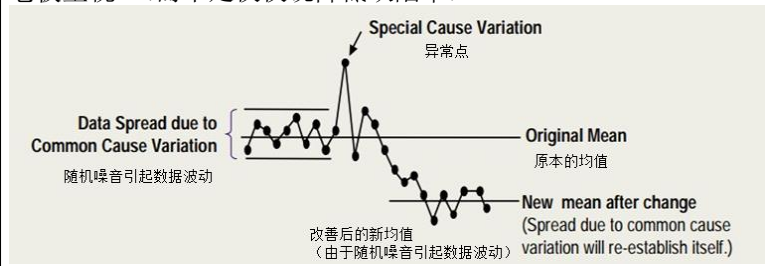


图 10-7

Chapter 11

分析根因

先用日本俱乐部里由 7 位年轻女服务员组成的 QC 圈在 84 年做改善的故事介绍根因分析的主要元素。

根因分析 (Causal Analysis & Resolution CAR) 的主要步骤:

1. 选择要分析的结果, 利用二八原则识别引起大部分问题的最主要因素
2. 分析背后的主要根因
3. 对应改进措施 (改进建议)
4. 评价改进效果
5. 总结成根因分析报告 (A3 报告)

可参考附件 A:“绅士俱乐部过程改进” 了解团队如何按以上步骤做过程改进的实例。

根因分析的误解

请千万不要误以为只要使用了根因分析的方法 (例如: 二八原则) 就能做好, 很多团队在开始时对其理解可能只是一知半解, 需要不断纠正和改进。

某公司过程改进组 (共 4 人) 对过去一年的项目进行了根因分析, 旨在改进交付质量, 减少交付验收时的缺陷数。

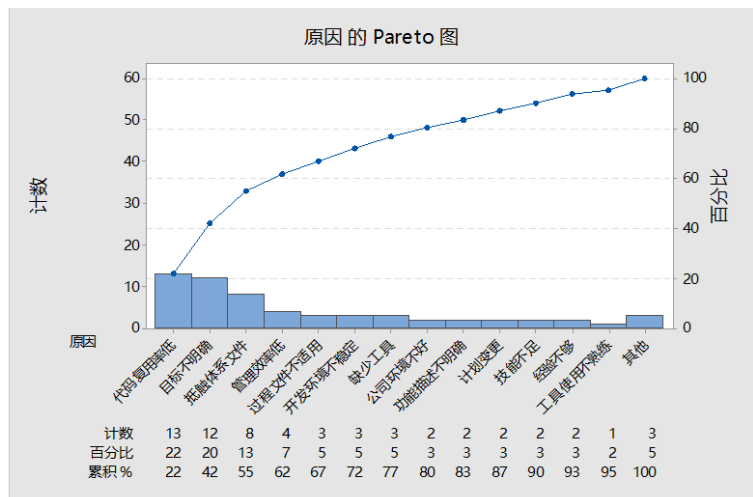


图 11-1

我：请问这帕累托图的依据？

组长：针对客户缺陷密度较高的现状，我们做了敏感度分析。发现需求引入的缺陷数跟它相关性最高，接下来我们使用鱼骨图分析，请 4 个人一起头脑风暴，共识别出十几项主要原因种类。然后按每人 3 票的原则，各自单独投票得出排序。最后根据二八原则，我们识别出前 3 项主要原因，接着，我们根据这 3 项原因又进一步发现，这些原因是导致需求缺陷的原因，包括流程图画不好等。

香港启德国际机场 1996 年航班延误统计：

Cause 主因	Freq次数
Late baggage to aircraft. 运输行李到飞机有延误	340
Bad weather. 恶劣的天气	240
Too few gate agents.登机口工作人员太少	66
Late cabin cleaners.清洁工打扫机舱晚到	21
Poor announcement of departures.出发通知混乱	12
Gate agents not motivated.登机口工作人员没有动力，影响工作	103
Late aircraft arrival to gate.飞机晚到	68
Late or unavailable cabin crews.空乘（飞机服务）人员迟到或因故不能上岗	21
Passengers bypass check-in counter.旅客绕过出登机牌柜台	62
Late passenger cutoff too close to departure time. 最晚乘客登机时间太接近起飞	44
Late food services.飞机餐饮服务晚到	37
Gate agents undertrained.登机口地勤人员训练不足	116
Desire to "protect" late passengers希望“保护”迟到的乘客	57
Late issuance of boarding passes.打印/发放登机牌发生延误	33
Late weight and balance sheet. 飞机重量平衡表（为了确保飞机重量平衡，保证飞机安全）迟交	40
Late or unavailable cockpit crews驾驶舱人员迟到或因故不能到岗	19
Confused seat selection. 座位安排混乱	66
Late fuel. 燃料延误	35
Gate agents arrive late at gate登机口工作人员晚到登机口	19
Late pushback tug.拖车延误	75
Checking oversized luggage.检查超大行李	20
Desire to help company's income.希望帮助航空公司增加收入	37
Air traffic control delays.航空交通管制导致延误	166
Poor gate location.登机口的位置不好	130
Mechanical failures.机械故障	26
Delayed check-in procedure. 登机牌柜台手续 / 过程发生延误	31
Gate occupied by a late departing plane. 停机位被前面晚点起飞的飞机占据了	22

图 11-2

然后我解释：

我们不应直接把几十条原因按他们对问题的影响度从最高排到最低做帕累托图，而应先对原因进行分类。以机场延误为例，我们需要区分哪些是不可抗力的天然原因（如天气原因），哪些是与闸口管理相关的问题，哪些是与机场打印登机牌相关的问题。例如，如果识别出与闸口管理相关的问题最多，后续就可以针对这些问题进一步分析根因。分析才能有针对性。（详见附件）

你们的分类和帕累托图依据的“数据”都不过是用数字来表达你们头脑风暴的主观判断，所以这种帕累托图其实与直接用头脑风暴做出的结论没什么差异。你们只是用投票数据使结果看起来“量化”了，但其实并没有客观数据支撑。你们的分类存在问题。例如只有“功能描述不明确”，难道“非功能需求描述不明确”（如性能、安全性等）就不需要考虑吗？

你们现在的分类有不少是重复的，同样一个问题有可能归属于 2、3 个分类。你们的度量操作定义也存在问题。例如，如何根据历史项目的数据判断哪些原因归属于“功能描述不明确”？

建议你们应该首先识别出与想要解决的问题相关的度量项，针对这些度量项收集数据，然后再做分析。收集客观数据才是根因分析的第一步。

按团队水平因材施教

“发现我讲过的方法总是需要重复解释和提醒，是不是我教的方法有问题？”某老师交流时问道。

“这很正常，绝不要怀疑方法本身，我也经常遇到类似的情况。实际上，这是一切学习的必经过程：起初只是根据学到的技巧初步应用到项目中，而要实现持久应用，不仅需要团队持续实验才能真正理解，还离不开‘奴、徒、工、匠、师、家、圣’这个不断学习的过程。”

下面是针对根因分析，与他分享各团队水平不同，辅导方式不同的故事。

=====

黄带水平

某家专门做企业管理产品的软件开发的 Y 公司：

总经理听完如能做好迭代回顾和根因分析，不仅可以大大减少后期返工的工作量、提升产品质量，还能够提高整体开发效率，很感兴趣，并说会要求工程部按这思路做改进。

8 个月后再次见面时，总经理说：“虽然我们现在已经要求团队多做回顾和根因分析，但每次迭代的系统测试缺陷数量还是降不下来。比如某产品线，每次迭代的缺陷数量大约在 50 个左右，也曾降到过 30 个左右，但不久之后又回到了之前的水平。能否跟我们团队谈谈，了解一下情况？”

===

我：第二条根因是什么？

团队答：编码人员把某个参数搞混了。

改进措施：提醒开发人员要注意，避免同类问题再发生

第一条是测试人员不清楚需求，没有设计出充分的测试用例，导致未能在测试阶段预先暴露问题

改进措施：加强人员的培训，更了解业务需求

我：测试本身是不会产出缺陷的，为什么你们识别测试是根本原因的源头呢？

===

以上是团队交流的节录。

从他们提供的缺陷数据、缺陷分析和系统测试报告等材料，发现他们并没有真正找到根本原因，只是对缺陷进行了分类，并针对缺陷暴露出的问题，提出了修正措施。

团队自认为在复盘时已经做了根因分析，但虽然有参加过根因分析培训，但还是误以为对应发现的问题都做了处理便分析了根因。如果他们听完夜总会例子后，再小组分析机场延误数据，就能帮助他们先了解根因分析的基础。

=====

X 公司主要为全国不同的医院开发医疗行业的软件产品。六年前，公司规模还不到五百人；四年前，公司办公地点扩大了一倍，占地 3000 平方米，总人数也增加到接近 900 人。尽管公司骨干比较稳定，但新人的流动性很大。公司制定了很多量化指标，如需求一次性通过率、开发速度、测试缺陷密度等，并不断

监控，但并未取得明显改善。

技术总监问：“请教一个一直困扰我的问题——如何激励团队，让他们更有积极性？”

“现在你们员工的收入底薪和奖金的比例大概是多少？”

总监：“大概七成底薪，其他的就按每人每年的表现分发。”

“主要依赖主管的打分吗？我完全可以理解为什么是这个结果了。因为在这种安排下，员工的收入既不会太差，但也不会特别高，所以他们自然缺乏动力去改进自己的工作表现。”

总监解释：“也不完全依赖主管的主观判断。我们制定了很多与绩效相关的度量指标，以根据这些指标客观地计算员工的奖金多少。”

“理解了，你觉得这样便能客观吗？一旦把度量与绩效/奖金挂钩，就会导致收集不到真实的数据。因为只要数据与收入有关联，高层需要什么数据，下面都会提供，这完全失去了度量的本意。收集度量数据本来希望像镜子一样，让团队可以看到现在的真正水平。团队成员相互合作非常重要，所以日本公司通常不会发奖金给某个人，而是发给团队。团队的奖金应该与团队为公司提供的价值（换算成金额）挂钩，这样才客观透明。大家可以预估到如果团队有了提升，能为公司赚多少钱，自己收入大概会增加多少。除了金钱上面的奖励，也不要忽略非金钱的赞赏，例如，定期让项目团队分享改进成功的故事，也可以帮助团队持续改善。你们每个开发团队里面都有明确的分工：需求、开发、测试等都有相关的度量指标。但岗位之间缺乏交流，导致团队的整体质量和效率没有显著提升，例如团队的缺陷密度和返工率一直没有明显下降。”

接下来，我讲述了 60 年代美国费城的 Weisbord 先生针对印刷公司订单管理部使用 XY 理论改革团队的故事：如何从每一步骤都要按小主管 (supervisor) 安排的传统管理模式 (X 型管理)，变为由 4-5 人自我管理的独立团队 (Y 型管理)，最终把每天处理订单量提升 50% 的成功改革故事。

总监说：“听完这个故事，确实感觉我们团队的管理模式类似于 X 型管理。虽然我们团队每两周进行一次迭代，但没有真正做到团队合作。测试人员只负责测试，开发人员只写程序，然后交给测试人员去测试。因此，从这个角度看，或许每个过程的指标，如测试和开发都很好，但团队的整体能力没有任何提升。”

“从这次观察来看，每个迭代中，系统测试的缺陷数保持在 40-60 个左右，六个月前是这个水平，现在依然如此，现在还是这个水平，没有任何进步。为什么？因为大家缺乏团队协作和互补的团队精神，导致没有改进的动力。各岗位只顾管好自己，确保指标达标，不被扣绩效就行。”因为我也对团队缺乏具体改进感到失望，所以不管总监是否高兴，我直言不讳地指出问题。

如果没有可信的数据，大家了解了根因分析的基础也不会促进改善，应先解决怎样能收集到真实数据这问题。

绿带水平

虽然 X 公司团队已经接受了根因分析的培训，并且在每次迭代回顾中都在使用，但依然遇到挑战。以下是项目团队分享会里的交流：“第一条根本原因写的是‘开发人员不理解需求，没有按照规格要求，导致后续测试不通过。’请举例说明。”

“我们这个项目是系统升级，针对医生在手术前的很多准备工作，不仅包括麻醉

师，还包括护士、设备等都需要准备就绪。而且大小医院要求不同，大医院分工很细，小医院可能由一个岗位负责多个任务。所以，要避免此类缺陷，首先要画出所有可能的处理流程图，明确有哪些不同的配置项和参数，并与熟悉流程的产品经理（甚至医生）确认。”

“这些内容是否都应该归入根因分析的范围呢？还记得我们之前讲过的 KJ 分析方法吗？（详见附件）所有团队成员需要用便利贴写下所有相关的因素和问题，并在大白纸上画出它们之间的关系，这才算是完整的根因分析。你现在写的“根本原因”只是浮在水面上现象。

再看对应的纠正措施：记得我们一直强调，改进计划也要像项目管理一样，用 WBS 将任务分解成不超过五天的可执行部分，并明确任务负责人。每个任务都必须有具体的产出物。如果只简单写成“一个月后由开发组长负责完成”，但由于缺乏实际行动，最终一个月后很难取得理想的效果。”

	根本原因	纠正措施	行动计划
1)	开发人员不理解需求，没有按照规格要求，导致后面测试不通过	开发与需求变更开发前的沟通，加强相关业务培训	7.17（一个月后），张 XX（开发组长）
2)	接口测试出错，没有使用正确接口，不清楚接口参数（2040、2021）	代码评审时检查接口相关检查	7.17（一个月后），张 XX（开发组长）
3)	开发没有充分自己测试好，很多不一致，但可能出现的情况，报错或显示效果不好，开发要加强自测（2419、2421、2432、2433）	对开发人员加强相关测试培训	7.17（一个月后），李 XX（项目经理）+ 王 X（系统测试）

表 11-1

“我们分析根本原因都应有要数据支撑，不应单依赖个人主观判断。这里第三项根因只是写了四个缺陷号，是什么意思？”

“因为我们没有时间把几十条缺陷都列出来分析，我们只能抽几条把缺陷号记下来。”

“你们总共多少时间做迭代回顾？”

“一小时左右。”

“明白了。确实时间不够，所以管理层必须给你们充分的时间，不然的话，无法做好根本原因分析。”

“第二条写‘没有使用正确接口’是什么问题？”

“这系统有很多外部接口与医院现有系统对接，有些缺陷是因为开发一测试用的接口版本不同，导致测试时不通过。”

“记得我们一直强调，开发、测试与客户的生产环境必须一致，因此这是一个关于环境的问题，接口只是环境的一部分。那么，为什么会出现版本错误？是否不应仅靠人工记录最新版本的对应关系？你们现在代码开发版本在 Git 管理；但相关文档，包括配置，在 SVN 管理。依赖人工记录这些之间的相互对应，很容易出错，这才是根因。”

虽然团队经过培训，已经理解了根因分析的原理，但还是不太了解如何用于实际工作中。这类的定期分享会，加上咨询老师的辅导会有帮助。

分析全过程

为北京某企业进行差距分析时，发现大量缺陷在系统测试阶段才被发现，因此建议加强迭代回顾中的根因分析。

质量经理：你说我们上次做了回顾还没有找到真正根因，但不知道如何改善。

我：数据是根因分析的基础，若要做好分析，首先要有充分和足够数据。

例如，团队只依据系统测试缺陷数据进行分析，由于没有缺陷对应的返工工作量的数据，只能从缺陷类型的数量来分析，这使得难以识别出影响最大的缺陷类别。

此外，没有考虑从需求设计开始进行评审、编码人员的自测，也没有关注各阶

段的缺陷排除率，结果大家只能从系统测试缺陷的症状进行分析，结论是编码人员疏忽或能力不足，而不会考虑这些缺陷是否可以在早期预防和避免。因此，在每次迭代回顾时，我们必须给团队足够的时间来收集数据，绘制全流程图以识别失效点，这对分析也有帮助。流程图不仅能帮助分析过程，还能帮助找出根因。绘制流程图后，就可以利用 FMEA 方法针对每个失效点找出根因和相应的纠正措施。

某公司专门为电信供应商开发客服管理软件系统。由于客户对软件质量要求较高，因此每次发布后都会对发现的问题进行分析。某些与软件开发相关的问题不易查明根因，例如某次发布后，发现报表中的数字对不上。然而，这些错误并不是开始使用时就出现的，刚开始使用时一切正常，但时间长了错误就显现出来了。

团队花费了大量时间分析整个过程，最终发现问题的根源在于某些内部接口未能充分考虑到传输的数据是否正常。部分数据虽然有误，但未被拒收，这导致系统在初期表现正常，但随着有问题的数据逐渐累积，最终引发了报表错误。

在讨论如何避免类似问题时，团队提出了以下改进措施：

```
graph LR; A[设计/编码] --> B[代码评审]; B --> C[单元/集成测试]; C --> D[系统测试]; D --> E[客户验收使用]; A --- A1[编码规范和培训]; B --- B1[增加检查项]; C --- C1[加强接口测试]; D --- D1[压力测试场景]; E --- E1[报表数据有误!];
```

图 11-3

按照系统产品集成的流程思路，可以参考 FMEA 的思路识别哪些环节可以预防问题的发生。为了减少类似问题的出现，可以在集成测试和系统测试阶段增加类似的场景，并加大压力测试的力度，以提前暴露潜在问题。在集成测试中，全面测试所有接口传输的数据也能有效防止问题发生。如果我们做好了前期的设计和代码评审（问题的根源往往与设计有关），也能预防类似情况的出现。例如，通过完善评审检查单的检查项，以后应该能够避免同类问题再次发生。

从这个实例可以看出，首先绘制总体流程或路线图，并识别每个环节的潜在风险，有助于更好地挖掘问题的根本原因。

如何分析与对应各类由于人员引起的缺陷

- 可大致分为 4 类跟人相关的缺陷和对应措施

类	分析后发现缺陷模式	可能的根因	解决办法
1	某些缺陷，发生模式是随机的，不容易看出哪类缺陷经常发生。	错误是由于疏忽造成的。	完善过程，防止人为错误。
2	某些缺陷，有些人员经常犯错，但另一些人员总是“很好”。	错误是由于缺乏技巧造成（能力、知识、技能等）。例如某些窍门只有部分人知道。	挖掘和推广窍门，避免出现小众才知道的秘密。
3	有些人员总是在各种各样的缺陷上容易犯错。	潜在的原因：故意不遵守标准。天生不能完成这类任务。缺乏培训。	激励措施。换人。提供培训。
4	某些缺陷，所有工人都容易犯错。	缺陷应该由管理层控制。	语言标准化；提供翻译，词汇总表。达到自我控制标准。

表 11-2

实例 1：

表 11-3 统计了 6 位技术人员，按 29 种缺陷统计：

缺陷类型	技术人员						总计
	甲	乙	丙	丁	戊	己	
1	0	0	1	0	2	1	4
2	1	0	0	0	1	0	2
3	0	⑬	1	0	2	0	⑮
4	0	0	0	0	1	0	1
5	2	1	3	1	4	2	⑬
6	0	0	0	0	3	0	3
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
27							
28							
29							
总计	6	⑳	8	3	⑳	7	80

表 11-8

分析：

* 虽然第 3 类缺陷出现共 19 次（最多），但大部分是源自员工乙，但他除了犯类型 3 缺陷外，他其他方面的缺陷数很少。很可能是他误解了里面某些关键过程（表 11-8 的类型 2）

- 第 5 类缺陷属于上表的第 4 类，所有员工都有犯错
- 员工戊的缺陷数最多，占了缺陷总数接近一半，是表 11-8 中类型 2 的例子

实例 2：来福枪故事

- 是上面类型 3 的标准例子

软件开发绝大部分靠人来完成，也可以从“人”的维度分析，例如：

- 下面是 22 位来福枪组装工人从 2 月到 7 月的缺陷数统计，他们每人负责完整安装一支枪：

（缺陷：试验后无法打开，取出空弹壳。导致必须从组件重新安装。）

组装工人	3月	4月	5月	6月	7月	8月	总计
1	4	1	0	0	0	0	5
2	1	2	0	5	1	0	9
3	3	1	0	3	0	3	10
4	1	1	0	2	2	4	10
5	0	1	0	10	2	1	14
6	2	1	0	2	2	15	22
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
17	18	8	3	37	9	23	98
18	16	17	0	22	36	11	102
19	27	13	4	62	4	14	124
20	6	5	2	61	22	29	125
21	39	10	2	45	20	14	130
22	26	17	4	75	31	35	188
总计	234	146	34	496	239	241	1390
%缺陷百分比	10.6	6.6	1.8	22.6	10.9	11	10.5
最好的5位	9	6	0	20	5	8	48
最差的5位	114	62	12	265	113	103	669
比率	13	10	∞	13	23	13	14

表 11-4

发现每月的缺陷率差异很大（从 1.8 到 22.6）；但发现 6 个月里最差 5 位与最佳 5 位缺陷数的比例差不多。所以不是个别人员能力的差异，而应是有原因。

观察最佳和最差工人的工作方法发现质量好的工人都会用金属打磨器对负责的逐渐打磨。最差的工人都没有这样做。

总结

利用机场延误数据做小组练习，了解根因分析的基础后，可以用 FMEA 分析全过程找根因，也可以从人员的缺陷分析找出与人员相关的根因。

是否表示如果能把把握好各种根因分析技巧，便可以避免问题或灾难发生？

太空穿梭机（航天飞机）灾难（Space Shuttle disasters) 1986: 挑战者号（Challenger）航天飞机升空后，于发射后的第 73 秒爆炸，机上 7 名宇航员全部罹难（相关视频可以在网上搜到）。直接原因是用来密封火箭液态氢气燃料缸的 O 型橡胶环在摄氏零度低温不能及时膨胀。但这事故并非偶然。升空前，供应商已经警告过低温会影响橡胶的性能；以前的实验也发生过橡胶环有些部分半径只能达到预期的 2/3。但管理层为了按计划预期升空都漠视这些安全隐患预警。

2003: 哥伦比亚号（Columbia）航天飞机在返回地球时，因为左翼的隔热保护胶在 16 天前升空时被固体火箭助推器外层脱落的乳胶击破损坏，导致航天飞机返航时进入大气层的第 1000 秒钟，在 35,000 米高空因过热解体，7 名宇航员全部罹难。

Ride 在 2003 哥伦比亚号灾难事故调查回顾说：我很诧异这两次事故的原因非常相似（她也是挑战者号事故调查组员），86 年引起挑战者灾难的陋习又再出现：为了赶上计划升空日期，而忽略了之前性能报告的发现，没有注意前线技术工程人员提出的安全隐患，也缺乏安全保障措施。其实从哥伦比亚号在 1981 年首次升空开始，每次都有出现升空时燃料缸外的保护乳胶掉落的故事，但管理层一直都没有关注，工程师因为赶进度，每次升空前，都没有时间预先处理好。

虽然调查报告有找出事故的根因，但过了几年后，管理层只关注项目的成本与进度，忽略了质量与安全，没有再注意调研报告指出隐患，事故还是重复发生。

NASA 资源丰富，内部不缺经验丰富的工程师，工程师都了解各种根因分析技巧，但管理层为了赶进度，忽略了多次质量预警，最终因为发生了这两次致命事故，航天飞机计划最终被叫停。所以如果领导只关注进度，忽视质量，缺少风险意识，团队都懂得应如何做好根因分析，也不会有作用。

附件

机场延误数据分析

行李延误数量最多（340），但如果按以下分组：

- 闸口
- 行李运输
- 飞机（包括机组人员，燃料等）

发现闸口相关的总数反而比行李运输高。

非机场可控	闸口	行李运输	飞机	登记牌	清洁、餐饮	其他	
240		340					
	66						
					21		
	12						
	103						
			68				
			21				
						62	
	44						
					37		
	116						
	57						
				33			
			40				
			19				
166				66			
			35				
	19						
			75				
		20					
	37						
	130						
			26				
				31			
			22				
总数:	584	360	306	130	58	62	1500
	39%	24%	20%	9%	4%	4%	

前 3 类的总数：83%(=39% + 24% + 20%)

FMEA 实例

(FMEA: Failure Mode Effects Analysis)

大家可能都经历过因为没有管理好时间而导致迟到的情况。以坐飞机为例，从离开酒店到登上飞机的过程中，很可能因为一些风险导致最后没搭上飞机。当你出发时，可能选择不同的交通方式，如出租车或机场大巴。如果你不能在起飞前 45 分钟到达机场办理登机牌，你就无法登机。但即使你拿到了登机牌，也有可能最后没能搭上飞机，因为机组严格执行起飞前 15 分钟关舱门的规定，所以你还需要在起飞前 15 分钟到达登机口。

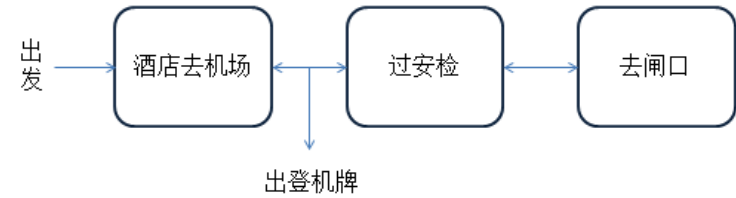


图 11-4 登机过程

要赶上飞机其实是个过程，中间有很多环节会导致最后失败，所以我们可以通 FMEA 过程分析来看如何减少失败的概率。

Process / Product Failure Mode and Effect Analysis (Process FMEA)															
Process or Product Name		1	Process Responsibility		1		FMEA Number		Page 1 of 1		Prepared By		FMEA Date (Orig.) (Rev.)		
Model Year(s)/Vehicle(s)		Key Date:													
Core Team:															
Process Step	Potential Failure Mode	Potential Effect(s) of Failure	Severity	Potential Cause(s)	Occurrence	Current Process Controls	Detection	RPN	Recommended Action(s)	Responsibility & Target Completion Date	Action Results				
											Actions Taken	Severity	Occurrence	Detection	
2	4	5	8	6	9	7	10	11	12	13	14	15			
酒店去机场	赶不上45分钟前柜台取登机牌	赶不上飞机	10	堵车	6	出发前向酒店前台预计时间	6	360	出发前（网上地图）查询预计时间（跟前台可能随变说	1/ 统计时间 2/ 预留上同查					
过安检	赶不上15分钟前到达闸口	赶不上飞机	10	寄存行李要开包检查 安检太久	2	出发前向酒店前台预计时间 自己看手表控制时间	8	160	必须起飞前35分过安检	1统计时间 2/ 换数字手表					
				登机牌遗失，要从打	2	自己带好	8	160	个人控制都放在一固定地方（某一口袋）						
去闸口登机				安检后去闸口很远	6	如熟悉机场，靠经验	2	120	预先问机场地勤人员，估计所需时间						

表 11-6 FMEA 例子

		Severity	Occurance	Detection
高	10	危险没有警告	非常高的 几乎不可避免的	不能检测到或非常低的检测概率
		丧失主要功能	很高，反复失败	远程或低检测概率
		丧失次要功能	中等的失败	低检测概率
		小缺陷	偶尔的失败	中等检测概率
		没影响	极低	必然的概率
低	1			

表 11-7 打分参考

以第一个失效为例：如果发生就会坐不上飞机，所以严重性最高，评分为 10。发生的概率比较高，评分为 6。是否容易预防、预警？由于不熟悉当地情况，加上主要依靠酒店前台提供信息，而有时他们也不清楚，所以评分为 6。RPN = 10×6×6=360。

从以上登机的例子可以看出，FMEA 是以整个过程进行风险管理的。比如在出发前，就要查询一下各个交通工具要花费的时间，例如，在出发前，需要查询各个交通工具所需的时间。在南京，从酒店坐地铁需要转车，要花费一个小时以

上，如果时间紧张就会来不及，那么宁可多花钱打车，以控制风险。
即使拿到了登机牌，还需要经过安检并到达登机口。有时机场很大，到达登机口也要花很长时间，因此需要提前问好路径并计算好时间，避免误点。
如果从安检到登机口需要超过 10 分钟，那么就需要在起飞前 35 分钟通过安检，才能来得及。这些都可以通过 FMEA 的形式把整个坐飞机的过程识别出来，找出各个阶段可能出现的问题，从而知道如何控制。
以这个坐飞机的风险为例，我自己就多次没赶上飞机。原因很多，但总结一下，都是因为习惯没有改过来。我应依据以往差点赶不上的经验，回顾一下，确保每一个过程都控制住，就不会后面再次出问题，这个和企业做风险管理概念一样。有度量才有管理。人和公司一样，很多事情看似是自己主动去想的，其实很多都是潜意识的习惯。如果没有设定一些量化的控制手段，就不会提高这方面的风险意识，还是会错过飞机。因此，我通过回顾以往几次搭飞机的情况，把每次到达机场的时间和到达登机口的时间记录下来（详见下面图 4）。
上面是机场柜台关闭（45 分钟）前到达时间，下面是关闸口前到达时间（分钟）（X 代表赶不上飞机）：

航班	NJ-SJZ	HKG-CD	CD-SJZ	SJZ-CD	CD-BJ
到机场	x	12	10	6	10
到闸口		1	2	1	x

表 11-8

最近一次赶不上飞机是因为未能在关闸门前到达，而之前也有两次类似情况——我刚到闸口，就到时间马上关闸了。
如果我把经验教训记下来，下次做好时间管理，就会避免后面的延误：有了数据，我们便可以更有效在回顾时做好根因分析 (CAR)。
以这登记延误风险为例，可以使用 FMEA 分析每个失效点，例如过安检（因没有预留足够时间）和从安检到闸口所需时间太长等的发生概率都很高（前者为 8，后者为 6）。
为了避免问题再发生，就要定一些具体的计划，最终希望把误点减到零。
在登机这个环节，可以利用以下有效方法/工具来帮助改善：
* 每次出发去机场前都查询各种交通方式的时间与风险（概率），预留足够时间，降低失效发生的概率
* 拿到登机牌后，计划好在起飞前 35 分钟通过安检
在多次错过飞机后，我发现传统手表没有正负 5 分钟的准确性，而使用电子手表可以精确到正负 1 分钟，这有助于更好地把控时间。



图 11-5 平常用于培训/评估计时的电子钟

经过这次误机，我就买了个电子手表，取代传统针式手表，希望对日后不迟到有帮助。

效果：这件事发生在 2019 年 9 月，之后我按这些计划行动，再没有出现赶不上飞机的情况。我每次都提醒自己必须在起飞前 60 ~ 90 分钟到机场，30 ~ 45 分钟前到闸口。分析的目的是提高个人风险意识，避免问题发生。

Chapter 12

软硬兼施

某次量化根因分析培训里，我跟各现场同学说：“前面我们说可以用鱼骨图分析根本原因，但我不建议在迭代回顾时用它找根因。知道是什么原因吗？请问你是读哪所高校？当时是否自己选的？”

“XX 大学。当时是我自己挑选的。”

“现在回想，是否觉得 XX 大学挺不错？”

“是的。”

“如果是自己选择的，都会觉得比其他好。1956 年，Brehm 决策影响实验（详见附件）验证了这心理。所以如果用鱼骨图分析根因，大家都会觉得这只是组长的分析，并非我自己的想法，后面便没有动力执行。”

让团队成员都积极参与

前面的粮食分配实验验证了团队只有兴趣执行自己集体讨论的分析结果，而非听从“专家”意见。所以若要团队做好迭代回顾，取得效果，要注意必须让大家参与，一起分析、讨论，才有动力采取行动。建议使用 KJ 分析法（详见附件）：每人都有一支水笔 + 便利贴，一起找根因，让每人都可以写上自己的意见。如果能让每位团队成员都有充分机会表达意见，不但能集思广益，做到更好的对应方案，也能提升团队后面执行改进方案的积极性。

以下经典心理实验解释了为什么 KJ 分析法，让每位参与者写便利贴这方法，能改善平常大家围着大桌或组长在白板上画鱼骨头等头脑风暴方法的不足。

想象你在大学读书时被邀请参加某教授的实验。你进入教室，里面已经坐了 4 位年龄较大的同学，都不认识。
教授投出下图：

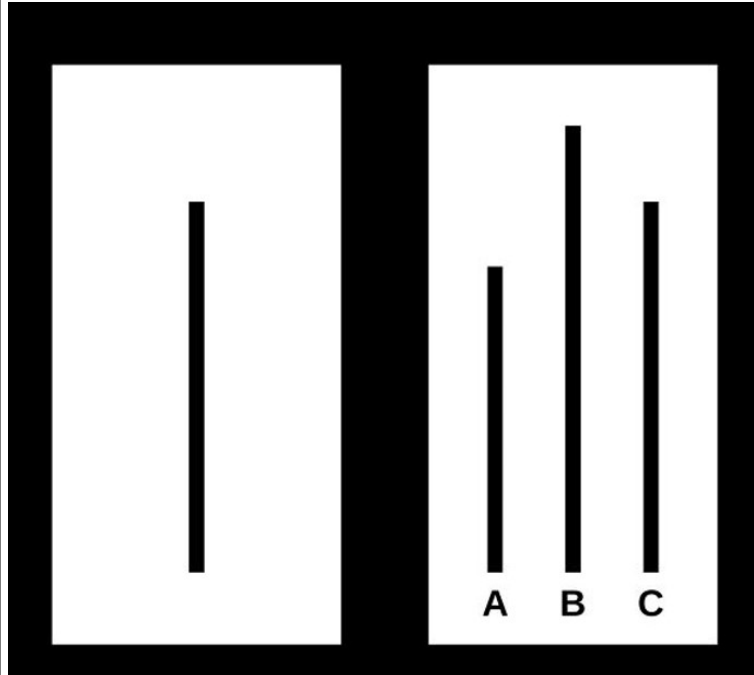


图 12-1

然后问：“右面 ABC 三条线，那条的长度与左面的一条最近？”

第一位用了 10 秒钟仔细考虑后说：“我选 B。”

第二位考虑后也说：“我选 B。”

第三、四位也选择 B。

现在轮到你（第五位），你会选什么？”

“C。”

“你选 C 的时候心里怎么想？”

“有些疑虑。虽然很清楚正确答案应该是 C，但为什么其他 4 位都选 B？会否有些他们懂我不懂的秘密。”

“这经典群众压力心理学实验从 50 年代开始已经重复做了几百次（详见附件），如果在真实实验环境下，如果前面有超过 3 位都选了错误答案，参加实验者会有 30% 错误率，证明大部分人都会受从众影响。”

做鱼骨图分析时都是某人（通常是组长）拿着水笔自己说自己在墙上的大白板上画，大部分的原因其实都是组长自己说自己写，其他团队成员都不会主动说自己发现的原因。但如果给每位参与者便利贴和水笔，大家都愿意把想到的写出来。



图 12-2

回顾流程

回顾流程冲刺可以按以下五个步骤进行，确保每位成员都全心投入参与，并通过分析形成行动，达到改进效果：

设置舞台——“热身”，获取对游戏规则共识

1. 收集数据（除了收集“硬”数据，如缺陷、工作量、进度偏差等，也要收集“软”数据，如团员感受）
2. 分析，找出根因
3. 决定做什么（必须明确下个迭代的具体改进行动到人、任务、时间）
4. 结束

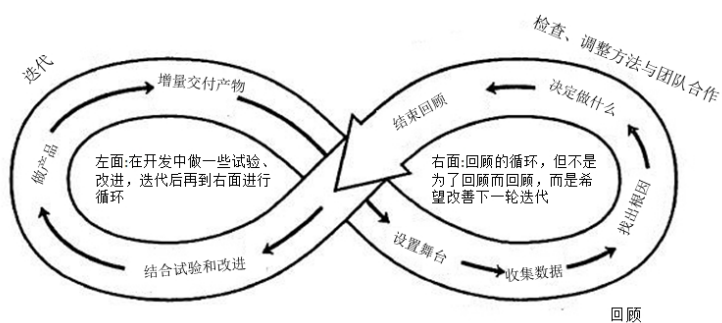


图 12-3

回顾时，也要想办法避免受到群众压力的影响，导致不能真正挖掘问题的原因。所以在回顾时需要大家放开，以下方法可以减少团队压力，让大家更投入，能畅所欲言：

- 回顾开始时（设置舞台），尤其是团队首次在回顾里做集体根因分析，建

议用互动游戏，让团队放开顾虑，全心投入讨论（例如附件里“游戏：应与否”）

让每位团队成员都有充分机会表达意见，集思广益，得出更好的纠正措施。

团队所有成员参与

整个团队参与分析，全面看问题。例如如果只是从测试人员的角度，他可能以为缺陷都是来自开发；需求分析人员（或产品经理）会觉得自己做得很好，因需求评审里没有什么问题被发现。但如果团队所有成员聚在一起（包括项目经理、需求、设计、编码、测试），把所有的缺陷列出来，让团队所有岗位一起分析，才可以全局看问题，才可能发现不少问题是因为需求没有明确，导致后面做出来的功能并不是客户要的。后面可扩大参与回顾的干系人，例如客户代表，可以更全面分析。

数据准备

有数据才有管理，但软件开发与工厂不同，很多数据必须依靠员工提供，

例如，某活动所花的工时，必须靠个人记录（具体步骤可参考“个人量化管理”：个人记录每天工时）。

但记录数据要花精力，如果不了解收集的数据，后面有什么用途，便难以维持不断收集数据的习惯。如果当大家都清楚到迭代回顾时会一起分析团队数据，并制订改进措施，就有动力继续迭代里统计数据。

也解释了为什么依赖公司级数据分析专员收集并分析团队数据，因通常几个月后才收到数据分析的反馈，公司难以收到真实的数值。

内部教练如何做好现场辅导

培训后过了 6 周，质量经理电话联系我。

质量经理：我们计划下周三与项目组进行迭代回顾。该团队有 7 到 8 人，两周前曾与另一团队使用便利贴进行了根因分析，确实听到了更多不同的声音。应该如何准备，才能让团队放开并有效地做好根因分析？

我：如果只有缺陷数量和工作量等“硬”数据，但缺乏如团队成员的感受、积极性、主动性等与人性相关的“软”数据，也难以真正分析出能够避免同类问题再次发生的根本原因。例如，在冲刺回顾时可以用时间表（Timeline）让大家回忆过去四周的经历，并用颜色点（Color Code Dots）让每位成员标记每个事件的感受，这是一种收集软数据的方法，详见附件：敏捷回顾互动练习。

要做好回顾，准备工作至关重要。例如需要准备大白纸，并确保有足够的空间。如果团队成员围坐在一个大桌子旁讨论，效果往往不佳。相反，我们在墙上贴上大白纸，并提供便利贴和大号、小号水笔等，当每个人都有自己的一支笔时，团队成员会更加投入。他们会走动、讨论、写字、画图，并不断交换位置、互相帮助，这样能更好地体现团队精神。因此，辅导的重点不是在各种技巧上，不是教他们每一步怎么做，只需提醒时间（时间的管理很重要。每次进行小组练习时，我都会用电脑投屏显示剩余时间（分钟），让团队即时掌握剩余的时间）。先

定好目标，让团队自主发挥与讨论，效果会更好。只要他们清楚需要做什么，大部分团队都能自行分工，比如有些人负责画帕累托图，有些人做总体根因分析。质量经理：你的意思是减少干预，尽量让团队自主思考。但因未做过，可否再举些例子？

我：正确。不如我反过来，建议你应该避免说什么：

1. 其实你们团队和我见到的其他团队相比好或者差（因这都是个人观点、感觉，非事实）
2. 本来只需要你们画帕累托图和鱼骨图，为什么你们还画了个脑图
3. 你们都已经讨论了很多细节，要立马抓紧时间做最后总结
4. 你们讨论好像都只是针对机场闸口的问题，是否应该也考虑其它

为什么要鼓励团队表达不同意见？

团队要改进，首先要确保成员充分理解彼此的差异。经过充分讨论后，团队成员找到共同点，才有机会一起改进。

如果大家在回顾时能够开放讨论，充分表达各自具体的看法，就能让大家看到全貌和问题的真正情况。

因此，内部教练应鼓励多样化的声音。心理学家 Asch 在 1951 年的从众实验中证明，只要有一个人率先提出异议，其他人也会陆续表达自己的意见。因此，内部顾问应确保每个人都有发声的机会。意见越多样化，大家越能了解真正的问题，找到大家认可的改进方案（详见附件：功能性分组帮助团队成长实例）。

例如，如果发现团队在大型会议上因人数众多而不愿发言，可以考虑分成小组讨论，这通常会更有效。

对于一些“慢热”的团队，需要给予他们更多时间，不要急于打破沉默。等一等，一旦有人开始发言，接下来的讨论就顺利了。

团队迭代回顾根因分析常见问题

1. 没有利用缺陷数据做帕累托图分析，不清楚应针对哪一类，不清楚哪一类问题最多，导致根因分析讨论没有针对性。有些缺陷分类没有定义好、不明确或同一缺陷归到几个类别，导致分析结果没有参考作用
2. 不清楚什么是根因，只找到表面看到的现象，例如人经验不足，对技术不熟悉，对业务不熟悉等。没有抓到系统的根本问题
3. 没有量化目标，只有定性的根因分析，下轮迭代难以判断效果能否达到
4. 回顾时间不够（足够的时间，最少 3 小时）

以上前 3 点可以利用培训，加强上章讲过的根因分析方法。

最后一点需要领导理解如果想要有效，必须收集数据，并让团员投入讨论。

研发总监：已经做了很多年过程量化考核，起初能给团队带来活力，因为至少比以前好，有路径可以量化他们，员工觉得做有所值，公司也看得见。前几年执行得很好，产出和上线频率比过去好很多。但是现在感觉逐渐拉不出差距来，因为大家都非常熟悉这套游戏规则。

我：理解，比如现在你们团队开始做迭代回顾，团队一起分析根因，持续改善。您觉得效果如何？

研发总监：挺好，但是还是感觉他们开回顾会，耗时太长了，每次都要 2 小时以上。

我：因需要团队收集数据、分析、讨论下迭代措施，所以通常要 3 小时，熟练后会快一点，底线是改进的节省应大于回顾的投入。

高层的支持是任何改进的重要成功要素。所以首先要让管理者赞同迭代回顾能为公司带来回报（省钱），可以节省的成本（工作量）大于回顾的人力投入；也要让他理解任何改变都有个混沌的过渡期，要经过几轮回顾后，团队才能自己持续改善。

如果回顾只能 1 个或 1 个半小时的话，就不够时间让团队针对缺陷分组分类、画帕累托图做分析，没有根据实际数据分析根因，也影响到团队没有动力在后面迭代花精力去收集数据。例如缺陷返工数据一直都非常难获取到的，也需要在迭代根因分析时给团队时间讨论统计，所以通常完整的迭代根因分析回顾不会小于 3 小时。要做好这块如果有内部教练可以更好控制整个回顾的节奏。不会在某一个环节耽误太多时间，也不会太长，也确保团队人员都全程投入讨论。

所以要团队做好迭代回顾，内部教练不仅仅教大家各种“硬”技巧，也要考虑影响大家的心理，必须让每位成员都积极参与，才有动力执行和以后继续积极参与。如果只是组长一言堂，反而会妨碍团队成员的动力。但是只依赖大家的经验和感觉，识别根本原因也不科学，所以也必须有数据支撑（硬的内容）。当然也需要高层支撑——提供资源和时间，如高层觉得整个团队花半天来做回顾太浪费资源，也无法推行。

下一章我们会分享一些团队对回顾的误解和常见问题。

附件

Asch 1951 群众压力实验

实验设计：随机邀请几位志愿者（包括你）参加，实验之前与（除了你以外）所有人预先说好，都选 **A**，并且要假装成经过深思熟虑。实验时，你是最后一位作答。

结果：

- 有群众压力时，大概 **37%** 会从众选择错误答案（虽然不同人有差异，但总体只有 **25%** 能坚持自己的正确答案）
- 个人自己作答，接近 100% 选对，错误率少于 **1%**

影响因素:

- 前面选择错误答案的人数越多，错误率越高:

1. 一位：接近 0% 答错
2. 两位：大概 14%
3. 三位：32%（详见下图）

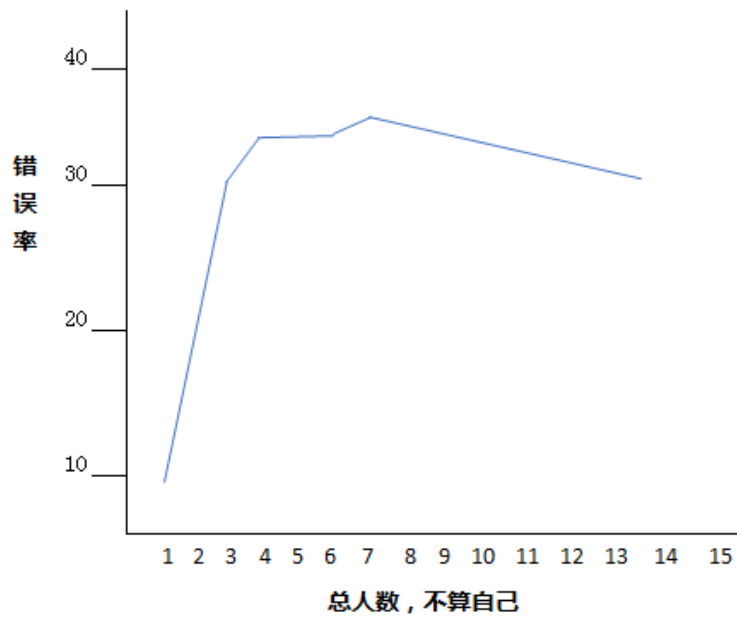


图 12-4

- 如果前面有一位“战友”选了正确答案，你的错误率就显著降低（大约原本水平的 1/4，看下图黄线），并让你感到与他更亲切

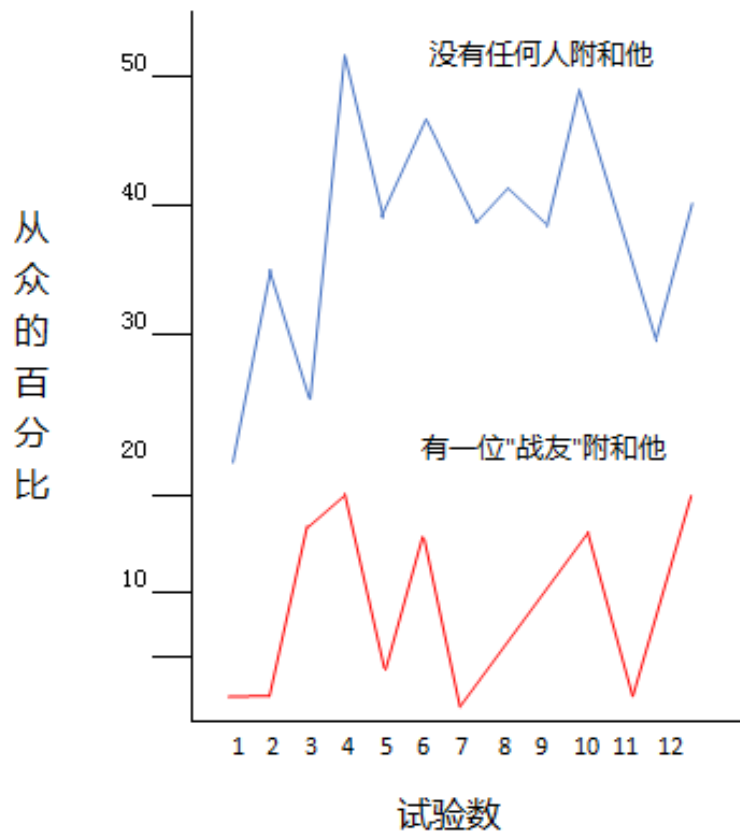


图 12-5

- 尴尬场景 (例如迟到), 你的错误率会更高
 - 实际你没有迟到, 只是让你觉得确实迟到了, 觉得尴尬
- 如果不是要你说出答案, 而是让你把答案写在纸上, 错误率会降低 1/3

Brehm 1956 决策影响实验

实验设计:

1. 提供 10 多种不同礼物, 包括台灯、多士炉、挂钟、收音机等
2. 请你按自己喜好对礼物排序。例如最喜欢的选一, 如果同样喜欢两件礼物, 可以不分高低, 写同一个数字
3. 但当你马上要离开时, 让你从两件同样喜好的礼物挑选其中一件, 让你拿走
4. 在你最终离开之前, 请你再对所有礼物按喜好排序

结果:

- 绝大部分人都会抬高自己选择拿走的那一件礼物排序, 调低非选择的那件

礼物排序

- 但如果不是自己选择，而是由老师选好给你，你后面的礼物排序就没有变化

结论：

- 人都会觉得自己的选择比较好
- 例如选大学、选汽车、选伴侣。如果是由你自己决定选择（非被安排），你会不会都会觉得自己的选择较好

KJ 分析方法步骤

让团队了解大家的事实，一起分析根因，一起做分析报告

- 大家都对着墙上的大白纸

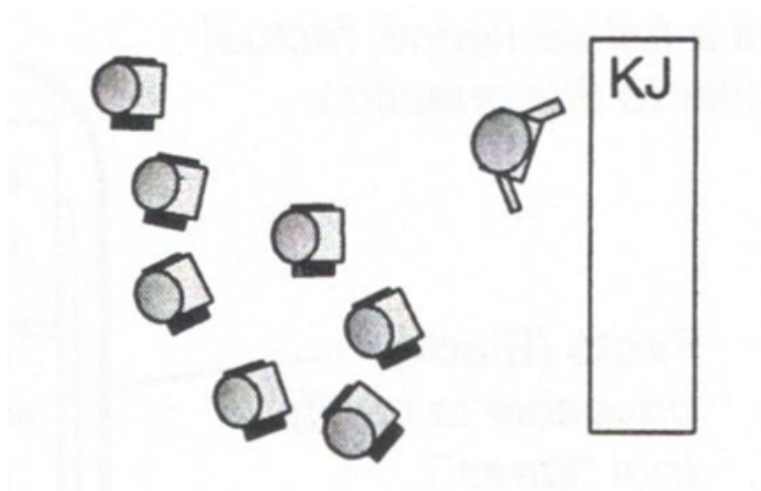


图 12-6

- 每人依据主题写“事实”并写在便利贴上，贴到大白纸上

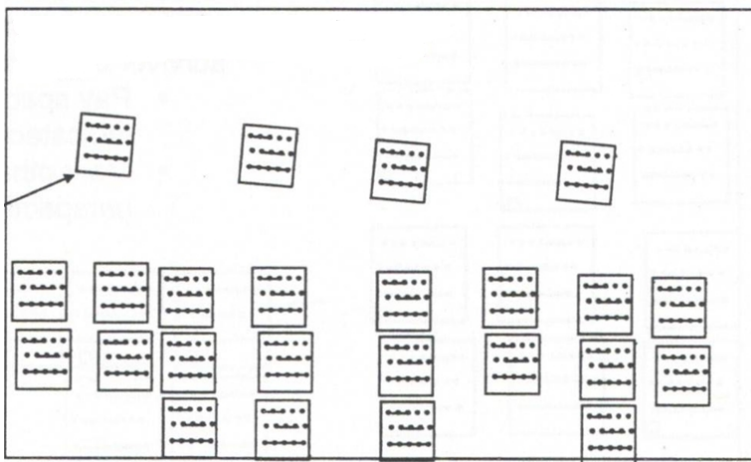


图 12-7

- 轮流移动上面的便利贴，经过 1~2 轮后，逐渐看到归类
- 大家一起挖掘隐藏在低层次事实中的更通用的重点

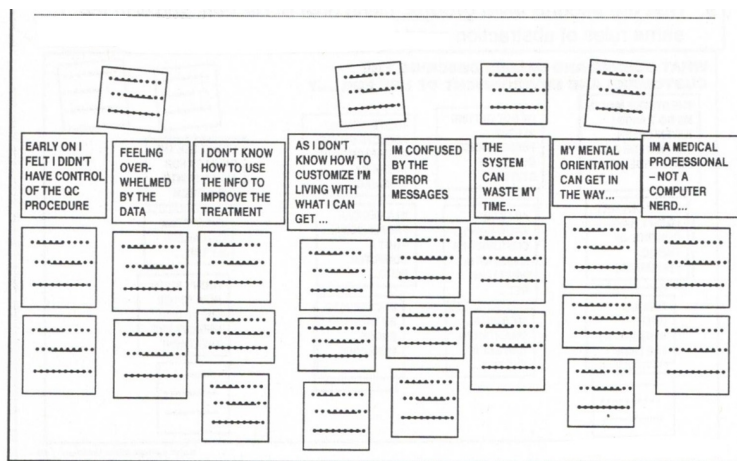


图 12-8

一种头脑风暴方法，每人把自己想法写在便利贴上，一起轮流把所有便利贴排放分组

1. 把主题以问题形式写在大白纸的头顶
2. 把相关的事实写在便利贴上面，用黑色
3. 搜集相关的事实，如果是一组人的话，分散到不同人手上
4. (团队) 查看描述是否不清楚，是否要整理
5. 每人轮流把相关的便利贴组合在一起
6. 当每人都已经调整过便利贴组合后，一起把每一组便利贴加上一个题目，用红色标识

7. 红色的标题下包含相关事实的内容
8. 把相关的红色题目（最好不超过 3 个）组合在一起
9. 给这大组一个题目——蓝色
10. 在题目下放上相关的红色小组
11. 把蓝色的题目和剩下红色的小组或单独没有分类的便利贴都在墙上放好，也可以用一些箭头描述它们的因果关系

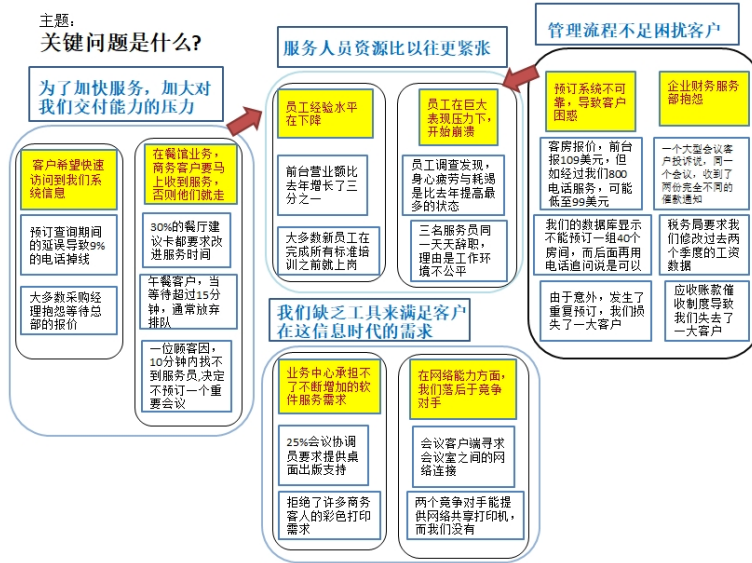


图 12-9

利用便利贴做 KJ 分析让大家都充分参与，不仅仅适用于迭代回顾。我每次培训，如果有时间，尽量最后腾出 1 个小时，让大家归纳刚刚学过的东西，按自己日常工作不足可以改善之处进行分析根因，并制定改进计划。本来上课时无精打采的学生都会立马精神过来，积极参与讨论。后面有些同学在填培训反馈表时，还建议应再多分配时间给学生互动讨论，减少讲课和个人做练习的时间。

游戏：应与否

在迭代回顾中使用。

目的

帮助建立有效沟通的思维模式。帮助参与者抛开指责和判断，以及对指责和判断的恐惧。

所需的时间

8 ~ 12 分钟，取决于小组的大小。

描述

在描述这些模式之后，参与者讨论这些沟通模式的意义。

步骤

- 1. 将注意力集中在“应与否”(见下图)，并简要阅读
- 2. 组成小组，每组不超过 4 人。要求每组用一对词来定义和描述，有 4 对以上的词组。如果有多个组有相同的词组也可以
- 3. 让每个小组讨论他们的 2 个单词的意思和它们所代表的行为，让他们描述各自对团队和回顾的影响
- 4. 每个小组向整个小组汇报他们的讨论情况
- 5. 询问大家是否愿意使用左面的方式



图 12-10

用角色扮演为做好回顾打基础

首次现场培训时，我都会要求角色扮演：学员用各种角色扮演什么是吵架、什么是交流，让观众和表演者更能感受到量化回顾先要有对事不对人的心态。

除了能提高学员的参与度，也能让大家能更好理解左面与右面的区分。

功能性分组帮助团队成长实例

系统，包括团队、公司的成长都依靠不断集合内部的差异，如果完全不能接受不同意见，就无法进步。基于心理学家 Yvonne Agazarian 系统中心理论 (Systems Centered Theory)，团队必须先从传统（按岗位、功能）分组 (Stereotype

Subgroup) 变为功能性分组 (Functional Subgroup)，才能成长。
要让大家觉得有共同点，基于这基础才可以听从不同意见。

场景实例

团队讨论如何完善社区里面的教育，大家讨论个人的希望。

A：好像我们大家的想法都很类似。

内部教练：可否举个例子？

B：我们都想为自己和我们后代有个终身不断学习的机会。

C：好像我们都没有人提过大学，不知道为什么。

D：但很多人负担不起大学学费。

E：我觉得每个人只要想受到教育，都能做到，我就是靠自身努力。最后完成大学教育。

F：（开始跟 D 形成小组）是的，但教育确实越来越昂贵，我估计我们当中不少人负担不起那些美国东岸名校的昂贵学费。

G：（有成见，觉得那些不能完成大学教育的都是没有动力，没有上进心）我还是觉得任何人，只要有动力都能做到。

如果没有人附和 E 的观点，就可能变成他的个人独角戏。后面有两个可能的发展场景：

场景一 C 附和了 E 的观点。

C：我的经历确实和 E 说的类似，辛辛苦苦最终完成了大学教育，但确实很艰苦。如果我当时没有舅舅的支持，肯定完成不了。

场景二没有人附和 E 的说法。

内部教练：有没有其他人也有经过自己努力完成大学教育的经历？

（注意：内部教练千万不能说：“有没有人赞同，人只要有动力都能进大学？”因为这只是个人观点，并非事实。）

A：我确实也完成了大学教育了，但也经历了很多困难。

G：我读了一年，因为再得不到奖学金，没有办法，只能退学。

到了这个点，整个分组就比较复杂，虽然生成了新小组，E 得到一些回应，但也有些人提出不同的经验，需要有说法把新小组归纳。

接下来，

C：我看来这个很简单，有些人能完成大学教育。有些人虽然很希望完成，最终还是完成不了，但不是所有人都想或者必须完成大学教育。所以我们的改进任务应该是帮助所有人学到他希望学习的东西。

有了以上 C 的说法，整个团队就可以进一步成长。

敏捷回顾互动练习 1: 时间表 (Timeline)

在较长的迭代、发布或项目回顾中使用此方法收集数据。

目的

促进大家记忆，从多个角度回看之前的工作。大家一起回看谁在什么时候做了什么工作，与当初的假设对比。看是否有什么规律？或者生产水平有没有变化？用来表达事实和每人的感受。

所需的时间

30 到 90 分钟，这取决于团队的规模和迭代周期长短。

描述

小组成员写卡片来标示那些很有印象、特别有意义或其他重要事件，然后按时间顺序贴在时间表 (Timeline) 上。教练鼓励团队讨论事件，以了解已发生的事实和感受。

变化：功能性泳道 Functional Swim Lanes

沿时间轴的背景线纵向绘制行 (假设你不打算直接在墙上张贴卡片，然后使用丝带或胶带划分行)。为每个部门或功能做一行。每组只会把他们的卡片放在自己组的泳道里。

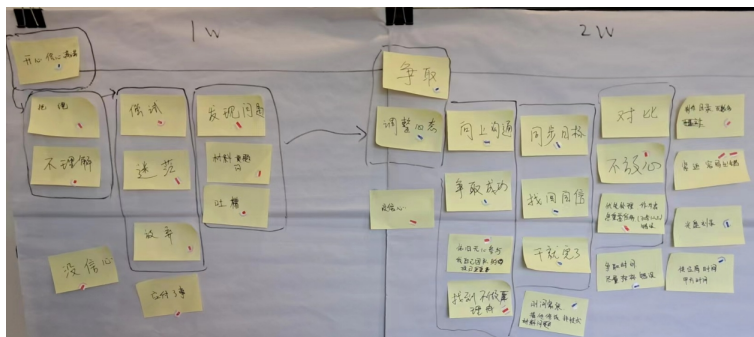


图 12-11

敏捷回顾互动练习 2: 颜色点 (Color Code Dots)

与时间表 (Timeline) 练习结合使用，在更长的迭代回顾中收集团员的感受。

目的

展示团队成员对经历事件的感受。

所需的时间

5 至 20 分钟。

描述

团队成员使用颜色点来表示在时间轴上事件的心情高低。当所有的事件都已经展示在时间轴上，团队也已经回顾后，每人使用颜色点来显示他们的情绪是高还是低 (见图)。

1. “我们已经一起看了事实，让我们也看看大家工作时的心情，状态”。作为开始之前说，让大家理解游戏目的

2. 给每个人提供两种颜色的点。每人从 7 到 10 个点开始，如后面不够用，可提供更多。解释哪种颜色代表情绪高，哪种颜色代表情绪低
3. 让每个人都用点来表示哪里情绪高，哪里停滞、衰弱或低潮

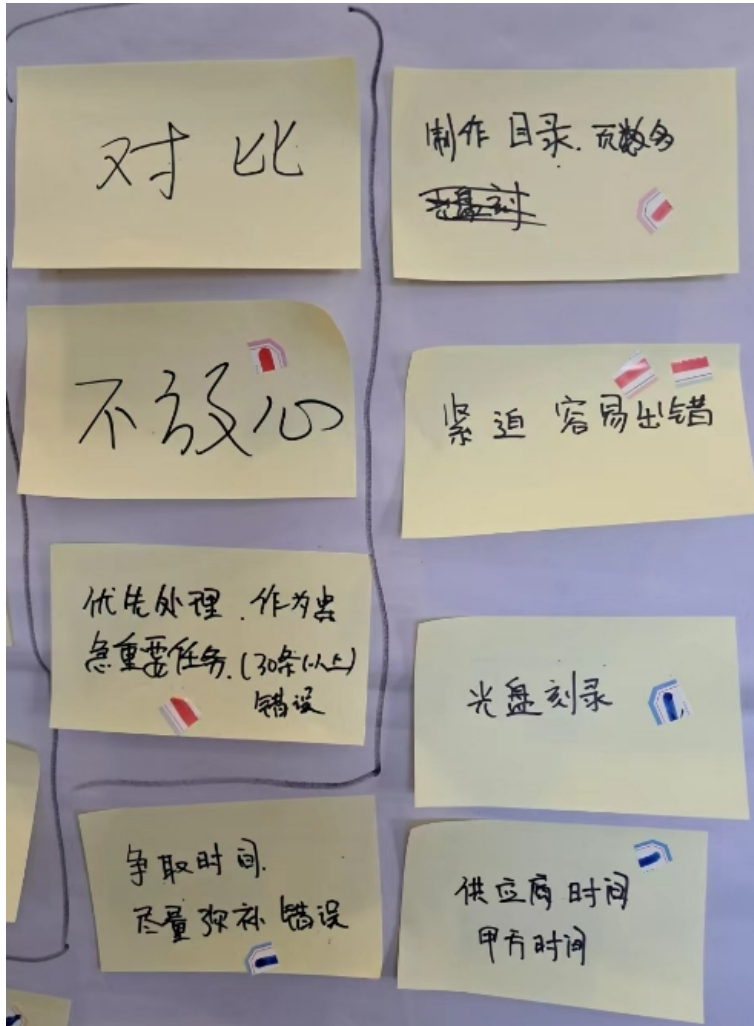


图 12-12

Chapter 13

从团队实验到持续改善

冲刺回顾是团队分析根因在下一轮冲刺改进的最佳时机，前面介绍了时间表 (Timeline)，颜色点等方法收集团队“软”数据，这章会针对“硬”数据。如果团队了解改正 (Correction) 和纠正措施 (Corrective Action) 的区别，和根因分析，便可用在回顾时分析数据，制定有效纠正措施，避免发生过的问题再发生。

收集哪些数据

美国某家医院的故事

1996 年，某家美国 Cook County 医院做了个试验。这家医院所在的地区比较穷困，很多市民没有买医疗保险，且医院床位紧张。因人口老化，也没有良好饮食习惯，这医院面对的最大问题是怎么判断病人有没有急性心脏病的危险？因为如果病人胸口痛，医生从他的背景、年龄、生活习惯判断有心脏病的危险，就要收他住院，但医院的床位有限。新来的院长 Reilly 很有经验，了解问题严重，首先要解决这个难题，因为针对心脏病的治疗的病床要花费 2000 美元一晚，一般心脏病要住 3 晚，而后面发现很多病人其实没有心脏病，浪费了医院的宝贵资源。他发现每个医生都会有自己的一套方式去判断病人是否需要住院，诊断的差异很大。每个医生要收集的数据也很多，心电图、血液、尿液、x 光，还有病人的历史、习惯等。因为院长是从东岸名牌医院过来的。他认识专家专门研究怎么有效去判断病人的专家。这位专家 Goldman 用了两年时间，针对心脏病猝死的诊断，收集了很多数据和验证，发现只需要针对 3 个紧急的风险因素，就可以准确判断：

1. 病人的肺部有没有液体
2. 血压上压 (systolic Blood pressure) 是否低于 100
3. 病人是否感到心绞痛 (patient unstable angina)

通过这样就可以判断病人是否需要住院。所以院长就依据这个做了实验。结果发现，平常用以前的方式，医生诊断准确度只有 75%-85%，但用了这个准确度就超过 95%。

很多人都误解度量分析必须像大数据分析一样，要有大量数据，用很多因子来

求出一个漂亮的方程式，才算成功。其实是反过来，如果想要收集的数据量越大，变量越多，量化过程改进越难成功，开发人员越抵触。所以我们必须要针对想改进的重点收集对应数据。

很多时候信息量多，但是没有针对性，反而不好。

好比病人去医院，都必须抽血、验尿、做心电图，收集各类数据让医生诊断，但是否信息越多越齐全就越好。从上面美国 Cook County 医院的故事看到其实不是，

小时候看病，中医是用望闻问切的方式；西医就是用听诊器听心跳、呼吸，看舌头、口腔，然后也是听你的问题描述，便可以诊断开药了。现在去医院排队看病，医生第一件事就给你开四五张各类的检查单，通过这些检查报告做诊断。虽然我是外行，但我觉得小时候那种不超过十分钟诊断过程不会比现代的起码两个小时的诊断质量差。”

如何建立并完善缺陷预测模型的例子

公司度量分析人员展示了一个大表，上面密密麻麻有五、六十个项目变量，包括缺陷、进度、工作量等等。

我问：为什么你们要收集这么多数据量？你们的改进目标是什么？

度量分析人员：我们要提高生产率、减少进度偏差，也要降低最终的客户的缺陷密度。

我问：如能尽早发现缺陷，加强前面的评审效率来减少遗漏到后面的缺陷，减少返工。如果大家都相信这可行，就应该专注这模型相关的度量，如每迭代的缺陷分布、与相关返工工作量等，让蒙地卡罗预测模型可以尽快跑起来。但如果对进度偏差，工作量偏差等都要收集，每轮可能要收集几十个变量，反而没有针对性。

我们看看这预测缺陷模型，比如你们有什么不同的评审方式？效果有什么不同？

度量分析人员：我们跟开发讨论过，确实有些区分，例如，如果我们在前期有做正式评审，质量都会好一些，但是如果没说的话，后面确实缺陷就多了。如果我们有明确需求，例如按活动、实体方式明确写出来，跟没说的话也会有质量的区分。有些项目有规定要静态扫描代码并记录缺陷，后期的缺陷数会较少。

我总结说：所以如果可以估计各种方式的缺陷排除率/引入缺陷率，便可使用模型预测最终会遗漏多少缺陷。可利用这蒙地卡罗预测工具，帮你比较各种方法，制定缺陷数目标。团队开始时先参考上一轮迭代数据，估一下预测模型参数，并一起头脑风暴讨论有什么新方法可以做提升，加进模型，蒙地卡罗预测模型可比较不同的组合，看看对缺陷率，缺陷排除率和工作量等有什么影响，团队便可参考模型的建议，最终选择下一迭代的配搭。

例如有些团队利用静态代码扫描预先找出代码缺陷，就可以做个实验，区分一下是否代码扫描后，系统测试缺陷会减少，排除缺陷的百分比是多少等。

开始的时候可能数据量不多，慢慢项目多了数据也可以看是否稳定，便开始变成团队的基线。所以开始时不要太计较准不准，而是尽快利用这概念开始量化管理。

如果团队能尽早发现缺陷并修复，后面返工量便可倍数降低，生产率自然也会提升，进度偏差也会改善。所以建议只针对能如何尽早发现并解决缺陷，减少

返工工作量设定目标，量化管理。

缺陷排除率

从需求到设计、编码、单元测试、系统测试、验收，整个开发过程大家都很熟悉，需求、设计、编码后可以通过评审来排除缺陷，但仅仅做评审不一定能确保质量。因为最终验收缺陷数取决于每个步骤能否有效排除当前的缺陷。

可用缺陷排除率 (Defect Removal Efficiency DRE) 来判断每一个过程的质量：

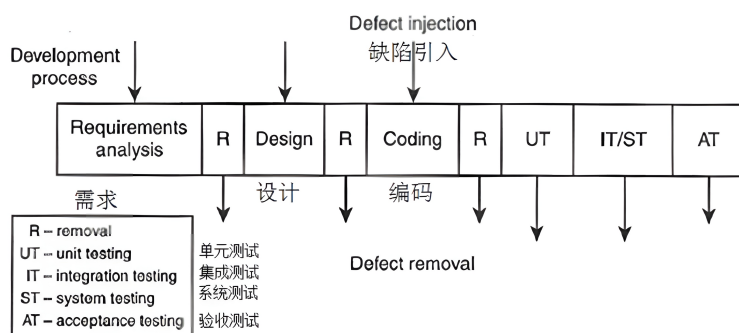


图 13-1

$$DRE = \frac{\text{Defects found by the QC activity 缺陷排除数}}{\text{Total defects in the product before QC 缺陷总数}}$$

如果要最终质量好，缺陷排除率就要高。但计算缺陷排除率，必须要等到整个开发完成才可以计算。

缺陷排除预测模型实例

- 团队分析迭代缺陷数据，缺陷是源自需求/设计/编码，得出以下分布表：

		Req	Design	Coding	
RR		1			
DR		1	2		
CR		2	3	6	
UT		2	5	10	
ST		3	9	16	
AT		1	1	2	
		10	20	34	

图 13-2

A/按缺陷排除率 (DRE) 公式，计算出各个过程的缺陷排除率，并把参数输入水晶球模型。例如第一行 Quality 那行：在底栋填 10 (共有 10 个缺陷是源自需求)；第二行：在第 4 栋填 10% (需求评审缺陷排除率是 10%)。(本来迭代缺陷参数都标 Infosys 识别。)

RR	$1/10$	10%
DR	$1+2/(10+20-1)$	10.3%
CR	$2+3+6/(64-4)$	18.3%
UT	$2+5+10/(64-15)$	34.7%
ST	$3+9+16/(64-32)$	87.5%
AT	$1+1+2/(64-60)$	100%

图 13-3

B/ 按以上迭代回顾缺陷分析结果，按以下输入框位置输入模板

1. 需求引入缺陷数 = 10 (3)
2. 需求评审缺陷排除率 = 10% (4)
3. 设计引入缺陷数 = 20 (2)
4. 设计评审排除率 = 10% (4)
5. 代码引入缺陷数 = 34 (4)
6. 代码评审排除率 = 18% (1)
7. 单元测试缺陷排除率 = 35% (2)
8. 系统测试缺陷排除率 = 88% (1)
9. 验收测试缺陷排除率 = 99% (3)

注：以上括号里的数字表示模板哪一栋，例如第一行需求引入是 (3) 代表在第三栋填 10。

	(1)			(2)			(3)			(4)								
	e1 E2-950												Zone 2 T2 U6					
需求	Requirements Development																	
	Worker Effort		LL	Area	UL		LL	Area	UL		LL	Area	UL		LL	Area	UL	
	Worker Realize		10	20	30		10	20	30		10	20	30		10	20	30	
需求评审	Requirements Review																	
	Worker Effort		LL	Area	UL		LL	Area	UL	WAR 需求 Warnings			Influence Layer					
	Worker Realize		10	20	30		10	20	30	10	20	30	10	20	30	10	20	30
设计	Design																	
	Worker Effort		LL	Area	UL		LL	Area	UL	WAR 需求 Warnings			Influence Layer					
	Worker Realize		10	20	30		10	20	30	10	20	30	10	20	30	10	20	30
设计评审	Design Review																	
	Worker Effort		LL	Area	UL		LL	Area	UL	WAR 需求 Warnings			Influence Layer					
	Worker Realize		10	20	30		10	20	30	10	20	30	10	20	30	10	20	30
代码	Code																	
	Worker Effort		LL	Area	UL		LL	Area	UL	WAR 需求 Warnings			Influence Layer					
	Worker Realize		10	20	30		10	20	30	10	20	30	10	20	30	10	20	30
代码评审	Code Review																	
	Worker Effort		LL	Area	UL		LL	Area	UL	WAR 需求 Warnings			Influence Layer					
	Worker Realize		10	20	30		10	20	30	10	20	30	10	20	30	10	20	30
单元测试	Unit Test																	
	Worker Effort		LL	Area	UL		LL	Area	UL	WAR 需求 Warnings			Influence Layer					
	Worker Realize		10	20	30		10	20	30	10	20	30	10	20	30	10	20	30

表 13-1



	① 修复缺陷 总工时	② 缺陷数	① / ②
RR+DR	16	4	4
CR+UT	80	28	2.9
ST	160	28	5.8
AT	80	4	20

图 13-4

C/从团队工时表估算各过程的缺陷返工工作量。例如最下面一行 Quality 那行输入: \$20,000 (因为验收测试缺陷平均修复工时是 20, 假定每工时成本为 \$1,000); 系统测试那行: \$5,800 (因为系统测试缺陷平均修复工时是 5.8)。

Zone 4 AE2:AL60		Costs	Unit Costs	Min	Most Likely	Max
Requirements Development	Effort	\$71,888	\$966	\$700	\$800	\$1,000
	Quality					
		Costs	Unit Costs	Min	Most Likely	Max
Reqs Review	Effort	\$8,416	\$828	\$700	\$800	\$1,000
	Quality	\$16,768	\$887	\$3,400	\$4,000	\$4,800
		Costs	Unit Costs	Min	Most Likely	Max
Design	Effort	\$67,683	\$798	\$700	\$800	\$1,000
	Quality					
		Costs	Unit Costs	Min	Most Likely	Max
Design Review	Effort	\$6,308	\$764	\$700	\$800	\$1,000
	Quality	\$27,002	\$1,418	\$3,400	\$4,000	\$4,800
		Costs	Unit Costs	Min	Most Likely	Max
Code	Effort	\$88,301	\$844	\$700	\$800	\$1,000
	Quality					
		Costs	Unit Costs	Min	Most Likely	Max
Code Review	Effort	\$11,682	\$886	\$700	\$800	\$1,000
	Quality	\$71,888	\$1,877	\$2,486	\$2,800	\$3,336
		Costs	Unit Costs	Min	Most Likely	Max
Unit Test	Effort	\$144,168	\$976	\$700	\$800	\$1,000
	Quality	\$83,814	\$7,643	\$2,486	\$2,800	\$3,336
		Costs	Unit Costs	Min	Most Likely	Max
Integration Test	Effort	\$4	\$826	\$700	\$800	\$1,000
	Quality	\$3	\$3,831	\$3,000	\$3,600	\$4,600
		Costs	Unit Costs	Min	Most Likely	Max
System Test	Effort	\$79,004	\$801	\$700	\$800	\$1,000
	Quality	\$124,782	\$7,303	\$4,830	\$5,800	\$8,870
		Costs	Unit Costs	Min	Most Likely	Max
Acceptance Test	Effort	\$52,786	\$834	\$700	\$800	\$1,000
	Quality	\$131,887	\$17,801	\$17,000	\$20,000	\$23,000

图 13-5

D/ 质量总成本：水晶球会依据质每过程的缺陷分布和单位成本分布，预估质量成本的分布。

质量总成本 + 工作（如评审，设计、开发、测试等）成本（注），得出总成本分布。

注：指具体做评审、测试等活动的工作量，填在模板里工时 Effort 那行
从水晶球得出总成本分布的 95 百分位数 (95% percentile) 为：\$905,022

E/ 团队分析根因后，团队找出以下 3 条改进建议：

1. 用正式专家评审加强需求评审，估计能把缺陷排除率从本来 10% 提高到 80%
2. 用评审检查单加强设计评审，估计能把缺陷排除率从本来 10% 提高到 59%
3. 用检查单加强代码评审，估计能把缺陷排除率从本来 18% 提高到 35%

- 问题 1: 用了这些新方法会对缺陷分布有什么改善效果?
- 问题 2: 是否 3 种改善都使用? 或只用其中 2 种更有效? (总成本最低)

把 3 种新方法和% 按下表加入模板: 水晶球会依据质每过程的缺陷分布和单位成本分布, 预估质量成本的分布。

1. 按本来迭代需求引入缺陷数 =10 (3)
2. 需求评审缺陷排除率 = **80%** (3), 需求评审缺陷排除率 = 10% (4)
3. 设计引入缺陷数 =20 (2)
4. 设计评审排除率 =**59%** (3), 设计评审排除率 =10% (4)
5. 代码引入缺陷数 =34 (4)
6. 代码评审排除率 =18% (1), 代码评审排除率 =**35%** (2)
7. 单元测试缺陷排除率 =35% (2)
8. 系统测试缺陷排除率 =88% (1)
9. 验收测试缺陷排除率 =99% (3)

最佳搭配是只选加强设计与代码评审, 需求评审不变。

改进后成本分布的 95 百分位数 (95% percentile) 为: \$874,854

- 如果我们把优化后的缺陷分布与本来迭代的缺陷分布比较, 能看出改进后, 本来在系统测试才发现的缺陷大多数可以预先在评审暴露, 降低返工工作量, 也可以预估评审缺陷范围, 如果实际评审缺陷数未能达到范围, 便可预警, 尽快纠正

如果没有预测模型, 我们只能主观估计会降低, 有了预测模型, 便可以估计缺陷分布的变化, 也能帮我们挑选最佳搭配, 并预估下一轮的缺陷分布范围。

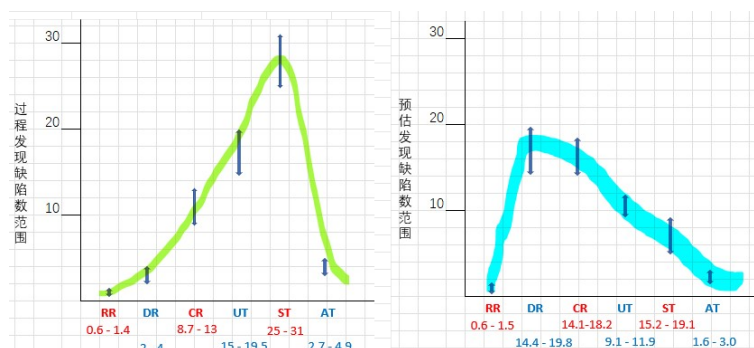


图 13-6

解读预测模型最佳搭配选择:

模型以总成本最低可以防止局部最优。例如需求评审没有选用排除率高的新方法, 因为新方法会使用较大评审工作量。如果用质量最优 (质量成本最低) 便必然会读选用缺陷排除率最高的方法, 但用总成本来比较便可以综合考虑工作量 (Effort) 和质量 (Quality), 避免局部最优。

由于不同项目甚至不同迭代冲刺的缺陷数和缺陷率各异, 难以进行比较, 但如

果方法和团队人员相似，缺陷排除率应该是可比的。通过分析以往的缺陷数据，针对最薄弱的环节讨论根本原因，并在下一个迭代中进行改进，提升缺陷排除率，并可以预估各个阶段（如评审、测试等）的缺陷发现数。例如，如果需求评审的缺陷数远低于预期，就应立即进行纠正，而不是像以前那样仅仅提醒大家要重视评审工作。

== ==

怎样开始

一天的培训，可以帮助他们了解根因分析的重点。

如何提升软件开发质量TrainingPlan		
9:30	用案例介绍根因分析的五个步骤与技巧:帕累托图、鱼骨图等练习	
10:00	根因分析小组互动	
11:00	休息	
11:15	迭代回顾场景(由公司内部教练做角色扮演)	
11:45	总结，并介绍缺陷排除率模型	
12:00	午休	
13:00	介绍缺陷排除率模型	
13:15	量化根因分析模拟练习，包括：	
	分析模拟数据，估计缺陷排除率与返工工作量	
	针对主问题，团队所有成员做KJ根本原因分析	
	从根因、讨论对应改进措施	
15:15	休息	
15:30	设定量化质量计划模拟练习	
	利用MonteCarla工具	
	对原本迭代数据做3点估算	
	再依据前面制定的方案	
	利用工具寻找最佳方案	
16:15	总结，确认团队后面的具体工作	

图 13-7

让团队了解以下重点：
改进目标：降低返工工作量（同时也会降低缺陷后期才被发现的数量）
分析根因：不仅要分析后期发现缺陷的分类与数量，还需对从需求到最终客户验收的全生命周期进行缺陷排除率分析，以找出最薄弱的环节并深入挖掘根因。利用数据进行互动练习是最有效的学习方式，因此建议从以往的缺陷数据中抽取 70-150 条进行小组分析。
预测效果：以福特 8D 根因分析框架为例，缺陷排除与返工预测模型可以帮助团队预估新方法的效果（例如，通过代码静态扫描加强代码评审）。

团队在开始从定性到定量管理的时候，可以在回顾时，分析刚刚迭代的缺陷排除率和缺陷引入，以前没有这种细化的数据，根因分析可能只看到系统测试缺陷的总数多少，用了这个排除率的分解，就可以让团队了解主要问题来自哪里，

针对性分析根因和对应措施。例如发现大部分的缺陷来自代码，就深入地去看代码出了什么问题，那一类代码缺陷比较多，针对这一类的缺陷讨论原因，下一轮做对应改进措施，并制定新一轮的量化目标。

量化回顾也让团队成员了解到自己的问题，更了解自己的质量和水平。

如何能简单有效地收集编码缺陷与返工工作量数据

最有效项目数据的方法就是每次迭代回顾时，同时看看本迭代缺陷的相关数据，这样团队立马可分析本迭代收集的数据（而非团队收集数据，让公司数据专员做分析）。很多团队说”我们都有评审代码，但问题都直接改正了，没有记录。“后面便不知道有多少缺陷，无法量化分析。如果要求开发人员按系统测试缺陷记录方式，在缺陷管理系统里面一条一条记，大家会觉得太花时间不愿意做。可以利用工具帮助，例如可利用 GIT 做代码评审记录，也可以在 GIT 系统里面跟踪评审问题。不需要把问题另外记在系统测试缺陷跟踪系统，但每条问题在 GIT 系统里都有痕迹。开发就可以从中数出大概共有多少代码评审缺陷。如果平常也记录了每天所花的时间，他用来修改 bug 的工作量也有统计数据了。

团队自己收集数据：此外，团队成员应开始每天或每周自行统计返工的工作量（包括评审缺陷，所有数据均记录在缺陷管理系统中）。

BUG #	系统测试BUG描述	开始	结束	总修复工时	备注
5384	当客户选护照（非大陆身份证），大陆地址留空，系统报错，无法继续	9:30-13:30 14:30-16:30 8:30-10:00 7:30-12:00		10	没有完整准备各种场景，分析需求有误
5198	输入错误结束日期，但系统没有报错（估计程序没有校验输入是否正确）	17:00-18:00 20:00-21:30		2.5	
5365	输入-9.99 查询，结果是阳性，不对	19:00-21:00 9:50-11:00 18:30-20:00		5	内部接口，运营没有处理

表 13-2

建议每人自己每天记录当天工时花在什么地方，比如花在缺陷的修复的是多少。迭代结束回顾时，每人看看自己的记录。团队迭代回顾时，因为在系统里面也有评审缺陷的跟踪也可以数出来缺陷是多少，从而可以算到平均一个缺陷花多少返工工时。这样就不会要开发人员花太多时间做记录，但回顾还可以收集到开发过程的所需数据。

从迭代复盘到持续改进

当某些团队已经能做好回顾和根因分析，也能利用数据改善下一轮的开发质量时，如何将这种做法转变为团队或公司的持续改进呢？

深圳有一家公司专门从事内部 IT 服务，因为对质量要求严格，新版本在正式投产（发版）前必须通过测试验收。公司每两周进行一次发版，并对发版成功率进行统计。因为都做了很久，依据以往水平，要求发版成功率不会低于 93（如想更了解这案例的数据变化，详见第 24 章里‘控制图的应用’）

虽然个别项目组的迭代回顾能够有效改善质量，但这些成果仅仅是高层关注下

的“种子”。尽管通过工作量和缺陷的改善能够显著减少持续浪费，但如果高层不关注并投入资源，那么这些“改善”可能只是短暂的闪光，很快大家就会回到以前的工作状态。

反之，如公司把过程改进当成项目来管理，定目标，定期监控，便能像以上案例形成标杆（基线），而且会像上台阶一样逐步提升。

注意：开始时，各团队水平不同，一般难以形成公司级标杆，但可以先形成项目组自己的标杆。

总结

有数据才可以谈改进，但收集数据花精力，所以必须有针对性，二八原则可以帮助识别应针对哪方面入手。软件开发应，软件开发团队花最多工作量的不是编码本身，用于修复缺陷工作量最多。软件开发有个特点，缺陷越后暴露，返工工作量越多，例如如果缺陷在客户使用后才暴露，可能总共要花 20~50 人时修正，但如果缺陷能在前面评审或单元测试发现和改正，就不会超过 1 人时。但很多团队，大部分缺陷都是在系统测试或验收时才暴露，前面评审、单元测试发现的缺陷都很少。

虽然很多团队的根因分析用了帕里拖图、鱼骨图的方式，这些都是基于主观判断的方式，没有真正利用基于事实的数据，而且很容易受从众心理影响，变成项目经理的一言堂，无法挖掘出真正原因，使用便利贴 KJ 方式可以帮团队，更好从事实总结出背后的主因。

团队所有成员一起，包括需求、开发、测试、项目经理等一起分析迭代的缺陷数据，可以计算迭代的缺陷注入和缺陷排除率，利用蒙地卡洛预测模型就可以把本来只是定性的一个改进目标变成定量，也可以满足 CMMI4 级的主要实践（详见附件）。

参考 Reference

1. Juran, Joseph. "Juran's Quality Handbook "(1999) 5/e
2. Gladwell, Malcolm. "Blink: The Power of Thinking Without Thinking "

Part VI

软件工程质量

当团队已经开始每次迭代做回顾，分析缺陷与返工工作量，按二八原则识别改进的针对点后，还需要了解软件工程，包括需求、开发、集成等的常见问题，才能有效分析，并制定下轮的改进措施。高层的理解与支持很关键，所以本部分内容除了让没有开发经验的读者（包括管理者）也能了解外，还解读敏捷开发背后的重要思路、原理。

部分章节最后会总结本章的实践如何对应 ACP、XP、CMMI 等，供正在做过程改进的团队和公司参考。

Chapter 14

精益与敏捷

看完某研发部总监的缺陷分析报告后，我问：“你们什么时候做分析？”
总监：“产品发版后。”他们最近一次发版是八月份，立项早在三月份已经开始。
我：“如果你们分析的目的是之后做改进，发版后才分析的作用不大。要想有真正的效果，就必须让开发团队每次 2-4 周迭代后一起做回顾分析。”
虽然总监说很赞同，但估计他心里想：“我们这几个月一直天天都很忙，中间怎么能腾出时间做回顾复盘！”
一条原则应能改变他的思路，从 2013 年起，这原则一直驱动我每天每月做规划。

2017 年

我有一位拥有 30 多年电机工程经验的老同学李先生，他对 Linux 感兴趣并持续研究，这几年又开始玩树莓派，对此也非常精通。
我找到他，希望在树莓派上安装 Ipython、Jupyter Notebook 等开源应用软件。
一直以来，李先生的理念就是尽量用最简单、最轻易的方法去解决客户的问题，不浪费资源。2017 年，我开始尝试建立 WIKI 服务器给客户使用，他推荐我用树莓派。
他说：“树莓派不仅省电而且快，也不知道你这个概念是否持久，我不会浪费时间——在大型服务器上安装 Linux + Mediawiki，如果你觉得这（树莓派）可以便继续用。如果觉得它性能不够，下一步我帮你换更大型的服务器。”
一直到现在，已经超过 8 年了，我们一直还是继续用树莓派来协助评估（树莓派从原来的 Pi2(1G 内存)，现在已经是 P4(4G 内存)，服务的客户也超过了百家。这证明他的思路是对的。
还有一个细节，当时他安装好 Jupyter Notebook 程序包后，一直没有后续工作。我问他什么原因？
他说：“很简单，我不熟悉软件工程，也不懂如何测试。肯定要找测试一下，我才知道有没有做错。这样可以避免无谓的返工。”所以我们就约好周日我回到办公室跟他远程测试。
李先生打开树莓派后，我用 VNC 客户端进入他的树莓派，打一些最基本的 Ipython 指令，验证没问题。

下一步我要跑一些比较复杂的、要用到 Pandas 包的 Python 程序，却发现还没有安装 Pandas 包。他马上到网上去找安装介质以及树莓派如何安装 Pandas 的方法。处理好后，我再跑对应的程序，包括之前的指令全部跑通。前后花了 2 小时，测试工作按原计划成功。

资源有限、用最低投入达到客户要求，不多做。每当李先生做完一步，如果没有通过测试，就不会浪费时间走下一步。

每小步验证

1983 年，本科最后一年我做毕业项目。导师建议我们参考一些常用的语言发声算法，尝试在专门做信号处理的芯片上写程序，读出一些文字或句子（与现在不同，当时这项技术还没有成熟，很多大学还在研究），虽然我预计有不小的技术难度，但觉得这很先进，便与另一位同学合作，开始制订项目计划。

我们全力投入，一边研究语言发声的算法，一边并行设计电子线路和软件等。我们用了 2 ~ 3 个月的时间做了整个电子线路板的硬件，同时还设计了整个软件架构，并使用计算机模拟，验证每个字实现算法后的发声效果。因为最后一年课程很多，时间过得很快，从 9 月开始用了 6 个月时间，软硬件的设计与开发终于都完成了。但是最终没有发出声音，更不要说能读出一些字和句子了，项目以失败告终。

还有一次失败经历让我印象深刻。在大学的第三年我有幸进入大东电报局 (Cable & Wireless) 电报业务运维部门实习，该部门当时在开发一套新的电脑系统以取代本来基于 UNIVAC 的电报系统，总工程师让我用半年时间，在一个微机上编程从那些电脑上收集重要信息，如果发现异常就警报。按照惯例我前 3 个月花精力做整个系统设计，还买了一些展示电子版，准备用来展示，但由于经验不足，半年后最终什么都没有展示出来。

到了 2000 年，在我兼读软件工程硕士课程时，开始接触到敏捷开发，才意识到两次项目失败的主因：不应该花大量时间去追求前期完美的设计，而是应该逐步迭代，先做一些最基本的简单功能，每小步验证，逐步优化。例如，我的毕业项目中应该先做出最基本的硬件、软件，起码能够发出声音，因为整个项目是开创性的，从未有过成功的实验。

敏捷大师 Dave Thomas 先生在 2015 年演讲里提出敏捷软件开发的核心是：

1. 向你的目标迈出一小步
2. 从反馈调整你的理解
3. 重复
 - 当两种或以上选择的价值大致相同时，选一条让未来更容易修改（软件）的路径。

这些经历让我体会到为什么我们在写程序时，必须先想一下该怎么测试。

每小步验证算是敏捷原则里“要频繁地交付”，但其实是精益（LEAN）的核心思路。

精益？是敏捷的概念吗？为什么 2001 年提出的敏捷宣言和 12 条原则都没有提精益？

很多敏捷团队使用“看板 (Kanban)”，其实并非源自宣言或 12 条原则，而是来自丰田的 Just-in-time——按客户订单拉动物料的采购。

Mary Poppendieck 具有多年编码经验，也为 3M 公司开发系统，完善生产流程，很了解精益的力量，2003-2010 年间出版了 3 本著作，提出软件开发也可以使用精益做改善（详见参考）。

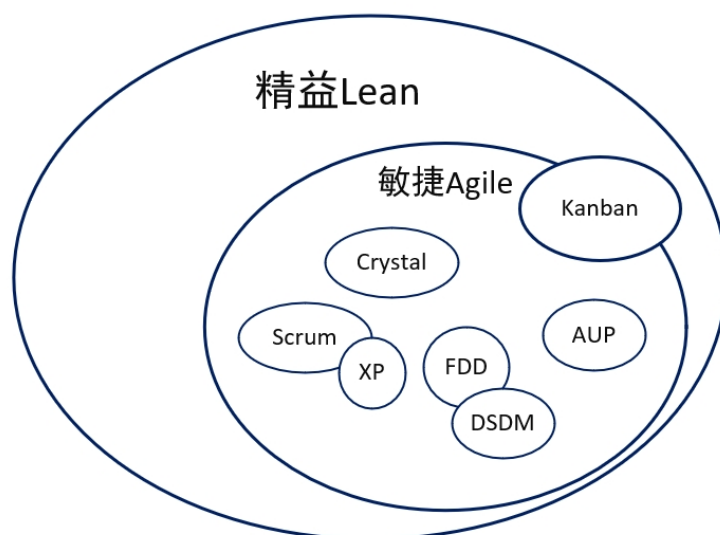
精益强调：

1. 减少浪费
2. 确保质量、零缺陷
3. 快速交付
4. 不要过早下定论，尽量延后承诺

虽然策划重要，但不是死板地按计划执行，制定几个月的长期计划，要灵活按迭代交付后再优化策划。第二章里谷歌创始人 Larry Page 拒收 CEO Eric Schmidt 的商业计划书“芬兰计划”，请他先与谷歌工程师聊聊。这和包括频繁交付 (Ship and iterate)、管好失败 Fail well、针对客户 (Focus on the user) 等谷歌文化，都属于精益。

丰田专注零缺陷、零库存，都是精益。

精益与敏捷是相对吗？敏捷宣言和 12 条原则主要是针对当时软件开发的各种不良现象；精益软件开发更强调整个系统的持续改善。两者是互补的，相互渗透，敏捷与精益之间的区分也逐渐模糊。所以敏捷开发演化至今已经包含了精益思路。PMI 的 ACP 把敏捷归入精益其中：



精益可以用于软件开发吗？完全可以但需要变通地理解。可以先从了解软件开发浪费开始。

浪费 (Waste)

在软件开发，除了缺陷外还有哪些浪费？除了软件开发的缺陷（对应生产的不良品）外，其他 6 类生产浪费都能找到对应：

工业生产	软件开发
过程中库存 (In-Process Inventory)	半成品 (Partially Done Work)
动作 (Motion)	任务替换 (Task Switching, Multi-tasking)
生产过多 (Over Production)	多余的功能 (Extra Features)
加工 (Extra Processing)	重新学习 (Relearning)
搬运 (Transportation)	交接 (Handovers)
等待 (Waiting)	延误 (Delay)

除了缺陷，和多余的功能（会在后面软件需求里讨论），其他的浪费的说明如下：

半成品

未完成的产出物包括：

- 1. 没有对应软件代码的文档
- 2. 没有测试过的代码
- 3. 没有文档说明的代码
- 4. 代码没有实际安装使用

任务替换

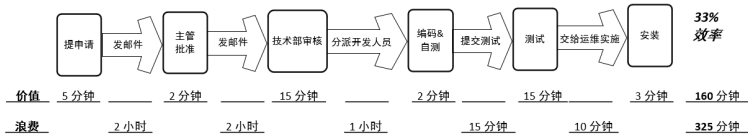
原因可能是要同时处理几个开发任务，导致无法集中完成任务；过后想完成时要多花时间，引起返工。

注意：请不要误以为不应同时做多个项目。例如，写文章或编程，如果没有思路无法进展，我会转做其他；生产率超高的作曲家舒伯特同时会创作多首作品。所以前提是策划任务时，要让每项任务有足够的时间“完成”。

延误 (Delay)

在软件维护，引起延误的最大原因是什么？通常是系统或流程的问题，并非开发人员的质量水平。

精益里有价值流 (Value Stream) 分析，可以区分哪些过程是产生价值，哪些是浪费。



例如：从客户提出需求到最终完成修正、使用，前后共需要 8 小时，但对客户有价值的过程只占 33%（2.5 小时）。如果能减少或省略无价值的过程，如发电邮到主管和组长等待审批，就能减少延误。

软件交接 (Handovers)

因为接受方不能完全理解，最终产生返工，导致浪费。例如开发完后交给其他人维护，如果他之前没有参与，便难以顺利交接。

重新学习 (Relearning)

开发是不断学习、创造的过程，因没有及时做记录，过后很多时候自己也忘记了，导致后面重复，引起浪费。

第二章里，Weisbord 先生印刷公司的订单处理部门便是一例：，在开始团队制前，任务都是按业务流程分工，会产生以下浪费：

任务交接 - 从输入订单，检查客户信用，发起内部生产订单等，各由专人负责，导致很多任务交接或任务替换。

半成品 - 从收到客户订单，到交付产品后，收到支票并内部对账，过程中的产出物需要等待下游处理。

后面变成团队负责制，团队自己处理所有相关任务便能减少这些浪费，也能减少整个订单的处理时间。

软件开发公司也类似，例如：

专门开发游戏的公司，为了加快开发出新游戏的速度也会从原本按功能部门分工改成团队负责制，每个团队负责一款游戏，除了能更快速开发出新游戏外，也能减少浪费。

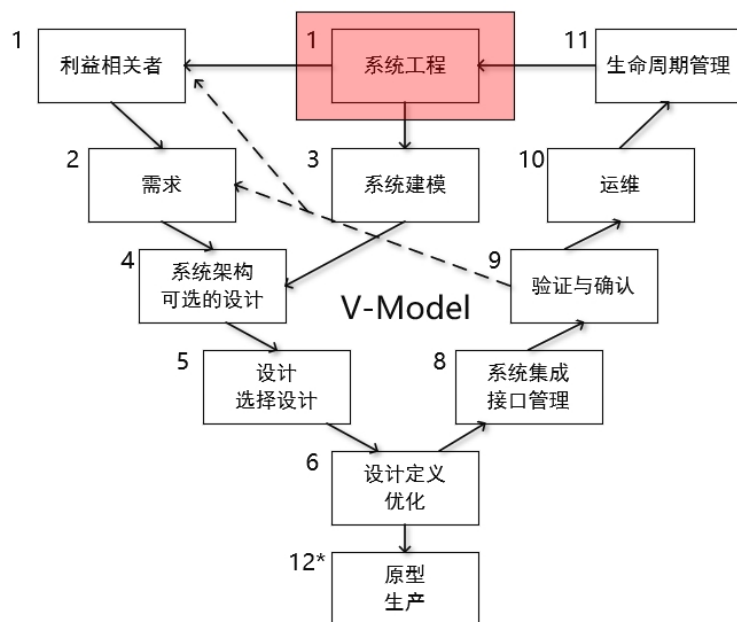
非开发人员如何应用精益

大家都想尽量避免半完成的任务，避免浪费。所以精益的应用范围很广。随便在网上搜索“精益”，便能看到“精益管理”、“精益思想”、“精益创业”等各类应用。例如，如果今周每天都有培训安排，只有晚上能腾出 2 小时左右完善这本书，我会把时间用于哪些改动不大的章节，因从历史数据，如果某章就须要大规模重写，会起码要 1 - 2 天才能完成，会尽量安排在周末做，尽量避免在零散的时间段里做，减少浪费。

例如金庸写长篇武侠小说都是每周在香港《明报》发布，最后累加成书。因每周每天可以腾出的时间有限，我自己也是使用精益思路写书：写完一章后先自己读一篇，修正错误；然后发到群里让朋友圈读，按反馈修正；尽量定期在 CSDN 发布，每 2 周发下一章，给自己压力尽早完成，但因之前已经印刷了 2 版，发现出现太多应可避免的错误，所以也不断提醒自己不能因进度而牺牲质量。

系统工程 V 模型

从 50 年代开始出现编程语言，软件程序越来越庞大，越来越复杂，难以维护。是否可以用系统工程思路来改善呢？系统工程主要用于比如飞机、汽车等复杂系统，要设计新款飞机，先从需求开始，按 V 模型，验证确认后可以最后交付。例如按系统工程的概念，我一个产品例如汽车，应该先有个产品分解的思路，汽车由车轮、引擎、油缸或者电池等各种组件组成，应先有总体设计，然后一层一层分下去，里面也可能包括一些软件组件。



虽然除了缺陷外，其它 6 类浪费也会有极大影响，但因这部分主要讨论软件工程，下章会从缺陷的角度，看看导致软件难以维护的原因与改善措施。

本章最佳实践对应：

- PMI、ACP
 - 里面持续改善 (Continuous Improvement)，价值驱动交付 (Value Driven Delivery)，策划 (Adaptive Planning) 等领域都能对上精益思路
- CMMI
 - 虽然没有特定实践域包含持续改善，但 PCM、CAR、MPM 等实践域都支持精益的持续改善

参考 References

1. Poppendieck, Mary, Poppendieck, Tom *Lean Software Development* (2003) Pearson Education

2. Poppendieck, Mary, Poppendieck, Tom *Implementing Lean Software Development: from concept to cash* (2007) Pearson Education
3. Poppendieck, Mary, Poppendieck, Tom *Leading Lean Software Development* (2010) Pearson Education
4. *Agile Practice Guide* Project Management Institute(PMI) (2017)

Chapter 15

TDD 与重构

从 50 年代起，软件系统越来越普遍，规模越来越大，越复杂。但延迟甚至失败的项目也越来越多。

如果我们沿用系统工程的思维，是否可以像造一架飞机一样，写好大型的复杂软件？

1968 年，Dijkstra 提出要 structured programming 方式编程，所有的程序都应该跟设计飞机一样，要先总体设计，然后一层一层分解下去，程序里不能随便用 GOTO 语句。七八十年代的软件开发几乎都是用这种思路去设计，但发现复杂项目还是经常失败或者严重延误。

```
{| class="wikitable" |-| 某软件开发公司为某电信公司提供让家庭用户线上付费  
下载电影的服务平台，开始时，这服务平台一直按需求变化有更新和修改，但越  
来越困难，直到 2 年前，因为这些代码像意大利面一样双互依赖，而且完全读  
不懂（而且原先的开发人员早已离职），维护工程师已经不敢动这系统的大部分  
代码，系统代码虽然还能使用，但功能完全不能修改，我们叫这些是“死代码”。  
|}
```

2001 年，敏捷大师提出敏捷开发，他们大部分（如 Kent BECK）都熟悉面向对象设计软件。针对当时软件行业的问题，提倡用敏捷开发的方式，让软件工程效率更高，质量更好。

Structured programming 那种从上而下的设计有什么问题呢？为什么不一定能适用于建立大型复杂的软件系统呢？

有两个好习惯能帮助团队免“死”，测试驱动开发是其一。

测试驱动开发 (Test Driven Delivery TDD)

在编码之前，应先写测试用例，然后才编码去通过这测试。写完代码后再补上测试用例，就失去了单元测试的原意。假如你是大学老师，教大学生编程，学生数量超过一百，不可能只要求他们就按你出的练习题写代码交卷，因学生写出来的程序会五花八门，如果老师像批改语文课一样改卷，会耗费很多精力，怎么办？美国麻省理工老师除了给学生题目，也会提供模版，包含单元测试的框架，类的头跟尾，并明确告诉学生，必须确保单元测试通过才可以交卷。这样

就可以让学生 DIY，自己先检查代码能否跑通，没问题才可以交卷，省了老师很多时间。

承包商必须按蓝图施工，确保建筑物合乎建筑师设计规范。在软件开发项目里，什么东西等同于蓝图？设计文档等同于蓝图吗？无论设计文档写得多么细，还是有多种代码设计方式能实现，所以编码本身才是蓝图。为了确保产品符合设计要求，软件设计师、架构师应不仅仅写设计文档，更应先明确分好类和单元测试用例，然后让初级编码人员去填上代码，才能确保软件符合设计规范。单元测试应该是设计的一部分。脚本式单元测试也有助于其他人读懂软件设计。

“我一直很认同单元测试的作用，我之前是要求开发人员写单元测试，用脚本自动跑，但编码人员说工作量太大，我也想不到什么好的理由，就没有再推行下去。”某技术部经理说。

改变工作习惯很困难，所以不能依赖下命令要求他们做单元测试，务必要他们自己心里觉得写单元测试反而会节省工作量。例如在每 2 周迭代回顾时，要求团队所有成员一起分析缺陷和返工工作量：

例如，系统测试平均返工工时 = 总返工工时 / 系统测试缺陷总数。

如何以数据说话，持续改善

有些管理层只关注进度是否延误，能否按原计划发版，不关心代码和设计的质量。他们觉得是否大部分缺陷都在最后系统测试才暴露、是否有充分的单元测试覆盖等都是偏技术的指标，无法关联到上层关注的指标，如生产率，进度偏差等。但如果改为统计：

- 修正一个缺陷的平均工时
- 开发一个需求功能的平均工时

便可与业务关联，如果能做好单元测试，就能提前发现并解决缺陷，减少返工工时；减少了修正的时间，总体交付功能的工时也会减少。

如何收集数据

资源有限，没有系统，没有问题，沿用精益原则，先基于现有资源实验、尝试。每天在个人的小本子上写上今天计划的主要任务，包括开始结束时间，按实际记录任务所花的工时，到周末把本子上的数记录到电子表，自我回顾，识别哪些是可以避免或减少的浪费，分类，然后参照上部分“如何做好迭代回顾”，利用二八原则识别主因，把主因和纠正措施写成 A3 报告分享。经过 2-3 个月后，结合每轮 A3 报告，总体回顾团队这段时间的改进量。

管理软件开发团队的困难

“最头疼的问题是当开发人员离职，其他人无法维护他写的代码。影响了客户对我们的信任。”某位有 15 年经验的项目经理说。

如果其他人无法读懂某人写的代码，以上的问题就很容易发生。定期完善代码能避免这类问题（一般人经验有限，虽然代码能跑通，但不一定能首次便能写好代码。）这方法叫重构 (Refactoring)，是避免‘死’代码的另一个好习惯。但

不要误以为重构是等到软件已经难以维护时，大规模重写，而是用精益的思路，每一小步改善。

例如把一些重复代码抽象出来，变成函数 (function)/方法来复用。目的是希望能改善代码的可读性，因如果写得不好，过了几个月连开发人自己都看不懂。如果想定期做重构，维护代码的质量，能自动运行的脚本式单元测试非常重要。因为如果没有单元测试，便无法知道改完以后会不会有影响，本来的功能是否正常，就不敢更改任何一行代码。

反之，有了单元测试保证，便敢一小步一小步进行重构。首要目标是让代码易读性，一般人也看得懂。例如先减少重复代码，或写清楚一些变量命名等。

设计

软件与硬件最大的区分就是改变功能很容易，更新代码后就可以做出新功能。要改变硬件的功能就必须整个更换。但软件项目经理很怕开发后期出现大量需求变更。

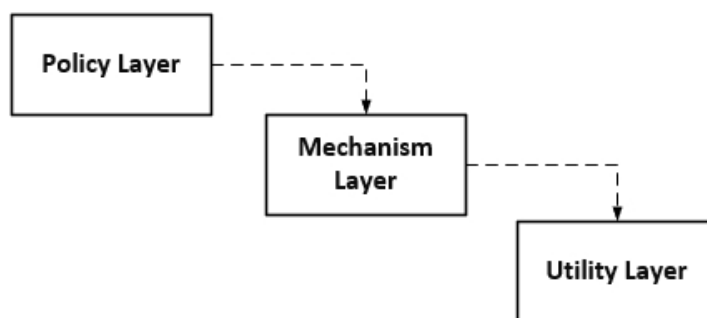
在土木工程项目，到了最后交付前还是会出现不少修改，例如管道、供电电线、插口等，项目经理习以为常。为什么软件本来比硬件灵活，反而怕出现变更？

系统包括上千万行代码，一般人难以理解里面设计的来龙去脉，反过来大家都很清楚大楼的喉管、电线等施工情况，也不会怕修改后影响大楼的其他部分。但大型复杂软件就不一定；如果设计不善，可能某功能修改会影响了另一个模块的功能！

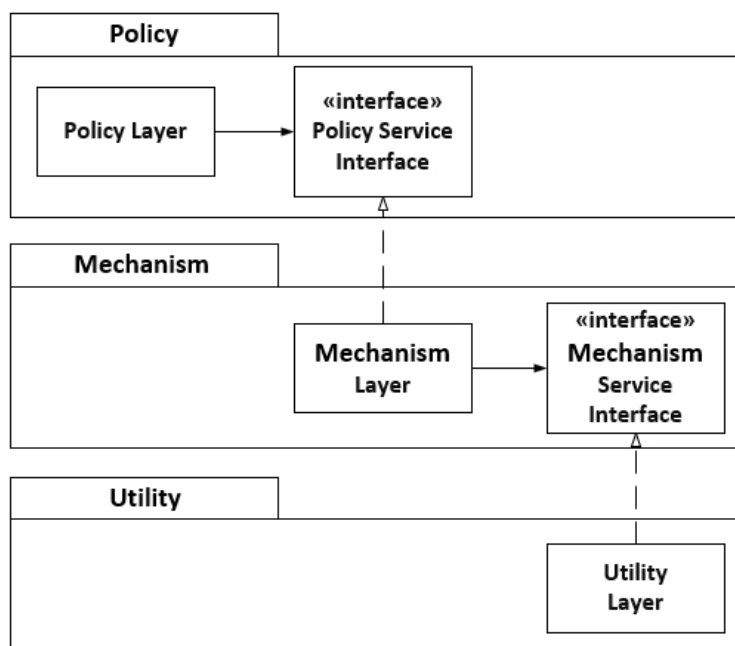
虽然 Alan KAY 早在 1966 年已经提出面向对象设计 (Object oriented design)，在 1972 年 Xerox PARC 推出全面支持面向对象设计的 Smalltalk 语言，但当时很多人不理解，结构化设计 (structured programming) 一直是主流。

虽然 80 年代有 Objective-C (1980)，C++ (1985) 等面向对象语言出现，直到 1995 年 SUN 推出 JAVA，面向对象设计才逐渐变成主流。面向对象如何能减少系统模块间的耦合度 (coupling)，让系统更容易修改、维护？

面向对象的设计，其实也是一层一层，从最高层的策略 (Policy) 到中间层 (Mechanism Layer)、最下层 (Utility)：



但我们不希望在设计软件的时候，上层依赖于下层的详细实现，避免因为下面的实现细节改动影响上层模块。如果用 UML 的表示，下图是面向对象设计方式：



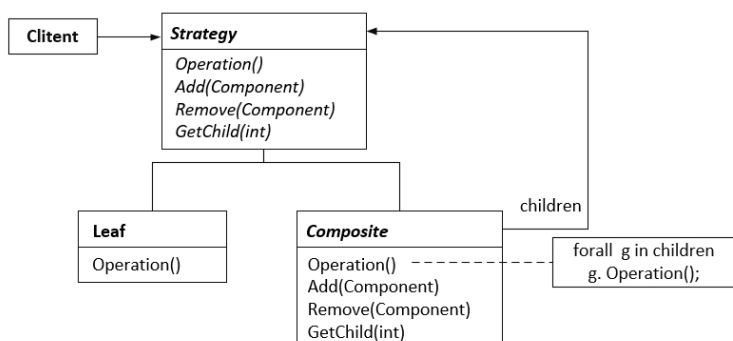
上层只看到下面的接口，不会直接在代码里具体放某一个对象 object，这样就可以避免下面详细的代码影响上层代码。上层可以利用接口调用到下层，但上层代码不会包含下层的具体对象 (object)，这样就保持了灵活性。

实例说明：MVC (model view controller) 框架把显示、模型、控制（例如经键盘输入）分隔开，大部分应用系统会用不同的显示方式表达数据，例如饼状图、柱状图等。

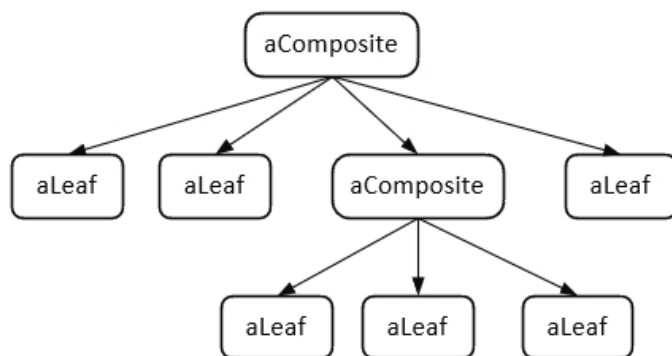
650px

按 MVC 框架，使用 view 里的模块做展示，里面包括各种图形展示方式或报表方式，但 view 跟模型数据模块是隔开的，双互独立。若要新增柱状图方式展示数据，只需要在 view 里增加支持柱状图展示的代码便可，其他模块不受影响。同样思路也可以用于 controller，我们不仅需要展示，可能需要接收不同的输入方式，例如传统键盘，但如果后面需要使用弹出模拟键盘，也可以只增加模拟键盘的代码，但 controller 主程序不受影响。

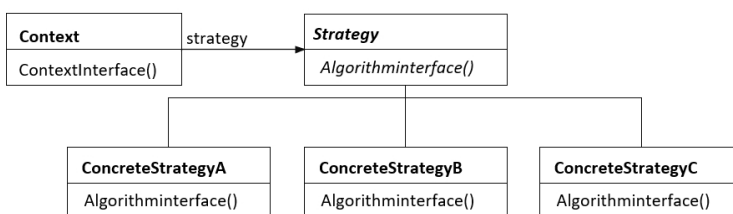
细节方面里面 view 跟 controller 的体现方式会有区分。view 通常可能会有多层展示，例如报表可能只使用饼状图，但有些报表可能要包含饼状图、柱状图、和表格等合成。但如果是刚才 controller，就只有某一种方式，不会拼起来用的。前者是一种设计模式，叫组合 (Composite) 模式。也满足开闭原则 (Open Closed Principle OCP) 软件应对变更关闭；对扩展开放。



aComposite 也可以包含另一个 aComposite，做成多层组合 (nested Composite)



后者叫策划 (Strategy) 模式。也满足依赖反转原则 (Dependency Inversion Principle DIP): 上层模块不应依赖于下层模块; 都应依赖于抽象; 细节应依赖于抽象, 应针对接口编程, 不针对实现。



所以重构不仅仅写好变量名称, 减少代码拷贝粘贴, 更应包括如何管理模块之间的依赖关系、耦合度等。

当我们做重构时, 应考虑这些原则和设计模式, 有点像乐高积木一样, 帮我们系统地设计软件。

希望任何功能的改变, 尽量不会影响上层的代码, 让整个系统更灵活, 保持总体架构容易维护, 更要让其他人能理解、读懂。

所以, 如果了解面向对象的原理, 再配上定期重构, 就可以避免出现死代码。这是长期学习过程, 除了培训和自学, 也可以利用评审促进。例如让经验丰富的

组长检查代码与设计，让经验少的编码人员尽早知道问题所在并改善。

“我很忙，哪有空做这些计划并记录？”

做这些记录，每天绝对不会占用超过 15 分钟时间，但好比锻炼马拉松，有度量才能知道今天跑一公里多少分钟，有没有进步或保持。我首次感到这习惯受用是十多年前考 CMMI 培训师，观察员（考官）会记录我每个模块实际花的时间，与计划对比。因为每天培训时间有限，必须在计划时间内完成。我一直保留这习惯：每天培训前，先在本子上写上计划时间，培训过程里记录实际时间，每天培训结束后在 XLS 里把计划与实际都记录。好处：我下次再培训同样内容就能更好策划实际的分配，也提醒我每个模块不能超时。这方法能适用于任何活动，帮助估算：例如，我发现常常低估一些编程和写分享文章的工作量（因少做，不熟悉），但一些重复性工作会越来越快。这方法也能帮助避免无限拖延。所以回报能远远超越投入的工时。

如果团队有每次迭代冲刺回顾，以上的个人数据可归纳为团队数据，团队所有成员一起分析，制定下轮的改进措施。当然，采用相应的工具和技术，可以更好的提高效率，多记录，少占用时间。

软件开发以外

能让读者继续读下去的书，必须有头有尾，前后贯穿，有节奏。当我准备这一版时，就发现前面的版本都只是原始资料，没有组织好，除了要调整架构提高可读性外，还有很多多余的文字，须要精简。

代码重构也是同样的过程。只是勉强写好通过测试，程序员应该像工匠一样，精益求精，最后写出大家能容易读懂的代码，并经得起时间考验，方便维护。

书写得不好，读者可以选择继续读下去，但程序没有写好，后果可能致命，所以管理者应关注代码的质量，不要以为程序通过测试，能按时交付，就算完事，只是开始。

本章最佳实践对应：

- CMMI
 - 单元测试：PI 2.4 , OO 设计：TS 2.1
- XP
 - 测试驱动开发 TDD、重构、编码标准都是 XP 里的实践

参考 References

1. Gamma, Erich , *Design Patterns: Elements of Reusable Object-Oriented Software* (1997) Addison Wesley
2. Fowler, M., *Refactoring*2/e (2019) Pearson Education
3. Martin, Robert C, *Agile Software Development: Principles, Patterns, and Practices* (2002) Prentice Hall

Chapter 16

评审与结对编程

客户：我们的客户以银行为主，他们很注重质量，所以一直很注重评审。他们对需求评审、代码走查等也很赞同，也能找到缺陷，对提升质量有作用。但他们最困惑的是通过设计评审很难发现缺陷。

我：你听说过敏捷的结对编程吗？

客户：听过，也给客户做过培训，但是一般管理层都接受不了用两个人干一个人的活，所以几乎都没有团队在实际工作中使用。

我：单做培训确实难以推动并持续，必须按 3 步走，按部就班才有效果，让我分享一下。

背景

4 月

现场结对编程培训之前 8 个月，首次见这家客户的总监。他们想用一年时间改进软件开发质量。事后知道原来他们有个超过 12 月的大型项目，因为验收时质量不达标，不能通过风险的验收测试，被拒收。因为公司是老字号，不想影响声誉，请求客户给机会让他们重头再做，客户同意但估计产生的损失超过 100 万，所以高层下定决心必须改进软件质量，避免类似情况再发生。

7 月

与客户签了 CMMI 服务合同后首先做差距分析，首先收集各领导的关注点。某部门经理最担心的是系统维护：如果有人员离职，新人要花很长时间才能上手。

技术总监更担心是有些交付后的缺陷会很花工作量：如某系统交付后开始使用的时候很正常，但过了 9 个月，用户量和数据量都增加了，偶尔会出现有些报表出错，我们架构师、工程师等，花了 3 个半月才发现其中某接口的参数有问题。另外发现开发人员水平参差，也没有做好单元测试和代码评审，代码可读性低，其他程序员难以理解，在分析后的汇报里，建议利用 TDD 和面向对象的设计原则改善编码规范，并加强培训，推行结对编程培训，提高代码的可读性，方便

日后维护。

最后发现很多项目资源不足，没有充分的时间确保代码的质量。

给管理层汇报

因缺陷绝大部分都是后期测试才发现、缺陷修改的总工作量很大（估计占项目总工作量 30%），建议加强开发单元测试。

因代码都是个别程序员负责，建议团队用结对编程，除了能提高质量也防止一人离开其他人无法跟进的风险。

也预警管理层开始时要预留 15-25% 时间给开发团队，当后面返工量减少，以后的总工作量会降低（生产率提升）。

2 天培训课

第一天讲面向对象设计原理，设计模式，测试驱动开发；

第二天讲结对编程：



Quote from an interview with
David Parnas in 2009

Q: What is the most often-overlooked risk in software engineering?

A: Incompetent programmers. One bad programmer can easily create two new jobs a year. Hiring more bad programmers will just increase our perceived need for them.

问：在软件工程常被忽略的最大风险是什么？

答：不称职的开发人员。据估计，美国需要的程序员数量超过 20 万。这完全是误导。这不是数量问题。我们有质量问题：一位糟糕的程序员一年内便很容易就会产生两个新岗位（的工作量）。如果我们有更多优秀的程序员，并且能够轻松识别他们，我们需要的就会减少，而不是增多。

“也有人说：“卓越的程序员比优秀的程序员，不仅是好一些，而是倍数级的差异。”

请问你们觉得团队里程序员的水平也存在很大的差异？”

培训开始，对应以上幻灯片，我问大家。

你们是否觉得：

- 交付后的软件难以维护

- 其他人难以读懂其他人的代码，员工离职后其他成员难以跟进和维护
- 代码总体设计不好，如缺乏良好架构，不灵活
- 系统交付后，有些缺陷需要很长时间才能修复好

如大家希望提高代码质量，必须从人入手。

如果你们团队也有同样问题，有什么好的解决办法呢？培训？

编码的最佳学习方式并非读书，而是动手做，所以单靠搞内部培训效果有限。有什么好方法帮助开发人员提升？

代码评审

就算评审规范做得很好能帮助初级程序员提升吗？很难，因为代码都已经写完了。

比如我跟另一位老师看团队的代码，他们都写了起码几千行。我们的结论就是：虽然代码的设计很有问题，但已经到了这个地步，除非重写，否则单靠修正部分代码并没有帮助。这也是代码评审常常难以做好的主要原因。而且大家都要面子，如果在评审里，直接说他设计有问题，也可能会影响到大家的关系，所以在设计评审时难以找出缺陷，这很正常。

团队主管看程序员代码只能把关防止严重错误，对程序员提升没有作用，所以评审不能有效保证代码质量，也不能帮助团队能力提升。

当写完了几千行代码后，就很难在评审时要求开发改动。程序员会觉得又不是不对，测试也跑得通，为什么要改。所以代码的质量问题应在写的时候就避免，这是结对编程的原理。你可以把结对编程看成是提高团队之间互相交流的培训方法。因为结对编程是在编码时针对具体问题集思广益进行解决，很容易落地，效果会比培训或评审好。

两个人互相讨论写代码才就是有效的培训，不能单挑单靠设计和编码规范和指南。

培训设计

有些人误以为：

- 结对编程必须要找一位合适的搭档，人与人之间是否合得来最重要
- 结对时只是在旁边看
- 以为仅仅是教同伴应怎么做
- 以为只是看，作为人手编译器，只看代码是否写对，能否通过编译

我们针对这些重点设计一天培训。

培训重点

要培训有效，必须利用练习，让学员动手动脑筋。

结对编程和实践说明
练习 #1 和讨论：代码阅读
结对编程说明
练习 #2 和讨论：设计和结对编程
练习 #3 和讨论：Code Review 和 Pair Programming
练习 #4 怎样开始结对编程：10 分钟结对编程
讨论：10 分钟正确结对编程
练习 #5：队伍结对编程
Pair-Solo Programming

练习一：用 10 条代码实例，每一条分 A 与 B 两种方式写，问学员那种更容易读懂，说明代码应怎样写才能让一般人容易读懂

练习二：先自己按面向对象做总体设计然后两人结对分享设计做出最终设计学员讨论结对是否有效

然后解释若要持久，必须轮流替换伙伴，也介绍各类结对方式的利弊，例如：

Public-private pair programming ; pair - solo programming

练习四：先 2 人试试结对

练习五：六人为一队，轮流合作写程序，让学生体验交换。（详见附件）

经验分享

结对编程需要 2 人对话沟通，例如某人写代码时，另外一个人提问题，有些不习惯编写代码边说话沟通，培训时，预先写好台词。如果想多了解结对编程如何沟通，可参考书的实例。

有些程序员比较内向，不想说话，我们会用角色扮演：先写好台词，让他们展示在结对编程时如何沟通。帮助他们先打出一步，鼓励尝试。

注：如果了解结对编程如何沟通，可以参照 Robert MARTIN 先生的保龄球游戏实例（参考书里第 6 章），了解两个人如何从需求、总体类设计、按 TDD 写单元测试用例、然后写代码，让它通过，一步一步最终完成整个程序。

总结

培训开始时用例子说明结对编程除了能帮助团队知识分享和提高团队精神以外，更能：

* 提高代码质量和可读性（编码规范）

- 提高代码复用
- 加强测试驱动开发

利用互动练习让学员亲身体会结对编程的原则：

1. 一边写代码一边说

- 2. 提问，如：这个方法是否归为这个类
- 3. 每人带纸和笔（最好铅笔）
- 4. 不断想复用

--- *** === *** ---

客户：我猜你说的 3 步是：

第一步，高层认同软件质量问题的严重性，为此交过学费，知道损失多大

第二步，管理层不仅了解软件的质量问题和改善措施，更需要内部立项做培训和投入资源（人力），做过程改进

第三步才是具体的培训课。因为缺乏第一步和第二步，所以之前做完了结对编程培训后，没有效果

我：厉害，完全对。结对编程只是改进的一种方法，如果高层不关注、不立项、不投入资源，不会有改进效果。

结对编程这方法在 70 年代已经开始，与 TDD、代码重构已经都经过验证能有效改善质量减少返工。所以结对编程本身没问题，要解决的问题是总体质量改进。

若要成功推行结对编程，必须先说服管理层，投入时间和资源，但投资会有好回报。

结对只是一种有效的改进方法，团队必须有每个迭代回顾，持续提升的好习惯，不可能单靠管理层下命令来推行，更好是团队自己有改进的动力，选择使用结对编程来提升质量，减少返工。

本章最佳实践对应：

- CMMI
 - 单元测试：PR 2.X 3.X 所有实践
- XP
 - 结对编程，代码共同拥有都是 XP 里的实践

附件

Ex 5

时间	组一	组二	组三
1	A 和 B 结对并打开信封X	C和D结对并打开信封Y	E和F结对并打开信封Z
	看清楚需求，然后把X掉了	看清楚需求，然后把Y掉了	看清楚需求，然后把Z掉了
	A和B 开始结对编程，A写B看	C和D开始结对编程，C写D看	E和F开始结对编程，E写F看
2	过了4分钟转换		
3	F和A结对编程X，F写A看	B和C结对编程Y，B写C看	D 和E结对编程Z，D写E看
4	过了4分钟转换		
5	E和F结对编程X，E写F看	A和B结对编程Y，A写B看	C和D结对编程Z，C写D看
6	过了4分钟转换		
7			
完			

Chapter 17

持续集成

为了避免客户验收前或使用后才暴露大量棘手缺陷，可能要花很长时间才能发现并解决，便应依据精益和系统工程的原则，把系统拆分成子系统/模块，先开发并测试子系统/模块、集成、再测试，按部就班地完成整个软件开发。

验收测试

验收要满足需求，需求要可验证，所以通过相应需求的测试成为验收通过的必要条件。需求确定后，就可以规范验收通过的准则，开始准备验收测试了。

很多 QA 人员以为验收测试只需要做黑盒测试：

- 把开发出来的系统看成一个黑盒，开发人员已经做好本身的所有自测（包括单元测试集成测试等），然后系统要通过我们的系统测试用例，包括功能与非功能。

常见问题

当开发人员完成了所有模块的开发，开始进行系统测试的时候，往往已经到了项目（或迭代）交付预期的后期，这时如果发现大量缺陷，开发人员没有时间做好修复，即便修复后，也没有充足的时间做回归测试（重新跑本来通过的测试用例，确保不会因为开发人员的修改，而引起新的错误）。

改善方法

要避免这种恶性循环，减少发布延期的概率，QA/测试人员就要关注前面评审和测试（如单元/集成）缺陷排除率，以降低质量成本的概念，尽早在项目的前期暴露缺陷并及时解决，不要等到系统测试时才处理。所以不能单靠开发人员自觉做单元测试/集成测试。

从开发人员的视角，他不会想主动用测试找出问题。最佳场景：开发完成，交给系统测试。没有发现缺陷，通过并结束，再开发新的模块。“精明”的编码人员思路：“如果对测试/缺陷没有要求，多严峻的开发进度目标，我都能轻易达成。”

所以要做好验收测试，QA/测试人员不仅仅是做最后的黑盒系统测试，而是要从单元测试开始，设定好测试通过准则 (Definition of Done DoD)，然后交给开发人员执行，要看到所有测试，从单元测试开始，都完全通过才算通过验收，以避免上面的问题。如果 QA/测试人员的技术能力有限，起码要明确所有测试的准则（测试用例）。反之，QA/测试人员仅做最后黑盒测试，不能按时交付/交付质量问题，始终难以解决。

测试策略小建议：

1. 在项目一开始便要开始写测试用例（在设计、编码之前）。
2. 代码每次有任何变动时，利用持续集成系统自动跑测试。
3. 有了自动化就可以轻松实现回归测试。
4. 以上不仅仅是覆盖功能测试，也包括非功能，例如安全性、性能负载量等。

目的：尽早让开发人员获得反馈，以此驱动他们能随时达到一个可发布的状态。自动化让测试人员可以集中精力针对一些特有的测试，减少重复性测试的工作量。

测试自动化

与设计开发人员探讨他们的产品集成过程：

问：你们有没有用一些自动集成的工具，例如 Jenkins 集成工作自动化，原因是集成很花时间，现在也越来越流行持续集成、每天集成，所以如果靠手工集成、打包是很花工作量的。

答：没有，我们还是手工。

问：你们在手工集成之前怎么确保这些模块已经可以？

答：我们有测试。

问：做什么测试，可否举些例子？

答：我们开发完以后，会输入一些查询，然后看是否能正常使用。

问：我不是指整个系统功能的测试。例如，要通过你们 Java 开发，有自动化单元测试，可以写脚本，如果你单元测试通过，就变绿，不然的话就红，你们有做吗？

答：没有，我们都是手工测试。其实我们现在到了产品升级、增加功能的维护期，记得当初首次开发的时候，我们开发人员是有用 Jenkins 自动化集成，但现在维护期我们就没有。

从以上对话可以了解到，很多开发维护团队基本就没有再做一步一步集成的过程，只是针对修改的功能，开发完就测试一下，然后便交到系统测试。

如果只是依靠对整个系统的集成测试：

- 不一定能找到所有问题。当我们改动了一些代码，却没有通过模块的单元测试，就很难确保模块的每一次改动都是正确的。所以，这将导致如果后

续确有问题的话，就很难定位或者难以修复。

- 如果没有单元测试的话，你其实是不敢改动代码的，不知道改完以后对不对。

所以单元测试很重要，也因为单元测试高复用度，所以需要自动化才能节省大量手工测试工作量。

集成的概念，就是先写自动化单元测试用例，然后才写代码。通过单元测试才可以进入下一步集成测试，两个模块之间是否通。到最后进行总的系统测试，一步一步去做。如果团队像刚才那种情况，绕过了那些步骤，直接做总系统或集成测试。就会有很多隐患，到最终的验收测试才被发现，付出大量的缺陷修复代价。产品集成很重要、很花时间，所以绝大部分的团队如果注重软件质量的话，都会把产品集成自动化。改了代码之后，每天靠脚本来通过单元测试，比如晚上自动跑产品集成发现的问题，第二天暴露给开发人员修改，修改后再集成看看有没有问题出现。有些公司除了要求单元测试通过以外，也会要求通过代码的静态扫描，道理都一样。只有用这种一层一层进行产品集成的方式，才可以有效确保软件的质量。正如 Cunningham、Fowler 等敏捷大师所说，没有做好集成过程里的测试会产生很多债务，产品交付给客户以后，因为质量问题导致还债的代价就更高（详见后面“技术债务例子”）。

持续集成

持续集成的道理很简单，每次提交代码做的修改，系统都可以自动构建整个系统，并跑通相关的自动化测试，如果发现自动构建出错必须立马停下来修改缺陷。（详见附件“持续集成步骤”）团队要开始持续集成，起码要具备以下 3 个条件：

1. 版本管理：所有的代码测试脚本、数据库脚本/构建脚本等都放在版本管理系统管理，不仅仅是代码。
2. 利用工具实现自动构建（例如 Jenkins），虽然很多 IDE 集成开发环境都有这方面的功能，我们建议还是要用命令式（Command）去跑构建脚本（像跑程序一样），每一次构建都能有详细记录（也随时可以重跑）。
3. 持续集成不仅仅依赖工具自动化，还需要整个团队高度付出、参与和自律，每个人频繁以每个小步交付他的开发部分。James Shore 先生在“Continuous integration on a dollar a day”文章里提出不一定依赖自动集成工具（例如 CruiseControl 是另一种类似 Jenkins 的集成工具）只要每个团队成员都做好自动化单元测试，每次提交代码都使用版本管理系统合并，也能做到持续集成。



不要以为每个开发人员每天提交（Commit）代码到版本管理系统就算做到持续集成了，还需要：

- 每次提交代码要确保已经测试过，没有问题。
- 所有部署发布后的问题都能在 10 分钟之内解决。

不然就不算达到持续集成的基本条件。所以每天提交只是持续集成的第一步。

所以要做到持续集成得符合以下 3 个条件：

1. 是否频繁把变更更新到主干去。
2. 有没有对应的自动化测试，确保集成后的代码是能通过所有测验。
3. 如果构建失败不能通过测试，必须立马停下修正代码，直到测试通过。

有人会质疑为什么要这么频繁去更新主干的代码，自己分析测试好不也一样？很多持续集成的团队只做到第 1 点，部分能做到第 2 点，但能把 3 点都满足的很少。

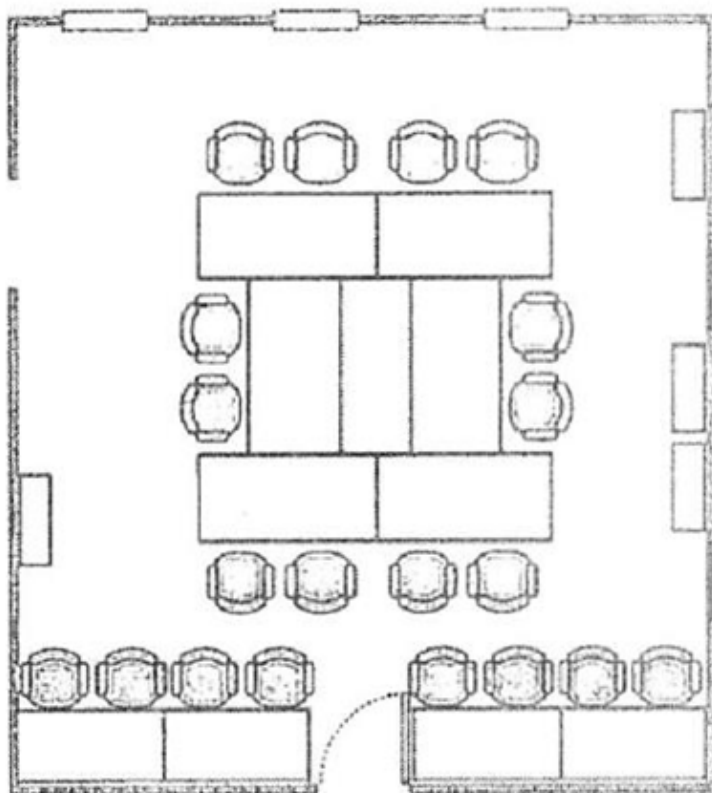
如果只是自己测试，不能尽早暴露团队成员之间的代码冲突；如果可以利用自动化测试，频繁进行自动构建的话，就能及时暴露这些冲突。而且，因为每次变更的范围比较小，所以比较容易修复，反之，如果等写了几千几万行代码后再测试，就可能积累了很多的集成问题，难以有效修正。自动构建有版本管理系统支持，我们可以知道每一次变更的内容，也可以回滚到之前某稳定版本。

团队协作、不断替换

当多位开发人员协作编码时，首先必须让他们做到“同步”，不然难以做到持续集成。

可以利用结对编程，如结对结对非固定，定期更换，更有效提升团队能力。

如果结对编程总是集中于某一位编码，这容易开始，但是很难持续，这种互相分享知识的空间就越来越低。通常我们有好几种方式去对工作分工来做结对编程。例如最简单的是结对跟单独的交替或者共用跟私有结对等，也要考虑工位的安排。例如下图，有些座位可以让他们做到一起结对。另外有些是让他们私人做编程，减少干扰。



信息辐射器

当团队的能力相当、节奏可以同步后，便可以让团队自己设定奖励/惩罚机制，然后在大家都能看到的墙上，使用日历监控每天的情况。

敏捷大师 Martin 先生的建议：如果哪位成员提交的代码破坏了团队构建，他便要穿上一件脏且臭的 T 恤，上面写着“我打破了构建”(I Break the Build)，并且要在墙上日历当天贴上红点，如果当天全部构件都成功，贴上绿点。团队首个月还是红点居多，过了 2 个月后便逐渐反过来，变成绿点较多了。

持续交付

早在 90 年代，XP（极限编程）的创始人 Kent Beck 先生带领瑞士某保险公司的团队做软件开发，他们当时已经可以做到每晚发布了。越来越多的软件团队（尤其是互联网产品类公司）已经按照“尽量缩短交付时间 (Cycle time)、尽快得到反馈”这样的思路在做了。

因为软件从完成编码到可以在客户投产环境成功部署，中间很多地方可能出问题，编码后必须经过构建 (Build)、单元测试 (Unit test)、集成/系统测试，最终在接近现场客户环境完成验收测试，才有信心能成功部署。

把软件系统分成多个子系统/模块，分到几位开发人员并行开发，各模块必须整合，并通过集成/系统测试，所以持续集成是持续交付的基础。

要做到持续交付，除了整个部署流程要自动化外，版本/配置管理也非常重要，不仅包括代码，也包括例如数据库 (DB schema)、脚本 (Script)、环境配置 (Configuration) 等，因为如果这些变动发生任何问题都可能会影响交付。

你可能会觉得持续交付太理想，难以达到。实际上有些面对全球客户的互联网公司 (例如 Amazon) 已经做到每天部署，但不要误会持续交付很容易做到，这些成功案例都已经经过多年的不断过程改进，并配合自动化工具，才能做到。

本章最佳实践对应：

- CMMI
 - 单元测试：PI 2.1, 2.3 - 2.6，包括集成策略、准则和集成报告
- XP
 - 持续集成 (Integrate Continually)

附件

持续集成

有很多开源的软件可以下载来用，选好你要用的持续集成 (CI) 软件后，你就开始要安装使用，希望利用工具来实现自动构建，跟安装其他软件一样，开始的时候会遇到困难，你可以在你项目的 WIKI 记录下来，让其他人知道，避免以后重复错误。接下来大家可以开始用服务器持续集成：

1. 当你准备好提交 (Check in) 你的最新变更时，要确保它是否能正常运行。
2. 它可以正常运行并通过测试，你应该把你的代码从你的开发环境提交到你的版本管理 (Version Control Repository) 系统去，也看看有没有更新。
3. 跑构建脚本和测试，确保所有过程都可以在你的电脑正常操作。
4. 如果你在本地构建成功通过，就可以把你的代码提交到版本管理系统。
5. 让持续集成工具自动构建你提交的更新。
6. 如果不通过，你要尽快在自己的机器上修改问题，返回第三步。
7. 如果构建成功，你就可以进入下一步：完成。

如果团队成员都按照以上的简单步骤，团队便有信心使 (开发的) 软件在任何电脑上，以同样的配置都能成功运行，一些持续集成的注意点：

1. 定时不断提交 (Check in Regularly)，可以想象你写了很多代码才发现问题的，后面你会花更大精力来找出问题。
2. 创建全自动化测试套件 (Create an Automated Test Suite)。以单元测试为例，很多编码人员觉得写脚本式自动化单元测试浪费时间，宁愿手工单元测试，因他们以为只要在写完代码后跑单元测试，证明代码没有问题。当后面集成/系统测试发现缺陷，只要修改代码的错误部分，并能通过集成/系统测试便可。但修改后的代码还能通过本来的单元测试吗？不一定。所以任何代码修改后，必须再通过整个部署流程 (Deployment Pipeline)，里面包括单元测试。如果利用流程自动化实现持续集成 (包括单元测试)，便可节省用于手工测试的工作量。如果编码人员了解这个道理，便不会再说自动化测试浪费时间了。(注)

3. 保持较短的构建和测试过程，如果可以把测试和构建过程缩短的话，就不会等到一大堆问题才要解决，就像我们把复杂的系统分成几个子系统，模块逐个开发的道理一样。
4. 管好自己的开发工作区，不要只做好代码的版本管理，配置管理应包括测试数据、数据库脚本、构建脚本、安装脚本等，因为每一块都会导致你的软件运作不成功。

(注：为什么单元测试很重要已在《TDD 测试驱动开发》里说明)

技术债务例子

员工：总监，我们过程改进是以高管的关注点出发的，请问你有什么需求？
总监：我们的产品刚发布后，客户做了软件安全性检测，发现有漏洞，这种情况就表示软件质量有问题，客户感受也很不好。我们后面也很难去修正，不知如何入手，你有什么好建议？
我：你们对产品有没有安全性的需求规范？在开始设计或者开发时，有什么对应措施？我们都很清楚，软件产品到了交付阶段，已经把各模块集成在一起了，如果这时候发现缺陷，解决起来是很困难又费时的。你现在只知道有安全漏洞，但不知道漏洞源自哪些模块，就很难知道怎么去改，改任何代码也可能会影响其他的系统功能。所以在交付后，才发现缺陷的返工工作量是很高的，可能要花好几天时间才可以正式的改正。但如果你们在开发之初，就设计好对应安全需求的单元测试、集成测试、系统测试等的自动测试用例，每次变更代码都在持续集成、自动构建里通过测试，便可以避免后面这种验收才出现问题。

有些编码人员争辩说：“写自动化测试工作量很大，我们也不是开发产品，都是定制化开发”。但当他们理解到这些测试不是到最后跑一次，他们才会理解不能靠手工测试，必须自动化。

只有依赖自动构建、自动集成，程序员才敢改动本来的系统的代码。如果没有单元测试、集成测试把关，只靠最后对系统做功能上的测试，无法知道中间的改动会不会导致一些本来良好的功能整出问题，最后便导致客户投诉有安全问题。当系统已经是维护阶段，因为大部分代码都不是维护工程师写，如没有这种一步一步的自动测试来保证，程序员根本就不敢改动任何一行代码，软件开发行业称为“死”软件。

参考 References

1. Humble, Jez. *Continuous Delivery*. (2010)
2. Shore, James. "Continuous integration on a dollar a day" (www.jamesshore.com)
3. Martin, Robert C. *Clean Agile - Back to Basics*. (2019)

Chapter 18

软件需求

需求并不是关于需求 (Requirements are not really about requirements)

大家去公共图书馆寄存物品，以前都是扫二维码开箱，有些图书馆升级了使用指纹识别。

“是否新方法比以前好？”我问年轻的开发人员。

“当然用指纹识别好。新技术！现在已经很少看到使用条码的系统了。”

“如果取物时指纹识别失败怎么办？以前还有条码纸条作为凭证，找管理员人工开箱处理，但用指纹识别的话，就无法证明自己是寄存那个人，甚至连寄存在哪箱都可能忘记了。而且指纹识别的错误率比条码高，所以虽然是新科技，成本也提升了，但未必带来价值。”

每次培训，我都会用这例子提醒学员需求并非只谈需求，必须为拥有它的人提供最理想的价值。有些产品经理混淆了解决方案与需求本身：“寄存需要用指纹识别开箱”，

但如果把需求写成：“让寄存者取得凭证，之后用来开箱”

设计师就不一定选择用指纹识别，它只是其中一种方案。

如果希望 IT 系统给用户带来价值，就需要从整个业务全面分析，有些需要由 IT 系统支持，有些不需要。

下面会举例说明。

很多团队都未能做好以下 3 点：

- 需求分析
- 识别关键干系人
- 非功能性需求

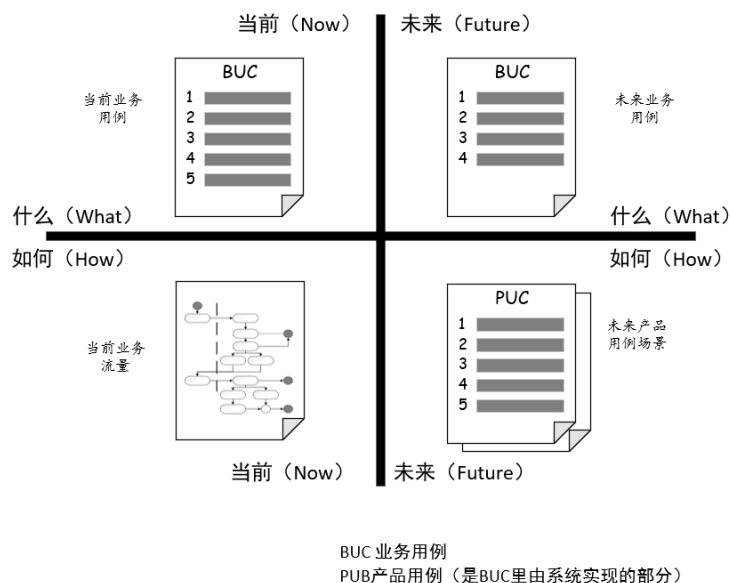
需求分析

业务用例，包括产品用例，可以帮助我们分析整个业务流程，确保能为业务产生价值。

使用下面的模型，可以有效地按以下顺序系统地分析与设计业务用例 (Business

Use Case BUC)、产品用例 (Product Use Case PUC)。

- 现在业务流程 (Now,How)
- 现在业务用例 (Now,What)
- 未来增强后的业务用例 (Future,What)
- 未来的产品用例 (Future,How)



例如：线上在超市购物，产品用例就包括处理买家的结账请求，连接信用卡数据，更新买家的购物历史和等待发货的订单信息。但业务用例就包括系统以外的其他配合工作，包括：超市员工在货架上按订单取货，联系物流公司把货品送到买家地址。

首先描述现在的处理方式（左下角）。

在左上角描述现在业务用例，柜台处理的方式，然后在右上角想象用了系统后，网上选购与下订单的未来业务用例，哪些流程应该在线上做更容易（右下角）。

“刚年底审查了我们某针对医院临床软件系统，发现整年开发的功能中，有 86% 都未被使用。”

我们从而了解到，把需求分优先级很重要，所以要制定开发什么功能，集中精力这些会被使用的 20% 功能，而不是浪费时间在开发完都没有用的 80% 功能。有人提出，其实可以反过来看，怎么筛选哪些功能不应该做，可能更容易。

但应怎么筛选呢？

某家专门做高端安防系统的 P 公司，为一家大卖场 S 公司开发一款廉价的安防系统，如下：

The house has sensors for doors and windows and sensors to detect motion. 该房屋的门、窗户都有传感器并可感应察觉动作

Residents enter a password, then press the On button, to turn on the system. 居民输入密码，然后按按钮，打开系统。

The system beeps while the resident leaves the house. 当居民离开房屋系统会发出声音

After the beeping stops, the system monitors the house for any disturbances. 当声响结束后，

系统就开始监控房子



When a disturbance is detected, the sirens turn on. 当检测到干扰，系统就会拉响警报

签订合同后两个月，开发团队已经开始设计和编程，S 公司的代表提出以下新需求：“增加了一个需求即能够很精确的定位入室的盗贼的位置；以及他的去向，还可以检测出该盗贼就屋里呆了多久。”

请问你作为产品经理会如何反应？

我的经验，有百分之八十的学员都会说让我和工程部先内部沟通，再回应是否能够实现？费用是否增加？加多少？

我说：“如果你很有经验，你应该立马问客户先生为什么要做这个需求，因为你不用问工程部，已经可以猜出来开发这个功能需要大量的成本和时间，但未必有价值，如果你当成问客户他也提不出对业务的重要性，你应当场拒绝。”

--==++++==--

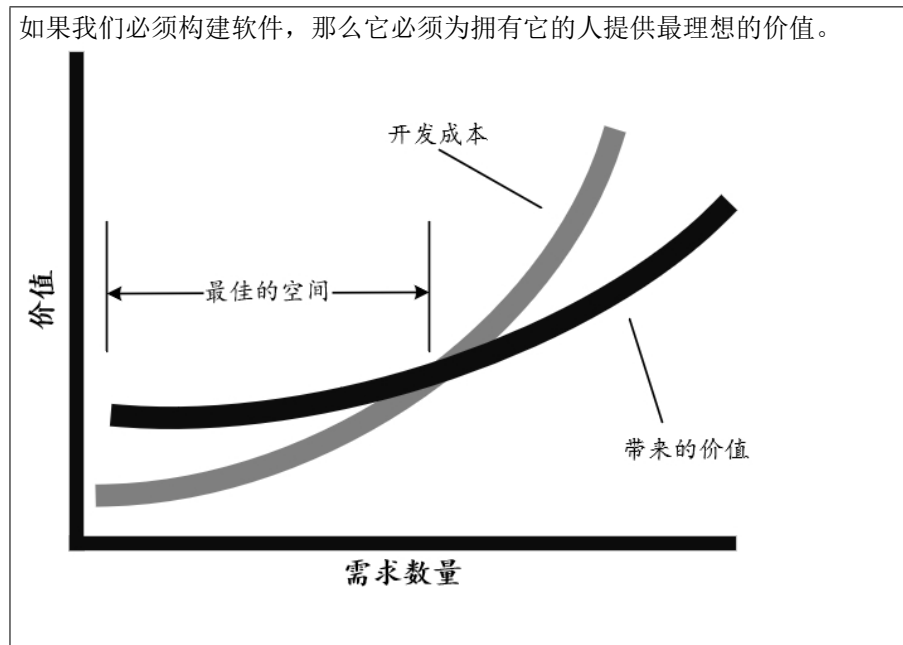
“增加一个缺失的需求去检测损坏的窗口”很多学生会立马问（因为刚刚听完第一条的解读。）：“客户为什么要做这个功能？”

客户说：“如果系统能够识别出破碎的窗户，便可以报上保险公司申请节省时间。”

但如细心想想，这需求其实是不可能实现，因为系统只能从传感器检测到波动的幅度，但无法判断是否破损（例如：不碎玻璃）。

从以上例子看到产品经理不仅仅是传话，要利用业务的知识，了解和过滤需求，拒绝没有价值的需求。

需求人员过滤了不合理的需求后，还是要分析成本和给客户的价值，对价值比成本高的需求分优先级。



以上面储物柜故事举例：

因为指纹识别成本高，但没有提供更多价值，所以不应该做。

用例与场景

用户故事针对“做什么”与“为什么”（“What”，“Why”）；场景针对“如何做”（“How”），例如普通市民用手机程序搜索下周一央视 7 台有什么节目。所以它们之间是互补，没有冲突。每个需求都应考虑各类场景，以银行个人用户用银行卡去 ATM 机取款为例：

正常情况：使用个人银行卡取 300 元

其他可选情况：从其他银行卡取款

异常情况：取款后没有及时取回银行卡，导致吞卡

正常场景是基础；但如果没有全面考虑各种场景便会导致最后开发出来的产品不满足客户需求。

实例：护照到期续证

比如我们护照更新，以前是要到柜台手工办理，如果我们把那个过程数字化，让市民可以在线做，不需要亲身到柜台办理。应怎样做呢？

某国家的做法是本来的手工填写模板直接变成系统页面（每个输入与手工表格一一对应），申请人在系统里按本来模板填写并提交，上传个人新照片，也是经过系统，我有一次尝试用系统线上填电子表单申请，但因表单很繁琐，有很多护照原有信息都需要重新填写，光是填那表就花了我接近一个小时，最大问题还不是在我花时间填手工表，而是最后要上传照片，因为照片像素高的话就很大，需要很好的网络才可以传得上，如果照片像素小，便导致模糊不清，不能通过。最后，一个半小时后，我用尽所有方式都还是无法传上照片，我最终放弃了

在线上提交申请，直接预约去在柜台做！

之前描述的整个过程只是把原有的手工步骤信息化，和原本的申请手续一样。但线上办理和在柜台现场办理不同，在现场你可以要求对方直接把照片给你，也可以要求对方提供老护照，但在线上办理，应很容易从系统里找到个人护照信息，所以很多本来在柜台要手工填写的老护照信息就不需要再填了。

客户：怎么可以简化整个过程？最困难应是照片的更新？

我：有些国家是这样做法，比如你申请续证，只需要填上老证件的基本信息，系统就立马能识别出本来的证件你确实有那个“旧”证后，便可立马提交申请。跟在网上购物一样，你确认过内容没问题，就在网上付费，然后打印申请表并在表上亲手签字，然后附上几张符合规格的照片，邮寄到政府机关。他做好新证件以后再邮寄回你。或者你自己到他规定的地点领取也可以。这样就能很简单地利用“低”科技解决方案，解决了刚才上传照片的困难。

从上面例子看到，我们应不仅是把那些本来手工的流程自动化，应该全局看要解决的问题本身，哪些过程自动化，哪些不应该自动化（比如传照片）。

甲方对怎么可以在线上做这个过程也没有概念，他只是知道，本来手工需要填表。

乙方也不知道，他只是做开发，也不知道有什么方面可以不用 IT 方式，而用其他方式更合理。

必须要一起探讨才可以有最好的解决方法。

以上例子，业务用例是线上续护照。但有些功能不靠系统处理，如上传照片，不归纳为产品用例，用其他方式手工处理。

以上只是考虑了正常业务流程，也要考虑各种异常场景才算全面。（请动脑筋，写出各种异常场景，附件里有以往学员的答案，供参考。）

所以作为业务分析师，你很可能要改变用户思考问题的方式，例如利用业务过程模型，配合场景与页面原型，与利益相关者一起探索问题的本质。软件系统必须为拥有它的人提供最理想的价值，构建软件系统本身（例如只是把现有流程自动化）不一定能解决客户业务问题。

识别利益相关者

和杭州客户领导吃晚饭，领导就跟同桌的项目经理开玩笑说：“你们刚刚完成内部项目管理自动化工具，做调研的时候好像没有找过我？其实我是其中一位经常要使用这系统的管理者，下面项目的监控、申请都是经过我，但我发现这个系统很不合用，对收集我需要的项目信息没有作用。比如没办法处理一些批准信息。”

从上面对话，可以了解如果没有全面识别项目的利益相关者，可能会影响到项目的成败。例如我接触一些有些具备开发经验的需求人员，但他们通常只注意功能需求的技术细节。

问他们：“哪些是你的项目干系人？”

答：“甲方有协调员，要访谈哪些人都是由甲方协调员安排，我们听他安排。” 所以为了避免未能识别所有干系人的风险，就要主动跟甲方一起策划。我们说沟通计划必须是甲乙双方一起合作做出来，而且会牵涉甲乙双方各个层次的人。

例如要听甲方出资人的需求，可能就要乙方的总经理出马，乙方的需求人员顶多可以跟甲方对口的项目组人员沟通。（如何可以做好识别干系人，并制订沟通计划，可详见附件。）

非功能性需求

虽然很多项目有性能、易用性等非功能需求部分，但缺乏可衡量的量化指标。

例如，多少用户量、数据量、什么平台与网络环境下的反应时间。

“客户没有提非功能的具体需求。”需求人员可能说。

“但没有明确可衡量的需求不等于没有要求。假如验收时甲方项目经理换了人，项目可能会因为反应时间太慢，甲方说不能接受，但因为非功能需求不明确，你们也无法证明产品满足需求，最终项目可能被拒收。所以建议你们也要写上具体可验证的性能需求，保障自己。如果客户没有具体要求，可以依据以前同类项目性能测试结果，制定容易达到的性能规格，成为需求模板。”

除了性能 (Performance) 外，其他主要非功能需求包括：

- 产品观感 (Look and Fell)
 - 产品的外观和感觉越来越受重视，例子：苹果 iPad MacBook 等都让客户觉得产品设计精简但高贵
- 可用性与易用性 (Usability and Humanity)
 - 线上门购手机 APP，能否容易找到喜欢的菜式、挑选并下单
- 操作环境 (Operational and Environmental)
 - 从肩高跌落时，产品仍能存活
 - 产品应在什么温度、湿度等条件下使用
 - 产品应节省电池寿命
- 可维护性和支持 (Maintainability and Support)
 - 产品应易于移植到 Android 和 iOS
 - 能简单、快速地从原有的产品导出资料，更新到新一代同类产品
 - 应翻译成各种外文
- 安全 (Security)
 - 权限 (Access)，例如：只允许哪些人使用
 - 隐私 (Privacy)，例如：防止打印任何个人及机密资料；防止未经授权的人进一步或二手使用
 - 完整性 (Integrity)，例如：确保传输的数据相对应
 - 审计 (Auditing)，例如：在法定期间保留所有交易的日记账。
- 文化 (Cultural)，法律 (Legal)

其他最佳实践

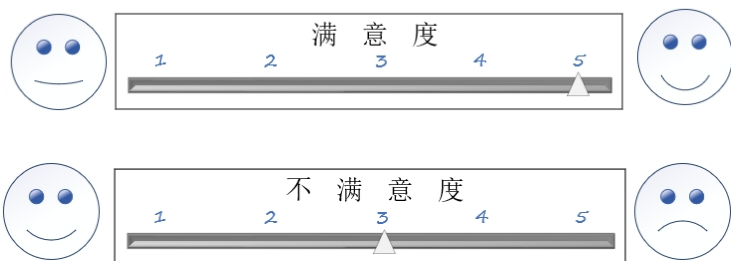
用户故事卡

用户故事卡片目的是让用户（业务）与开发沟通的桥梁。

需求卡片除了包括需求描述，理由，验收标准外，还应有以下内容：

- 顾客满意度/顾客不满意度：

用两个数比单纯用优先级能更全面反应客户声音。



（如想了解更多为什么要这样分，请看附件里的“客户声音:Kano Diagram”）

- 来源：每项需求都应可追溯到源头. 例如需求是哪个人, 什么岗位提出.
- 冲突与依赖其他需求：是/否；确保需求之间的一致性。
- 独立的需求编号：

因为设计、编码、测试用例都应需求相互对应，有明确编号才能对应。

（用实体卡片, 团队难以互动、分享，有些公司采用系统记录与跟踪需求。电子卡也应包括以上内容。）

做好需求评审

很多团队都有做需求评审，但因为没有主要关注点，起不了质量把关的作用，可以利用以下的检查单提醒需求有没有犯了这些错误，尽早改正。

评价和验收的准则可包括：

- 是否明确，陈述清晰、恰当
- 唯一识别符合架构方法和质量属性的优先级
- 可实施
- 可测试
- 可追溯到来源
- 可实现与业务价值 (Value) 挂钩（镀金：未带来价值）
- 是否由客户确定为优先事项
- 超出范围，与项目目标无关
- 不完整
- 不一致（与其他需求有冲突）
- 不正确
- 有二义性
- 属于解决方案（未全面考虑各种可行方案）

总结

上面简述了 3 类需求常见问题：

- 需求没有按价值过滤与分析，分优先级；没有全面考虑各种场景
- 没有全面识别利益相关者

- 没有明确描述非功能需求，并可验证

和 2 条实践：

- 用户故事卡
- 需求评审

都可以与 CMMI 模型对应：

- 需求分析：RDM 3.5, 3.6, 3.2 (场景)
- 干系人：PLAN 2.4, MC 2.2 (沟通的策划与监控)
- 非功能需求：RDM 2.2 (客户需求，包括功能与非功能需求)
- 需求卡片：RDM 2.2 , 2.4 (客户需求与活动或工作产物之间双向可追溯)
- 需求评审：RDM 2.3

附件

利益相关者

利益相关者计划检查单 (**Stakeholders Plan Checklist**):

- 1) 列出所有潜在的利益相关者 (List all potential stakeholders)
:1a. 类型/分组 (What are the base Segments)
:1b. 可否再细分 (Any sub-segments)
 - 2) 把她们分为 F (友好)、I (忽略)、U (不友好)
:Assign F (Friendly)、I (Ignore)、U (Unfriendly) to them
 - 3) 他们最关注什么?
:What is important to them?
 - 4) 学习目标 (Learning objectives):
: 针对某利益相关者，我们需要了解什么?
:For each stakeholder, what will we need to learn?
 - 5) 怎样沟通 (How)
 - 6) 什么时间 (When)
- : 抽样计划 (Sampling plan)
- 如何招募 (How to recruit)

如何能获得承诺 (How to get commitment)

[针对以上第 1 至第 3 项，参阅以下实例/解读]

实例/解读 (**Examples / Explanation**)

某公司专门设计、开发新一代手提电脑产品。

1) 全面考虑各类利益相关者 (Stakeholders)

出资人 (**Sponsor**): 出资人为产品的开发付钱

顾客 (**Customer**): 顾客购买产品。必须对他们有足够的了解，理解他们认为什么有价值，所以会购买什么产品。

用户 (User): 确定用户的目的是为了理解他们所做的工作, 以及他们认为哪些改进有价值。
在开发消费产品、大市场软件时, 应该考虑用一个“假想用户”。假想用户是一个虚拟用户, 它是大多数用户的原型。

类型/分组的例子如下:

未来笔记本电脑的潜在用户:

大类:

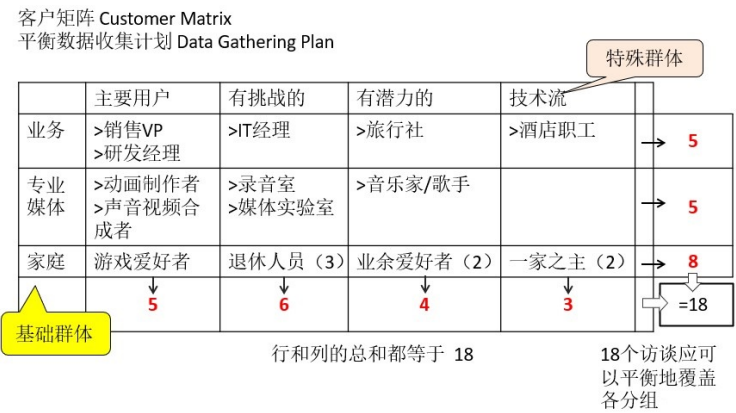
商业人士 (Business)

1. 媒体专业人士 (Media Pro)
2. 家庭用户 (Home)

细分:

主要用户 (Lead User)

1. 有极高要求者 (有挑战的 Demanding)
2. 潜在用户 (有潜力的 Potential)
3. 追求技术完美者 (技术流 Tech-Phobic)



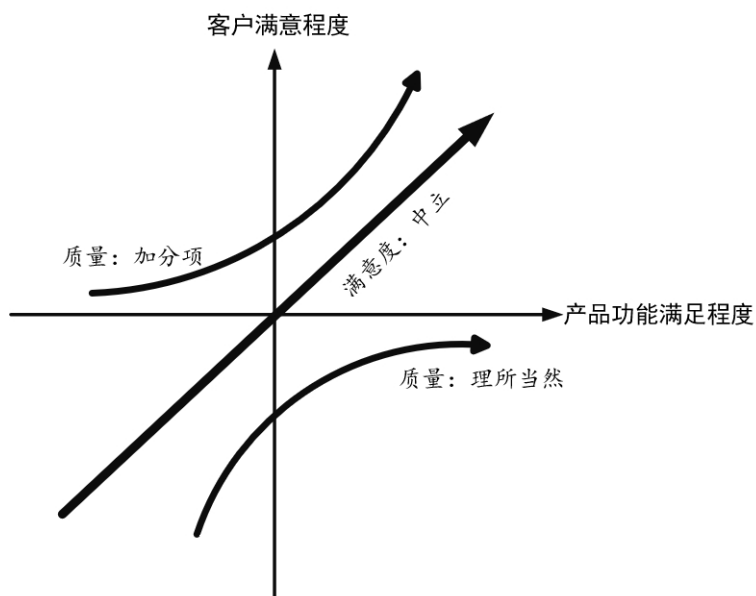
- 2) 策划包括哪些用户 (User inclusion strategy)
- 依据设计人员会怎么对应, 把不同识别出来的利益相关者分成 **F**、**I**、**U**
- 例: 以针对笔记本电脑创新产品
- F(Friendly)** 友好, 比如家庭用户、商业用户、媒体专员
- I(Ignore)** 忽略, 比如忽略残疾人士
- U(Unfriendly)** 不友好, 比如黑客、小孩 (禁止他下载游戏、玩游戏)
- 3) 他们注重什么 (主要关注点)

干系人	角色/概况	质量关注重点
商业用户	长期出差者坐长途飞机做演示保护某些秘密文档	待机时长屏幕清晰度安全性
专业媒体	创造性工作; 并要协同。录音和录视频	数字带宽 (声音和视频电脑速度和内存
家庭用户		

- 以上有那些在调研之前已知, 其他有那些需要挖掘。

客户声音：Kano Diagram

可以用下图分析用户对需求优先级：



解读上下两个箭头应怎样看：

* 下面的箭头代表理所当然 (Take it for granted)，如果缺乏，客户会很不满意，包括觉得是理所当然 (例如满意度：中立 1，非常不满 5)

- 上面的箭头代表是加分项 (Attractive)，如果包括会非常满意，但如果缺乏不会觉得不满意 (例如满意度：非常满意 5，不满意度：中立 1)

所以“需求卡片”用两个系数：顾客满意度 + 顾客不满意度，能更好判断某功能属于哪类功能需求。

线上续护照的异常场景

以下是部分异常场景：个人信息类

- 信息不符，无法识别
 - 生僻字姓氏，系统无法识别

付款

- 支付失败
 - 直接经银行付款
 - 经渠道支付验证出错

系统类

- 系统间接口兼容性问题，提交失败

- 系统出现故障，无法登录或无法上交
- 浏览器兼容性问题

特殊情况

- 法定假日，不接受申请
- 紧急情况：申请人遇到急病，亲人海外死亡等紧急情况，需加急处理
- 信息错误：申请表信息填写不完整或填写错误，需补正
- 照片不符合规格

特殊人群

- 申请人属于未成年人
- 父母国籍不一致，无法线上判断国籍

参考 Reference

1. Beck, Kent , with D. West. "User Stories in Agile Software Development"
, Ch.13 of *Scenarios, Stories, Use Cases: Through the Systems Development Life-Cycle* edited by F. Alexander(2004)
2. Robertson, S. "Mastering the requirements process. " (2006) 2/e

Chapter 19

协作改进整个系统

“专注用户，从顾客的角度生产产品。”

某次现场培训，我分享了丰田汽车故事后，请学员分组讨论，写出对工作最有帮助的 3 条。以上是某组的第一条，我请他们解释说明。

“我是做开发的，反思时发现其实不清楚我的工作对客户有什么作用。”一位年轻的编码人员说。

因为编码工作大部分都是由产品经理或项目经理过滤后分配，所以如果你随便问身边的编码人员，70-80% 的可能不知道用户为什么要这样做，要开发的功能有什么价值。

这位年轻编码人员的分享让我想到 2 年前某公司要推行需求与研发分离的故事。

需求与研发

质量部经理：以前需求是研发团队的一部分，很多时候就会倾向于从研发团队的角度来看需求。公司希望分开后可以更加独立，更多地从客户的视角看需求，也希望需求可以与研发相互制衡。

我：你们把需求与开发分家以后有没有遇到什么问题？

质量部经理：遇到问题可不少，例如研发团队会说——需求人员把需求写得太简单了，写得不清楚，无法开发。而需求人员却说，要写到多详细才可以开发，你可以给我一个标准吗？甚至有些较极端的研发人员，明知需求有问题，但是还是按需求的描述做开发。公司领导希望利用标准化流程和相关度量指标去控制过程。我就是负责制定这些指标的，但有个问题，比如我们觉得研发做出来的产品应该得到客户的认同、让客户满意。所以客户满意度应该很重要。但研发组长就拒绝了这指标，他说——现在我们只做研发，不负责需求，也控制不了需求的质量，为什么最终那个产品的满意度要由研发负责？而如果要求需求人员对最终的客户满意度负责，体现在 KPI 指标，他们会说——我们只做需求，开发做出来的产品客户不满意，为什么让需求买单？

听起来两边都理。

我：如果我们希望只通过某些指标监控过程，比如代码评审缺陷率，希望提前发现错误，尽早修正，反而会影响总体效果。团队可能因花大量精力评审，影响

了交付速度，反而会导致客户的不满意。如果客户对产品不满意，开发肯定有问题，所以只追求某些过程指标没有意义。人很聪明，会想出各种满足指标的方法。

例如管理层希望加强代码评审改善产品质量，开会时跟开发与测试人员说：“如果评审的缺陷数量能超过指标，便会按超过的数量发奖金。”

会后，测试不服气，私下跟开发说：“估计你们很快可以轻松地变成百万富翁了。”

这经典滑稽故事有点夸张，但能点出只追求指标的问题。

就好比如果评审或者测试的缺陷数越高奖励越多，聪明的团队可以很轻松地变成百万富翁。

质量部经理：我们那些团队就很懂怎么针对指标作假。

公司觉得今年改革之前做新产品开发项目的立项太宽松，缺乏有效的管理和监督，这导致很多新的研发项目都是没有回报或者失败，浪费了很多资源。所以现在用新方式，所有立项的研发项目，都应该由中央委员会审批才能立项和开始做，我觉得这个倒是挺好的。

我：你觉得好吗？

你听过苹果电脑 (Apple) 吗？苹果公司本身就是由两人创办：一位是乔布斯、另一位是沃兹 (Steve Wozniak)，沃兹本来是惠普的工程师，在 1975~1976 年间研发出了苹果电脑 (Apple 1)，他本来希望获得惠普高层的支持，申请内部立项，但多次都被拒绝。乔布斯看到这个产品后，觉得很有市场潜力，就跟沃兹跑市场，最终获得 Byte Shop 老板 Paul Terrell 的首张订单，购买了 200 套苹果电脑 (Apple 1)。当时 Apple 1 都由沃兹手工焊接而成。创新产品都必须具备以下重要元素：

1. 了解市场需求的人员，乔布斯就是这个角色
2. 研发工程师沃兹

两者相互合作才把苹果产品做成。如果把需求跟研发切开，估计苹果的传奇故事永远都不会诞生。正是两人的团队合作，才会有个人电脑这个新市场。

创新不能用传统的工厂化流程，靠一些指标去管理。

惠普公司很优秀，内部肯定不缺资深专家，评判指标也应该很完善，为什么沃兹的内部申请立项，多次都被拒绝？惠普一直都是专注于做度量和测量仪器，但个人电脑是全新的概念，所以单靠现有指标是无法鼓励项目创新。发明全新的东西必须是需求配合技术开发工程师，再加上与客户的接触和确认，才会成功。

编码以外

这不仅仅是编码与需求的问题。测试的主要目的是希望内部先发现问题，不要等到交付给客户才暴露。

B 公司专门做餐饮系统产品，2 年前为某企业提供食堂系统。因公司一直都只为餐厅、菜馆等提供系统，测试工程师专门做了研究，发现食堂与一般的餐厅不同，因为要支撑大量员工在午餐时段点餐，通常都只提供几类套餐，简化流程，所以便向产品经理提出。但他没有接纳，还是按照客户提出的系统菜单 (menu) 要求开发。系统交付后，过了几个月，系统最终还要改造、简化，只保留几类套餐。

因为需求由产品经理负责，测试人员只能按需求制定测试计划，无法影响需求，导致走了弯路，引起本来可以避免的返工。所以应赋能与前线团队，包括开发、测试等，前期让团队一起全方位地讨论给客户的价值，减少浪费。

小结

从以上 3 个故事看到员工只管自己负责的任务、过程，导致：

- 团员不清楚最终对客户的效果和价值（例如开发）
- 每个过程的指标都很好，不一定代表总体效果会理想
- 容易引起各自为政

最终做不出优秀产品，所以戴明博士的经典 14 点里第 9 条强调“打破部门间的围墙”：

”Break down barriers between staff areas.”

先让我们看看，跨部门合作的团队如何能为公司增值。

改进整个系统 (System Thinking)

促进团队跨部门有效地合作，为公司增值，顾问 John SEDDON 建议从下面 5 问题开始：

问题	描述
目的 (Purpose)	组织的长远目的
能力 (Capability)	当前系统的能力
客户需求 (Demand)	需求的特点、特性
浪费 (System conditions)	有哪些浪费？原因？
流程 (Flow)	工作流程

下面按这 5 条分析某 B2C 场景——美国西南航空。

西南航空 (SW Airline)

公司成立于 1971 年，开始时只有四架波音 737 飞机，为德州 (Texas) 的 3 个城市 (Houston, Dallas, San Antoni) 提供服务。

Purpose 目的

当时的 CEO Jim PARKER 说：“我们其实是一家服务公司，客户是否满意非常重要，因为我们主要针对州内的短途飞行，我们主要的竞争对手不是航空公司，

而是客人自己开车。一线员工要应对各种突发情况，及时反应。因员工都知道客户满意最重要，所以面对客户的一线员工可以现场自己做决定，不需要向上级申请。我们相信员工必须有良好的工作环境才愿意服务好客户。所以在西南航空，我们把员工放在第一位，第二位是客户，股东是第三。

Capability 能力

其他大部分航空公司更注重能否把座位全部卖光。但西南航空了解如全部坐满，反而会影响客户满意度，所以西南的座位占用率都比其他航空公司低。西南航空反而更注重飞行期间的准备时间，因为飞机在机场地面上的时间是不会为客户或航空公司产生价值。某些更大的航空公司因为更注重座位占用率，希望尽量坐满，导致要处理大量乘客和行李，反而引起航班延误，不能按时起飞，影响了客户满意度。（还记得某美国航空公司，因为乘客超载，航班无法起飞。最终要保安人员进入乘客机舱，强行把某乘客拉走的视频吗？）

Demand 客户需求

1978 年，美国总统取消了对航空公司的规范（例如包括票价等），很多小的航空公司被淘汰，西南航空高层也感受到竞争激烈，经营困难，必须卖掉一架飞机，但这样会导致减少提供的航班数量，客户会不满意，公司的收入也会减少。

System conditions 浪费

前线员工一起想办法，建议可以把飞机落地到起飞时间，从原本的 30 分钟压缩到 10 分钟。因为都是短途航线，飞行时间只需一个小时，把准备时间压缩了 20 分钟便能以 3 架飞机满足原本按 4 架飞机设计的航班安排。

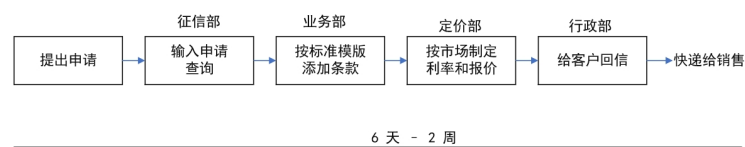
Flow 流程

从飞机落地到准备好可以起飞，中间包括各部门人员的协作：行李、闸口、飞机汽油、餐饮、驾驶员和航空服务员等，但日常操作中会发生各种变化，如天气大雾、航班延误、机器故障等。西南航空一直让前线员工以客户服务优先为原则，遇到问题就现场自己协调，无需管理经理批准。虽然不容易，一线员工可以通力合作，克服困难，满足客户服务需求，让客人满意。

(如想多了解 SW Airline 故事，参考 Poppendieck 第 1 章)

IBM 信贷 (IBM Credit)

以下是 B2B 的业务流程改造例子。公司主要为购买电脑的客户发放贷款，但因为要经过不同专业审核，从销售提出申请到收到报价单需要 6 个工作日到 2 周。



前线销售觉得这时间太长，有风险（因为过程中，客户可能受竞争对手影响，改变主意），希望尽量缩短审批时间。

高层特意召集所有相关人员，一起模拟从头到尾的过程，发现贷款实际工作只需要 90 分钟。一起复盘讨论时，发现当初设计流程时以为每个环节都需要专业人士处理，但其实没有这么复杂，于是把贷款工作改为由 1 个人处理，并提供他一些工具与模板，发现经过培训后，整个审批流程确实可以由一个人处理。这不仅减少了人手，还把贷款单的处理效率增加了 100 倍。

（如想了解更多 IBM Credit 故事，参考 Hammer 第 2 章）

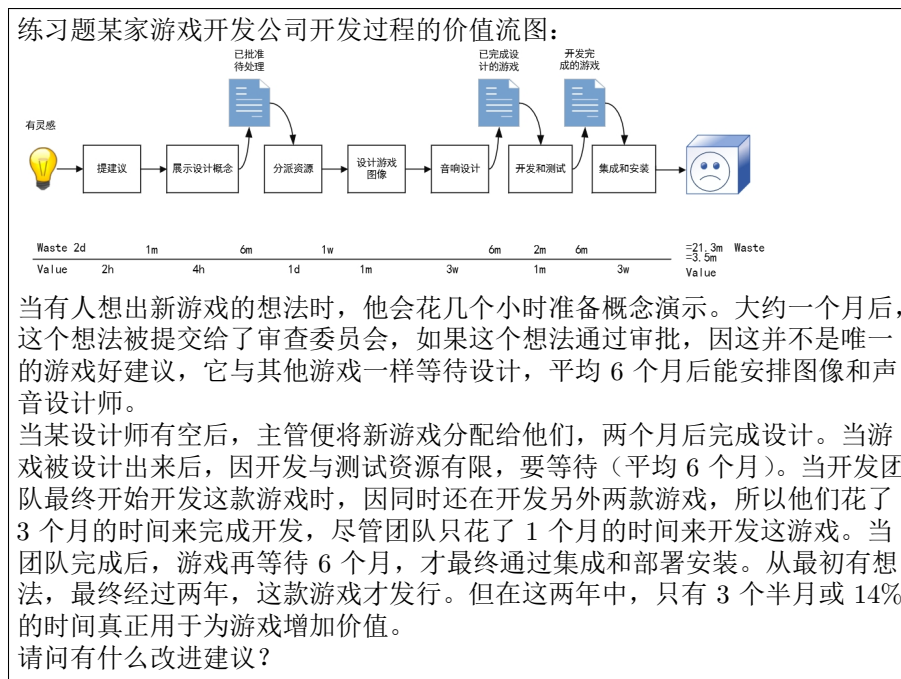
Value Stream Map 价值流图

从以上两个例子看到重新设计业务流程能节省时间，这种思路能用于软件开发吗？

前面十四章提到软件开发过程有各种浪费，我们可以用价值流分析，先画出当前流程，识别哪些过程有价值，哪些是浪费？然后分析原因，优化过程。

价值流 (Value Stream) 是精益的过程分析方法，先画出从头到尾全流程，然后识别哪些是有价值，哪些是浪费；然后分析看如何能减少浪费。

下是我用于精益价值流培训的小组练习题，欢迎尝试提出你的解决方案（答案在附件）：



总结

正如戴明博士说，要清除部门间的围墙，一起合作为客户服务，所以不能单靠每个子过程最优，更重要是从头到尾的效果，是否让客户满意，并减少浪费。

但是否设计好总体的优化系统和流程便能得到理想效果呢？下章继续。

附件

练习答案

这个案例出现了三类浪费：

1. 过程中库存、半成品 (In-Process Inventory)
2. 交接 (Handovers)
3. 任务替换 (Task Switching, Multi-tasking)

公司为了精简，成立了跨部门团队，每个团队只开发一款游戏，团队就可以自己把握从头到尾的整个流程，避免中间交接，例如：刚开始第一个跨功能团队，能在四个月内开发出一款新游戏，是之前开发时间的六分之一。

参考 References

1. Poppendieck, Mary, Poppendieck, Tom *Leading Lean Software Development* (2010) Pearson Education
2. Hammer, Michael , Champy, James *Reengineering the Corporation* (1993) Nicholas Brealey Publishing

Chapter 20

从 AI 到最佳设计

“我们现在推行 AIGC，服务端不需要 UI 交互设计的用 AI 自动产出代码，你建议的结对编程、TDD 等是否还适用？”

这两年 AI 确实很火，是报纸、杂志的热门话题。例如，HBR 杂志从 2024 年 9 月至 2025 年二月份 3 期，里面有接近一半文章都与 AI 有关。朋友圈晚饭或在飞机场候机厅我都提过以下对话：

“未来人类的工作估计全都会被人工智能取代！”

“以前以为有些专业，例如医生，会比较放心，难以被 AI 替代，但听说好像连手术也可以由机器人来做了。”

越来越多开发团队使用 AI，希望提高效率。如何能有效地利用 AI？应如何适应这新技术发展？详细讨论这些之前，先分享两个故事：

两个故事

某欧洲城市巴士公司为了提升效率管理层引入了奖金计划：按计划不延误，给司机的奖金越高。

因为巴士按区收费，售票员也要查看下车乘客有没有正确按规定买票，公司会随机随机派人抽查，如发现售票员违规（包括售票与查票）会扣奖金。因为收入提升，司机或和售票员都喜欢新的奖金制度。售票员在巴士后方有固定位置，指示司机是否可以开车离站。但当交通繁忙时，因乘客太多，太忙，售票员可能来不及回到车尾位置指示开车，这类延误会影响到司机的奖金收入。售票员大部分是少数民族新移民，司机大多是来自传统蓝领阶层，经常发生纠纷，有时候甚至会双方动手打人。请问有什么解决方案建议？

改善奖金制度？例如：平均分配奖金？请双方代表坐下来协商？一起讨论解决方案？

最终解决方案：

因为发现主要的繁忙车站数量比巴士数量少，所以安排售票员守在车站，在巴士到站前预先售票，售票员也可以在车站查看下车的乘客是否买了正确的车票；也可以在车站指示司机是否可以开车。非繁忙时间售票员可以正常回到巴士里

工作。

---+++-

美国一家历史悠久的彩色印刷公司，主要为高端的杂志和出版社印刷彩色照片或者油画等。公司在美国俄亥俄州有 8 条生产线，虽然销售还是很理想，但管理生产的副总裁 VP 要解决一个头疼的问题——过去 5 年生产线的生产率逐年下降，甚至已经开始没有利润。通过检查设备、机器是否老化、设备缺乏维修等，发现这些都不是原因。最后发现主因是虽然增加了大量新产品，生产量却很少，而每一轮生产之间的准备时间却要增加很多，便请顾问看看如何优化生产的时间安排，希望能尽量减少准备时间。

顾问先用电脑对生产进行模拟，发现如果改善策划，确实能提高生产率。但顾问同时也发现产品线中绝大部分的利润来自 8% 的产品，它们构成所有生产的 9 成。另外很多“老”产品，销售量很小，基本没有利润。发现这情况后，顾问跟生产副总裁提建议，但被拒绝——“卖什么产品，不是我们管生产的范围。”就请他问管市场部的副总裁，副总裁虽然没有拒绝，但担心影响客户满意度也不愿意取消老产品。因为这家老公司承诺客户永远不会停掉某一条已经提供过的产品线。

顾问还是觉得问题的核心不在调整生产，建议修正销售人员的分成安排：基本薪水不变，但他们的分成会按销售是否有利润挂钩。如果他们多卖一些有利润的产品，他们的收入就会提高。但如果他们多卖一些没利润的产品，对他们的收入没有帮助。人事部对这个建议很感兴趣，愿意找美国某区域做试点，试点发现有 65% 没有利润的产品在整个试点区内就没有销售，但有利润的产品就提升了 18%，销售人员的收入加倍，然后公司的总收入也上升了一倍。

---+++-

以上两个故事的经验教训：

- 改善系统的某部分不能改善整个系统
- 解决问题应从各种视角看，如果仅仅某个专业来分析，难以找出整个系统的最佳方案

在软件开发公司也有类似例子，比如专门给医院做系统的公司发现 80% 开发出来的功能没有用上，浪费资源。改善开发流程并非问题的核心，而应针对怎么去挑选并拒绝没有价值的需求，公司组织里面的功能互相依赖，好比我们人体器官相互依赖，不能单靠优化某一部分改善整个系统。

向煤矿工人学习

战后 50 年代的英国煤矿，在机器化前，煤矿工人都是小组手工运作，伙伴互相帮助，后面利用机械化大规模采煤，类似工业生产的生产线一样，本以为设计好整个系统，配上工人，就能大大提升效率。但发现并非如此，机械生产开始时，按技能要求分工，如钻洞、切割、修建隧道、挖煤、清理等，每人负责一小块固定的工作，轮班（详见附件的图 2），发现工人常常生病，出勤率也不理想。必须安排后备人员，因某些岗位缺人会影响整个过程。

因煤矿是天然产生，煤层（分布、厚度等）会按地形变化。例如，如果最后挖煤的人发现前面负责挖洞工人没做好，挖煤便很困难。机器化之前都是团队制，互相依赖，互相帮忙解决问题。但现在工作细分后，当挖煤的工人遇到困难时

便求助无门，很无奈，导致大量缺勤、生病。

针对这不足，有些公司采用多功能团队制，让团队可以灵活按需要转换工种，不是只做某一块工作，像机械化前互相合作，发现效果很好，生产力从以前的 78% 提升到 95%；旷工率从 20% 降到 8.2%。（详见附件）

从以上研究发现，不能仅仅靠科技设计生产线，也必须考虑人的因素，E.Trist、F.Emery 等学者提出社会技术系统 socio-technical systems (STS) 的概念，工人不是生产机械的延续。

1936 年的经典黑白电影《摩登时代》(Modern Times)，卓别林演电影中的工厂工人，每天不断地做一项工作：拧螺丝钉。最终顶不住烦躁的工作，发疯了，借此讽刺工业生产线生产的问题。

软件开发也一样。

前面章节提到有些公司从需求到测试每个环节分工明确，某些岗位只做需求，某些岗位只做设计，误以为每个环节做好，总体的效果一定会好，忽略了活动间相互依赖。例如需求写得模糊会导致开发和测试出现缺陷和返工。这些都要等到测试和验收时才发现，但正因为每个工种独立，做开发或测试的很无奈，因为心里都知道，问题是出自需求，但她控制不了。所以敏捷开发强调要小团队各人互补。

例如：测试报告：

因为测试只能从他的视角看到有多少是系统测试找出的缺陷做分析，没有考虑其他过程，如评审，需求等，不清楚缺陷的真正源头，报告对减少后面缺陷与返工没有帮助。但是如果所有团队成员，包括需求、开发一起分析缺陷的排除率，就可以更全面的看到问题了。

“我们确实有类似以上的问题，但如果要改善整个系统有什么好办法？”

“请问你现在电话的一些功能，例如：

- 按键音频电话、呼叫等待、呼叫转移、语音邮件、来电显示、电话会议、免提电话、快速拨号
- touch tone phone , call waiting, call forwarding, voice mail, call ID, conference call, speaker phone, speed dial in memory

是什么时候发明？”

以上功能都是 1951 年在美国贝尔实验室创造出来的。例如，按键音频电话机之前的电话都是拨号盘式。现在可能只能在某些古典酒店大堂看到，而且都是假的。

1951 年，贝尔实验室的会议室

“全美国的电话系统昨晚被彻底破坏，一无所有了。” 召集紧急会议的副总裁很严肃，跟所有部门经理说。

“你们各位最好相信我刚才这番话，如果到了中午你们还不相信的话，我就辞退你。”

所有经理都莫名其妙，想“今天早上我才打过电话，这位副总裁出了什么问题？”

过了一会，副总裁面露笑容，原来他是跟大家开玩笑。

然后他解释说,“很多美国人都相信我们实验室是全美国最优秀的研发实验室。请问贝尔实验室对电话系统的最大贡献是什么?”

“转盘式 (dial) 电话”



“同轴电缆 (coaxial cable), 经过大西洋连接美国和英国”

“但这些伟大发明都在 1900 年前, 都是在座的所有人出生之前, 你们都只是对电话系统做了些改善, 没有什么更大的贡献。

要做出有突破的优秀发明, 先有以下 2 假设:

1. 假定现在的系统已经被彻底破坏, 从零开始想象希望电话系统有什么功能和好处? 而且只是针对现在, 不是五年或者十年以后。
2. 不要顾虑有任何困难, 先从客户的角度提出。

但还要考虑 2 个限制条件:

- 技术方面是否可行, 我们不希望只是一个科幻小说的场景
- 在现在环境可操作, 因为我们不可能改变环境, 包括所有的法律、规章制度等

我们这么多人不可能一起设计, 先把你们分成 6 个小组, 包括:

- 城市内的短途通信
- 城市间的长途通信
- 交换机
- 电话机等

现在大家开始分组讨论。”

上面提过 (包括按键式电话) 的多种现代电话功能, 都源自在当天分组互动工作坊里电话机小组的头脑风暴讨论。(工作坊时间有限, 组员先从用户的角度, 会遇到什么困难, 想象有了新功能想法后, 还是要与工程部交流, 探索可行性。) 以上是 Dr Ackoff 教授 1951 年的亲身经历, 这次经历促进他在 2006 年出版《最佳设计》(Idealized Design, 详见参考)。他提倡要做好设计应:

- 大家天马行空没有任何顾虑和限制, 不考虑现在系统
- 不去分析现在系统的不足
- 工作坊形式交流与头脑风暴: 没有公司岗位级别的区分, 每个人都可以畅所欲言
- 因为这些经理都有参与开头的讨论, 后面他们才会支持并投资资源实现这些新发明

这最佳设计思路也适用于改善过程。我会经常利用在这类工作坊来帮助企业做过程改进, 强调 4 点工作坊原则:

* 包括整个系统 Whole system in the room.

- 从全局“全象”策划本身行动 Global“whole elephant” context for local action.
- 关注未来和共同点, 而不是问题和冲突 Focus on future and common ground-not problems and conflicts.
- 自我管理, 对行动负责 Self-management and responsibility for action.

很类似《最佳设计》里提倡重点, 因背后都是基于 STS 学者研究发现的原理。

经验分享

必须找到了合适的利益相关者参与, 必须整个系统的代表在一起, 才能包含各种视角, 例如, 发现原来开发或产品经理是不理解现场实施人员面对的问题, 测试人员也不知道开发人员面临的问题等。但当大家都聚在一起的时候, 就可以交流、讨论如何改善整个系统。

主要部门负责人都要在, 如果他们不在的话, 所有新的思路会因为没有获得资源和支持, 都很难推进。反过来如果相关人都有参与讨论的话, 他们后面就有动力参与推行, 因为觉得这是自己有出主意的东西。

也打破了公司里面的高低层的关系, 每个人都可以自由发言, 提意见。

如果刚才贝尔实验室的场景只靠高层副总裁定个年度计划, 设定目标给一层一层去做创新, 你估计会在短期内发生这么多新的创新变化吗?

不可能。个人的想象力有限, 当然贝尔里面的人员每个都是行业精英, 所以更容易在适合的环境, 从整个系统看问题, 做出突破。

开发团队如何应对 AI 大趋势

“AI 实在太厉害了。AI 围棋, 初级, 它让我 6 子, 我还是赢不了。我以前一致认为电脑下象棋打败人类可信, 但围棋太复杂了, 一直不信电脑能学好这玩意。”我老同学聚餐时说, 他从初中已经懂围棋, 有超过 40 年下棋经验。



越来越多人问人工智能帮公司提升生产率吗？我不懂人工智能，用手机下载了 POE 做一些查询，会导出一个很好看的报告，后面我再去细看的时候，发现里面有的信息是错误的。

例如，2025 年 2 月初问：“Eric Trist Biography”



虽然格式包装都很专业，但内容有误：第一天：发现去世年份时 2019 年是错误的。第二天：同样再询问，（1909-1993）是正确的，但出生地点不是伦敦，他是剑桥大学毕业，非伦敦大学。与能网上免费搜索到，如维基百科或 Britannica 出的内容相差甚远。

AI 也可以自动生成 PPT 报告，并配上声音读出来。一些超过二十分钟的 PPT，我通常看不到两分钟就离开了，因为太模板化，没有人气，就和打电话给人工智能客服一样，你会尽量想找个真人来跟你对话。

对比 18 分钟的 TED 讲话，讲者可能会先跟你开个玩笑或者挑某一件事引起你的兴趣，然后中间也会有停顿，有声调的高低，最重要会和现场的人有各种交流沟通，眼神、面容等，这些都是机器暂时无法替代的。也正是这些元素来吸引你，继续听完他的 18 分钟讲话。

以上实验暴露了 AI 的不足，因为它是靠大量搜索同类的文章信息，提供“最佳”答案，但中间可能缺乏检测过程，产生错漏，但客户（人类）可能误以为电脑出来的都是正确。所以虽然 AI 能很有效解决有明确规律的问题，如围棋，但难以取代人的思考与交流。而且与 AI 交流如进入了虚拟世界，难分真假。

所以有些研究发现程序员虽然用了 AI 自动生成代码，虽然代码多了，好像生产率提高了，但是缺陷数量也增加了。2024 年的研究，分析了使用 ChatGPT 自动生成 JAVA/Python 程序，共四千个程序：发现 40%-89% 产出“正确”代码，看语言、难易度、是否有类似“老”程序供参考等因素。用静态扫描“正确”代码，发现各种代码异味（详见附件）。

很多时候是初级的人员没经验，误以为自动生成的代码都是正确的，没去校验，就会出现问题。

所以大家不用担心 AI 会取代开发人员工作，因 AI 只是有很大潜能的工具，类似 80 年代 IDE 能提高开发效率，但无法超越人类，而且要小心使用。因为 AI 自动出来的代码有不少缺陷，更需要我们前面提到的单元自动化测试，更需要团员间的评审，结对编程等确保代码的质量与可读性。更需要有环境让团队成员一起不断学习、不断交流、一起进步。最终人才最重要。（2023 年，有家出名北京软件产品公司，我建议他们要有自动化单元测试，但开发主管说我的代码都是剂枪人自动生成，无法做到。所以他们内部集成、系统测试时，有些产品会以 4 位数的缺陷发现。）

所以丰田汽车的大野先生在五十年代引进机器人自动化的时候也特别小心，他不会以为买了机器人就能提升公司的生产率，必须要工程师想好怎么利用机器人辅助工作，确实能提高质量和效率才会批准购买。我们也应该向大野先生学习，不要误以为用了 AI 就能提高效率，更需要看怎么可以辅助编码人员、开发人员、测试人员做好他们的工作。

如有固定规矩，偏重复性的工作，机器人和 AI 能发挥作用（如下棋、酒店送外卖到房间）；但要有创新的工作最终还是靠人。不要轻信短期未来人类会被人工智能取代，没有工作。

要做最佳设计还应参照 1951 年已经有公司成功使用的方法和原则。

AI 只是一类新科技工具，每个人要更好学习提升才可以跟得上市场的竞争的最佳对策。

所以 HBR2024 年 9 月份一篇文章提到公司不应误以为 AI 是可以帮公司创造独特的竞争能力，原因是你可以用 AI，你的竞争对手也可以很容易复制。（详见参考）

总结

虽然本部分分章节讨论需求、设计、编码、集成、评审、测试各过程的最佳实践，但若想改进应先设定总体目标，用多种视角看整个系统，再定各过程如何配合，好比建筑师按你的新花园别墅要求（如 2 层 4 房 2 厅，预算在一千万内）一样，必先做总体别墅设计，而不是先分别房间、客厅。

附件

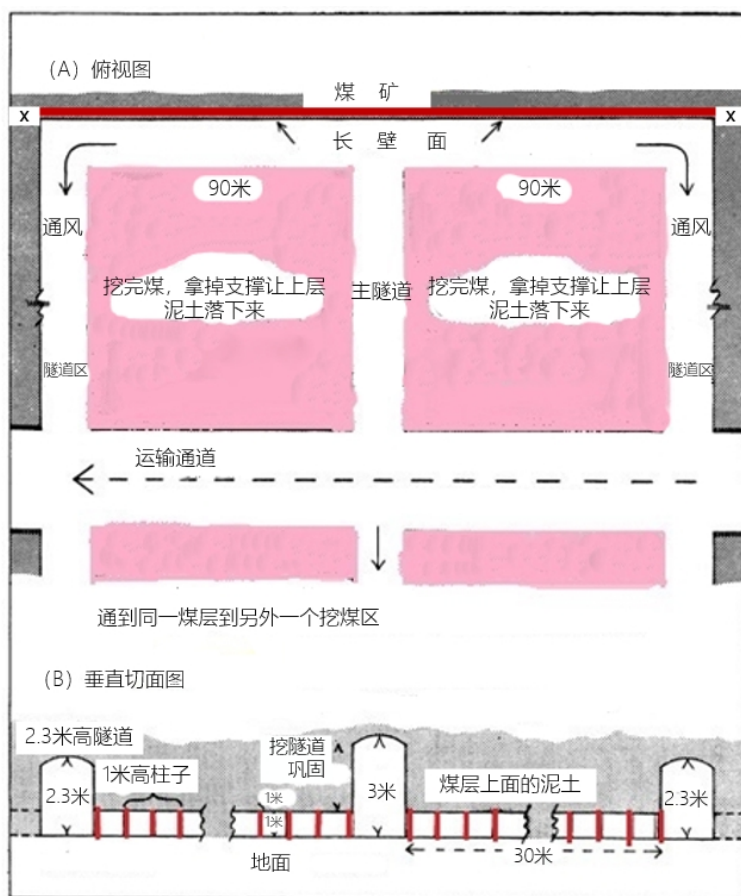
英国煤矿生产问题的研究

在机械化之前，英国采煤是以小组作坊式工作，通常是两人一组，再加上一位做整理的助手。小组会直接跟矿场合作，专门负责某一块煤矿，小组以互补的形式做各种工作，自己管自己，独立性很高。因为采矿工作非常危险，所以选择伙伴很重要，都希望有稳定的伙伴关系，不会轻易换人。当某工人受伤或者去世，他的伙伴都会尽力帮助他的家人。这种小组作坊式工作方式后来被机械化生产线模式取代，但机械化生产却引起很多新问题。

先了解一下长壁（Longwall）开采法是怎么利用机械化大规模开采煤矿。

英国的煤层一般不深，最多可能 1 米左右，上面覆盖着泥土。用长壁机械化采煤方式就可以大面积采煤。

图 1:



(上图 A 部分是煤矿俯视图，B 部分是俯视图中长壁面从左面 X 到右面 X 直线的切面图，能看到是左、中、右 3 条隧道)

共 180 米宽的长壁是为了大面积采煤，(看上图)为了通风和运输，会有 3 条隧道，之间距离 90 米，中间隧道是比较高，大概有 3 米 (9 英尺)，左右隧道比较矮，大概 2.3 米 (7 英尺) 高。这些隧道都是固定，让人或机械可以在中间通过。看上面平面图，左右 2 块 90 米宽的方形面积都是已经采完煤的泥土。从下面的切面图 (B) 可以看到中间的和两边两个隧道。每个采煤的煤层，只有 1 米高。也可以看见 1 米高的柱子，柱子是用来顶住上面的泥土，防止泥土在煤挖空以后塌下来。

长壁 (Longwall) 如何操作：

沿长壁面 X 到 X180 米的直线有两条平行的过道，都是 1 米宽。贴着煤矿哪条一条只有 1 米高的通道，是让采煤工人爬过去工作。旁边有一条传输带运作的通道，也是一米宽，一米高。每一米都会有柱子顶住上面的泥土。当长壁 180 米的煤挖完后，会继续同样往煤矿开 2 条通道，拆掉原先的柱子，搬到新建 2 条通道使用。原先 2 条通道的上层泥土会塌下来。你可以想象挖煤工人沿着一米

宽一米高的通道挖煤，放到旁边的运输带，工作环境多么恶劣。

工程师设计了 7 个工种，请参照下面的轮班表。每循环会包括 3 个轮班：

- 第 1 个轮班主要是做转动和切割和清理，比如让那些切割的煤有空位落下来，让后面那些采煤工人容易可以采到煤。通常第一个轮班都是下午班或者晚班。
- 第 2 个轮班就是巩固隧道和安装传输带，通常是下午班或者晚班，这 2 个班总共就会大概有 20 个工人。细分到不同的工种，比如有 2 位只钻洞、2 位只切割、8 位专门管理通风隧道的巩固工作，分工很细。
- 第 3 个轮班主要有 20 位采煤工人操作。第 3 个轮班只有早上班和下午班，所以他们需要依赖前期技术人员做好准备工作，然后自己按大概 9 米的范围去采煤。收入是按采煤量计算，比如 20 位采煤人每人负责 9 米，加起来刚好等于整个长壁的 180 米。

分工设计很科学，如果都按正常操作，每次循环可以采到 200 吨煤。

图 2：工人分工和轮班

轮班	隧道	长壁面区 (180米)
轮班1		钻洞 (2) →
		切割 (2) →
		清理 (4) →
		拆传输带 (2) →
		建传输带 (2) →
轮班2	巩固隧道 (8) ↑	
轮班3		↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑
		采煤 (20) 每人负责长度 (9米)

但是你估计，这种机械化的大规模连续采煤操作会有什么问题？生产力效果怎么样？

以上的工作关系很密切，例如：

- 如果洞钻得不够深，采煤工便采不到煤。
- 如果切割没做好会导致采煤工可采空间不足，影响产量。
- 如传输带没有安装好，不成直线就导致后面的传输停顿，采煤工无法操作。

各种状况都可能发生，加上煤矿是天然的，本身就有很多变数：例如地层有断层，或者地下天然气等。各种人为因素或天然因素都影响采煤工能否正常采煤。加上采煤的工作环境很恶劣，导致旷工问题很严重。例如某采煤工因前面 2 个

轮班工作没做好，导致煤太硬，采不了，就需要用机器、工具去采，很辛苦，效果也不好，导致他工作 2 天就放一天假，然后也不补班。有些轮班因为人手不够，剩下的采矿工人就需要在地下多做两三个小时，才可以把整个长壁做完。最后当有些轮班时，采矿人数确实太少了，其他人也撂担子，导致无法正常开工。因为采用机械化大规模采煤的各种问题，有些矿场开始引入自主团队组织架构，取代原本的轮班分工（下面称为传统分工）。

表 1 是两个不同的组织架构，技术都一样，也是在同一个环境。左面用原本长壁的方式设计工作分工，右面的加上自主团队负责制分工。左面矿工只需要做一项简单工作，比如采煤，与其他工人没有任何关系（在前面已详细举例说明）。右面就需要组员合作，协调应对采矿的各种特殊情况。从这个例子看到同样一个采煤技术，可以有不同的组织架构支撑，效果完全不一样。所以团队组织架构必须与技术部分相配合，不能单独设计工人工作。

表 1:

	Conventional传统分工	Composite自主团队
Number of men人数	41	41
Number of segregated tasks 任务数	14	1
Mean job variation for members 每成员的平均任务/变化数:		
Tasks work with 要处理的任务:	1.0	5.5
Main tasks worked 主要任务:	1.0	3.6
Diff. shifts worked 不同的轮班:	2.0	2.9

从下面表 2 可以看到右面的组织架构更灵活，生产率比传统的单一工作设计高。右面的自主团队架构更能适应变化万千的采矿、采煤环境。采煤的步骤安排，也必须按照按环境的不同来应对。但如果是左面的硬性单一分工安排，缺乏这种灵活性（除非是专业性强的技术工种，必须把工作细分, 每个人做某一件事)。但在采煤机械化、批量生产这种技术没有这个限制，所以采煤工人经过一些学习可以兼任其他工作。

表 2:

	Conventional传统分工	Composite自主团队
Productivity(%) 生产率	78	95
Ancillary work at face(hrs per man-shift) 辅助工作(小时/每轮班)	1.32	0.03
平均后备人力/总人力 (%)	6	no
Shifts with cycle lag 轮班延迟 (%)	69	5
最长连续轮班数（没有轮班有问题导致取消）	12	65
average per cent of coal won each day平均每天获得的煤炭%		

团队组织架构除了让工人更好做好采矿工作以外，也可以更好满足工人的个人心理需要，包括互相支持，有团队合作的概念。但是如果按左面传统的工业化分工，就缺乏合作。导致很多采煤工只能单打独斗，孤立无援，导致心理压力很大。这个也可以从表 3 的缺勤率看得出来，会更容易无缘无故请假等，背后主

因是分工设计导致工人心理压力太大，承受不了。团队互相协助的环境可以减轻这方面的压力。

表 3:

	Conventional传统分工	Composite自主团队
Absenteeism (% of possible shifts)旷工率(可能轮班数之百分比)		
Without reason 没有理由	4.3	0.4
Sickness or other 病或其他	8.9	4.6
Accidents 意外	6.8	3.2
Total 总数	20.0	8.2

ChatGPT 自动生成代码质量研究结果重点

2024 年发文的研究分析了 4066 个 ChatGPT 自动生成的 JAVA/Python 程序，发现 2756 个正确（约占 68%），1082 个结果不正确，177 个编译出错。

- 成功出正确结果% 与程序难易度相关，容易的程序成功率最高，难的成功率最低，JAVA 与 Python 一样
- Mann-Whitney U 测试 JAVA 与 Python 间的差异是否显著 P-value 都大于 0.05，差异不显著（详见下表）

表 1: LeetCode 的通过率准确率

Pass@1	低 Easy	中 Medium	高 Hard	总 Overall
Python	0.890	0.674	0.400	0.664
Java	0.860	0.710	0.468	0.691
P-value	0.346	0.654	0.535	0.471

- 也取决于这程序是否新出现，越新越容易出错（因 ChatGPT 难以从网上找到参考例子）:
- 下图最右是 2023 年 1 月后，最左是 2021 年 7 至 12 月
- 难度越高，新旧间的差异越大

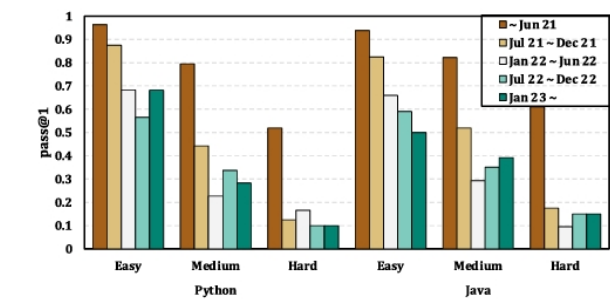


Fig. 5. Pass rate by difficulty and time period.

- 程序越难，发现代码的质量问题越多

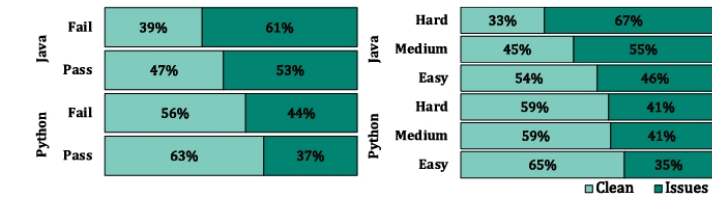


Fig. 7. Code quality distribution by difficulty and language for passed and failed tasks.

- 下表是编译错误、输出结果有误、代码异味与维护性、性能等问题 (issues) 按语言、难度的分布：
 - 代码异味例子：定义了变量，但没有使用

表 3：按难度级别和编程语言的问题分布

	Easy (501)		Medium(1,064)		Hard (468)		Pass (2756)	Fail (1310)	Sum
	P	J	P	J	P	J			
Compilation and Runtime Error	7 (1%)	8 (2%)	37 (3%)	32 (3%)	46 (10%)	47(10%)	0 (0%)	177 (14%)	177 (14%)
Wrong Outputs	47 (9%)	60 (12%)	290 (27%)	260 (24%)	229 (49%)	196 (42%)	0 (0%)	1,082 (83%)	1,082 (27%)
Code Style and Maintainability	174 (35%)	230 (46%)	431 (41%)	588 (55%)	194 (41%)	313 (67%)	1,243 (45%)	687 (52%)	1,930 (47%)
Performance and Efficiency	1 (0%)	2 (0%)	20 (2%)	16 (2%)	6 (1%)	6 (1%)	0 (0%)	51 (4%)	51 (1%)

- 利用 chatGPT AI 功能自动修复，成功率按程序语言、难易度和错误类型分布：（下图）
- 从 20% 左右起，最高能达到 60%

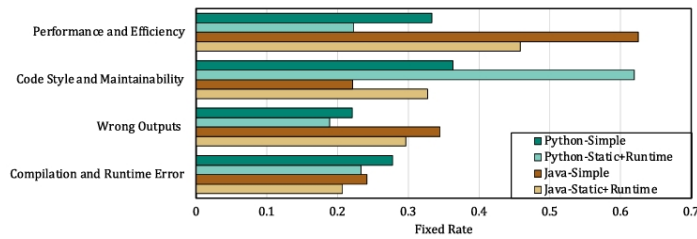


Fig. 8. Comparison of fix rates for different feedback types and code quality issues.

参考 References

1. E.L. Trist and K.W. Bamforth, 'Some Social and Psychological consequences of the Longwall method of Coal-Getting, Tavistock Institute (1951)
2. F.E. Emery and E.L. Trist, 'Socio-technical systems', in C.W. Churchman and M. Verhulst (eds.), "Management Science, Models and Techniques, vol. 2, Pergamon (1960)
3. Ackoff, R.L., J. Magidson and H.J. Addison, *IDEALIZED DESIGN Creating an Organization's Future*. Wharton School Publishing (2006)
4. Barney, J.B. and Reeves M., 'AI won't give you a New Sustainable Advantage.' Harvard Business Review Sept-Oct, 2024.
5. Liu, Y. et al. 'Refining ChatGPT generated code: characterizing and mitigating code quality issues.' ACM Trans. Software Eng. Methodol. Vol.33, No.5, Art.116 June 2024.

Part VII

度量与分析

当团队开始养成每天收集数据的习惯，在迭代回顾时分析数据，公司便可以开始准备度量与分析。（注）

要做好度量与分析，跟过程改进一样，要针对目标，制订度量计划。

本部分开头会以问卷调查为例，介绍度量计划的主要元素，然后会介绍假设检验、控制图等数据分析的基本功。

请不要误以为度量与分析必须由管理层驱动。若要团队能从定性升到定量管理并能持续，而且有效果，必须从团队开始，而不是靠管理者总体策划。

Chapter 21

取消量化目标

“我们想找个质量评价标准，打磨了一年，从网上搜到这个代码缺陷率标准就用了：

目的

- 通过数据督促开发提高代码质量，提高产品质量。

CMMI 定义

CMMI 级	千行代码缺陷率 #
1 级	11.95‰
2 级	5.52‰
3 级	2.39‰
4 级	0.92‰
5 级	0.32‰

千行代码缺陷率 = (bug 数 / 代码行数) * 1000‰

目的：希望有些具体量化目标，促进团队提升。请教你的意见？”认识多年的质量经理问。

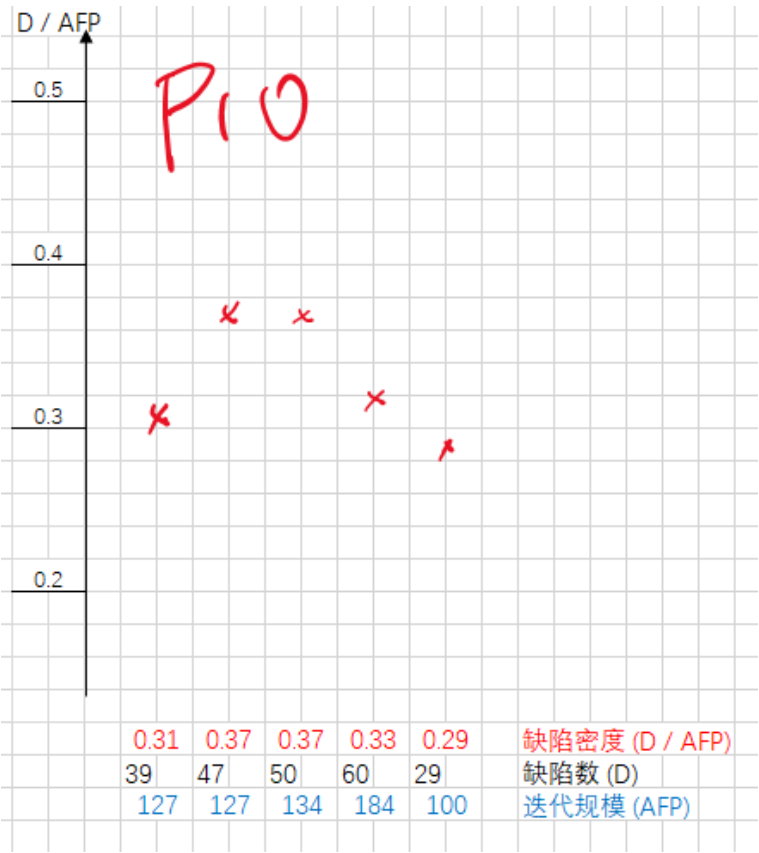
你们一直有系统管理缺陷，应该从历史数据看到不同产品线，对应不同市场，和团队能力不同，有不同的缺陷率范围。（假定说是内部系统测试阶段，同样也适用于验收测试阶段。）

对于某些优秀团队，他们可能已经超越了这个目标；但某些初级的团队会难以达到。

某些项目经理会想：“公司总部质量部制定的目标必须达到，达不到怎么办？”导致出现一年前跟你讨论过的情景：.....我们很注重量化质量管理，搞度量分析，但大家制定算法、原理就开始作假。现在我们主管对度量很反感.....。更重要的应是让团队从过去迭代的缺陷数据统计现在系统能力范围，如果系统已经稳定，如果没有变化，系统不会达到超越现在能力范围的目标。

下面是两个项目的迭代缺陷密度趋势图，请问你觉得那个项目更合适制定量化改进目标？

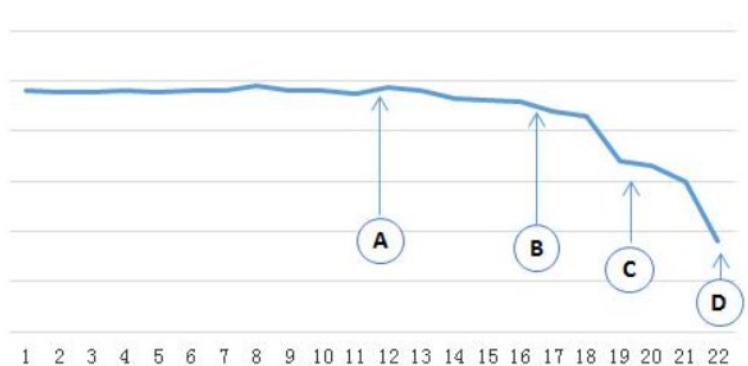
250px



质量经理答：“右面 P10 应该可以，左面 P2 的数据太散了。”
所以如果现在系统不稳定，没有能力，公司制定目标也起不来促进作用：如果公司目标与当前水平差很远，团队会放弃，或报些假数据应付；如果很容易达到，便会反而妨碍了团队制定更高的目标。
所以戴明博士的经典 14 点里包括“取消量化目标”这条：:”Eliminate numerical quotas for the workforce and numerical goals for management.”

建立基线

戴明博士每次质量演讲都会介绍如何使用控制图：假设一家生产矿泉水的企业（通常一瓶水 500 毫升），为了确保每次机器装的水不会太多，也不能太少，每一轮生产都会随机抽 5 个样本，然后记录抽样的均值和范围（最大和最小）。
下图是过去 22 天均值的变化：

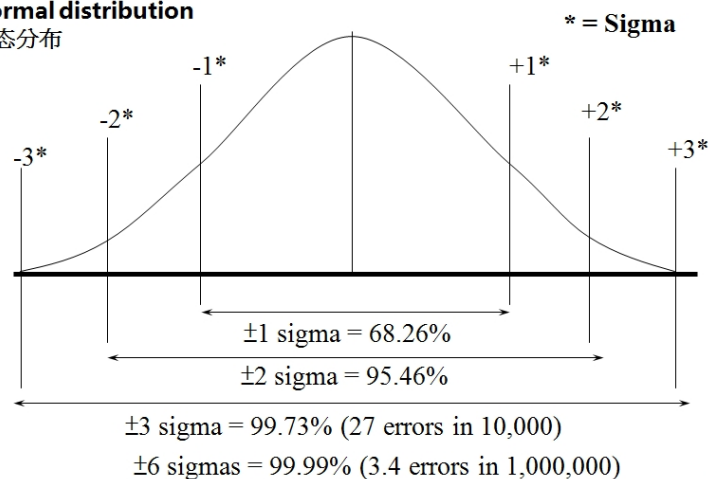


肯定是后面比之前的下跌了，但是如果等到最后 D 点才行动就太晚了，已经跌了太多，但是在那个点开始有变？

这也是 20 年代美国电话公司要解决的问题：很多电话线路与设备都埋在地下，调配维修工作很困难，如何能尽量减少无用的调配工作？

公司也利用生产线大量生产电话设备，需要调试生产线的各种设备参数，但工程师发现难以把生产线调试到稳定状态，如果按系数低了调高，或者高了调低，反而会越调越乱。针对这难题，Dr Shewhart（当时在 Bell Labs 工作）发明了控制图，用来区分是过程的噪音还是信号。下面是统计学的正态分布图：

Normal distribution
正态分布



正态分布，正负 3 个标准差包括了 99% 以上的面积。按中心极限定理，任何分布如果随机抽样够多的话，它的均值就是接近正态分布。所以控制图按样本平均值的标准差，用方程式计算上限和下限，如果后面有些点超出了上下限范围（因上下限是按正负 3 个标准差出来，所以任何后面的点超出这范围，它发生的概率是少于 1%。）利用控制图帮我们区分那些波动是噪音，那些波动（异常点）是信号。（参考附件，了解各种常用于生产管理的控制图，和中心极限定理。）

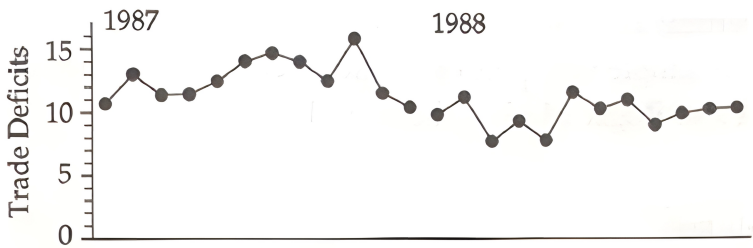
注意：虽然上面用正态分布来理解为什么选上下 3 个标准差为控制图上下限，但 Shewhart 先生的控制图方法没有规定或假定数据必须是正态分布。

控制图的应用例子

看下面 1987~1988 美国国家总贸易逆差每月的数据趋势图，就更好了解美国贸易逆差这两年的变化。

美国按月贸易逆差，1987 ~ 1988 (\$, 亿美元)

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1987	10.7	13.0	11.4	11.5	12.5	14.1	14.8	14.1	12.6	16.0	11.7	10.6
1988	10.0	11.4	7.9	9.5	8.0	11.8	10.5	11.2	9.2	10.1	10.4	10.5



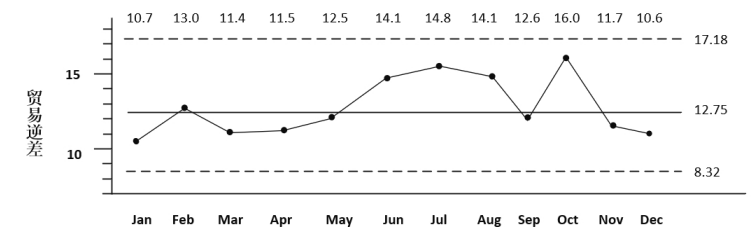
2 年的美国贸易逆差数据有变化吗？

有些人会觉得有变化，比如中间 1987 年 10 月就特别高，后面 1988 年的 5 月也特别高，应该是不正常。

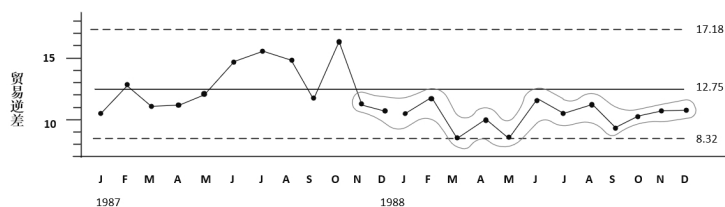
但其他人可能看不出来，反而觉得好像是后面那些数比前面低了。

要判断分析数据有没有变化，很难单靠主观识别，需要有工具帮助。控制图便可发挥作用。

不是说 10 月份特别高吗，但是如果用控制图发现它还是在范围之内。如果我们多看两个月——11 月和 12 月，发现贸易逆差又跌下来。



我们继续看，发现 3 月有异常点，后面在 5 月份也有异常点。如果我们再看后面月份的逆差数据，看见明显比前面降低。

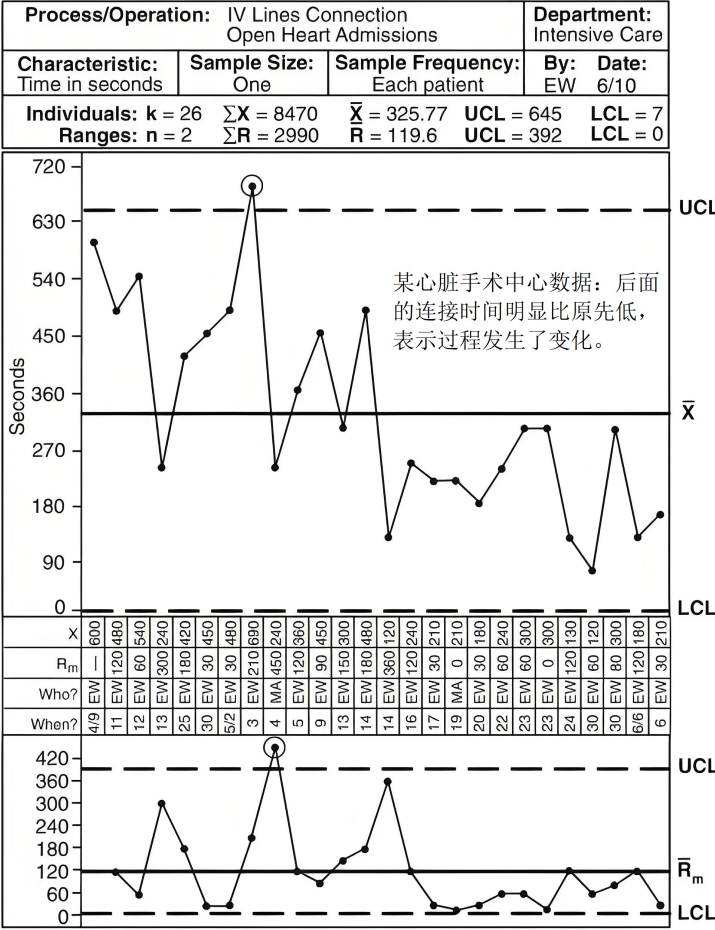


控制图帮助我们识别数据的变化是噪音还是信号。例如 87 年 10 月份的 16.0 只是噪音（未超出上下线），但后面 88 年的 3 月份 7.9 和 5 月份的 8.0 是信号，表示逆差有根本改变，应重新按数据制定上下限。注意：不应把两年的数据的均值与范围（极差）作为基线，应该是以后面降低后的那些点，才能真正反映当前的水平和分布。

如果不是工业生产，不一定有这么多数据去随机抽样、求均值，怎么办呢？可以用 ImR 图 (或 XmR)，比如下图就是某个医院做外科心脏手术所花的时间的统计，想看看手术的时间是否稳定，因为外科手术特别危险，越长时间那患者的存活机会就越低，也看到是有异常点。

IV Lines Connection Time

连接静脉导管时间



(ImR 控制图相关公式，详见附件)

ImR 控制图可以用于软件开发。比如一些已经进了维护期的产品都很稳定，你就可以用控制图来识别有没有变化。但是一些全新的过程可能变化很大，可能控制图就不一定适用。但如果是客服收多少投诉，每天都是很稳定、连续的，应该可以用。

软件开发应用实例

某企业非常关注交付有没有延误，每月都收集各个部门发生投产延误的次数，除以项目发布投产总数，统计并每月汇报高层。
从 2021 年初开始使用控制图，计算上限与下限 (0.98 , 0.93)，3 月份统计发现系数是 0.92 (0.92 是按时交付率，所以是越少越差)，超出了下线范围，改进小

组启动根因分析。

每月的统计数是从 7 个模块（事业部）汇总来，改进小组利用 80/20 原则，识别出贡献最大的头两个模块已占总数之 73%，便专门调研这两个模块。使用问卷、面谈等，找出主因：

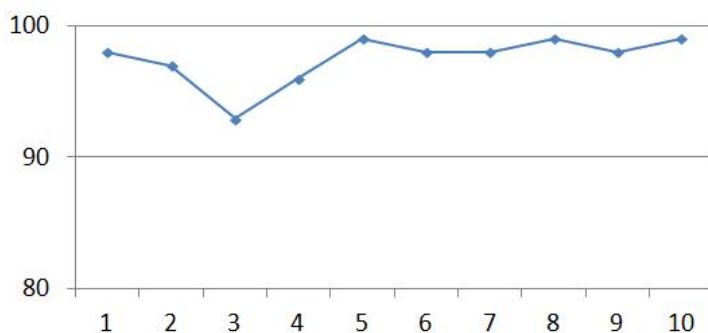
1. 新进项目经理较多，对里程碑管理、关闭条件等要求不清楚。
2. 项目有点难，和用户沟通效果不佳。

对应改进行动：

- 对项目经理的里程碑管理过程进行改进。包括：
 1. 培训（讲解项目管理工具的使用和里程碑管理要求）。
 2. 考核（明确如果出现里程碑红灯的考核标准）。
 3. 预防（建立专项沟通小组，对存在进度风险的项目，提醒项目经理尽快推进，项目经理和用户提前沟通，准备变更）。

我问：效果如何？

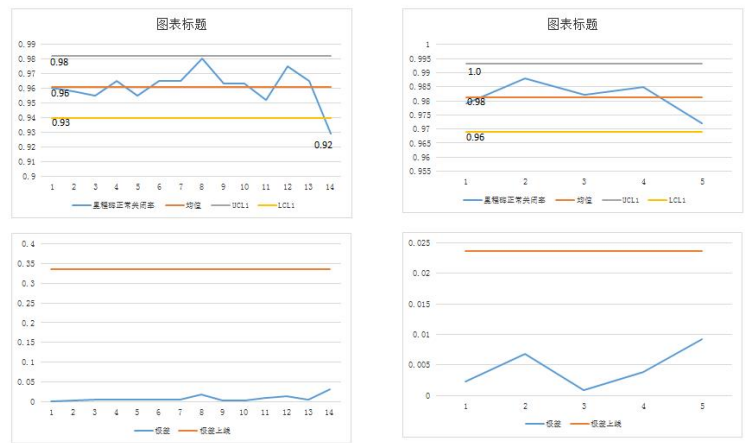
过程改进组长：请看下图今年到 10 月份的趋势图，从 3 月份的低点后面就明显改善了。



我：建议你用控制图方程式计算，从 5 月份到现在的统计图。

第二天，过程改进组长展示之前之后的两个控制图：

下面左图是 3 月份 0.92 与之前一年多的控制图，右图是改进后 5 月份开始的控制图。



我：从左右控制图看得出改进后的下限从本来的 0.93 上升到 0.96。除了画图，也可以计算过程能力指数 (Cpk) 反映过程能够满足客户要求 (Cpk 计算公式，详见附件)。

正与你说公司本来要求不能低于 0.93 (客户要求规格)，所以 3 月份之前的能力指数 $Cpk = 1.0$ 。但现在过程收窄了， $Cpk = (1-0.93) / (1- 0.96) = 1.75$ 上升了。因公司看到提升，便收窄了规格范围，要求把下限调高到 0.96，你们的能力指数 Cpk 又变回了 1.0。所以 Cpk 可用来代表过程满足客户要求的能力系数。

过程改进组长：从 5 月后数据更新的控制图一直都很稳定，没有异常点，是否表示我们就没有提升的空间，下限 0.96 已经是最佳状态？

我：不一定，你 3 月份分析时，不是说利用二八原则，识别出两个影响最大的板块吗？

现在你的控制图是把所有板块的总数的控制图，建议你试试针对那两个影响最大的板块，画他们的控制图看看。有机会能看到一些异常点。

第二天，过程改进组长：确实看到有较大的波动，但还没有超出上下限，我们会继续观察。

以上实例帮我们了解：

1. 基线

从历史数据得出基线范围能帮助我们区分后面哪些波动是自然噪音，哪些是信号。

信号表示过程可能有基本变化，需要更新基线。控制图帮助我们能做好这区分。

2. 过程能力 (Process Capability)

- 识别。
 1. 过程的声音 (过程的自然波动范围)。
 2. 客户的声音 (客户规格上限下限)。
- 计算过程能力指数 (Cpk) 反映过程能够满足客户要求。

3. 细分

- 不仅仅看总的综合控制图
- 从总图细分，细分后的控制图可能发现异常点信号 (详见附件有一个细分例子)

总结

如公司管理者依赖主观判断，定一个“合理”量化目标，反而会影响了团队的改进。更应按团队的历史数据，使用控制图，先制定团队的基线，让团队自己分析根因（也可以配合改进组的辅导），制定纠正措施后，才会有具体的提升目标。

附件

中心极限定理 (Central Limit Theorem CLT)

当总体 (Population) 均值为 μ ，标准差为 σ

随机样本均值

$\bar{X}_n = \frac{\sum_i X_i}{n}$ 的: (n 为每轮抽样样本数量)

均值 = μ

标准差 = $\frac{\sigma}{\sqrt{n}}$

- 样本均值的分布随样本数量增加越来越接近正态分布

检验统计量计算值 $Z_n = \frac{\sqrt{n}(\bar{X}_n - \mu)}{\sigma}$ 接近正态分布。

如何画 **ImR** 控制图

- 1、计算单值 (X) 平均数 (X-bar)
 - 2、计算移动极差 (mR moving Range) 均值
 - 3、利用以下方程式，计算 X 的上下限
 ::UNPL=X-bar+(2.66*mR 均值)
 ::LNPL=X-bar -(2.66*mR 均值)
 - 4、利用以下方程，计算移动极差的上限
 ::URL=3.27* mR 均值
 - 5、如果 X 或 mR 有超出范围，表示有过程变化的信号，过程不稳定
- Note:

UNPL=Upper Natural Process Limit

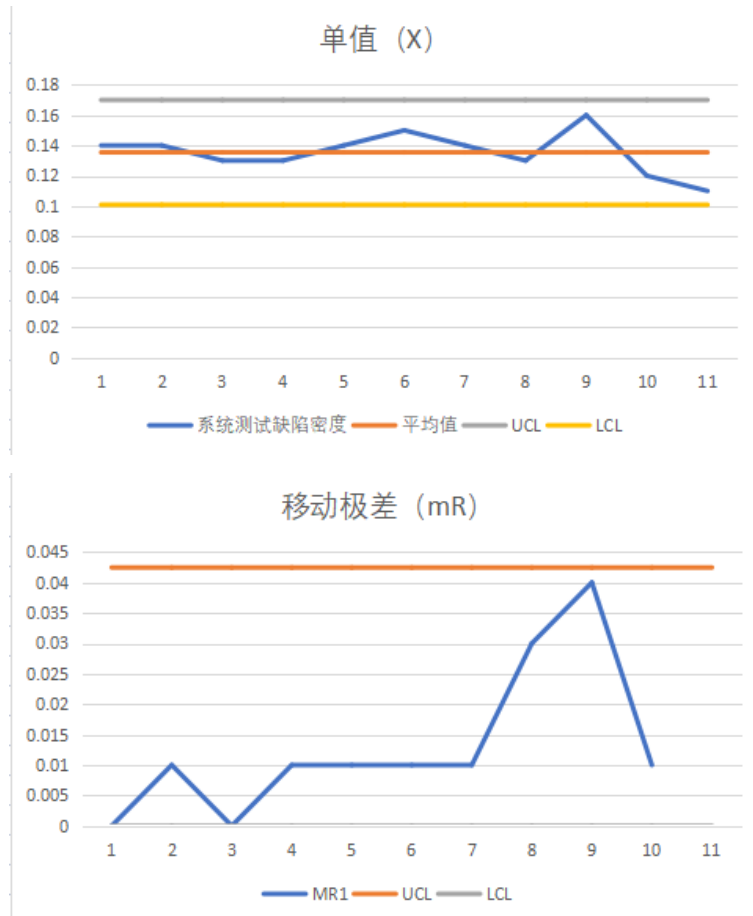
LNPL=Lower Natural Process Limit

URL=Upper Range Limit

例子:

: _20210929131748.png

- 1、计算单值 (X) 平均数 ($\bar{X}$) = 0.135 [(0.14+0.14+...+0.11)/11]
- 2、计算移动极差 (mR moving Range) 均值如第一移动极差为 0.14-0.14= 0; 第二移动极差为 0.14-0.13= 0.01; 10 点的均值 = 0.0128
- 3、计算 X 的上下限: 用上面公式 $UCL=0.135+2.66*0.0128=0.17$, $LCL=0.135-2.66*0.0128=0.10$
- 4、计算移动极差的上限用上面公式 $UCL=3.27*0.0128=0.042$, $LCL = 0$
- 5、下面 2 图已经加上上下限与平均线:



各种控制图

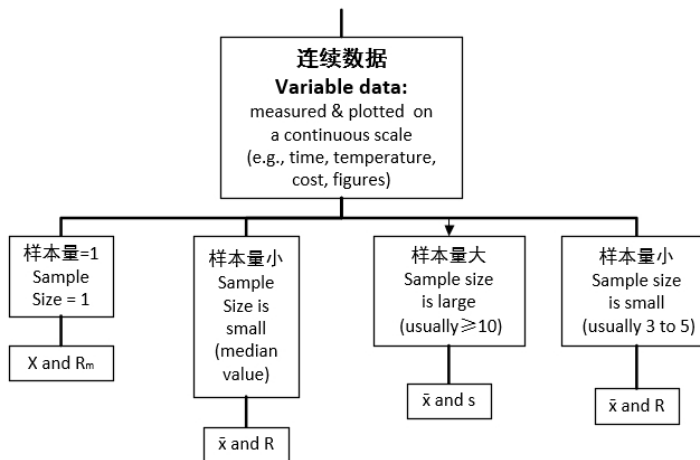
生产线用连续数据的控制图有两部分:

- 均值或中位数
- 标准差或者范围

两个图都要看。通常一些数据本身的价值有异常点的话, 很可能它的标准差变化也会有异常点。

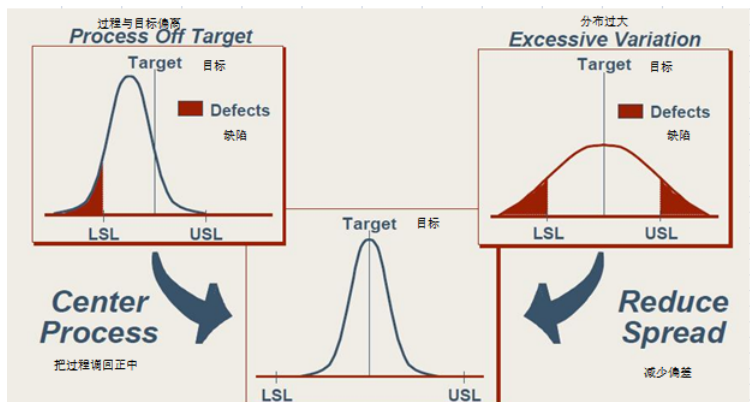
下面就是各种对连续数据的控制图方法，大部分都是按抽样样本的大小制订，比如抽样的样本数大于 10，我们就可以用平均值和标准差来做控制图。

基于样本多少选择恰当的控制图

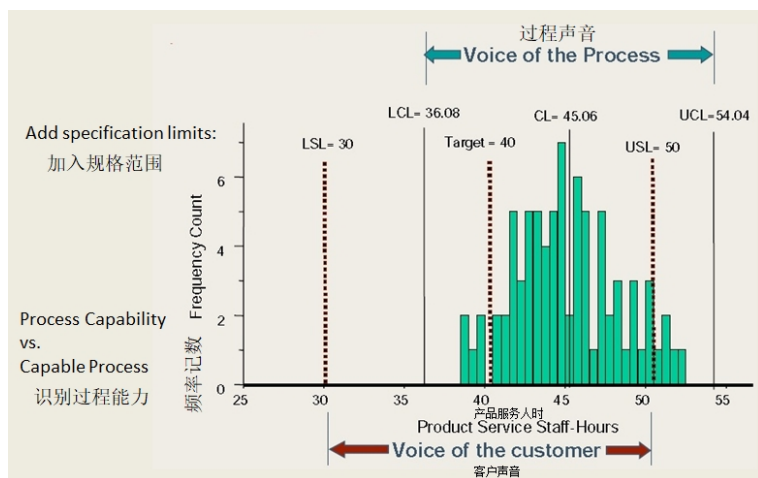


过程能力 (Process Capability)

过程能力反应能否满足客户要求:



中间的图就是客户的声音，USL 和 LSL 是客户要求的规格上下限，右上图就代表过程分布太宽，只有中间部分能满足客户规格要求，左图虽然过程变化范围满足，但偏离了，也不能完全满足客户要求。下图看到过程的范围是不能完全满足客户的规格范围要求:



以下方程式可以计算过程能力指数。 $C_p = \frac{USL - LSL}{6\sigma}$ ，不考虑是否偏移，只考虑分布范围。

如果考虑偏移，计算 $C_{pk} = \min\left[\frac{USL - \mu}{3\sigma}, \frac{\mu - LSL}{3\sigma}\right]$ (选其中最少数值)
指数越大表示过程越能满足客户规格（客户声音）。

参考 References

1. Wheeler, Donald. *Understanding Variation The Key to Managing Chaos*. (SPC Press, Inc., 2000)
2. Florac, William A. with A. Carleton. *Measuring the Software Process: Statistical Process Control for Software Process Improvement*. (Addison - Wesley, 1999) SEI Series in Software Engineering

Chapter 22

教小孩统计分析

“过去半年，我们几个团队在每次迭代回顾时都统计了一些数据，总共有 70 多条，后面有什么数据分析？”

“你们的目的是什么？”我问。

“希望预测下一轮迭代的工作量。”

“确实可以利用项目历史数据做预测，但请问你记得以前高中学过的统计吗？”

“统计学很高深啊！我都还给数学老师了。”

“若要深入了解和研究统计分析，确实要具备很好的根底，但如果只是应用——利用统计分析解决日常的问题，便简单多了。统计里有很多可用的工具，虽然不用深究背后的原理，但还是要知道怎样应用。以下是我如何教小孩统计分析的故事。”

从考试成绩开始

孩子：爸爸，你是做什么工作的？我明天要在班上介绍自己父母的工作。

我：帮客户做统计分析。

孩子：统计分析？

我心里想你这初中孩子，要如何给你解释统计分析呢？

我尝试用一个他可以想象的场景来介绍——假如下面是你班里同学的数学成绩，你会用什么方式把这些成绩表达出来？

孩子有点迷茫。

=== 20 名学生在数学考试中的分数 (满分为 100 分，按大小排序) ===

30, 35, 37, 40, 40, 49, 51, 54, 54, 55

57, 58, 60, 60, 62, 62, 65, 67, 74, 89

我：任何样本数据，都可以用图加平均值、范围等来表达。

举例：我们可以画茎叶图 (Stem and Leaf Plot)(详见附件) 表达这些学生成绩：

文件:MA_FA2 1.0.1.jpg

把 20 组数从最低排到最高。排在第十的学生和第十一的学生的平均数 56，叫中位数 (Median)。排第五和第六成绩的平均值 44.5，就是 Q1 (1/4)。排第十

五名和第十六名的平均值 62，就是 Q3 (3/4)。
这个例子中茎叶图再加上 Q1、Q2(中位数)、Q3，[44.5、56、62] 便可以总括表达这 20 名学生的成绩分布。也可以用下面的柱状图/直方图表达：

500px

也可以利用 Q1、Q2、Q3(四分位数) 画箱线图来表达 (详见附件)。
除了中位数，也常用平均值 (Mean) 来表示数据的中间趋势，平均值就是把所有数据加起来，然后把总数除以数量，比如上面 20 名学生数学成绩的平均值是.....

孩子充满自信，打断我说：这个平均值简单，我懂。

我说：好啊，你这么懂平均值，我问你一个小问题？

```
{| class="wikitable" | 问题：5、10、350、355 的平均数是多少？
```

```
|}
```

孩子拿了计算器算出：180。

我：你知道那些数是代表什么吗？

孩子：不知道。

如果这些数是代表角的度数，它的平均值应该是零，而不是 180。这是统计分析很重要的概念——你必须知道那些数的背景，否则你的分析没意义。如果你把那 4 个数字当成是距离来算均值是 180，与 4 个角度数求均值的结果就完全不一样了。

从以上例子，你就知道了解问题背景 (Context) 很重要。

孩子：这些统计分析有什么用？

我：例如有了这成绩的分析，你就可以知道自己的成绩在整个班是处于什么位置？也可以帮你比较不同科目的成绩。

比较 3 种教学方法

45 名学生被随机分成 5 个大小相同的组。两组 (A、B) 采用目前使用的方法 (控制组)，另外 3 组 (C、D、E) 采用 3 种新方法。实验结束时，所有学生参加了一次标准测试，结果 (满分 30 分) 见表 1。关于教学方法的差异，可以得出什么结论？

表 1

45 名学生的测试结果									
A 组 (控制组)	17	14	24	20	24	23	16	15	24
B 组 (控制组)	21	23	13	19	13	19	20	21	16
C 组 (赞赏组)	28	30	29	24	27	30	28	28	23
D 组 (责骂惩罚组)	19	28	26	26	19	24	24	23	22
E 组 (不理睬组)	21	14	13	19	15	15	10	18	20

我：你可以用刚学过的四分位数 (Q1、Q2、Q3) 配合箱线图表示每一组数据分布，然后看看有没有差异。这样你便可以比较 5 个班的成绩。

他过了 15 分钟就画了 5 个组的箱线图：
500px

我：能看到显著区分吗？

孩子：看得出是 C 组最好，D 组也比较好，E 组就比较差。

我：是的，但如果我们只是比较 5 个组成绩的平均值，没有考虑每一班成绩范围，便不能全面比较。例如：上面数学考试成绩的 Q1、Q2、Q3 是 (44.5、56、62)，如果 C 组的数学成绩是 (32、57.5、63)，你会认为 C 组比你们好吗？虽然中位数比你们班高，但 Q1 比你们低很多，所以不能单看中位数（或平均值）比较。

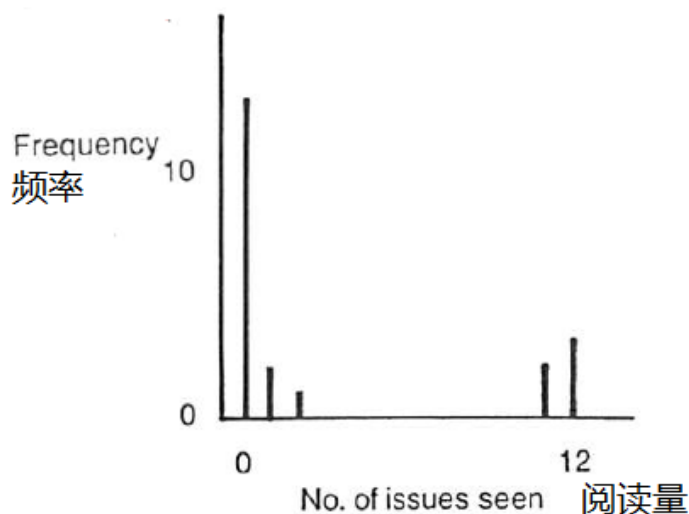
看到他对这些挺有兴趣，我便问他如何表达以下数据。

20 人一年内阅读月刊的数量

0, 1, 11, 0, 0, 0, 2, 12, 0, 0

12, 1, 0, 0, 0, 0, 12, 0, 11, 0

我看他开始使用同样方法，我便说这些统计数据跟前面学生成绩的分布不一样，它一头一尾最高，这表示大部分人要么就是不看杂志，要么就是每个月都看，这种分布就不合适用刚才那些方法去表达，只能说它有两个高峰，或者叫众数 (Mode)，再配上一个柱状图。如果我们用中位数或者这些数的平均值，或三分位数来表达的话，反而是误导读者，不能正确的表达那些数据的分布。



当总体 (Population) 包含非常多数据时，就只能抽样，用抽样来估计 Population 分布。但是要注意，如果样本不是随机抽样 (Random Sample)，可能会导致出来的参数偏离、产生误差。

我看孩子一头雾水，但我记得他很喜欢研究二战的战斗机，我就用下面这个例子说明，这样抽样出来的结论是没有意义。

取样偏差（Survivor's Bias）

错误的抽样会导致错误结论。

第二次世界大战时会对于那些没有被击落的战斗机，研究哪个部分被德国军队的攻击最多，针对这些部位去加强防护，希望增高飞机的存活率。你同意吗？我们只是抽样了未被击落的飞机，被击落的都未被抽样。一个比较正确的抽样是所有两类都抽样才比较合理。

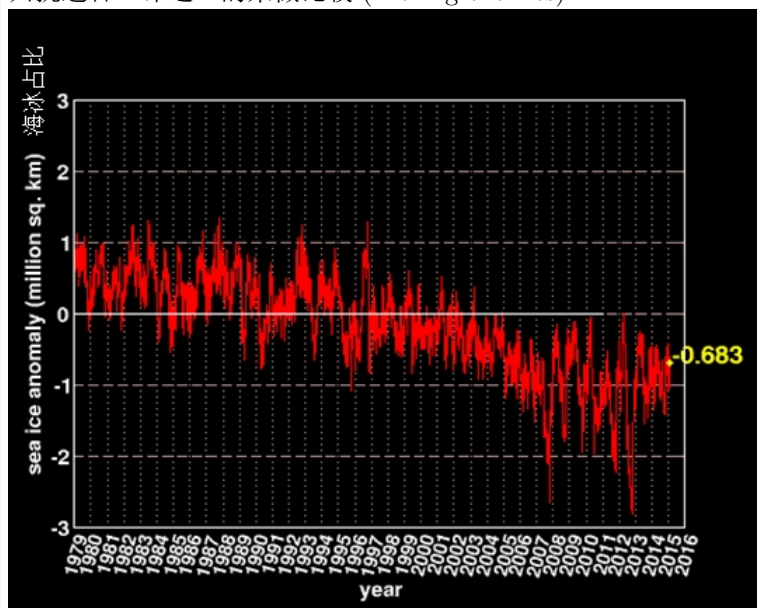
其实被击落的飞机哪个部分被击中更重要，但是我们无法得到那些抽样，只可以从没有被击落的飞机去看，这是抽样的错误。所以分析得出的结论没有意义。

二战时期战斗机



我：你看看下面例子，便会更了解随机抽样的重要性：

只挑选合“味道”的来做比较 (Picking cherries)



从上面 80 年代到现在全球海洋冰量的统计很明显看到地球在不断地暖软化。但你还可以抽两个时间，例如：2012 年 4 月份的冰量比 1988 年 6 月份多，来说明“全球变冷”！

孩子对统计分析好像越来越感兴趣。

我：现在你开始知道什么叫统计了吗？明天可以跟老师讲故事了吗？

孩子追着问：可不可以再给我一个练习题？我觉得这统计学也没什么困难，我还可以把你给我的那些题目拿给我同桌小李试试，看他懂不懂（男孩总是要当英雄，出风头）。

我：好啊。下面的身高统计数据，请你用刚才的方式来汇总一下，看看你学会没有？

20 名妇女参加某种疾病研究，她们的身高（以米计）分别为：

1.52, 1.60, 1.57, 1.52, 1.60, 1.75, 1.73, 1.63, 1.55, 1.63

1.65, 1.55, 1.65, 1.60, 1.68, 2.50, 1.52, 1.65, 1.60, 1.65

他便拿着纸，用刚才学过的方式计算中位数、三分系数等，蛮有自信地画图、交卷。

我：你认识 2.5 米高的女性吗？

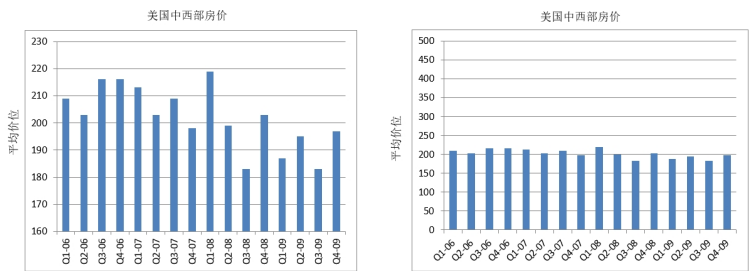
小孩：好像没有。

我：所以里面的 2.50 肯定是不对。你为什么没有疑问，直接去做？统计分析必须要判断数据是否正确、是否合理。

如果数据本身不可靠，那么怎么分析都没用，我们称之为“垃圾进，垃圾出”（Garbage in, Garbage out）。

我：有很多人利用图表误导读者，所以我们看“统计专家”的分析报告时要小心。

美国中西部房价趋势图



上图是 1906 年到 1909 年美国中西部每个季度的房价。如果我们看右面那个柱状图，你会认为房价波动很大。但是如果同样这组数据看左面柱状图的话，你就会觉得没有太多变化。所以我们要注意看左面坐标有没有标明数字。不应该仅仅看图。

虽然数据都一样，但如图形不一样，传达的信息便完全不同。

孩子：你说的我都懂，很简单。

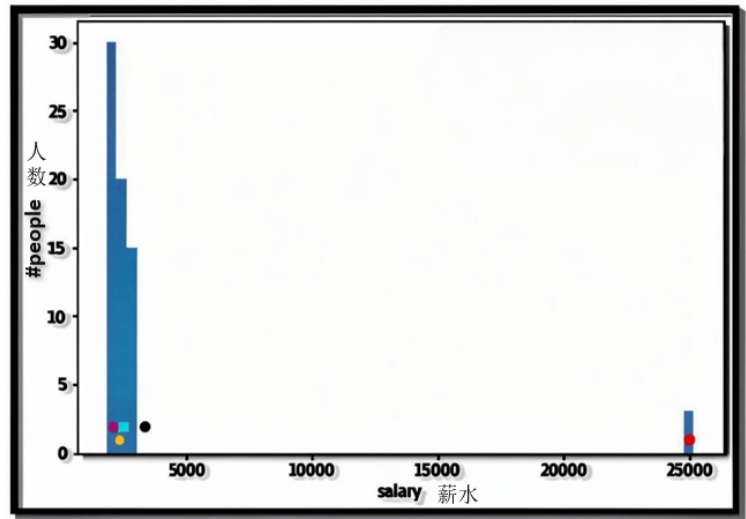
我：有很多用统计分析骗人的故事。你不是说要今年暑假找暑期工赚钱，下面是一个去工厂找工作的故事：

{| class="wikitable" |-| 小李刚中学毕业去工厂找工作。

小李问人事：工厂员工薪水是多少？

人事部经理面带笑容地跟他说：我们工厂员工平均月收入大概 3300。

小李进了工厂后，才发现绝大部分的工人月薪是 2000。有多年经验的工人才可以拿到 2400。但有少数的管理层的平均月薪是 25,000。



工厂员工平均收入是每月 3309，所以人事部经理没有“骗”小李。

}

分析例子：连续与分组数据

第二天，孩子乖乖地来找我。
他说：爸爸，今天我跟老师说了你的工作是做统计分析，老师听到就发我一些高年
级的生物实验数据，希望帮他做些分析，看两类实验有没有差异。你可以帮忙吗？

蚕豆植物 —— 双样本 T 检验

以下数据比较 10 个剪枝和 10 个生根样本中某化学物质的浓度：
剪枝:53, 58, 48, 18, 55, 42, 50, 47, 51, 45
生根:36, 33, 40, 43, 25, 38, 41, 46, 34, 29

我：你可以同样用昨天学过的方法（如柱状图）比较两组数。
孩子就按这个汇总了两个实验数据的分布。

浓度	
15 - 19 20 - 24 25 - 29 30 - 34 35 - 39 40 - 44 45 - 49 50 - 54 55 - 59	
剪枝	1 1 3 3 2
生根	2 2 2 3 1

我：你应该也看出剪枝样本有一个很明显的异常点——18。你应该问老师，18 是否错误还是值得相信的数？如果 18 是不对，你就很明显看到上面剪枝的平均浓度比下面生根高。如果不能“看”出来就可以用“双样本 T”检验比较两组数据是否有显著差异。但是如果有刚刚那种异常点，如直接用双样本 T 分析，因为第一组数据被那个 18 拉低了，得出结果是两组没有显著差异。所以还是直接看数据的分布更实在。

我：我们在统计学可以使用假设检验（注），其中包括双样本 T 检验比较、方差分析 (ANOVA) 等，就是针对这类统计数据分析。但是在你这个实验数据，从画图一眼都能看出有显著区分，就不需要再采用假设检验，因采用那些方法不会得出额外的信息。

我接着跟他说：有些时候统计数据不是连续数据，像学生成绩、浓度、高度等，而是分组数据。例如下面这些国外人种眼睛颜色与头发颜色的数据，这种数据我们就需要用列联表 (Contingency Table) 来识别这两个变量之间有没有关系。

头发和眼睛颜色的不同组合

592 人的头发和眼睛颜色的不同组合：

表 1	头发颜色			
眼睛颜色	黑	深褐	红	金
棕	68	119	26	7
蓝	20	84	17	94
淡绿褐	15	54	14	10
绿	5	29	14	16

如果我们利用表 1 的数，把它那些每一行、每一列进行求和，我们可以看见每列或者每行的分析结果。例如棕色眼睛的占大多数，但金发例外，金发的大部分是蓝眼睛。其实也看到金头发的那列跟其他的一些分布很不一样。比如我们看每行的话，也可以看到棕色眼睛跟蓝眼睛的数量差不多。但在金头发和黑头发的列分布就不一样了。统计分析的卡方检验也可以帮我们分析分组数据间有没有相关。但刚才这类简单 4×4 数据表，计算出表 3 列联表便足够，卡方检验没有带来更多有用信息。

实际 (Obs.) 数量表:

表 2	头发颜色				
眼睛颜色	黑	深褐	红	金	总计
棕	68	119	26	7	220
蓝	20	84	17	94	215
淡绿褐	15	54	14	10	93
绿	5	29	14	16	64
总计	108	286	71	127	592

实际 (Obs.) vs 预计 (Exp.) 数量表 Contingency Table:

表 3	头发颜色								
	黑		深褐		红		金		
眼睛颜色	Obs.	Exp.	Obs.	Exp.	Obs.	Exp.	Obs.	Exp.	Total
棕	68	40	119	106	26	26	7	47	220
蓝	20	39	84	104	17	26	94	46	215
淡绿褐	15	17	54	45	14	11	10	20	93
绿	5	12	29	31	14	8	16	14	64
总计	108		286		71		127		592

注：双样本 T 检验、方差分析 (ANOVA)、卡方检验等都属于假设检验 (Hypothesis Testing)，附件里用某卡方检验为例，说明如何利用计算出的 p 值做判断。

总结

分析数据前应先用图或数量描述数据分布，包括：

- 平均值、中位数、四分位数 (Q1、Q2、Q3)、众数
- 柱状图、箱线图、茎叶图等

下表总结本章对应描述性统计的注意项：

步骤	上面实例
	这些数字代表什么? 什么单位 (角度、还是距离)?
策划收集数据	怎样取样? 避免取样偏差
确保数据质量	2.5 米高女性? 异常数据?
初步数据分析 (描述性)	平均值、四分位数 (Q1、Q2、Q3)、柱状图、箱线图
数据分析	如果能简单看出来有显著差异, 便不需要用假设检验, 或其他统计分析方法
解读、沟通结果	当我们读统计分析报告要小心, 注意是否被骗

了解每个变量的分布后, 便可按数据, 分析变量间的关系:

注意项	例子
分析两组数据间有没有显著差异。如果能简单看出分布的差异, 比只用假设检验更有洞察	双样本 T 检验
头发眼睛颜色列联表	
确保数据质量	2.5 米高女性? 异常数据?

从以上例子看到要有效分析, 应按数据本质挑选合适的方法, 而不是墨守成规, 像机器人一样按规定分析。

附件

如何画茎叶图 (Stem and Leaf Plot)

以文中 20 名学生在数学考试分数为例:

1. 把分数从小到大排序
2. 从上而下画一直线
3. 第一个数 30, 左边写 3, 右面写 0 (叶)
4. 排第二是 35, 因 5 大于 0-4, 新一行, 左边写 3, 右面写 5 (叶)
5. 37, 因 7 属于 5~9, 所以同一行, 在 5 右面写 7 (叶)
6. 最终便可以简单利用数目字形成一个横放的直方图

箱线图 (Box and Whisker Plot)

500px

箱线图包括 5 个数:

- Minimum 最少 (Q0)
- 1st Quartile (Q1 , 25 percentile) 箱子的左面边线
- Medium 中位数 (Q2 , 50 percentile) 箱子中间的线
- 3rd Quartile (Q3 , 75 percentile) 箱子的右面边线
- Maximum 最大 (Q4 , 100 percentile)

有些箱线图直接把两头指到最大与最少, 不展示离散点 (Outliers)。

Interquartile range (IQR) = Q3 - Q1

卡方检验 (Chi Square test)

研究三家医院 (A、B、C) 在 3 类最终导致患者死亡感染的主因 (外科手术 Surgical Site、肺炎 Pneumonia、血液 Bloodstream)，分布是否有显著差异？

	C1-T	C2	C3	C4
	hospital	SurgicalSite	Pneumonia	Bloodstream
1	A	41	27	51
2	B	35	3	40
3	C	169	106	109

{| class="wikitable" | - | 假设检验的步骤

零假设: A 哪家医院与死亡主因没有显著关系

挑选对应的检验方法后，可以利用工具计算出的 P 值来判断。

:: 如果 P 值大于 0.05，就不能拒绝零假设。

500px

卡方检验得出 P 值等于零，所以可以拒绝零假设，医院与感染主因有关系。|}

- 如不用工具，也可以手工计算卡方值 (Chi-square)，首先使用以下公式计算每个列联表的预计值 E(Expected Value)，

$$E = \frac{(\text{row sum}) * (\text{column sum})}{\text{grand total}}$$

再用下面公式从预计值与实际值 O(Observed value)，计算卡方值：

$$\chi^2 = \sum \frac{(O-E)^2}{E}$$

- 计算得出卡方值 = 30.696 (注)，比关键值 9.5 高

(对应 DF=4 (=(3-1)*(3-1))， $\alpha = 0.05$ ，从 χ^2 统计表得出 9.5)，

所以拒绝零假设，医院 (Hospitals) 与感染种类 (Infections) 之间相关。

: 注：接近工具自动算出的 30.755。

参考 References

1. Guttag, John V. *Introduction to computation and programming using Python*. (MIT Press , 2021)
2. Chatfield, Chris. *Problem Solving: a statistician's guide*. 2/e (Chapman & Hall, 1995)
3. Bluman, Allan. *Elementary Statistics*. 10/e (McGraw - Hill , 2018)

Chapter 23

做问卷调查

小时候看病，中医是用望闻问切的方式；西医就是用听诊器听心跳、呼吸，看舌头、口腔，然后也会听你讲述有什么不舒服，便诊断并开药了。现在去医院排队看病，医生第一件事就给你开四五张各类的检查单，通过这些检查报告做诊断。虽然我是外行，但我觉得小时候那种不超过十分钟诊断过程不会比现代的起码两个小时的诊断质量差。”

度量与分析同理，我们应针对最相关的度量数据（也容易收集到）来诊断，才有效，还省钱。所以当我看到公司度量分析人员展示数据大表，上面密密麻麻有五、六十个项目变量，包括缺陷、进度、工作量等，我会问：“为什么你们要收集这么多数据？你们的目的是什么？”

数据分析应先制订计划——依据目标收集哪些数据、如何收集、如何存储数据，准备后面如何分析数据。

“你们有做过问卷调查吗？”每次教度量分析，我首先会问。

有些学员为某快餐连锁做过问卷调查：研究哪些因素影响客户满意度；有些为公司做过产品的问卷调查：研究客户重视哪些功能。

他们讲完后，我解释：因为问卷调查的问题不能太多，必须针对调查目的和要解答的问题，设计问卷。度量分析也一样，需要针对目的和问题，制定度量计划。

案例：为某全国快餐连锁做问卷调查

这家连锁快餐店，全国有 3 万员工，但员工流失率严重（接近百分百）。人事部估算聘用、培训、建档案等，每个新员工要花费 300 美元成本，每年由于员工流失就产生 1000 万美元的成本，差不多等于整个集团的盈利。

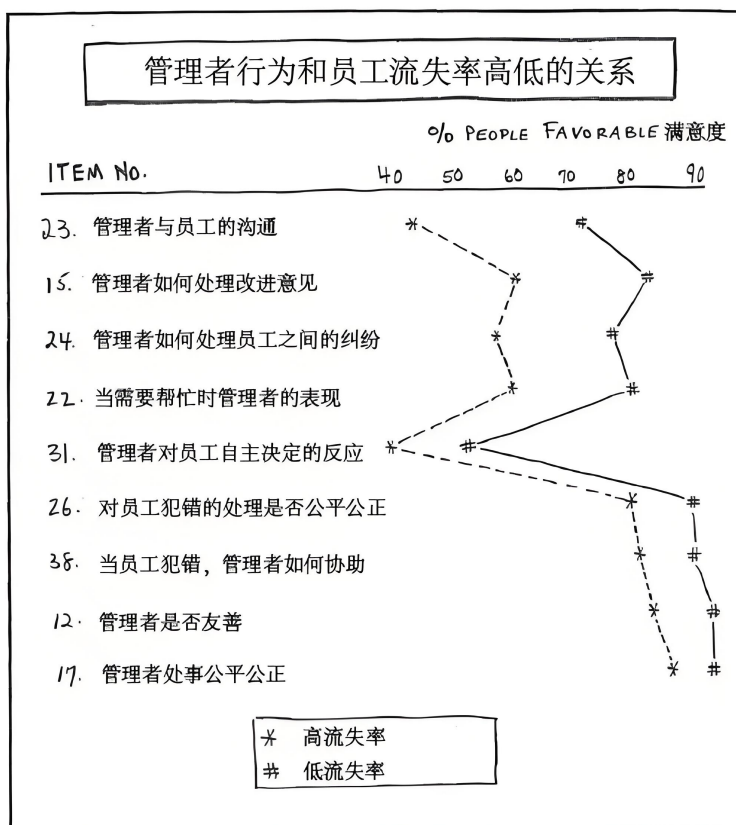
虽然高层一直知道，但觉得大量员工流失率是快餐业的通病，所以没有主意。咨询顾问针对这问题，详细查看流失率的分布，发现不同店铺的员工流失率差异很大，从最低 6% 到最高 800%。

详细分析

抽看其中 100 家餐厅，把每年流失率在 120% 以上的当成高，60% 以下的当成低，分析数据发现在各种职位：前厅、后厨、维修工人等。发现员工流失率与工种无关，主要是跟餐厅有关。

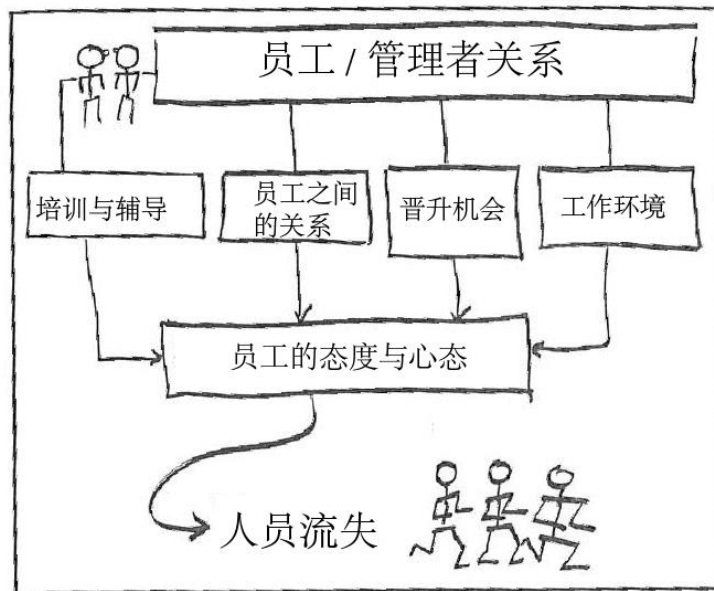
研究高流失率的原因

- 随机抽样 10 家餐厅，4 家属于低等（每年流失率 40%~60%），2 家属于中等水平（60%~120%），4 家属于高等（每年超过 120%）
- 做问卷调查，有 39 条问题
- 初步分析发现高与低流失率被抽样单位，发现它们的员工的年龄、被餐厅聘用了多少年、工种、是否轮班等分布都很类似，所以这些都不是原因
- 主要原因是在管理者的态度。比如低流失率餐厅的员工，在 39 条问题里，关于管理层的 31 条都比高流失率餐厅员工较高分
- 低流失率餐厅的员工觉得他们得到较好的辅导，经理比较乐意帮助他们，甚至更有机会升职
- 低流失率餐厅的员工更满意工作和薪酬（其实无论高与低流失率餐厅的工资水平大致一样）
- 下图是 9 条最有代表性的问卷结果：



从分析到行动

下图针对主因，总结了关于管理者的态度如何影响餐厅员工流失率，汇报给管理层：



因为经理与员工的关系影响到培训、员工的成长机会、晋升的机会、员工之间的关系等，直接导致员工流失。

针对发现开始做实验

对应纠正措施：

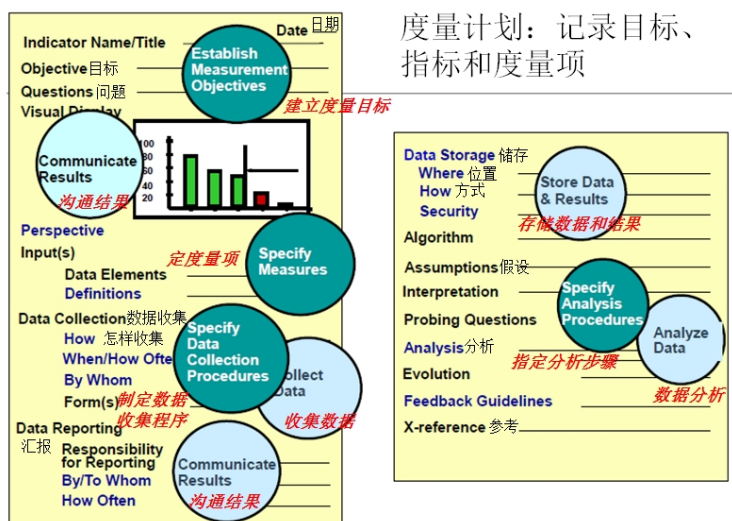
- 为某区域的餐厅经理做针对性培训，也让他们看到从问卷调查，他们店的成绩与全国其他餐厅的比较
- 在 4 天培训里，针对问题解决、领导能力等做培训
 - 培训形式不仅仅是老师讲课，大部分是利用角色扮演：首先列出来他们希望新员工要学到哪些点，然后用角色扮演，在新员工入职培训里面如何执行
 - 让学员自己评判效果或者表现如何
 - 学员也有机会跟高层经理交流，可以反映一些他们在自己餐厅单独解决不了的问题

培训后效果

- 发现在 27 家餐厅中，参加过培训的 24 家后期员工流失率显著降低。例如其中一家有 29 位员工，以前一年离职 37 位，但培训后，没有流失。整个做实验的区平均下来，流失率减少了 50%。

度量分析计划

从以上案例看到如何针对改进的目标设计问卷、收集数据，利用数据分析帮助找出根本原因，制订针对根因的改进措施，然后评判改进效果。先基本分析，比如以什么维度去分析，哪些因素相关、哪些无关；针对不同餐厅，做问卷调查，验证了本来的假设问题——是否经理跟员工之间的关系影响到流失。问卷调查证明了这个关系后，就按对应改进措施做实验，最后看效果。



度量计划应包括（看上图）：

度量目标为何要度量，要解答什么问题。

度量项收集什么度量（对什么对象，问什么问题）。

收集数据，分析数据用什么形式来度量，如何分析，这个是在度量前要想好，有些很重要的因素忘记问，后面无法补。

以下注意项让我们可以避免一些度量分析常见问题：

1、明确度量目标。希望改进什么，然后依据目标或者预估影响结果的可能因素。例如是否跟管理者的领导和沟通能力相关。然后依据这思路，制订问卷内容。

如果有一堆历史数据，但都不知道哪些度量与分析的目标关联，就浪费了。所以度量计划首先要明确度量分析的目标再定需要收集哪些数据。

2、度量项的操作定义。如果定义不清楚，可能不同团队因为理解不一样，提供的数据就不同了。

* 如何确保数据的质量。因为如果数据本身不对，后面什么分析都没用。如果有系统记录，可以利用系统验证数据是否正确。但反过来，如果数据都是凭经理在项目结束最后填一个表给你的话，就会怀疑这些数字有多少的可信度。

3、数据怎么获取。尤其是在软件开发中。如果他们没记录缺陷或者工时，很多数据就无法在回顾时获得，没有数据就没法做后面的分析。然后在做计划时，要想假如那些数据收集到之后要怎么分析？如果问卷没有收集一些餐厅的基本数据，比如区域、员工数量、入职时长等基本数据，后面就无法用这些维度去分析。

4、数据存储。为了方便后面的统计分析，应该形成一个数据表的格式，比如一个项目一行，每一列表示不同的变量，方便后面累加数据，可以不用修改就直接分析了。

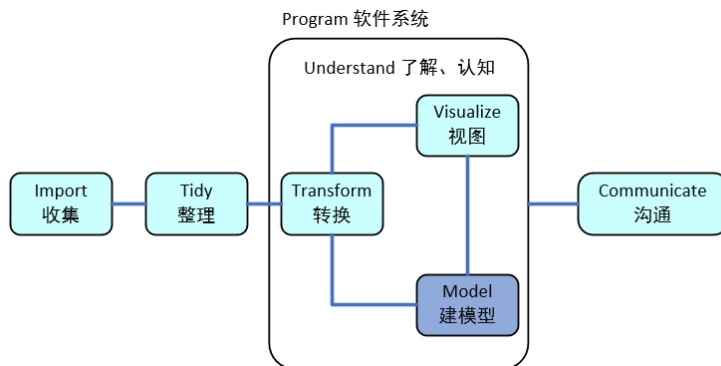
总结

从以上例子看到 2 类分析：

- 描述性 (Descriptive) 分析
 - 例如 100 家餐厅中，流失率与哪些因素相关，哪些无关
 - 可以用上一章的统计分析技巧
- 验证假设 (Inference) 分析
 - 基于问卷调查，发现有 31 条问题都是高流失率的餐厅比较高，识别出人员流失跟员工的心态相关——培训与辅导、员工间的关系、晋升机会、工作环境等是主要因素
- 也可以利用上一章的假设检验分析方法分析数据

之前绅士俱乐部里的根因分析只是依赖头脑风暴、鱼骨图找出根因，从快餐连锁案例看到如何利用数据分析，包括问卷调查，依据客观数据找出根因和纠正措施。

下图描述了数据分析的主要步骤与关系：



上面提到的描述性、验证性分析都归属于“了解”里，前要先收集并清洗数据（例如，某些数据缺失）。敏捷开发团队也可以分析每个迭代的数据，例如缺陷、返工工作量等，制定下一个迭代的纠正措施和量化目标，提升生产率和产品质量。但很多团队只是比较以往实践与计划的偏差，没有利用数据预测未来。如果有足够数据量（如 100 组左右），团队便可以开始利用历史数据，建模型做预测。下一章会用例子介绍如何利用各种机器学习方法，帮团队做预估。

本章最佳实践对应：

- CMMI
 - 管理性能与度量：MPM 3.1 - 3.6

- 原因分析与解决: CAR 4.1 、 4.2
- ACP
 - Problem Detection and Resolution (Domain VI)

参考 References

1. Weisbord, M.R. *Productive workplaces revisited : dignity, meaning, and community in the 21st century*. (Jossey- Bass , 2004)
2. Kuhn, Max and J. Silge *Tidy Modeling with R : A Framework for Modeling in the Tidyverse*. (O'Reilly , 2022)

Chapter 24

预测模型

“按你当初想使用项目历史数据做预测的目的，假如已经按做问卷调查思路，对应目标收集了足够数据量，便可以用数据挖掘分析数据，建立预测模型。”

“但是否要懂编码，我毕业后已经很少编程了。”

“理解，可以先简单用 XLS 配合 web GUI 程序开始，简单感受一下整个过程。如果觉得有用，后面可以利用程序脚本做分析，效率会高很多。不用过于担心，现在数据分析的编码易用性已经改善了很多：一般人也能用上，可选的模型也多了。我可以简单展示如何使用 XLS，配合 web GUI 程序开始。为了让你也可以容易重现，我用一套网上大家都可以下载的“公共”数据，展示过程，”

从数据开始

这套数据共有 81 个软件项目数据（下载网址，详见参考），里面的变量与定义如下：

变量	描述
id	项目 ID
Project	项目信息
TeamExp	以年为单位的团队经验
ManagerExp	以年为单位的经理经验
YearEnd	项目结束年份
Length	项目持续时间（以月为单位）
Effort	实际活动是用人时来衡量的
Transactions	事务是系统中基本逻辑事务的计数
Entities	实体是系统数据模型中实体的数量
PointsAdjust	调整后的功能点（项目规模）

目标：希望建立预测模型，基于项目的特性，预测项目的工作量。

步骤

0) 用 GUI 程序读取 81 行项目数据表, 程序展示读取数据表的其中 4 行项目数据:

600px| 无

1) 先展示数据, 查看数据是否齐全, 发现缺乏 4 个经验数据 (-1), 因为会影响分析, 而且缺的数量不多, 我们就用变量的中位数取代。

600px| 无

展示变量的基本统计: 最大、最小, 中位数等, 查看数据范围:

600px| 无 2) 并且看到其他的数据都正常, 包括工作量、功能点数等, 用柱状图看变量的分布, 发现以下变量都是偏左, 我们使用取 log 对数, 产生新的变量, 简单把它的数据拉到中间, 避免影响分析。(有更正式的统计分析方法, 比如 Box-cox 转换, 我们简单直接取 Log。)

600px| 无

3) 使用热力图和相关性 (correlation), 查看变量之间的相关系数, 避免出现 2 个变量强相关, 会影响数据稳定, 只能取其中之一。例如功能点和调整功能点强相关, 只取调整功能点。

600px| 无

4) 用散点图看 2 个变量之间是否有关系, 例如工作量和项目的时长强相关, 时长越长工作量越多, 功能点也类似, 是否跟不同语言有关系? 从散点图看到 1、2 语言和接近, 语言 3 的分布好像与 1、2 不同。

600px| 无

600px| 无

600px| 无

按变量与 Effort 的关系, 先挑选关系最高的变量建模。(注意: 虽然时长 (Length) 的关系最高, 但它与工作量 (Effort) 都是要等到项目结束才有数据, 所以时长只可以是 Y 因子, 不能用来预测。)

5) 把模型预测的 LogEffort 结果, 放进 XLS 表单, 计算各个 RMSE 和 R-square 值。

600px| 无

挑选最佳

要比较不同模型, 先要判断不同模型的质量, 我们希望预测项目工作量是连续数据, 不是分类数据, 可以用 RMSE (Root Mean Square Error), 把预估的结果和实际结果的差异, 平方的均值再开方, 数字越小越好, 也可以看 R-square 系数, 它是预估结果与实际结果的相关系数, 如果 R-square 是 1 代表 2 者完全一致, 0 代表 2 组数据没有任何关系。(注意主要看 RMSE, 因为它是直接比较预计与实际的差异, R-square 系数可能会被一些极端的点影响, 系数高不一定代表准确。)

因为我们希望预测工作量，所以需要先把数据分成训练数据和测验数据，从训练数据建立预测模型，预估测验数据每个项目的工作量与测验数据本身的数据量比较，判断模型的质量。

线性模型: 如果 X 与 Y 呈现直线关系 (看散点图), 可以用拟合线回归分析, 得出 $y = a + bx$ 的直线方程。如果有 3 个控制变量 X , 也可以用回归分析得出 $y = a + bx_1 + cx_2 + dx_3$ 的多元回归方程。ElasticNet 是一种对基本回归的优化, 神经网络、随机森林、Cubist 都归属于非线性模型, 不能用上面拟合线回归来分析。

我们按变量与工作量相关性, 挑选变量, 建立预测模型, 先选最相关的 2 个变量, 然后陆续加变量:

1. PtsAdjust + TeamExp
2. PtsAdjust + TeamExp + MgrExp
3. PtsAdjust + TeamExp + MgrExp + Language

从加 Language 开始, 再增加变量对模型预测准确性没有改善, 下面只针对前 2 类的结果做分析。

650px

- 上图左面是选用功能点与团队经验, 红色是第一组测试数据 (#1), 黑色是另一组 (#2), 只有随机森林模型 (RF) 比较接近, 其他的模型预测都有很大差异
- 右面加了经理经验为第三个变量, 但对预测没有改善, 虽然也是 RF 差异最少, 但还是比 2 个变量多

从以上结果看到不同的模型预测不同抽组的准确度有明显差异。例如在线性模型里, PCR 虽然准确率不如多元回归和 ElasticNet, 但对两组不同测试数据的准确率相差很小, 更稳定。

非线性模型, 如随机森林 (Random Forest) 偏差也小。模型对某组测试数据和训练数据能预测很准确, 但若换了另一组测试数据便不准确, 叫过拟合 (overfitting), 但因预测是未知, 模型能非常“准确”地预测某组测试数据反而会不利于做预测。例如如果变量键之间相关性高就会导致预测结果不稳定情况。针对线性回归, PCR 会先用主成分分析 (PCA) 把所有 X 变量转化为相互不影响的新变量, 防止相互影响, 然后才做回归分析。(从变量相关性矩阵看到, TeamExp 和 LogPtsAdjust 之间相关性是 0.27; TeamExp 和 LogEffort 之间相关性是 0.22; LogEffort 和 LogPtsAdjust 之间相关性是 0.69, 所以 TeamExp 和 LogEffort 间相关性不低。)

从以上 2 组数据分析结果看到不能单依赖一组测试与训练数据来评估模型预测准确性, 须要从多种抽样搭配的预测结果才可以估计这模型的预测准确性的范围。(有很多自动化方法, 例如 10-Fold, Cross Validation: 把所有数据分成 10 组, 不断地抽取其中一组作为测试数据, 其他为训练数据, 然后看预测结果的分布。)

R-square 的分析发现也类似（与 RMSE 不同，R-square 是越大越好）

650px

从上面的简单例子，看到如果数据质量不错，不需要要收集很多变量数据，只需要 2 个关键变量，便能有效预测。

虽然可以简单先用 excel 加图形网页比较各种机器学习模型的预估效果，也不需要学统计分析的语言，因为我们假定只需要用标准的参数，不需要纠结每个模型的参数要如何调整，实际上差异不大。但熟悉后，肯定不会一直用这种简单方式，来分析数据与建模，最终还是会上脚本和语言。

例如，数据准备：处理无效数据，求对数等建模前的准备等，虽然都可以使用 XLS 完成，很花时间并容易出错。随机抽样选测不同试数据搭配，交叉验证，如 10-Fold Cross Validation。

自动化

从上面的分析看到，要选出最佳的预测模型，并非一步到位，会受不同数据、不同模型、选择哪些变量影响。须要反复做多轮分析。既然要重复做，必然回想如何使用脚本自动化。不想单靠手工操作，专门针对统计分析的 R 语言具备这类功能。

用 R 语言自动完成不但会更准确、更快速。更重要是，所有实验的参数与步骤都有记录，并可以自动重现（不依赖手工记录），例如，想抽 20% 的数据作为测试数据（本来大概 10%），改一个参数，重跑便可。

R 语言是专门针对数据分析的开源语言，是由全世界的数据工程师一起贡献，并一直在发展。工程师可以针对某类分析需求，写一些分析模型包 (package)，加强语言的数据分析能力，虽然这能让 R 的能力不断加强，但各个包的格式、参数、配置函数格式都不一样，要学习就很难。

针对这需求，R 的 tidyverse 让分析工程师用统一格式调用各种不同的预测模型（不用记住每类模型的语句格式规范，无论你用选择什么模型都可以用同类的格式。）

但 R tidyverse 也是依赖少数几位工程师开发出来，资源有限，所以培训材料或者用户手册都有限。虽然我已经接触 R 语言 5 年了，但一直没空深入学习，对 R 的语法、结构一知半解，难以独立写出 R 脚本。以前我在这方面一直有比较大的困惑，因为不知道如何尽快学会语法，跑通脚本程序。例如按书本的程序，有些没说怎么去改动数据里面的某些非正常数据，虽然网上搜索能找出一些代码举例，但不是我需要的。

AI 不断成熟，开始可以降低学习难度，例如，我只需要对 AI 讲讲述以下需求：

R tidymodel parsnip random forest machine learning via ranger R code example

系统就会给出代码例子和原因解释，比以前在网上搜索轻松多了。虽然如此，但千万不要误以为 AI 就可以代替学习，还是要知道 ranger 是其中一种引擎，才能得出有用的代码范例。

总结

“听你讲的过程让我想到以前高中和大学时，做实验、收集数据、交报告，觉得以前学的东西最终有机会用起来了。”

“基于真实数据分析，找到背后的原理，这是科学精神。高中、大学短短几年，时间有限，更重要是毕业后能否持续学习，才能跟上科技的发展，不落后。你有兴趣吗？”

“兴趣是有的，我估计我们团队的同伴也会感兴趣，但问题是数据哪里来？”

“记得我们前面讲敏捷迭代回顾时，可以收集缺陷排除率和返工工作量吗？然后我们当时介绍用蒙地卡罗模型，模拟各个过程的缺陷范围。蒙地卡罗模型模拟过程，假定变量都是正态分布，也假定缺陷排除率是比较固定，但其实这些假定都不一定正确。但是我们确实相信这些结果加上一些关键的可控因素，例如团队的经验或者评审的力度、准备等等，但会不会有影响，都可以利用数据来分析。如果你们两周一个迭代，总共有五个团队，半年就有 60 套数据。然后针对你们预期的目标用这些方式去分析，便可以开始建立你们自己的预测模型了。而且加上人工智能 AI 的辅助，就不用花大量时间去处理那些语言的格式，AI 依据要求可以快速给你出一段可以使用的代码脚本。如果预测变量是连续数据（如工作量、缺陷密度等），X 变量也是连续数据，可以直接使用这刚才演示的 c 程序（如果有分类数据，可以使用哑变量，如前面语言分 123 类）来分析自己团队的数据。”

“但我们项目都很忙，哪里抽得出时间做这些研究分析。”

我沉默了一会，然后面带微笑与他继续说：“千万不要误以为自己每天工作很忙，都已经饱满。还记得我们说过软件开发中有很多浪费，可以优化节省；例如团队最大的工作量（成本），是花在不断返工、修改后面才发现的 bug。但如果团队抽时间做好每个迭代回顾、分析数据，在下一个迭代完善，就可以节省时间。如果你们领导了解你们花时间做迭代回顾能最终提高团队效率，节省工作量（成本），他肯定愿意让你们做。但有些领导忙于业务与市场工作，误以为团队的效率饱和，没有改善空间，所以须要先简单估计做迭代回顾能节省多少。因为分析数据后有改善，你们就可以减少很多没有价值的返工工作。高层也要了解这是个先试点再推广的漫长过程（要改变人的习惯很难），开始的时候需要交学费，但大家学会后就有回报，对公司有价值。也可以帮团队建立继续学习的文化，不断探索新的东西，而不是仅仅是当‘码农’，按规范编写代码。前面提过美国 David PAMAS 在 2009 年提出：‘一位糟糕的程序员一年内便很容易产生两个新岗位（的工作量）。如果我们有更多优秀的程序员，我们需要的就会减少，而不是增多。’科技进步，学习新软件技术的门槛越来越低，有能力和动力的工程师，不断更新、学习，才能跟上科技发展，对自己，对公司都能快速增值，不会担心被淘汰。”

参考 References

1. 数据 (dataset) : <https://www.kaggle.com/datasets/toniesteves/desharnais-dataset>
2. Philippe J.S. De Brouwer *The Big R-Book* (Wiley , 2021)
3. Kuhn, Max and J. Silge *Tidy Modeling with R : A Framework for Modeling in the Tidyverse*. (O'Reilly , 2022)

以前我们用统计分析、假设检验、基本统计那些技巧，但现在科技发达，学生其实不太需要懂背后的统计分析原理，但分析的原理还是要清楚的

Part VIII

总结

前面的迭代根因分析只归属于问题解决，不算是创新，这部分会先回顾和分析个人和企业的创新案例，探索创新的要素。

个人的创新能力是基础，26 章北京手记让大家认识到为什么创新能力会因人而异。

最后 27 章会利用实例回顾人类从 20 世纪至今创新的根与翼，并探索软件开发团队能如何继承前人的根与翼持续改善。

Chapter 25

创新：从个人到公司

丰田不断改善，最终实现了许多人最初认为不可思议的目标，如“零”缺陷和“零”库存。这使得丰田成为现代汽车生产的标杆。这些改善都只属于解决问题 (Problem Solving)，并非创新（即使团队迭代回顾时针对缺陷排除率分析根因也不能算是创新）。
下面将先回顾个人创新的要素，然后探索团队创新的方式和成功的关键要素。

400 年前的重大数学发明

如果你要求 345 乘 456 等于多少，你可以立即使用电子计算器甚至手机，一秒钟就可以得出准确答案。但在 70 年代之前（还没有电子计算器），是怎么计算呢？如果用手工计算，很费时间。60 年代，读理科的学生都使用计算尺 (Slide Ruler)，通过移动计算尺上的滑动部分来求得答案。

400px

对数 (Log) 是计算尺背后的原理。（如果不用计算尺，也可以查对数表得出答案。）

对数是 16 世纪纳皮尔先生 (John Napier 1550 – 1617) 用了超过 20 年时间“发明”的。

对数的简单例子：我们能在草稿纸上计算出“10,000×100”的结果，同时，我们也可以通过将“10,000”和“100”中的计算“10,000×100”的话，使用乘法会更快，但如果数字位数较大，需要手动计算时，使用加法运算会更加简单。纳“这不就是指数运算的法则吗？”按照指数运算的法则“ $a^m \times a^n = a^{m+n}$ ”来思考的话， $10,000 \times 100 = 10^4 \times 10^2 =$

在 16 世纪的欧洲，数学研究还处于早期阶段，纳皮尔先生发现对数能够将原本复杂的乘法或除法转化为加法或减法进行计算，这项工作非常不容易。为了实现这个方法，他凭借个人努力，花了 20 年的时间手工计算出相关的对数表。现在我们还可以找到他当时算出的对数表，用现在的计算机验证，8 位数运算的头 7 位还是准确的。

纳皮尔先生的对数表：例如第一行列出 $18^\circ 30'$ 的正弦值为 0.3173047，对应
对数值为 0.111478920。

550px| 无

当时为什么要计算这些复杂的正弦相乘呢？因为纳皮尔先生还研究球面三角学，可以利用他的纳皮尔公式探究角度与球面弧（距离）之间的关系。16 世纪的欧洲国家主要通过航海，发现新大陆进行发展。但海员在大洋中如何确定自己的位置呢？船员在大海中只能依赖天上的星星，利用六分仪估算出船只的经纬度。使用纳皮尔公式，他们就能够估算出地球球面点 A 和点 B 之间的弧长。纳皮尔先生通过研究球面三角学，发现了角度与球面弧（距离）之间的关系公式，如下所示：

$$\cos \theta = \cos \varphi_A \cos \varphi_B \cos(\lambda_A - \lambda_B) + \sin \varphi_A \sin \varphi_B$$

海员只要知道两点（A 和 B）的经纬度（角度）便能使用以上纳皮尔公式，估算 A 和 B 点之间地球球面弧长。

例子——北京和巴黎距离：

A: 北京中心位于北纬 $39^\circ 54' 20''$ ，东经 $116^\circ 25' 29''$ 。转换为纬度 39.905556，经度 116.424722。

B: 巴黎转换后纬度 48.8583，经度 2.29451。

$$\begin{aligned} \cos \theta &= \cos \varphi_A \cos \varphi_B \cos(\lambda_A - \lambda_B) + \sin \varphi_A \sin \varphi_B \\ &= \cos 39.905556^\circ \times \cos 48.8583^\circ \times \cos(116.424722^\circ - 2.29451^\circ) \\ &\quad + \sin 39.905556^\circ \times \sin 48.8583^\circ \end{aligned}$$

$$= 0.767103 \times 0.657923 \times$$

$$(-0.40881) + 0.64152 \times 0.758084$$

$$= 0.28$$

$$\theta = 1.287$$

$$(\text{rad}) \text{ AB 距离} = \text{地球的半径 } 6,378 \text{ KM} \times 1.287 = 8,208 \text{ KM} \quad (\text{实际距离是 } 8,217 \text{ KM})$$

当时没有电子计算机，他发明了对数来解决这类计算难题（对数不仅仅用于航海，还有助于天文学的计算）。对数帮助我们解决复杂的计算问题长达 350 年，直到电子计算器出现才退出舞台。

创新不仅需要独特的思维和洞察力，更重要的是能否有持久的毅力实现创新。什么驱动他花 20 年时间独创对数呢？

Robert Fritz 回忆：“60 年代，当我在 Boston Conservatory 学音乐作曲时，开始思考学的不应仅仅是对位、和音，更他毕业后继续做音乐创作，有一次参加创新专业人士聚会，这些专业人士中有作家、画家、音乐家和建筑师等，他发于是他举办培训课以帮助不同背景的人提升创新能力，让各行各业的人都可以学习什么是创作/创新。几年后，他把

原创精神 Spirit of Creating

真想不通是什么原因驱动我能够在一个半小时内，在山西省图书馆一口气读完了《且听风吟》(日本小说家村上春树先生 1973 年的处女作品，共 40 章 144 页)。(快速读完的原因之一是：我没有图书卡，而图书馆会在 5 点钟准时关门。)作者描述了自己亲身经历的场景——在酒吧与老朋友喝酒、听电台播放流行歌曲、购买唱片以及邀请女朋友吃牛排等情景。

虽然没有惊天动地的结局，也不是一气呵成的大作风格，相反，里面的场景很散乱。虽然出现了少女裸体、女朋友自杀和作家跳楼等情景，这些都不足以打动我。但总觉得这本书与众不同，很有特色，例如作者本身，以及书中的每一位角色都挺有性格，展现了独立的思维，都是活生生的“真”人。我不仅看到了他们的表面，也感受到了他们内心深处的世界。

创作并非被他人驱使，而是源自内心对创作的渴望，渴望创作出世界上最优秀的作品。只有认为自己的创作具有重要价值，才会尽全力努力创作。最终，希望自己的作品拥有独特的生命力，并受到他人的赞赏。

除了《且听风吟》，我还读过《挪威的森林》，也是采用第一人称叙述的手法，以作者渡边君的视角逐步展开各角色：例如他好朋友本月的女友直子，大学同学绿子，在疗养院照顾直子的玲子等。直子的优美、优雅、娴静，绿子的活泼烂漫，都是通过渡边的亲身经历逐步细化而来，就像使用显微镜观察一个细胞：起初由于焦距还没有对好，一片模糊，但随着焦距的调整，可以一步步看清细胞的每一个细节。每个故事的发展也时不时地出人意料，就像一部优秀的话剧，观众永远无法预料下一个桥段……这种情节的安排吸引着读者继续追读，让他们不愿停下来。

《海边的卡夫卡》于 2002 年出版。在这本书里，村上先生设立了卡夫卡和中田两条线，奇数章以卡夫卡的故事为主线，偶数章则以中田的故事为主线，两条主线齐头并进。起初两条线没有直接的交叉点，但后来它们产生了交集。像我这样的非文科读者可能会觉得两条独立的情节主线会很奇怪，但当它们开始产生交集时，便会感到耳目一新——从没有过类似的小说。

文艺创作，无论文学或音乐，不同于数学、科学，给作者很多原创的空间，贝多芬的创新推动了西方音乐从古典时期向浪漫时期的过渡。

贝多芬 (1770 – 1827)

我们都听过他的第五交响曲《命运》。贝多芬的早期交响曲风格与海顿和莫扎特相似，但到了 1805 年完成的第三交响曲《英雄》，他完全打破了以前交响曲的传统。

之前古典时期的交响曲。都按不同的曲式，如小步舞曲式 (Minuetto)，写作。首尾乐章按奏鸣曲式 (Sonata form)——先展示主题，然后做各种变种奏，最后主题重现。大多是 3 到 4 乐章，绝大部分乐章都不超过 10 分钟。

英雄的首乐章便已经超过 16 分钟，整个交响曲共 4 乐章，时长超过 45 分钟。除了在形式、时长有突破外，英雄交响曲还把前后四个乐章串联起来，“讲述”某英雄的故事，感情丰富，大大提高了音乐的深度，成为了以后浪漫时期交响曲的新标准。

1824 年完成《第九交响曲》，他在最后的第四乐章中引入了四声部合唱，这在传统的交响曲中也是前所未有的。

== ==

2025 年 4 月初，我在香港大学的小音乐厅，近距离听 Quatuor Ebene 演绎贝多芬降 B 大调弦乐四重奏 (Op 130) 和赋格曲 (Große Fuge Op. 133)，听完后立马站起来鼓掌，真心感谢他们的高水平演出。这演奏也解答了我的疑问：“大家都说贝多芬的最后 3 首弦乐四重奏很特出，但到底有什么创新之处？”前面五个乐章都各有特色，虽然已经远远超越了之前莫扎特、海顿等的作品的风格与形式，但依然有古典风格的韵味，安祥和优美的如歌慢板第五乐章后进入最后赋格曲第六乐章，贝多芬充分利用弦乐特性，能很好奏出各种音量大小，这乐章像火山爆发一样，带给听众前所未有的新感受。要做到这效果也依赖 4 位乐手的完美搭档，不仅在这长达 16 分钟的乐章演奏中把听众引入一浪接一浪的高潮起伏，并能带出赋格曲各声部的主题交接，让我们现场听众无法想象它是 200 年前（1825 年）的作品。

赋格曲在巴洛克时期很流行，巴赫 J. S. Bach 把赋格曲带到巅峰。赋格曲虽然很有特色，但因需要每声部按序加入，作曲很困难，所以虽然贝多芬在之前的最后一首大提琴奏鸣曲和第 31 首钢琴奏鸣曲都有使用赋格曲式，但都不长。（大提琴奏鸣曲的最后赋格曲乐章较长，才 4 分钟。而且因钢琴依赖敲打弦线，与纯弦乐乐曲比较，音量变化和感情的表达力度都有局限。）在这弦乐四重奏的最后乐章，贝多芬虽然采用赋格曲式，但没有完全按照巴赫的严格规定来写，加入了很多创新，所以能让听众觉得很新鲜。

原来背后还有个故事，在维也纳首演时，听众虽然喜欢前 5 个乐章，但完全不能接受这古怪的最后乐章，贝多芬碍于出版社压力，被迫删除改写最后第 6 乐章，所以我们很少机会听到这个采用赋格曲为最后第六乐章的原来版本。

贝多芬对音乐创作的远景目标——希望创造前所未有的作品（他觉得当代的作品，甚至包括海顿和莫扎特的作品，离这远景还差很远）。由这崇高的目标驱动，加上他自身的音乐演奏和作曲能力，贝多芬不断突破，甚至连耳聋也阻挡不了他的创作力（他从 25 岁开始听力减退，到 45 岁已经完全失聪），将音乐创作推向了新的高度，对后来的作曲家产生了深远的影响。

贝多芬追求完美，他有极大的魄力，通过持续不断地创新和完善，才取得了超越常人的成就。

Fritz 发现大师们都拥有以下共同的特点：把握自己的命运，和主动追求个人目标。许多人很有才华，但为什么他们中的大多数却不能成为大师？他用下图解释。除了要对未来有远景目标外，了解现状同样重要。如果不觉得有差距（张力），就没有改进的驱动力。有些人有理想，但缺乏计划与行动，便只是梦想。

600px

这个创新驱动模型不仅仅适用于音乐创作，也适用于数学。例如纳皮尔先生为了解决正弦相乘的困难，发现可以转换成对数进行计算，把乘法简化为加法 (VISION)，为了实用，又花毕生精力计算出了对数表，也是从张力转化为行动 (ACTION) 并创新的例子。

从贝多芬和村上春树的故事中，我们了解到创新首先是由不满足于现状驱动、并由此长期不断完善的结果。如村上先生说：并非受人之托才写小说，而是因为“我想写小说”这种强烈愿望，深刻感受到这种内在动力，才不辞劳苦地努力写小说。

从个人到公司

您身边的人群中可能有超过 50% 的人都在使用这家公司的产品，涵盖各个年龄段。该公司的产品功能广泛，包括音

苹果公司 (Apple)

我和全世界千千万万人一样每天都在使用苹果产品：iPhone、iPAD 平板电脑和 iTune 网上搜索音乐歌曲，要了解苹果的成就，就不能不谈乔布斯 (Steve Jobs) 了。他的一生充满传奇，很多故事可通过阅读《Steve Jobs》一书进行了解。让我们先回顾苹果如何成为大众音乐市场巨人的故事。

= = =

音乐播放器

自 80 年代末 MP3 格式问世以来，将歌曲数字化开始流行，市场上也出现了一些可以随身携带听音乐的小型设备。然而，由于技术限制，这些设备都存在不少问题。他们比较了当时市场上常见的播放器，发现或多或少存在各种问题，例如容量有限、难以按照个人喜好进行歌曲的排序等，如果想要从 CD 上下载歌曲到电脑，然后按照自己的喜好顺序在播放器上播放，过程相当复杂。乔布斯本人很喜欢音乐，他看准了这个市场，并希望将苹果定位为个人电子设备（包括音乐播放器）的综合中心 (Hub)。首先要解决的是音乐播放器的便携性和存储容量这难题。当时，小型显示器、锂电池和小型硬盘都尚未普及。几个月后，他们找到了合适的解决方案：东芝刚刚研发出一款直径只有一英寸的超小型 5G 硬盘，形状像个大银币。但如果将这些功能都集成到播放器中去实现，因为播放器的屏幕尺寸太小了，实现起来非常困难。他一直非常关注产品的易用性，当乔布斯看完内部软件研发工程师首次展示正在开发中的刻录软件后，他站到白板前画了一个框，说：“你们设计的软件就应该只需把歌曲拉到那个框里面，然后点击一个按钮就可以刻录。”当时软件工程师被吓呆了，觉得很难做到。这反映工程师往往不会从用户的角度来看问题，乔布斯后来要求播放器软件都必

须给不懂技术的人使用，让她在 3 个步骤内，不需要查看说明书就能完成任务，这成为他的测试标准，如果做不到，他就不会接受这软件。播放器的设计也有创新，比如通过转动面板上的轮子来选择歌曲，用户只需转动轮子就能方便地跳到要听的歌曲，这样的设计极为便捷。iPod 的产品设计也体现了苹果公司一贯的设计理念——简洁。

产品采用白色设计，包括耳机在内都是白色（当时，大部分耳机还是黑色的），甚至连充电器也是白色的。设计师 Jony Ive 解释：“白色能够赋予产品高贵感，使其与其他廉价的播放器有明显区别，用户会觉得这样的设计有艺术感。”

350px

乔布斯也非常注重产品包装，iPod 的包装让你觉得：“在拆包装时，觉得有一件很贵重的东西在里面”。

400px| 无

虽然定价不低：399 美元，很多人觉得定价过高，但是 iPod 在 2002 年正式推出市场后一直热卖。

= = = =

从 2001 年到 2014 年苹果大概卖出了 4 亿件 iPod，2015 年后还继续卖出 50,000,000 件。iPod 播放器也成为了苹果的主要收入来源。2006 年，也就是苹果推出 iPhone 的前一年，iPod 的销售收入占了公司总收入的 40%。下图展示了从 2002 到 2013 年，iPod 的销售数量变化和收入占比变化：

700px

700px

iTunes 网上商店

苹果的 iPod 加上 Mac 电脑的 iTunes 应用软件很成功，但这只解决了从网上下载音乐并在便携播放器听歌的技术问题，还有更严重的问题需要解决——盗版，2000 年开始，越来越多网站提供免费音乐下载，这不单影响了唱片销售（2002 年销售额下降了 9%），也侵犯了音乐创作者的知识产权。各大唱片公司都急于解决这一问题，但大多数公司缺乏所需的技术能力。乔布斯也十分重视这些创作者的利益，他预见到如果没有知识产权的保护，就不会有人再有动力录制高质量的音乐作品（就像没有知识产权保护，就没有软件和电脑公司愿意投资进行创新）。

因此，他基于 iPod 和 iTunes 音乐播放能与电脑结合的特点，向各大唱片公司展示了在线平台如何运作、如何防止盗版。由于唱片公司缺乏相关的技术，很快就有 4 家唱片公司表示有兴趣，愿意合作：利用 iPod、iTunes 等技术，筹建统一在线平台服务，使消费者可以在网上购买并下载音乐。

从 2002 年开始，乔布斯一直忙于周旋各类问题：除了技术方面格式问题要解决（例如各有自己的格式，需要统一），也有些唱片公司想保护自身利益。

2003 年 4 月，iTunes store 正式推出市场，用户可以在网上用关键字搜索任何一家唱片公司的唱片或歌曲，也可以搜索相关的唱片。乔布斯的业务模式很简单，他从以前的经验知道首先要通过低价吸引大量用户使用，如果收费太高，只是小部分人使用，就没有动力吸引新唱片公司加入，恶性循环。他的策略很成功。2003 年苹果刚开始发布 iTunes 网上商店时有 200,000 歌曲可以下载购买。他本

来预计可能需要 6 个月的时间才能售出 100 万首歌曲。但实际上推出 6 天就已经卖了 100 万，到 2003 年 12 月，已经累计卖出 2500 万首歌曲。

2004 年 7 月份共卖出一亿首歌曲（2003 年底的 4 倍）；2005 年 11 月（两年半后）苹果成为全美国十大音乐零售商之一，其他都是传统零售商，例如 Walmart。随着网上买歌商业模式越来越普遍，传统零售商越来越困难，到了 2010 年 2 月，苹果成为美国最大的音乐零售商。

因为网络技术发达，现在不再是当年的网上传下载，变成用 streaming，客户还是只要每个月交几十块钱就可以随时随意在网上搜索并听到自己心爱的歌曲，这商业模式也保护了艺术家、唱片公司的利益。

公司如何创新

从以上故事看到与 400 年前相比，创新的速度发生了巨大的变化。在过去，像纳皮尔那样仅凭一个人独立完成创新可能需要 20 年的时间。到了 2000 年，一切都变得非常迅速，评估团队只用 3 个月就推出了 iPod 播放器，并在两年内创建了 iTunes 网上音乐平台，帮助苹果公司为个人客户提供数字中心（Digital Hub）平台服务。客户可以使用电脑选择喜爱的歌曲在 iPod 播放，也可以下载最新的歌曲，非常方便。这些就是我们 2000 年代看到的创新，这些创新不可能靠一个人独自完成，而是要团队合作。

公司创新与几百年前那些大师的创新最大的区别：需要整个公司所有人的协作，并确保他们的目标一致。这意味着整个组织的各个部门都必须协调合作，同时又要细分目标，以便不同的人负责执行不同的任务。

纳皮尔先生花了 20 年时间写了 3 本数学书，总页数不到 200 页（其中一本还是在他去世后出版的），贝多芬的产量也不算高，一年只出版几首作品。这些大师都是靠个人的努力完成整个创作过程。但是现代市场变化越来越快，要成功，公司必须快人一步，但因依赖团队合作（各有专长），也需要协调大家的工作，确保各个层次的目标与公司总目标一致。

=== 企业管理提升计划 ===

1. 识别公司的总目标。
例如在下一年，要一年内生产 7 个新的产品系统。
2. 描述公司的现状。
例如现状是：平均一年只能产出 3 个新产品，有 2 个开发团队。但开发人员的能力有限，质量也不好，导致用了不少精力，去维护已经完成的产品，没有时间来开发新的，招新人也不容易，比较难找得合适的，也没有时间培养新人，扩大工作。
3. 形成具体行动计划。
例如简洁、精简的开发流程，增加 1 个开发团队，加强代码评审和编码能力，来减少缺陷导致的返工，要跟市场部密切合作，确保新产品满足客户的要求，给客户带来价值。
但真正公司的从上而下总计划会更具体。
4. 当我们定好明确的最终结果，也了解真正的现状，有一些行动计划，下一步是具体到每活动的完成日期和负责人。

600px

同意可以用以上模式帮助公司创新吗？

我们看看乔布斯另外两个创新故事。

万事起头难

1985 年 9 月，乔布斯被迫离开苹果公司后，自己投资 700 万美元，创立了一家专门针对高校工作站 (workstation) 市场的新公司，名为 NeXT。乔布斯开始对公司的期望特别高，当时他看到这市场主要被太阳 (SUN) 和 (DEC) 公司占有，但这些工作站产品都是以输入命令为主，与 Macintosh 用鼠标操作不同，所以易用性不高，所以他希望创造前所未有的新平台，让那些高校的学者与研究人员可以简单地使用电脑模拟各种场景来做研究。

公司开创 90 天后就在某度假村开首次头脑风暴会议，讨论如何可以在两年后推出产品，以赶上学校放暑假购买电脑的时机。三个月后再次讨论时，大家都意识到这个目标太高，难以实现。但乔布斯没有放弃，他一直用自己的魅力说我们可以按大家的专长达到这个目标。

1986 年过去，1987 年又过去了，但新产品研发仍然没有显著进展。直到 1988 年，公司才进行了第一次长达 3 个小时的产品演示，但产品仍然处于初级阶段，没有彩色显示，并且速度较慢。尽管由曾为苹果推销 Mac 的全国电脑零售商做全国营销，但一年仅售出 360 台，与预期相差甚远。（在 1984 年，同一家全国零售商却为苹果售出了 40 万台 Mac。）

1990 年，到公司的第二次产品发布时，新产品开始用了彩色显示，速度也得到了提升。然而，尽管有所改进，该产品仍然被认为只是初级产品，尚未完全成熟。

由于销售情况不理想，乔布斯再次投入了 500 万美元的资金，但很快就用光了。幸运的是，有一位外部投资者——德州企业家 Ross Perot 先生，他对乔布斯非常信任（他觉得以前没有来得及投资苹果，一直耿耿于怀），因此投资了 2000 万美元进入 NeXT。

1993 年，公司开始裁员。600 人的公司，起初只裁减了几十人，随后又裁掉了 200 多人，接近公司总人数的一半，随后又有 200 多人被裁，最终公司将整个硬件部门出售给了日本佳能公司，只保留了操作系统部分 NextStep。

1996 年，因为苹果公司急需新操作系统支持公司的新产品开发，最终挑选了 NextStep，以 4300 万美元收购 NeXT 公司，乔布斯也得以重返苹果公司（最初以顾问身份）。又经过了 15 年的努力，乔布斯带领苹果公司再度走向辉煌。

从濒临破产变为回报最好的投资

乔布斯在 1986 年用一千万美元投资 Pixar。当时，Pixar 只是 LucasFilm 旗下的一个由 30 多人组成的电脑部，由《星球大战》导演 George Lucas 创立，主要用于电影动漫效果的制作。乔布斯对美术领域一直很感兴趣，他通过 Alan Kay 的介绍去参观了 Pixar，被公司的产品深深吸引，认为其技术领先并远超同行。乔布斯最初只是一名投资者，并没有过多干预公司的软硬件产品开发，但乔布斯雄心勃勃，觉得公司开发的产品有特长，也希望产品能像苹果的 Mac 一样，卖给动漫专业人员做 3D 设计。尽管产品的功能强大，但使用起来仍需要复杂的命令操作，不太适合一般的动漫从业者（Adobe 推出的软件，虽然功能不如 Pixar 强大，但更易于上手）。乔布斯投入了更多研发资源，改善产品的易用性。为了推广 Pixar 产品，他开设了很多零售商店。

虽然乔布斯对 Pixar 公司的投资不断增加，然而产品的销售情况一直未见起色，

公司陷入了资金短缺的境地。到了 1988 年，他被迫采取了紧缩开支和裁员措施。直到 1991 年，与迪士尼合作制作 Toy Story《玩具总动员》，Pixar 才得以起死回生。乔布斯在这之前已经投入了高达 5 千万美元的资金，相当于他从苹果公司股票出售所得收入的一半。因为与迪士尼的成功合作，Pixar 公司得以在 1995 年（与 Toy Story 首次公演同年）成功上市，获得了 15 亿美元投资。随后，Pixar 与迪士尼继续合作，推出了 5 部动漫电影，都非常卖座。2006 年，迪士尼为了防止 Pixar 与其他公司合作，以 74 亿美元的价格收购了 Pixar，也使乔布斯成为迪士尼的最大独立股东。

后来的事实证明，乔布斯认为一般动漫从业者会喜欢使用 Pixar 的 3D 建模软件是错误的，这只是他的一厢情愿。但他的另一个希望：如何结合电脑数字技术和艺术做创新，确保梦想成真，Pixar 与迪士尼合作制作新一代动漫片，从 1995 年的 Toy Story 开始，把动画片技术提升一个高度。（关于 Pixar Toy Story 制作故事，请看附件）

从以上两个案例可以看出，任何企业的创新往往会面临难以预料的挑战。因为市场的不确定性，超过 90% 的创新产品可能最终以失败告终，因此前面的企业策划模式不一定能成功。也印证了马云先生的名言：“今天很残酷，明天更残酷，后天会很美好，但绝大多数人都死在明天晚上”。

成功要素

回顾 iTunes 电子商店和之前 iPod 成功的故事，以及乔布斯与 NeXT 和 Pixar 的故事，可以总结出以下成功要素项：

灵活、敏捷

从物种进化历史可以看到物种可持续的原则：

- 最强壮/巨大的物种也会灭绝（如猛犸象）
- 只有那些能够快速适应环境变化的物种才能长期传承下去

从 60 年代开始，商业环境快速变化，也只有能快速适应的公司，不断求变才能持久，成为百年老店。

从乔布斯的 NeXT 和 Pixar 故事可以看出，创建新产品或服务涉及诸多不确定因素，失败的可能性很大。因此必须随时调整策略以适应变化。如果发现最初的想法或计划没有产生预期的效果，就应立即做出改变。

这个原则适用于许多科技公司，比如谷歌，它也有很多失败的项目。谷歌的宗旨是“Fail Fast”，意思是如果一个项目在半年到九个月内未能达到一定的客户量，就会立即终止该项目。如果等年底才全面评估某项产品是否持续，可能已经为时已晚。

专注 (Focus)

资源有限，公司应该专注于自身能够做好的事情，而将那些可以外包的任务尽量外包。举例来说，对于生产方面，苹果公司专注于发挥自身的专长——产品设计。（请不要误解苹果公司对生产的态度，它的产品生产都是按照 JIT 原则组织的，只是不直接雇佣生产工人。）

乔布斯回到苹果后，吸取了过去十年在 NeXT 和 Pixar 的经验教训。当时苹果公司内部存在着一百多个研发项目，乔布斯深知资源的有限性，因此开始对这些项目进行整合。他只保留了在市场上表现最优秀的项目，解散了那些不必要的研发团队，只保留与公司发展愿景相关的研发项目。

乔布斯 2010 年被访问时说：“我们选技术，只挑选有发展潜力的技术，如果什么都支持、都研究就会耗费宝贵资源。”

A 级团队成员

Pixar 公司虽然小，但每位都是行业精英。

我在美国的堂弟 Sywa Sung 从小就对动画人物情有独钟，家中也都是星际旅行、星球大战等的模型。他和其他 3 位设计师是公司外聘参加《超人总动员》制作设计的原始核心团队。其他 3 位都是行业中的精英。他说：“Pixar 公司在动画片行业非常出名。核心制作团队，包括艺术总监和生产设计师等都是当时行业中的精英。”《超人总动员》非常卖座，美国本土票房收入已经超过 2 亿美元，也得了奥斯卡奖。Pixar 公司这些动画片，能有这么好的票房并得奖，不是没有原因的，因为公司有名气，可以吸引到当时

因此，当 A 级成员遇上能力相当的团队伙伴时，能够激发更大的创造力，产生 1+1>2 的效果。

苹果公司于 1984 年推出的 Macintosh，产品在技术能力和设计方面（如视窗、鼠标等），远远领先于之前 IBM 推出的 IBM PC。该产品的开发团队成员也都能力超群：其中 Burrell Smith 负责硬件线路板设计，Andy Hertzfeld 等人负责 Mac GUI 操作系统的设计，Bill Atkinson 负责 Graphics 应用软件 QuickDraw 和 HyperCard 的设计，Joanna Hoffman 负责市场推广，Bruce Horn 负责 Resource Manager 应用软件的设计等。然而产品推出后，

乔布斯从 Pixar 的经验，深明高端球员 (A Players) 只愿意与高端球员搭档的道理。但为了吸引和留住这些精英，公司必须给有吸引力的报酬。

例如，乔回苹果不久就特别要求董事会修订员工股权安排（因当时苹果股价已经很低，之前的员工股权安排已经没有意义。）

极高目标，要做大做强

在杭州某商务酒店看到十几条浙商名言，写在墙上，其中一条如下：
600px

乔布斯创立 NeXT 时是世界第一的工作站供应商，所以生产线是按每月能生产一千台设计！他重返苹果后，一直秉持着以创造世界第一的产品为目标，这在后来被证明是成功的，例如 iPod/iTunes(2001)、iPhone(2007)、iPad(2010) 等。这种思路使得苹果公司成为了美国市值最大的科技公司。

有非常高的要求，用心做

每位团队成员都应该追求卓越，不仅仅是完成任务，而是要创造出优秀的、世界一流的产品。只有这种抱负才能激发每个人付出最大的努力，把事情做好。乔布斯本人就是一个典型的例子，他判断有价值事就会全力以赴去做到最好。举例来说，在推出 iTunes 电子商店之前，他就不断地邀请各种音乐家、歌手和音乐制作人参与其中。他每次都会想尽办法去介绍他的好东西，有一次小号手 W. Marsalis 到家拜访，乔布斯就问他：“你喜欢谁的作品？”

Marsalis 说：“贝多芬。”

他就一起展示如何在商店里面可以容易搜索到贝多芬的作品，Marsalis 后面回顾说：“其实我一直都对电脑不太感兴趣，但乔布斯他一直全心全意，全程用了两个小时很细心地给我介绍。过了一会，我的注意力不是放在他的产品或者电脑上，而是他本人，我深深地被他的激情和专注打动了。”

例如苹果的首席设计师 Jony Ive 用简洁设计思路，让消费者觉得苹果产品有艺术感，非一般电脑：

300px

Ive 的父亲是位优秀工匠，从父亲那他学到要做一流工匠必须对质量有极高要求。

领导决策、快速行动

索尼一直以其在电子产品领域的创新而闻名，例如 Walkman，同时也拥有唱片公司；创建网上音乐平台的主要条件它都具备了，但为何索尼做不到，苹果却成功了呢？其中最大的原因在于索尼是一家庞大的企业，由各个事业部门组成，每个事业部门都要追求自身的盈利。正是这个原因，当需要各团队之间进行紧密协作时，许多精力就被耗费在部门之间的协调上，最终以失败告终。相反苹果公司不是这样的，它只关注整个公司的总体利益，而且乔布斯对产品有着非常高的要求。

例如在设计 iPod 时，工程部和软件开发部门完成了 30 个原型，乔布斯与大家讨论并同意后便立即确定设计方案，随后立即着手实施。（其他传统大公司，如飞利浦、西门子，绝对不会仅在一个会议里做这么重大的设计决策。）

总结

苹果的案例可以被视为将敏捷开发升级至企业级的最佳实践案例，从前面的故事中可以看出，整个公司都在按照精益的思路不断改善。

团队合作至关重要，而团队成员的能力要求也很高，这样才能在最短的时间内做到最好。强调岗位之间的相互合作，不是将工作分得很细，比如你负责测试，我负责开发，各自只需要做好自己的工作，而是要求团队成员相互协作，最终确保整个项目或产品的成功。不应计较是否在自己的工作范围之内，应该确保成员之间相互有效沟通，即使在需求或开发方面是最优秀的，但并不代表产品能做到最优。有些开发团队会抱怨说：“其实我们的开发工作做得挺不错的，但是由于需求混乱，我们无法做到最好。”

但是因为公司架构原因，部门之间各有自己的指标，导致各功能之间不能相互合作做好。

从乔布斯的故事中了解到，高层对产品质量有极高的要求这一点至关重要，敏捷开发也同样。如果敏捷团队不注重产品质量，只关注按时交付，最后只能生产出一些不达标的次等产品。但反过来，当团队具备足够的能力，并且大家都有抱负、有追求，敏捷开发可以帮助团队快速响应不断变化的需求，就像苹果和乔布斯一样，成为世界第一。

1983 ~ 84 年，乔布斯（28 岁）在苹果公司带领 Mackintosh 开发团队时，就不会向预计交付期限低头。如果他觉得产品未达到质量要求，他宁愿延后。他甚至鼓励团员要精益求精（千万不要以为你在他管理的团队可以偷懒）。

如果没有合适的人才和领导，没有苹果公司这样的企业文化，就永远无法打造出世界一流的产品。这个道理无论在国内还是国外都同样适用。

= = =

附件

Pixar 制作第一部电脑动漫：玩具总动员 (Toy Story)

John Lasseter 出生于西岸 Hollywood，从小就非常喜欢卡通片，从加州美术学院（迪士尼出资创建）毕业后就进了迪士尼制作动画。和其他刚刚毕业的动画师一样，他渴望创造像当代《星球大战》那样的高质量作品，然而，他们的部门主管都非常保守，按照传统方式管理，经常发生冲突，最终被公司开除。恰逢 Pixar 计划自制短片，以展示公司的软硬件技术实力，他就被招聘了进去（动画师都要经过多年的专业培训，收入不低，为了避免管理层的质疑，Lasseter 在入职时的职位是“界面设计工程师”）。

由于乔布斯对艺术情有独钟，而 Lasseter 又是公司里唯一懂艺术的人，因此自从公司被收购开始，两人的合作一直非常默契。

1986 年，公司决策要制作 2 分钟动画片展示公司的最新 3D 技术。他就按自己所长，按他天天相对最熟识的桌灯制作动画短片。

同行看完短片初稿后反馈说：“虽然是 2 分钟短片也必须讲故事”。

“短短 2 分钟，怎么可能讲故事？”Lasseter 想，但他还是按这建议，改成做成两把桌灯，一大一小，大的是父亲，小的是小孩。相互争夺一个气球，争来争去，最终气球被弄破了。（大家可能都看过，因为迪士尼 Pixar 制作的动画片通常都会在正片播放前放映这个短片。）

尽管在 NeXT 的工作繁忙，但乔布斯仍然抽出时间与 Lasseter 一起参加了 1986 年 8 月份的动画片大会 (SIGGRAPH)。结果这个短片大受欢迎，并获得了大会的最佳动画片奖。

乔布斯事后领悟到：Pixar 公司唯有这个短片才真正融合了艺术元素，而不仅仅是科技的展示。将艺术与

1988 年，尽管公司资金极度紧张，一直依靠乔布斯不断的注资，但他仍然为 Lasseter 的动画片部门拨款 30 万美元制作短片，但叮嘱必须做出优秀的作品。不负所望，《Tin Toy》得了 1988 年短片奥斯卡（学院奖），迪士尼高层想招聘 Lasseter 制作电影，但他不愿意离开 Pixar。迪士尼与 Pixar 开始谈制作玩具总动员 (Toy Story) 动画片。尽管乔布斯是一位谈判高手，但由于 Pixar 是一家小公司，急需新项目来维持运营。经过几个月的商务谈判，双方签署了商务合

同。由于迪斯尼拥有绝对的强势地位，Pixar 只是迪斯尼的合同工：迪斯尼拥有所有人物和电影片的版权，而 Pixar 则从票房销售中获得 12.5% 的份额。此外，迪斯尼有权继续以这个主题与 Pixar 合作制作两部电影，但迪斯尼拥有选择权，可以随时终止合作，而无需支付任何赔偿。

《玩具总动员》从 1991 年开始制作，但因为迪斯尼的负责人将里面的主角吴迪 (Woody) 变成一个有个性但缺乏人性的牛仔，导致团队辛辛苦苦做出来的初稿在 1993 年 11 月被迪士尼高层叫停。然而乔布斯并没有解散团队，他认为仍然有希望继续合作，并鼓励团队按照 Pixar 原有的思路改善动画片。经过 3 个月的努力，他们再次向迪斯尼高层展示，重新获得迪士尼的支持。1994 年 2 月，他们同意继续制作，但由于已经发生了大量的返工，最初的 1700 万美元预算无法完成了，乔布斯认为这些超支应由迪斯尼承担，因为这些超支是因为迪斯尼动画部门的管理失误引起的，但迪斯尼管理层不同意。最终经过多轮谈判与协商，调整了预算。乔布斯本来没有太多参与玩具总动员 (Toy Story) 的制作，但后来他越来越投入，后期初稿出来后，他每次邀请亲友们到家里看更新后的动画片。有亲友回顾说：“每次去乔布斯家都要看他展示动画片的最新版本。虽然可能只是优化了不到 10%，越来越觉得有点烦。”

1995 年乔布斯被迪士尼邀请参加大型动画片宣传活动，他觉得可以依赖动画片制作让 Pixar 公司获得投资资金，所以同时筹备 Pixar 公司在 Toy Story 开片同年上市 (IPO)。然而许多投资公司对此表示怀疑，因为 Pixar 公司在过去的 5 年里一直处于亏损状态。乔布斯回顾道：“当时确实有些紧张。也有人劝我应该等到第二部电影上映后再进行 IPO。”但乔布斯解释道：“我们迫切需要这些资金，这样我们才能与迪士尼就第二部电影进行谈判。如果我们没有资金，就无法在谈判中争取到更好的合同条款，我们将永远只是迪士尼的合同工，没有发言权。”《玩具总动员》大获成功，首周票房达到 3000 万美元，已经超过了全部制作成本，随后一直保持着当年票房最高的排名，在美国本土的票房达到 1.92 亿美元，在国际市场达到了 3.62 亿美元的票房，同时影评也给予了极高的评价。

参考 References

1. Fritz, Robert. *The path of least resistance for Managers*. (Berrett - Koehler Publishers, 1999)
2. Isaacson, Walter. *Steve Jobs*. (Simon and Schuster Paperbacks , 2011)

Chapter 26

北京手记

北京是全国的文化之都，来到北京后，晚上有空时我便去欣赏各种表演节目。连续三晚，我听了一场音乐会，看了两部话剧。

国家大剧院管弦乐团演奏马勒第三交响曲

到第六乐章的最后五分钟，两位定音鼓手齐声敲出“咚、咚、咚、咚”，仿佛为步兵擂鼓（又像赛龙舟的鼓手），与台上百余位乐手配合无间——包括 6 位打击乐手、40 位铜管和木管乐手。每位乐手都全神贯注地盯着台前挥动的指挥棒，没有丝毫松懈，将全场听众带入最后的高潮。

在大剧院音乐厅，几百名听众已聆听了近 100 分钟的演奏，经历了第四乐章女中音的独唱，以及第五乐章合唱团与女中音的合唱。乐章之间没有片刻休息。

回想上一次的类似体验，是在香港大学音乐厅欣赏贝多芬第 29 钢琴奏鸣曲（“Hammerklavier”）的演出，全曲长达 45 分钟。钢琴家演奏结束时，全场听众与演奏者仿佛刚跑完一场马拉松，彼此都想互相拥抱，庆祝共同完成了这场非凡之旅。

论家中拥有多么昂贵高端的 HiFi 音响设备和大屏投影，都无法替代现场音乐会的感受。

非常认同美国中提琴家 Kim Kashkashian 接受 YouTube 采访时所说的：“古典音乐的未来取决于现场听众与演奏者共同参与的音乐会（而不仅仅靠录制唱片、视频或 DVD）。”

正如话剧一样，这种互动才赋予了表演生命力。

马勒的第一交响曲大获成功，为他后续交响曲的创作奠定了蓝图：

- 使用超大规模的乐队
- 以歌曲作为主题
- 摒弃传统德国交响曲的格式，例如奏鸣曲式（sonata form）

他希望通过第三交响曲实现自己的创作理想。在奥地利萨尔茨堡（Salzburg）郊外阿特湖畔度假时，他终于从周围的高山湖泊、鲜花巨木和万物生灵中汲取了

创作灵感。

最初，6 个乐章都带有标题：

- 牧神苏醒，夏季昂头阔步地迈进
- 草地上的花儿告诉我
- 森林里的动物告诉我
- 人类告诉我
- 天使告诉我
- 爱告诉我

但在 1902 年首演前，他删去了所有标题：“希望听众直接从音乐中去感受，不受标题的干扰。”

马勒的交响曲创作印证了此前提到过的弗里茨（Fritz）创新理论：一切创作都源于一个崇高却尚未实现的理想，感受到差距（张力），从而激发计划与行动。

话剧：进入黑夜的漫长旅程 (Long Day's Journey into Night)

话剧表演结束后，我一看手表，已经快 10 点了。不知不觉中连续看了将近 150 分钟，没有片刻休息，现在才觉得屁股有些酸痛——因为坐得太久了。

虽然全剧只有一家四口——父母和两个儿子——但通过夫妻间、父子间、兄弟间间接连不断的对话，让观众从他们早晨的笑脸背后，渐渐感受到每个人的痛苦、无奈与绝望。尤其是父亲与小儿子 Edmund 在深夜交心时，父亲诉说自己小时候一家人从爱尔兰移民到美国的贫困经历，后来虽然赚了钱，却深知赚钱不易，结果被家人误解为“守财奴”，认为他不顾家。那一刻，我实在忍不住泪水，掏出手帕擦了擦脸。

这部戏是美国经典剧目，作者尤金·奥尼尔（Eugene O'Neill）生前三次荣获普利策奖。这是他自认为最好的作品，在他死后的 1956 年出版，并在 1957 年再次获得普利策奖。

如果只是借用别人的故事，内容很难有深度，必须植根于个人真实经历。奥尼尔年轻时考入著名的普林斯顿大学，一年后却因酗酒被劝退。正因如此，他才能如此深刻地刻画《进入黑夜》中长子 Jamie——一个酒鬼——的想法、心态以及酗酒的缘由。其实，他不过是借剧中四个角色，将自己的亲身经历传递给观众罢了。

田纳西·威廉斯（Tennessee Williams）的成名作《玻璃动物园》（The Glass Menagerie）也是如此，写的就是自己的家庭：轻度残疾的妹妹找不到工作，母亲想尽办法为她寻找伴侣等。

话剧：完美陌生人

虽然票价最贵（300 元），却是三晚中最平淡的演出。七位演员很专业也全情投入，但却未能打动我。这部剧改编自 2016 年的一部意大利电影，讲述了七位老友（包括三对夫妇）晚上在其中一家聚餐的故事。晚餐时，他们玩了个游戏：将手机放在桌上，公开分享收到的消息和电话。结果像炸弹接连引爆——揭露了家庭矛盾、混乱的男女关系等。起初觉得有趣，但揭秘越多就越感觉麻木，总觉得这些故事像是导演凭空想象，缺乏真实感。我也不喜欢演员全程佩戴麦克风

讲话，像在听晚间电台广播，少了与观众互动的氛围。

创作泉源

音乐、话剧等艺术形式为创作者和表演者提供了无限的创作空间。

从 400 年前莎士比亚五花八门的喜剧与悲剧，到 250 年前莫扎特的歌剧、交响曲、协奏曲中千变万化的主题变奏，无不让观众耳目一新。（近代披头士 Beatles、Bob Dylan 的歌曲也同样深深吸引着我們。）因此，我特别喜欢买票去看现场演出，亲身感受创意带来的震撼。如果我们观察小孩，会发现他们在成长初期想象力丰富、充满创造力，为什么长大后反而缺失了呢？

“这是教育的后果。”美国宾夕法尼亚大学沃顿商学院（U of Penn, Wharton Business School）的 Ackoff 教授说，“主要原因是他们要应付学校里的考试和功课。”

在学校，学生面对试题时，第一反应是猜测老师想要的正确答案。学校和老师会不断“教育”孩子追求标准答案，拿满分。

比如： $3 + 5 = ?$ 读过书的人会立刻回答 8，而不会思考问题的场景。如果不是十进制，答案可能是 10 或 12。

只有通过多听、多读、多看，不断拓宽视野，才能提升个人的原创能力。但千万不要局限于自己专业领域的内容。

有人觉得奇怪，为什么香港大学的数学系归属文学院，而不是工程学院或理学院？虽然许多工程和科学发明都要依赖数学，但数学与哲学、音乐也并非毫无关联。著名数学家伊藤先生曾说：“数学其实能帮我们发现大自然的壮美与真理，比许多哲学理论更贴近现实。”所以，我们不应给自己设下框框。

不断学习

要让学生有学习的动机和动力，最佳的方式就是让学生当“老师”来指导学生。回顾我 20 年前教授 PMP 和项目管理，以及 2018 年讲授 ACP 敏捷课程时，为了准备教材、给学员上课、回答问题，我在这个过程中确实加深了自己对敏捷项目管理的理解。

学习可以分成以下阶段：

- 数据 (Data)
- 信息 (Information)
- 知识 (Knowledge)
- 理解 (Understanding)
- 智慧 (Wisdom)

从理解数据信息，变为智慧。

培训过程中，与学生互动，解答他们的疑问，让我能仔细深入思考，从单是理解，提升到实际应用。

反之，有些大学生学编码，利用 AI 技术，搜索对应老师题目的答案，便交作业。（程序可能连编译都不通过）

大家不一定都有机会讲课，但都可以记录与分享经验教训。

这些真实经历，如果没有及时记录下来，后面就很容易忘记。假如我没有把刚

刚连续看演出的感受记在纸上，过了一个月后肯定都变得模糊。所以我也尽力每天把过程记录下来。

若想不断学习，除了多听多看外，还要多思考、多写，因为如果没落到纸上都只是模糊的印象。大家有每天写日记的习惯吗？

现在科技发达，比 120 年前民国前要用毛笔写字轻松多了，但当时很多伟人还是每天不停写日记。我们现在可以利用语音识别等技术，只花 15 到 30 分钟便可以把每天经历的重点记下来。

提示：虽然每天记录、写草稿，估计最终能有十分之一留下来发布已经不错。

在美国，越来越多年轻女性参加以前只有男性的体育运动比赛，导致很多女性因缺乏准备、锻炼和热身，会腿部前十字韧带受损，做紧急手术。记者 Cynthia Gorney 就想按这个主题给纽约时报杂志写专题文章。整个过程像漏斗一样：先把各研究、访问得到的相关资料放进漏斗，到了交稿截点前开始整理资料，把零散的资料组织成有架构的文章，分章节、如何开头、如何结尾等。然后杂志编辑会按读者群体的关注点精简初稿，比如她初稿也包括研究过程中她觉得挺有兴趣的替换膝盖手术，她甚至特意飞到 Cincinnati 观察医生做手术，但编辑觉得这些跟主题无关，非读者关注，把这些部分都删除掉。经过多轮的过滤、修正，最后在杂志出版的文章 4000 字，共 7 页。（详见参考书）

但是确实要靠这去芜存精的过程才可以保证发布文章的可读性和吸引力。

前面说过乔布斯在大学演讲的时候给学生提 3 个故事，第一个故事分享当初在大学学美术字（Calligraphy）时只因为很感兴趣，没有细想有什么用，到后来在苹果公司为 Macintosh 个人电脑设计字体时，才发现能用上。他用这故事说明很多时候过后回顾才知道以前那些点可以串联起来，对后面有作用。我也希望能在日后能用上之前自己的听、看、读、写。

500px

和上面话剧、音乐会一样，必须与其他人交流，不断完善才能出好东西。总的来说，越接触各领域大师的创作，就越觉得自己渺小，但只要知道自己是今个月比上个月有进步，比半年前有进步就可以了。

不是每个人都可以成为乔布斯或马勒、尤金·奥尼尔，但都可以成为本职工作的专业知识工作者。

我喜欢看演出的另一个原因是想感受他们的专业性：台上的音乐家、指挥、歌唱家、演员，和幕后的导演、戏剧作家、作曲家等。

专业和业余的区别：已经从多轮经验教训改善、精益求精，能让客户完全放心。要专业，除了具备本职工作的知识技能，也需要有前瞻性。要不断改善，除了具备相关知识以外，更重要是态度：必须有自我不断完善的愿望（self-motivation）和个人规范（discipline），只有这样做才可以保持自己的专业地位。

在当今竞争激烈、需求不断变化的大环境下，企业每位员工都要准备好自己，做专业 XX 师。员工要专业，也需要上面提到多听多读多看多写，打破现有的框框，全局了解客户的要求，动脑筋分析，不断改善，不然这些都只是梦想。

参考 **Reference**

1. Kramer. Mark and W. Call "Telling True Stories. " (Penguin , 2007)

Chapter 27

根与翼

中国社会一直非常强调家庭价值观，希望实现家族的持续传承，家族有族谱，代代相传的关系对每个家庭成员的成长产生深远影响。我们每个人都只是人类进化过程中的短暂过渡。父母普遍希望把最好的东西传承给下一代。然而我们需要问自己，什么东西能够持久，仅仅是财富和金钱吗？

继承大量财富，但如果不懂得如何正确利用，可能反而会削弱个人的竞争力，适得其反，最终成为社会的蛀虫。

《七个习惯》的作者史蒂芬·柯维（Stephen Covey）在书的最后一章里提到，我们可以传承给下一代并具有持久价值的，只有 2 样东西：根和翼。

“There are only two lasting bequests we can give our children - one is roots, the other wings.”

根

回顾人类过去的 130 年间，虽然经历了两次世界大战浩劫，但也催生了许多前所未有的创新：

- 电灯（煤气灯）
- 汽车（马车）
- 飞机
- 摩天大厦
- 半导体，集成电路
- 计算机
- 互联网
- 移动电话

我们现在使用的许多发明，实际上都是“发明者”基于前人的经验不断演化而来的成果。我在爱丁堡大学的古乐器博物馆见到过各种奇形怪状的古乐器，它们都是现代乐队中铜管和木管乐器的前身，如双簧管、单簧管、巴松管、圆号和小号等。就像达尔文先生在《物种起源》中描述的物种进化一样，经过优胜劣汰，最终演化成现代的乐器。

例如我们熟悉的口琴和手风琴，其实是欧洲人在中国传统乐器笙的基础上改造而来的。

人类创新并不仅限于前述领域，这只是我们看到的凤毛麟角，在医疗、电影、音乐、数学等众多领域，同样展现出了非凡的创造力。这些都是祖先留给我们的宝贵财富。

正因为每一代人传承并发展了前人的成果，我们才能取得现在的成就。这，便是我们的“根”。

翼

除了“根”的传承，祖先还留给了我们“翅膀”，赋予我们自由思考、打破负面基因传承的能力，而不仅仅是简单地将基因传递给下一代。

通过下面的两个故事，分别介绍两种不同的策略。

飞行实验

自古以来，人类就渴望飞翔，东西方对此都曾有过诸多尝试，但大多以失败告终。传说明代火箭专家陶成道在凳子上绑了 47 根火箭，并绑上风筝进行飞行试验，最终却因爆炸丧命。15 世纪，意大利的达·芬奇也曾尝试模仿鸟类翅膀设计飞行器，但也仅仅是停留在图纸上，并未实现。

莱特兄弟（Wright brothers）在 1903 年成功进行了机动飞机的首次飞行。在 12 月 17 日的第四次飞行中，飞机飞行了 300px。

莱特兄弟并非工程学或数学方面的天才，但他们为了实现“飞翔”的梦想，不断从失败中吸取教训并持续改进。1900 年，他们首次试验滑翔机，但效果并不理想。

1901 年，他们将机翼面积从 15 平方米扩大到 26 平方米，并成功飞行了 120 米，但发现试验数据与计算不符。到 1902 年 9 月，莱特兄弟已完成了 1000 次试飞，其中空中飞行时间最长达到了 26 秒。正因为有了这 1000 多次试飞的经验，1903 年成功试飞后，他们继续完善设计。

到 1905 年 10 月，飞机已经能够在空中飞行 39 分钟，并能在飞行员控制下完成转圈。

飞行就是在不稳定的大气条件下操控一台同样不稳定的机器，充满了各种变数，因此极其困难。然而，莱特兄弟通过不断的试验和数据收集，不断优化，最终实现了突破。

莱特兄弟的飞行故事告诉我们，他们的成功绝非偶然。除了强烈的意志和拼搏精神，更重要的是善于从失败中汲取经验教训，并通过持续的试验和数据收集，不断优化，最终实现了突破。

量子故事 Quantum Theory

1905 年，年仅 26 岁的爱因斯坦发表了《狭义相对论》，这或许是大家耳熟能详的成果。然而，同年他还发表了一篇开创性的论文，解决了物理学中的另一大谜团：当金属片受到光照时会释放电子，但若光的频率低于某个临界值 ν_0 ，则无法释放电子。这一现象无法用传统物理学解释。

爱因斯坦基于物理学家马克斯·普朗克（Max Planck）1900 年提出的量子理论，提出了突破性的假设：光由光子（photon）组成，每个光子携带的能量 $E = h\nu$ ，其中 h 是普朗克常数。只要光子的能量超过金属原子的逸出能量 W （一个固定

值), 便可释放电子。这解释了为何光的频率必须高于 ν_0 , 而与光的强度无关。在高中化学课中, 我们可能好奇为何元素周期表中, 氢 (H)、锂 (Li)、钠 (Na) 等位于表的左侧且具有类似的化学性质。波尔 (Niels Bohr) 的原子模型为此提供了答案: 电子分层排布, 第一层最多容纳 2 个电子, 第二层最多容纳 8 个电子。锂的电子排布为 (2,1), 钠为 (2,8,1), 它们的最外层均只有一个电子。这种电子层结构也解释了元素在三棱镜中折射出的特征光谱线: 这些光谱线对应的是电子在不同能级之间跃迁时的能量变化。

然而, 在更深入的有机化学学习中发现, 原子的结构远比上述描述复杂。电子不再像行星围绕核心旋转, 而是以电子云的形式分布在原子核周围, 其位置无法精确确定, 只能用概率描述, 有些分布的形状很奇怪, 如下图:

:L01.jpg

这一新的电子理论始于 1925 年, 当时分为两派科学家分别提出改进模型:

1. 海森堡 Heisenberg (波尔的学生) 和波恩 Born 提出用矩阵数学解释的方程式
2. 薛定谔 Schrödinger, 基于德布罗意 de Broglie 提出电子可以用波动方程描述的理论, 于 1926 年发表“薛定谔方程” (Schrödinger equation)

虽然两派互不相让, 觉得对方不够完善 (其实两派理论之间没有冲突)。

650px

上图是 1927 在 Brussel Savoy Conference 的合照:

普朗克 Max PLANCK (前面第一排, 左一) 用了 6 年时间研究黑体热辐射, 1900 年提出量子理论 (解决了以前光波理论无法解释的现象)

1. 爱因斯坦 Albert EINSTEIN (前面第一排, 左四) 基于量子论, 提出光子
2. 玻尔 Niels BOHR (第二排, 右一) 提出原子架构模型, 解释了元素周期表和氢的光谱
3. 德布罗意 Louis de BROGLIE (第二排, 右三) 电子有波的特性
4. 海森堡 Werner HEISENBERG (前面第一排, 左四) 与波恩 Max BORN (第二排, 右二) 1925 年提出用矩阵数学解释的方程式完善玻尔 Bohr 原子模型的不足
5. 泡利 Wolfgang PAULI (最后一排, 右四) 也在 1924 年提出不相容原理 Exclusion Principle, 原子中的每层轨道最多只能有 2 个电子, 如果 2 个, 它们的自转必须相对 $m_s = +\frac{1}{2}$ or $-\frac{1}{2}$, 来解释氮 (He Helium) 的特征光谱线
6. 薛定谔 Erwin SCHROEDINGER (最后一排, 右六) 1926 年提出薛定谔方程, 用波来解释原子架构
7. 狄拉克 P.A.M. DIRAC (第二排, 右五) 1928 年结合相对论, 和不相容原理, 提出更完善的方程式 Dirac Equation, 整合了 Heisenberg 的矩阵数学和薛定谔方程

从 1900 年到 1928 年, 不到 30 年的时间内, 主要由 9 位理论物理学家 (后来都获得了诺贝尔奖) 提出的量子理论帮我们理清了原子结构, 并提出不同的模型以解释元素和原子的特性。这些研究帮助人类更好地理解原子的物理特性, 并能更准确地解释各种化学现象, 如化学键的特性及为何各元素可归属为金属、

绝缘体或半导体。
要了解 4 个量子数的应用，可以依据下表列出某元素的电子分布：

$n =$

能量级，例如：1，2，3

$l =$

轨迹类，从 0 到 n-1, 例如：0=s , 1=p , 2=d

$m_l =$

具体某轨迹，例如： $l = 2$ 里有 5 个不同轨迹 $m_l = -2, -1, 0, +1, +2$
例如，X(Cl Chlorine 氯) 17 个电子的分布如下：

650px

也可以使用周期表，从左到右，从上而下，逐一把电子加上，左边 2 栋是 s 层，最右边的 6 栋是 p 层，中间的 10 栋是 d 层。例如 Chlorine，首先 1 层 2 个电子，得到 He；然后 2s 层 2 个电子，2p 层 6 个电子，得到 Ne；然后 3s 层 2 个电子，最后 3p 层 5 个电子。

700px

并可以利用 Schrodinger 公式，估计围绕原子核每层里电子分布的概率，得出以下 s 层，p 层，和 d 层的三维分布。例如：p 层的 3 种轨迹之间没有重叠；d 层的 5 种也一样。

650px

因为这些物理学家在 20 世纪初 30 年内所作出的贡献，大家更清楚每个元素的电子配置与周期表，这些已经是高中物理化学的基本教材。量子理论的重要性不亚于过去几百年间传统物理学家，如伽利略、牛顿到法拉第、麦克斯韦等的贡献。

== ==

虽然我们都继承了祖先的根与翼，但并非每个人都能为人类的持续发展作出贡献。要实现这一目标，需要具备哪些关键元素呢？

To be a powerful transition person, one must first change his inner first.
要成为有影响力的过渡者 (a powerful transition person)，无论采用哪类策略，我们必须首先进行由内而外 (只有在内部发生转变，任何人才能在自己可控的范围内进行创新和持续改善。) 以上源自 Covey 先生的《七个习惯》。

“魔术师”乔布斯 1997 年重回苹果，13 年后再把苹果带上高峰

1985 年，乔布斯被迫离开苹果公司，随后创立了 NEXT 公司。直到 1997 年，苹果收购 NEXT，乔布斯重返苹果（当时苹果正处于破产边缘）。到 2010 年，他成功将苹果公司带上美国市值最高的科技公司，但这期间也经历了多次失败。乔布斯在他的传记中说：“是什么力量推动我前行？我认为大多数创新者（Creative people）都希望向前辈致敬，感激他们对人类的贡献。

我并没有发明我用的语言或数学。我的食物并非由我自己生产，衣服也从未亲手制作过。我所做的一切都依赖于前人的积累。许多人都渴望回馈社会，在历史长河中留下属于自己的一笔。尽管我们无法创作鲍勃·迪伦（Bob Dylan）的歌曲或汤姆·斯托帕德（Tom Stoppard）的戏剧，但我们可以用自己的方式表达感激。试图用自己的才能表达内心感受，表达对前人的感激并做出贡献，这就是推动我的力量。”

共通点

为什么莱特兄弟、提出量子论的物理学家以及乔布斯，分别能够在航天、物理和电子产品领域实现突破性的重大贡献？他们的故事有两个共同点：

- 专注——例如：
 - 专心研究飞机
 - 解决物理难题，更好理解这个世界
 - 结合新技术研发针对个人市场的领先电子产品
- 长期锲而不舍——从萌芽到成功的经历充满挑战，期间困难重重，必须保持内心的坚定。也正是这种坚定使他们成为具有影响力的过渡者，延续了人类发展的根与翼。以乔布斯为例，从被迫离开苹果到东山再起，历经 25 年，通过推出 iMac、iPod、iPhone 和 iPad 等创新产品，最终让苹果再创辉煌（这种公司级的突破，往往需要更长时间的积累与努力）。

从量子论的发展看到创新，除了要有超凡人才的专注和不断努力外，还有以下特征：

- 创新不是从零开始，而是基于之前的理论基础再突破
- 要抱着怀疑和批评的精神，看到不完善之处，例如：
 - 爱因斯坦一直反对玻尔学派提出的量子力学不确定性原理，直言：“上帝不会掷骰子！”。他试图通过设计实验来证明这一理论的缺陷
 - 海森堡认为薛定谔的波函数方程（wave function）并不能完整描述原子中电子的所有特性，因而不完善的
 - 薛定谔 Schroedinger 反而觉得海森堡 Heisenberg 的理论太抽象

现在仍未形成一套完美的理论，能解释量子论的不足，但量子论已为我们带来了深远的影响，使我们能够理解许多过去无法解释的化学与物理现象。

==

“你的例子大多是超级明星级别的人物，“根与翼”这一概念未必能够完全适用于我们这些普通人。”

“为什么你认定自己不能成为有影响力的过渡者？”我问，然后讲我的故事：

“2000 年，我被公司辞退，沦为失业汉，当时我 39 岁。

从 2006 年开始，香港政府鼓励软件开发公司使用 CMMI 模型做评估与改进，并提供资助。我便开始专注于软件开发 CMMI 过程改进的培训与评估，但香港市场很小，大陆市场巨大，但从未跑过，我从长三角地区开始，与大陆公司合作提供 CMMI 服务。2010 年通过美国面试，成为 CMMI 高成熟度主任评估师。2016 年，我开始深入比较各种敏捷开发方法。2018 年，正值美国 PMI 推出 ACP 个人证书，我开始与培训机构合作教授 ACP 课程。学习前辈在敏捷、精益和软件工程的贡献，成为我的软件工程的根，没有他们，我不可能从零自创出来。

不固步自封，因市场环境变化，从传统模式改变为精益思路，是我的翼，这些都是我自身根和翼的经历，让我走过 24 年的独自创业之路，也帮助我出版第一本书。

所以任何人，只要把心态转过来，并能锲而不舍，都有机会。”

“好吧。但你说这些跟敏捷开发、持续改善有什么关系？我没有期望能成为爱因斯坦！”

“请问我们前面讨论过的例如估算、需求调研、开发流程、持续集成、测试自动化、配置管理等。是否有改善的空间？”

“听你这样说，反观我们团队，我立马想到有很多地方需要改进，为什么你提议只针对减少缺陷引起的返工做改进？”

要理解背后的原因，先听听质量大师 Dr Juran 回顾过去质量改善的成功要素：

- All Improvement takes place Project by Project 所有改进只能按项目逐个进行
- Backlog of Improvement projects is huge 潜在的改进项目非常多
- Quality Improvement does not come Free 质量改进并非免费
- Major gains come from the Vital few projects 主要效果（收益）来自少数重要项目
- Quality improvement extends to All parameters 质量改进可扩展到各类参数（如生产率、进度、安全性、环境等）

（以上源自 Juran 博士《Juran Quality Handbook》第五章质量改进。）

所以，如果从一开始同时对 4 到 5 个方向进行改进，反而会失去聚焦，难以尽早看到改进的效果。

但如果只针对降低缺陷引起的返工，目标明确，不仅让大家更容易接受，也更容易扩展到各个过程，例如需求、开发、产品集成，甚至配置管理。

因为能否引起高层对改进的兴趣非常重要，所以与企业领导首次见面时，我都会讲述团队应如何分析迭代缺陷数据，以此开始定量管理。

TED 演讲

今年被质量经理邀请参加他们的质量沙龙，但只有 30 分钟，我就用了 TED 演讲思路，用了 17 分钟讲软件开发公司的常见质量问题和改进措施。

你觉得下面 4 种工作，哪一类工种占的工作量最多？

1. 编码与代码设计。
2. 交付后的所有工作，包括维护、更新与缺陷修正。
3. 交付前的评审、静态扫描、测试与缺陷修正。
4. 项目管理与监控。

从 2012 年 Capers Jones 对美国软件公司的统计中看到，编码只排第二（占开发工作量的 25%），而测试、评审和缺陷修改的总工作量则排第一（占 30%）。你选 1. 编码，其实大部分耗时的不是写代码，而是修正错误代码。软件开发有其特点，缺陷发现越晚，修复所需的工作量就越大。缺陷如果在早期被发现，可能只需要花费半小时或 1 小时进行修复。原因很简单，当已经集成为一个十几万行的系统，里面有很多组件。如果在完成集成后使用时才发现 BUG，首先很难找到出问题的代码（原因很简单，当已经集成为一个十几万行的系统，里面有很多组件）；然后，为了验证代码修正是否有效，必须先测试单元模块是否通过，再测试集成后是否正常，最后还要通过系统测试才能交付给客户。相对而言，若能在自测、集成测试和评审阶段时尽早发现并解决问题，将节省大量时间。

然而，大部分缺陷都是在系统测试或集成测试时暴露的，有些产品线的缺陷数甚至达到四位数。如果能够将这些缺陷减少一半，将为公司节省近一半的工作量，同时也能改善产品质量，对吗？

道理大家都知道，但为什么没有改善？怎样才能改变现状，提前发现缺陷？这种改进绝对不是通过下达一个命令或领导开会讲讲就能做到的。我先讲一个故事：在一家上万人的大型公司中，质量经理表示公司一直非常注重量化管理，为此制定了各种 KPI 考核团队，每月由主管分析数据。开始时确实有些改善，但过了一段时间后，效果就不明显了，甚至发现数据造假的情况。所以，我告诉质量经理，如果希望实现量化管理，就必须从团队提升开始，而不是仅靠总体数据分析和制定公司级改进策略。度量与分析最主要的作用在于帮助团队做好根因分析，制定纠正措施，并在下一轮迭代中进行提升。例如，你们现在按 2 到 4 周进行一次迭代发布，确实也有一些团队在复盘，但他们不理解纠正措施与改正的区别，通常只是针对个别缺陷进行修复，结果仍会因为未修复系统的基本问题而导致缺陷和事故重现。然而，如果整个团队在每轮迭代中分析数据、找出根因并制定纠正措施，并在下一个迭代中验证这些措施的有效性，就能养成良好的习惯。最重要的是，领导要给团队足够的时间进行复盘。要完成这种根因分析的复盘通常需要 3 个小时，因为要收集数据，并进行分析和讨论。

我们可以先从分析缺陷排除率入手，帮助团队养成复盘分析根因的习惯。缺陷有真实数据，让团队分析为什么需求引入的缺陷这么多？引导团队使用“五个为什么”方法找出根因。同时，团队成员需要了解当前的质量水平。通常，问需求人员自己的质量水平如何？他们会说需求评审中发现的 BUG 很少，就代表质量好，但这其实是一种误解。他们在评审过程中没有发现并不表示需求没有缺陷，等到客户发现时会更糟糕。

我们还可以利用一些预测模型和统计分析的方法。在迭代中，不仅要让团队制定纠正措施，还要设定定量化目标。例如需求评审应该发现多少缺陷？单元测试应该发现多少缺陷？如果未达到预期目标，说明可能还是会遗漏不少缺陷。通过设定这些目标，可以更好地推动团队进行前期的评审和单元测试。如果团队开始养成分析根因并制定纠正措施的习惯，而领导又关注团队的持续改进，时不时地询问“下次迭代是什么时候？改进了多少？”，就可以扩大根因分析的范围。除了分析缺陷外，还可以把根因分析扩大到其他改进方向。那么，如何开始呢？

只是把小孩扔进海里，他是学不会游泳的。因此，建议先进行培训，举办为期两天的工作坊，利用一些真实数据带领大家进行根因分析的互动练习，然后安排公司内部教练辅导团队，帮助他们在迭代中做好复盘。

东北虎

任何提升都需要内心思维的转变驱动。改变固有的思想和习惯并不容易。如果个人、团队或企业缺乏基于事实（数据）持续改善的思维，敏捷宣言、12 条原则和各种方法（如 SCRUM、XP、ACP），和相关的培训和技巧培训，最终都无法在敏捷开发中产生实际改进效果。（如首章 A 公司故事）

但如果个人和企业领导不满意当前的工作状态，内心相信必须改善质量、有危机意识，认为这工作重要，那么相关的方法与技巧反而次要。敏捷团队在改善过程中，或许不必单纯依赖敏捷原则和方法，甚至可以创造出更合适的方法与技巧。

团队和企业要健康成长依赖以下主要因素：

* 高层要制订很高的量化目标，也要提供资源（包括给团队时间）。

- 改进要有专注。
 - 例如针对如何减少缺陷返工，尽早在评审和前期测试发现缺陷
- 有能力与动力的团队成员。

有竞争才有进步，不知道自身的不足，缺乏危机感往往是改善的最大阻力。

东北虎属于濒危物种，国家组织人工饲养予以保护。

专家曾试图将这些人工饲养的东北虎重新放归野外，但这一任务极为艰巨。它们是否能够在野外独立生存？要确保老虎在野外能独立生存，它们必须能够自行觅食。在自然环境中，鹿是它们的主要猎物。然而，

环境是影响团队动力的重要因素。看完这部纪录片，我不禁联想到一些本来充满抱负和动力的年轻程序员，他们同样深受环境影响。如果大家只顾加班赶进度，忽视质量，习惯于大量缺陷在测试阶段才暴露的生产方式，如何能期望这些年轻程序员注重代码质量呢？就像保护区试图将人工饲养的东北虎放归野外一样，这些新人在几个月后便适应了大环境，形成习惯，再改变就非常困难。所以若想团队做好敏捷开发，为公司带来价值，首先管理层必须意识到当前的不足：如果不立马做改进，便难以保持公司的竞争力。

总结

在软件开发领域，我们这一代人继承了自 20 世纪 60 年代以来硬件和软件的各种创新成果，为我们的发展提供了坚实的“根”。

敏捷宣言则指出了传统的瀑布式开发和计划驱动开发的局限性，为我们插上了“翼”。对于有能力的团队，这些方法提供了快速开发高质量产品的机会。

“请问你们这几年举办了多轮项目管理培训，有什么改进？”我与内部评估组成员一起按 CMMI 模型实践她无言而对，我接着说：“虽然你们培训计划，考试打分，满意度调查，培训平台，内部讲师等都有，每

所以无论
是个人、还是公司，
是改善、还是创新，
如果这个月没有比上个月有可衡量的提升，便应问自己为什么。

千万不要以为这个过程会一帆风顺，只要按计划执行就能达到目标。实际上，在实施过程中肯定会遇到各种无法预料的问题。只要明白当前的水平、分析根因、采取纠正措施并持续改进，就能不断接近目标，如丰田 7 个习惯中的：“持续改善永无止境”——就像龟兔赛跑中的乌龟一样。

丰田汽车在 90 年代初有 9 万员工，每年总共有 450 万条改进建议，平均每员工每年提出 50 条。但如果能坚持不断回顾、改善，应该能像乔布斯一样讲出你的动人故事。（详见附录 C：与毕业生讲故事）

参考 References

1. Covey, Stephen R. *The Seven Habits of Highly Effective People*. (Fireside Book , 1990)
2. Isaacson, Walter. *Steve Jobs*. (Simon and Schuster Paperbacks, 2011)
3. Juran, Joseph M. *Juran's Quality Handbook* 5 ed.(McGraw - Hill, 1999)
4. McEvoy, J. P. *Introducing Quantum Theory* (Totem Books, 1996)

Part IX

附录

Chapter 28

附录 A：绅士俱乐部过程改进

原因分析与解决

如果不想问题重现，便要分析背后的根本原因，并采取对应纠正措施。

实例

80 年代，美国学者 Dr Wheeler 到日本做培训，晚上好客的日本人请他去绅士俱乐部 (Esquire)。他发现年轻的女服务员腰上都有一个英文 Q 字牌子，便好奇地问原因。原来这家俱乐部刚完成几个月的过程改进，得到政府奖励金，所以都挂上这个 Q 牌（Q 代表 Quality 质量）。

Dr Wheeler 把这家俱乐部的过程改进故事总结成书（见参考），以下是重点内容：

Esquire QC 圈

QC 圈成立于 1984 年 12 月，由 Sakae 领导，下面的女服务员。

当时，Sakae 43 岁，有七个月的经验。除了她，圈内 7 女服务员的年龄从 19 岁到 23 岁不等。

用二八原则（Pareto 图）识别主要源头

- 清酒 (Sake) 与啤酒 (Beer) 是最大的源头。

500px

600px

每月啤酒损失量

600px

每月清酒损失量

600px

结算出平均每瓶啤酒 232 日元，每公升清酒 1,653 日元

600px

啤酒和清酒每月总损失（日元）

600px

从 10 月份到次年 2 月份，啤酒和清酒的平均每月损失约 49,000 日元。

建立目标

目标：把损失减半（少 50%）：从目前每月平均损失 89 瓶啤酒和 17 升清酒，降到（不超过）44 瓶啤酒和 8 升清酒。

根因分析 **Root Cause analysis**（鱼骨图分析）

- 大家头脑风暴，讨论分析啤酒和清酒损失的主要原因。
- 根据服务员、工作流程、其他等三个主线分析。

这三组构成因果图的三个主要分支。

600px

鱼骨图每个分支的主要内容如下：

600px

600px

600px

600px

基于上面的鱼骨图分析，大家讨论并想到了以下主要理由/原因：

- 没有具体的固定位置放置账单。
- 依赖别人。
- 处理热清酒器不当。
- 价格计算有误。
- 分工有误/没有分配好工作。
- 准备不足。
- 对客户桌结算（喝了多少并收拾空酒瓶）频率不足。

注意：以上每一项都是问题的根本原因，很多人误以为针对问题本身，找改正措施便算根因分析。“5 W

改进措施

- 要确保留票据在桌子上，不带走（在繁忙时段，在纸上标记消费了多少矿泉水、啤酒和日本清酒，贴在总单的后面，利于计算总账）。
- 制订规程来分配各个区域的工作。每小时每个区域负责人必须检查一次所有范围内的票据。
- 送热毛巾的那位服务员负责记录新加入的客人。
- 谁接单谁负责记账，如果她要求其他人来协助记录，必须确保沟通好，检查是否已经做好记录。
- 管理者必须对新成员提供培训，如新员工小组学习。
- 以区域分桌，使大家清楚谁负责哪桌。

改进效果

600px

改进后啤酒损失降低到平均每月 20 瓶

600px

改进后清酒损失降低到平均每月 0.6 公升

600px

结算出平均每瓶啤酒 232 日元，每公升清酒 1,653 日元

600px

600px

以日元计算，啤酒和清酒的损失从每月平均 49,000 日元减少到 5687 日元，减少了 43,401 日元，即 88%。这一降幅远超过最初设定的 50% 的目标。

A3 报告

很多给管理层的报告都非常厚，信息太多导致管理者难以消化，所以丰田要求所有管理报告都必须精简到一张 A3 纸。照片中服务员手上的就是 A3 报告——包含了根因分析的重点。例如你可以看见其中有鱼骨图。

对应 CMMI 实践

文件:CMMIHM 02.png

- 利用二八原则，针对清酒、啤酒分析问题
- 使用鱼骨图头脑风暴，识别主要根因
- 对应 7 条主因，制定纠正措施
- 收集改进后 3 至 5 月份数据，与之前 10 至 2 月份比较，发现啤酒、清酒的损失量与总金额都有显著改善
- 汇总成 A3 报告（A3 纸是 A4 的一倍，含 4 面 A4。）

参考 References

1. Wheeler, Donald J. : SPC at the Esquire Club (SPC Press 1992)

Chapter 29

附录 B：过程改进工作坊

背景

某国企事业单位因以往发生了不少系统问题，管理层便决定花一年时间做改善，提升软件开发质量。

我和咨询师给这家单位做了第一次现场的首次培训和 CMMI 差距分析后，咨询顾问发给我他制定的短期和长期（一年）的改进落地计划。

咨询师：因为客户年底项目都很忙，抽不出时间，所以我们在年底前计划辅助改进组长编写相关的过程文件和模板。

我：这家客户是希望持续改善，后面能有质量提升的，上次我们现场不是和他们朱主任强调，如果要做真正的改进就需要在一月底做个 3 天工作坊，制定出改进的目标和行动计划初稿，加快每组改进的速度。12 月应尽快向高层申请立项，正式启动这个过程改进项目，包括预估如能把遗漏到客户的缺陷减半，估计能节省多少质量成本。工作坊要有效果，准备与策划很重要。但我看你的计划还只是针对各个过程做辅导，编写过程。不然，评估后还是和他们 3 年前做完 CMMI 一样，没有任何改善，一年后发起人看到没有任何效果，肯定会怪责我，我们两无法控制他们公司内的资源，所以必须尽早预警，告诉他们看到的风险。

咨询师：我也想趁这个机会做个案例，但我没有经验，可否指点一下这种改进落地的重点？

我：我也想一起合作在大陆做个成功案例。持续改进的道理很简单，就是如乔布斯谈持续改善时说——质量提升的道理其实很简单，是一个重复的过程，然后我们需要不断去看，有哪些无效的环节要省略，哪部分要重新设计，不断试验、提升，就这么简单。重点是所有的提升都应该是科学化的，有数据而不是泛泛而谈。——听起来很简单、容易，但其实困难重重。因为要改革便要改变工作习惯，改变习惯很困难。所以关键并非教他们如何写好过程文档，提供模板和检查清单。我刚刚在本月中为一家香港公司和它广州离岸开发中心两地一起做了两天工作坊，从这个故事你可以理解到要工作坊成功必须关注哪些要素。

案例

公司背景

案例公司的母公司在香港，子公司在广州，主营业务是为客户提供不同的 IT 方案。

除了 Jackie 领导的 PMO 是一个支持的组织外，公司还有 4 个小组，其中第 3 组是最成熟的，它专门做政府 XX 管局的项目，已经有很成熟的改进小组，定期开会讨论。组员会讨论包括如何降低缺陷，有什么客户问题要解决等。因为比较成熟，我自己参与不多，下放给第 3 组的组长 Joyce 管理。现在提到的每一组，其实人数都在 100 以上，赶上一家小公司的规模，每年收入也是起码一亿港元。第 2 组专门是针对商务客户做外包性开发。

第 1 组主要是做香港政府的大型项目。虽然我们一直推动香港的政府机关用敏捷，但是碍于很多原因，大部分项目还是瀑布的。虽然有时是用类似迭代的方式，需求调研后就开始逐步分迭代去交付。暂时没想到这些团队里面有哪个可以做领导，带领改进组，这组的人因为是做政府项目，也比较难推动的一组。最后就有个人数较少的第 4 组，专门是做数据调研，数据挖掘的，所以他们人工比较贵，也非常忙。

:Asl 1.jpg

与高层电话交流

因为与总监 Tommy 已认识超过 10 年，我拨通后便直接问：我上周到了你们广州开发中心，你是否希望后面能有过程相关的质量度量，以数据说话？

答：很希望。公司这么多年（51 年），肯定会有些员工没有什么进取心，虽然比较无奈，但他一直服务客户，客户也觉得很满意。所以如果有一些客观的度量，我们在年底考核的时候就可以启用。

我：很好的思路，但要小心这些指标，千万不要细分到个人，这样会影响度量本身的正确性。考核指标必须能客观衡量整个团队的成绩才有意义。现在你们已经有的度量指标是哪些？

答：1、团队的收入。每个部门还是要负责盈利的；2、人员的使用率。我们有 TPMS 项目管理系统，就按在它上面填的东西来计算人员的使用率；3、我们每 3 个月会做客户满意度调查。

我：你有没有发现除了客户满意度以外，缺少了项目质量的一些客观指标。我建议争取过程改进小组对每个组都开始以客户缺陷降低返功率为目标，从数据看团队是否能做到改进。因为你们一直都有用缺陷管理系统统计缺陷，所以那些数据应该是有并且准确的，可能缺少就是一些返工工作量，后面要想怎么可以收集到？

答：我赞同。

我：你们现在有做迭代回顾吗？大概多少时间？

答：1 个小时左右吧。

我：不够。如果我们要做根因分析，起码要 3 个小时，你估计可以增加吗？

答：比较难。比如客户部门要求我们的开发人员的时间只能用于做他们指派的工作。如果我们要求把回顾的时间加长，客户就会说这个为什么你们自己改进的事，要在工作时间做？要我买单？但是我们另外有每个月的一些评审会、回顾会，一般都是讨论一些各方面的问题，不仅仅是技术的，这个会通常就有 2

个小时了。

我：可不可以在这段时间改成 3 个小时，但是不是平常的开会形式，而是让团队讨论、找根因。就按你的安排，请你们定好每一个改进小组的人员，然后每个月定期监督改进的进展，让他们跟进目标。我们就以缺陷改善减少返工为主要的改进目标，另外再配上做好配置管理。因为现在配置管理，4 个组之间差异很大，比如第 3 组是最成熟的，过程步骤都很规范。但是做数据分析那组基本上就没有做配置管理。你们公司虽然有 50 年历史，但没有保留太多分享知识库。所以每个小组也要在过程里面，把经验教训、根因分析报告、中间的过程等写在类似 Wiki 的一个平台。

答：我们其实有很多平台提供给员工，当时他们就没有动力去写去分享。

我：这个也正是在过程改进小组要推动的，他们不仅要达标，也要把整个过程记录下来，留下数据，给其他人参考，成为公司的资产库。

答：我计划 9 月初去广州，到时候就正式成立这些小组，然后你们的咨询老师可以在启动会后开始安排培训和辅导工作。

我：一言为定。

一星期后，召集所有部门领导远程会议

会议开始先让总监 Tommy 跟大家说过程改进的目的，接下来我用 35 分钟讲以下改进故事：（共花了 45 分钟）

- 展示项目大部分缺陷都是到后面系统测试阶段才发现
- 如能尽早发现解决，能节省大量返工的浪费
- 项目团队都没有在迭代复盘分析根因，制定针对缺陷返工的纠正措施
- 所以针对缺陷排除的改进不仅仅提升质量，也减少无谓的返工，提升工作效率

会后，我请每个部门提供改进小组成员，由公司内部协调员统筹。

晚上我接到其中一位领导电话：“年底大家都非常忙，本来我们不是说好不需要每个部门提供 EPG 参与今次 CMMI 评估，为什么现在你跟总监说我们要派人？”

我解释说：“请不要把 EPG 与这次要成立的改进组混在一起。对 EPG 的要求没有变化。改进组专门针对团队与企业的持续改善落地。因大家都认为 EPG 是针对评估需要的安排，目标是通过评估，拿到证书，所以大家都想尽量节省人手。但如果改进组能帮助部门节省人力和成本，是长期的持续改善，他们无法拒绝。”

用 2 天工作坊促进改进

2 天工作坊实况

第一天工作坊从 9:30 开始，先安排香港、广州的学员同步玩小组传乒乓球游戏，让大家动起来。10 点总监 Tommy 先发言说明改进的目的、他的希望。接着由我解释每人手上材料：工作坊的 4 个原则，2 天的安排等。

400px

10:50 香港广州共 5 组便开始讨论按资料做第一个互动练习。

第二天 15:15，每个团队 ABCDE 组最后一个小时准备行动计划，要在 16:15 和

总监汇报，每个组全体全心投入最后冲刺。

依据之前总结和分享的共同点，写上具体的短期和长期目标和行动计划，按照我们的模板也配合他们广州的开发团队或测试团队的一些输入，加在他们的报告当中。

16:00，总监进入香港大会议室，专心听完每个小组的分享后，提出一点很重要的概念：“从管理的角度，所有的改进必须还是为团队省钱。假如你可以降低缺陷、减少返工，应该可以节省一些本来的人手，就可以做好你们的质量改进、模板或者提升工作，而不需要再要公司增加人手。所以对公司对员工都是有利的。”

所以我会不断提醒各组，所有的目标必须量化，并且要跟我们的 2 天工作坊的主题——提高质量、降低缺陷、降低返工工作量相关。整个发布和改进目标就有方向了。

我问每组的部门经理，觉得这两天怎么样？

“第一次有这种抛开所有日常繁琐事务，专心两天工作坊的体验，这两天也首次听到一些比如广州的情况，以前是没注意到的，所以得益很大，也很值得去花这个时间。”

因为我一直也跟总监有交流，他也觉得这种方式可以体现出他希望每个改进小组可以按自己的想法定自己的目标和计划，而不是由他硬推。

会后跟进

第二天结束后，我们立即用相机拍下每个部门的目标与行动计划，并按模板记录在电子表单，另外再在 Wiki 平台上建立香港和广州的 Wiki 页，让大家上传 2 天的相片和心得。Wiki 上也能下载 2 天的练习与参考资料。

香港照片与分享 Wiki 页节录：

650px

广州照片与分享 Wiki 页节录：

650px

同时发邮件给发起人和 4 位部门经理，感谢他们全程参与外，也强调必须每月跟踪进展，不然便会浪费了大家 2 天的辛苦投入。

回顾

我深深的体会到，工作坊培训要有效果、要成功，管理层的参与非常重要。如果没有约好手握资源的总监 Tommy 来参来、听汇报，如果没有每组的组长负责人在现场推动，只有广州的开发人员，或者每个团队自己内部的小组长、组长参与的话，整个 2 天浪费了。因为你谈什么都没用了，没有主要干系人的亲身参与，后面肯定就没有下文。如果这个 2 天的工作坊只是单独在广州开意义也不大，广州只负责开发、交付、测试一部分，他们也没有全局的概念。比如很多时候跟客户打交道、听需求还主要靠香港的员工。幸好我在工作坊的前一周，经 PMO 在午饭前召集 4 位部门经理，除了确保他们自己参加和也让他们领导报上会参加的关键人物。

保障出勤

在香港，因为有 PMO(项目管理办公室) 主任 2 天全程参与，他很有经验，负

责确保学员全程度参与。如果确实有客户会议要提前跟他申请，不是随便的来去。我们一开始也提醒大家，在 2 天里面尽量避开电话骚扰之类，确保小组成员可以专心参与。这很重要，确保每个小组是注重这件事。因为很多公司，之前的培训都是“兴趣班”形式，反正搞了培训，你有时间有兴趣来听听，效果都不好，也没有什么成果。有公司领导确保出勤，并尽量抛开每天各种骚扰电话，才能专心投入这个工作坊。

总结

如果领导关注质量改进并支持，首先要估计改进能带来多少收益，估计能减少多少返工成本，减除投入的成本，包括人力资源、培训、辅导等，然后申请立项，才可以把改进当成正式项目来策划和监控。

把所有关键利益相关者都拉到一起，用 2 到 3 天的工作坊培训，促进部门尽快制定改进目标和行动计划，挑选项目做试点，并预估何时可以取得效果，再大规模推广。

因为大家第一次都没有经验，不知道如何入手，工作坊可以让所有人一起讨论和制定改进目标与计划。本来可能需要几个月来回讨论修正，现在只能压缩到 2、3 天。

也因为所有干系人都要参与制定目标，他们会更有动力去按行动计执行。

附件

互动工作坊 (workshop)

原则

- 高层的支持 (Top Level Buy-in)：高层参与整个过程，也表示高层的重视。
- 从整系统入手 (Improve Whole Systems)：要企业改变成功，仅仅抓一个小模块是没用的，要求岗位相关人员都参与，让所有人都看到整个相关系统。
- 破冰-让人了解需要改变 (Unfreeze - need for change)：因为是工作坊形式的互动讨论，经过 2-3 天可以让所有相关人都了解改变的前因后果，和参与决策
- 整体参与 (Everybody solves problems)：整个团队都参与到讨论中，包括高层。而不是传统的高层策划再向下推广，不然没有参与，就没有承担的心，提高学员对改进的承担 (commitment)。
- 自己动脑筋：主持人/领导不会直接告诉你答案，而是通过一些讨论和分享的方式，引导学员自己找出痛点，分析并找出解决方案；每组都可以天马行空地出主意，但也会面对来自不同等级人员和评委的残酷批评
- 做实验/试点/游戏 (Experiment/pilot)：把平常要半年到一年的改进项目压缩到 3 天，但关键的重点都一起讨论过。但也好像一次有奖游戏，激励团队的参与

特点

- “把所有的相关者凑在一起 ‘Whole System’in the room’——所有相关人员都参与。例如：项目管理过程改进参与者不仅仅是项目经理，还有利益相关者，包括团队成员、供货商、客户代表、上下游、其他支持类部门等，可能都要参与

（目的：所有与系统相关的人都能对全局有概念，而不仅仅看到自己的部分）

- 顾问/老师不直接给答案/方案，只是引导、帮助学员找出方向
- 用分组竞赛形式提高成员参与的积极性
- 所有人都可以提出批评和意见，不仅仅是从上到下的决策/教育，接纳所有的思路和意见（像一个镜子）
- 分 4-8 组，8-10 人一组，需要有一个大而光亮的会议室，方便小组互动，大白板和架子，各种颜色的水笔，墙壁可以贴上小组的白纸等
- 培训期间里都最好不受外界打搅，例如每人都上交手机

成功关键要素

1) 预先要有充分准备：

- 建立行动委员会负责准备这次活动，活动的具体设计，在公司推进整个准备活动
- 主持人/协调人-对整个策划/准备/设计做辅导, 现场活动包括 (~ 6 - 7 天):
 - 2 天的策划会议
 - 1 天详细设计
 - 2-3 天工作坊现场辅导（包括事前的场地准备）
 - 1 天的事后回顾/跟进

2) 参会人员是否能够代表各相关角色（能否包括整个系统，整体参与？）

3) 高层是否全力支持和参与？

Chapter 30

附录 C：与毕业生讲故事

2005 年，乔布斯获颁史丹福大学荣誉学位，他在典礼上跟毕业生分享了三个故事：

故事一：

高中毕业后，我 17 岁进了学费昂贵的私立里德大学，但我只读了半年，一方面觉得学非所用，另一方面也不忍心花掉父母一辈子的积蓄，就选择了退学。但我并没有离开学校，而继续在学校里旁听感兴趣的课。我没有收入，就睡在同学宿舍地板上，同时靠捡玻璃瓶、可乐罐挣点小钱糊口。因平日吃不饱，每个星期天，我走路到距离七英里的一所印度寺庙去吃一顿施舍饭。

大学的美术字（Calligraphy）课程很有名，我去旁听后就立即迷上了。尽管当时我还不知道它将来会有什么用，但是在设计苹果的麦金托什（Macintosh）计算机时，我想起了当年在大学里旁听的美术字课程，并为 Macintosh 个人电脑设计了很漂亮的字体。

十年后回首，这些生命中的小点都串联在一起了，尽管在之前我无法预见到它们会串联起来，只是在事后回顾时才能看到。因此我们必须相信这些小点会在生命中的某个时刻连接起来，才有信心根据自己的信念勇往直前。___NOTOC___

故事二：

在我 30 岁的时候被苹果公司赶了出来，当时感到非常失败，没有面子，也对不起其他的创业者。然而由于我对电脑和 IT 仍然充满兴趣，我决定自己创业，继续做产品研究，专注于在工作站方面开发世界一流的产品。回顾过去，如果我当时没有离开苹果，那么我可能就无法体验后来我做产品开发的许多概念。现在回想起来，我一点也不后悔。最重要的是，你必须对你所做的事情充满热情并爱上它，无论是工作还是寻找伴侣，这一点都是相同的。

“有些时候，生活会拿起一块砖头向你的脑袋上猛拍一下，不要因此失去信仰。我很清楚，支撑我一路走下去的，是那些我所爱的东西。你需要找到你的所爱，工作如此，爱人也是如此。你的工作将会占据生活中很大一部分。你要相信这份工作是伟大的，你必须先热爱它；你只有坚信自己所做的是份伟大的工作，才能怡然自得。如果你现在还没有找到，那么继续寻找，不要停下。只要全心全意地去找，在你找到的时候，你的心会告诉你的。”

故事三:

我 17 岁时,听说应把每天当成你生命最后一天。所以我每天都会对着镜子问自己,回顾是否已经用好当天,做最重要的事情,如果连续几天都否定,就必须马上调整。现在回想起来,如果我能坚持这种想法,就可以不断地激励自己只做真正重要的事情。每隔几天,如果我发现时间被浪费在了不是最重要的事情上,就会立即进行调整。

“死亡是每个人必然的终点,没有人可以逃脱。死亡是生命中最好的发明,起新陈代谢作用。记住,每个人都会在不久后死去,这对我产生了重要的影响:当我做重要决策时,所有的顾虑,如其他人的期待,对失败的恐惧以及没有面子等,相对于死亡来说就都变得微不足道了。当你意识到生命的有限时,就没有理由不听从内心的指引,专注于最重要的事,因为其他一切都不再重要了。”

回想我自己过去的人生旅程，也能说出我的故事：

故事一：

我预科考大学的成绩非常出色，除了语文外，物理、化学、数学全是 A，进香港大学选读任何专业都应没问题。我父亲没上过大学，受传统观念的影响，极力劝我读医科（在我祖父母那一代，香港大学可选科目不多，学费高昂，一般家庭难以负担，所以父母都希望子女学医，毕业后收入稳定，社会地位也高）。但我从小对生物和化学的兴趣不大，反而对数学情有独钟，本想去麻省理工读计算机专业，但负担不起。当时香港大学还没开设计算机系，最终我选择了电子电机工程。尽管父亲强烈反对，甚至威胁说若我不选医科就跳海，但我经过深思熟虑，还是坚持了自己的选择。

由于香港的工业不发达，对科技重视不足，电子工程毕业后很难在本地找到好的发展机会。因此在校期间和毕业后的十多年里，我时常会怀疑，若当年听从父亲的建议选了医科，或许会有更好的发展。

大学毕业后，我在夜校读管理文凭时，首次接触到各种管理理论，如 XY 理论，但不甚了解。

1998 年，我在香港富士通工作时开始接触软件工程，对许多概念不甚了解但却感兴趣，于是兼读了软件工程硕士，开始学习敏捷开发、面向对象编程、软件易用性和软件质量等。

2010 年，我阅读了麻省理工学院教授 McGregor 的经典著作《The Human Side of Enterprise》，才能了解 XY 理论的原理。开始涉足组织开发领域的各种实验与研究，还看了心理学的公开课视频。

2017 年，我接触大数据分析，发现要理解其背后的原理，必须掌握线性代数（又称矩阵代数），便开始在网上搜寻相关视频。2018 年，我在麻省理工学院的公开课（MIT OCW）中找到了数学教授 Strang 主讲的线性代数课程，共 34 讲。我很欣赏 Prof. Strang 的教学风格，他用粉笔在黑板上边写边讲，于是我一有空就看视频。

我一直保留着读预科时的《矩阵代数》，作者正是 Prof. Strang。这本参考书至今仍很实用，内容无需更新，因为数学不像工程或技术，原理永恒。除了重读这本书，我还买了 Prof. Strang 为本课程新出的书，花几个月看完了近 30 个讲座。

如今回看，当年我选读工科还是对的。如果选了医科，我就没机会在五年前重学线性代数、统计分析和大数据这些我感兴趣的课程，也无法在过去二十多年在大陆长期出差，做培训、咨询与评估。

从预科起的经历，不仅仅帮助我的咨询、培训工作，也为我写书打好基础。起初只是出于兴趣，事后回顾才能看出每一点的贡献和意义。

故事二：

我曾在 40 岁那年被公司开除，当时是富士通的业务经理。

起初我很迷茫，觉得特别沮丧——从刚毕业时的大学精英，一下子变成了无业的失业者。现在回想，如果没有在 40 岁的时候从零开始，估计我如今也会像其他同学一样，在 50 多岁就不可能再找到工作，被迫退休。

故事三：

十多年前，我观看了话剧《最后十四堂星期二的课》，又读了原著《Tuesdays with Morrie》。与家人去爱尔兰度假时，我开始思考生命的有限性，意识到人无法掌控命运，随时可能面对死亡，所以必须抓紧时间，把想写的东西尽量记下来。（Covey 先生在经典著作《Seven Habits》中提到的习惯二——“以终为始”（Begin with the End in Mind, Principles of Personal Leadership）有个场景：你参加自己的葬礼，看到家人站在亲友前，读出你的一生。）

我开始录音，写分享文章。起初因中文基础差，写作速度很慢，只能试着写些简短的文章。但随着阅读和写作能力的改善，开始能组成目录架构，并开始了解好书要像交响曲一样，要有节奏，并能前后呼应。

每个人的成就，不在于看他是否毕业于名牌大学，而在于能否找到他热爱的事业，全力付出做到最好，并定期回顾。因此，我们应该把每一天当作人生的最后一天，只专注于最重要的事。
